Understanding ML: The Experiments

Train / validation / test, cross-validation, learning curves, bias–variance experiments, hyperparameter tuning, and why the test set must not be touched: what is manipulated, what is measured, what the curves mean, what people do wrong

Fundamentals of Machine Learning, midterm

2026-11-03

What this document is about, in one paragraph

Every method of Lectures 1–5 is judged by the same experiment: fit on some data, measure on other data, and vary one thing at a time. The thing we vary is called the *manipulated variable*: the amount of data N, the capacity (degree, depth, k, λ), the number of training epochs, the number of trees, the split protocol. The thing we measure is always a *risk* (an average loss) on a set of examples, and the whole art is choosing *which* examples, because the examples that chose the model cannot also grade it. This document describes each of the standard experiments as a procedure you could run tomorrow, says what the plot looks like, how to read each shape, and what people do wrong. Formulas marked [on your sheet] are on your two-page sheet; do not memorise them, understand what they measure.

Contents

1 The one experiment behind everything **2**

2 Train, validation, test: the split **3**

3 Why the test set must not be touched **6**

4 Cross-validation **8**

5 Hyperparameter tuning **10**

6 Learning curves: vary the amount of data **13**

7 Bias–variance experiments **15**

8 The complete protocol, start to finish **17**

1 The one experiment behind everything

Risk on a set of examples

A model f and a loss ℓ give, on any set of examples S, the *empirical risk* $\widehat{R}_S(f) = \frac{1}{|S|}\sum_{i\in S}\ell(y^{(i)}, f(x^{(i)}))$ [on your sheet] and, on the whole population, the *population risk* $R(f) = \mathbb{E}[\ell(Y, f(X))]$ [on your sheet]. The population risk is what we want (performance on the next case) and can never be computed; every experiment is a way of estimating it with an empirical risk on examples the model has not used.

Purpose. To replace the unknowable number $R(f)$ by a number we can compute, without fooling ourselves.

Impact. The choice of the set S decides whether the number is honest. On examples the model never saw, $\widehat{R}_S(f)$ is an unbiased estimate of $R(f)$; on the examples that chose f, it is optimistic (Section 2).

The repeated-sampling thought experiment

Imagine drawing a training set $\mathcal{D}$ from the world, running the whole learning procedure, and getting a fitted model $\hat{f}_{\mathcal{D}}$. Now imagine doing it again with a fresh $\mathcal{D}'$: you get a different $\hat{f}_{\mathcal{D}'}$. Every quantity in this course that has a hat is a random variable through $\mathcal{D}$. "Bias", "variance", "optimistic", "unbiased" are all statements about what happens *on average over such repetitions.*

Purpose. To make "sensitive to the sample" and "honest estimate" precise: they are averages $\mathbb{E}_{\mathcal{D}}[\cdot]$ over training sets.

Impact. Any experiment you run once gives one draw. Repeating it with different seeds, folds or bootstrap samples is how you see the variability that the theory talks about.

The generic learning experiment

Setup. A dataset, a learning procedure with one knob h (degree, k, λ, depth, epochs, number of trees, or the amount of data N), a loss, a metric.

Held fixed. Everything except the knob: the same rows in each subset, the same preprocessing, the same seed where randomness enters, the same metric.

What we manipulate. The knob h, one value at a time, over a grid.

What we measure. Two numbers per value of h: the risk on the examples used for fitting (*training* risk) and the risk on examples not used for fitting (*held-out* risk).

What the plot shows. Two curves against h. The training curve, and the held-out curve above it. Where the two sit, how far apart they are, and whether they still move, is the diagnosis.

How to read it. Three questions in order. (1) Are both curves high? The procedure is too restricted or the representation is poor (underfitting). (2) Is the gap between them large? The fit depends on the sample (overfitting, high variance). (3) Are the curves still moving as the knob increases? You have not reached the regime where the knob stops mattering.

The four curves you will be shown, and what is on the horizontal axis

Curve	Manipulated variable	Typical shape	What it diagnoses
Validation (complexity) curve	capacity: degree, depth, $1/k$, $1/\lambda$	training falls monotonically; held-out falls then rises (U)	the best capacity; underfitting on the left, overfitting on the right
Training curve	epochs or boosting rounds	training falls; held-out falls then rises	when to stop (early stopping); same U over time instead of over size
Learning curve	amount of training data N	training rises toward a plateau; held-out falls toward the same plateau	bias (plateau height) versus variance (gap) versus data-limited (still converging)
Ensemble-size curve	number of models M	held-out falls then plateaus	when more models stop helping; the floor $\rho\sigma^2$

Mixing these up is the most common reading error: "the error goes down with more" means nothing until you say more *what*.

Key insight. Every experiment has the same skeleton: hold everything fixed, vary one knob, measure the risk twice (fitting examples versus unseen examples), and read the two curves. The method you are testing only decides what the knob is.

How the professor asks it

Q. *A colleague reports that "the error decreases when we increase the model". Explain why this sentence is not yet a result, and what two numbers you would ask for.*
A. It is not a result because it does not say which error. Training error *always* decreases (weakly) when capacity increases, because a larger nested family contains the smaller one's best fit [on your sheet]; so a falling training error carries no information about new cases. Ask for the held-out error at each capacity, and for the gap between the two. If the held-out error also falls, capacity was the limiting factor (underfitting); if it rises while the training error falls, the extra capacity is fitting the sample (overfitting).

2 Train, validation, test: the split

The three subsets and their roles

Split the available examples into three disjoint sets. The *training set* fits parameters. The *validation set* chooses between alternatives (hyperparameters, features, model families, thresholds, stopping epoch). The *test set* is used once, at the end, to report the performance of the single chosen model. [on your sheet]

Purpose. Each subset answers one question, and each question would be answered dishonestly by the examples used for the previous one.

Impact. Training examples give an optimistic number for the fitted model; validation examples give an honest number for *each* candidate but an optimistic number for the *winner*; only the test set gives an honest number for the winner, and only once.

Why the training set grades too kindly, and the validation set does not

Let f be *any fixed* model and S a set of n i.i.d. examples. Then $\widehat{R}_S(f) = \frac{1}{n}\sum_i \ell(y^{(i)}, f(x^{(i)}))$ is an average of n i.i.d. copies of the random variable $\ell(Y, f(X))$, so $\mathbb{E}[\widehat{R}_S(f)] = R(f)$: honest. [on your sheet]

Now let $\hat{f}_{\mathcal{D}}$ minimise $\widehat{R}_{\mathcal{D}}$ over a family $\mathcal{H}$, and let $f^*_{\mathcal{H}}$ minimise R over $\mathcal{H}$. For every sample, $\widehat{R}_{\mathcal{D}}(\hat{f}_{\mathcal{D}}) \le \widehat{R}_{\mathcal{D}}(f^*_{\mathcal{H}})$ because $\hat{f}_{\mathcal{D}}$ is the minimiser. Take expectations: the right side is $R(f^*_{\mathcal{H}})$ by the honest-estimate fact (since $f^*_{\mathcal{H}}$ is fixed). And $R(\hat{f}_{\mathcal{D}}) \ge R(f^*_{\mathcal{H}})$ pointwise because $f^*_{\mathcal{H}}$ is the best in the family. So

$$\mathbb{E}_{\mathcal{D}}[\widehat{R}_{\mathcal{D}}(\hat{f}_{\mathcal{D}})] \;\le\; R(f^*_{\mathcal{H}}) \;\le\; \mathbb{E}_{\mathcal{D}}[R(\hat{f}_{\mathcal{D}})]. \quad \text{[on your sheet]}$$

The training score of the fitted model sits *below* the best achievable risk, and the true risk of the fitted model sits *above* it: the training number is optimistic by at least the whole gap. The one word that made the difference is *fixed*: the validation set grades a model that was fixed before the validation examples were looked at, so for it the first line applies.

Honest versus optimistic, seen directly

Setup. A simulated world where you know the truth (e.g. $y = \sin(2\pi x) + \varepsilon$), a model family (polynomials of degree 5), and a large pool of fresh test points.

Held fixed. The family, the noise level, the test pool.

What we manipulate. Nothing but the random training sample: repeat 200 times, each time drawing a fresh training set of $N = 20$ points.

What we measure. For each repetition, (a) the training risk of the fitted polynomial on its own 20 points, (b) its risk on the fresh test pool, and (c) the training risk of one *fixed* polynomial chosen before any data were drawn.

What the plot shows. Three histograms. (c) is centred on that fixed model's true risk. (a) is centred well below the centre of (b).

How to read it. The distance between the centres of (a) and (b) is the expected generalisation gap $\mathbb{E}[G]$ [on your sheet]; it is positive, and it grows with capacity and shrinks with N. The histogram (c) shows that the problem is not "training data", it is "the data that chose the model".

The split protocol, as a procedure

Setup. N labelled rows and a deployment question ("will this flow next month be an attack?", "does the model work on a new hospital?").

Held fixed. The rows in each subset, written down *before* any model is fitted, and never moved.

What we manipulate. The split rule, chosen from the deployment question: random when new cases are independent draws from the same population; *stratified* random to keep class proportions in each subset; *grouped* when deployment meets new users, devices, patients or sites (every group entirely in one subset); *temporal* when the model predicts the future (train on the past, validate on later, test on the latest). [on your sheet]

What we measure. Metrics on validation during development; metrics on test once.

What the plot shows. Not a curve: a table, one row per protocol, same model. The random-split number is the largest, the grouped number smaller, the temporal number smallest.

How to read it. The protocol that mirrors deployment gives the only number that predicts deployment. The others are not wrong arithmetic; they answer a question nobody asked.

Intrusion detection: the same dataset, three verdicts

Flows recorded from 50 devices over 6 months. Random split: flows from every device and week are in both train and test; the model can learn "device 17 is the compromised laptop" and scores 99.5%. Grouped split (hold out 10 devices): the model must recognise attack behaviour, 96%.

Temporal split (train months 1–4, test months 5–6, during which a new attack tool appeared): 90%. Only the last number describes next month. A bank predicting churn "next quarter" is in the same situation: features must come from before the quarter and the outcome from after it, so the split is temporal.

Choosing the split sizes

With 10,000 rows a common split is 60/20/20. The validation set must be large enough that the differences you want to detect are larger than its noise: the standard error of an accuracy estimate near 0.9 is $\sqrt{0.9 \cdot 0.1/n}$, which is 0.03 for $n = 100$, 0.0095 for $n = 1{,}000$ and 0.003 for $n = 10{,}000$. Comparing two models that differ by half a percent on 100 validation rows is reading noise. With few rows, use cross-validation (Section 4) instead of a single small validation set.

The leaks: using tomorrow's information today

Leakage is any use, during development, of information that will not exist when a new decision is made [on your sheet]. Each form inflates the held-out number, sometimes to perfection.

- **Preprocessing fitted on all data.** Standardising, imputing, selecting features or reducing dimension with means, variances and scores computed on train+validation+test. Fix: fit every preprocessing step on the training rows only and reuse the fitted $\hat{\mu}_j, \hat{\sigma}_j$ unchanged on validation, test and deployment [on your sheet].
- **Duplicates and near-duplicates** in several subsets; **the same device, session or patient** in train and test. Fix: deduplicate, then group.
- **Features created after the outcome**: "ticket closed as intrusion", "number of analyst comments". They predict perfectly and are useless at decision time. Fix: for each feature ask "at what time does this value become known?".
- **Shuffling a time series** before splitting: the model sees the future of the series it is tested on.
- **Hyperparameters chosen on test results** (Section 3).
- **Stratifying across groups**: scattering one device's flows across subsets to match class proportions destroys the grouping.

Training set versus validation set versus test set

	Training	Validation	Test
Used for	fitting parameters θ	choosing h: λ, k, depth, features, threshold, epoch	reporting the final number
How often looked at	every iteration	every candidate	once
Honest for	nothing after fitting	each candidate, not the winner	the winner
Becomes dishonest when	always	you pick the minimum over many candidates (winner's curse)	you use its result to change anything

Key insight. The data that choose a model cannot grade it. The training set chooses θ, the validation set chooses h, so each of them grades its own choice too kindly; the test set chooses nothing, which is the only reason its number can be believed.

How the professor asks it

Q. *You standardise all 10,000 rows, then split 80/20, train logistic regression and report 94% on the 20%. Is the 94% trustworthy? Explain exactly what leaked and how large you expect the effect to be.*
A. Not fully. The means and standard deviations used to scale the training rows were computed with the test rows included, so the test rows influenced the representation they are then graded on. For standardisation on 10,000 rows the effect is usually small, because two sample means differ little; but the same mistake with feature selection (choosing the 20 most correlated features on all rows) or with imputation by a fitted model can be large, and in cross-validation with few rows it is routinely several points. The fix costs nothing: split first, fit the scaler on the 8,000, apply it unchanged to the 2,000.

Q. *Explain why a model that scores 99.5% on a random split of device-level intrusion data may score 90% in deployment, and name the split that would have predicted the 90%.*
A. The random split places flows of the same device, and the same weeks, in both subsets, so the learner can exploit device identity and period-specific patterns that recur in the test rows: a shortcut that holds inside the collection and not outside it. In deployment the model meets new devices and later weeks, where the shortcut is absent. A temporal split (and, for new devices, a grouped split) removes the shortcut from the evaluation and would have shown the lower number before deployment.

3 Why the test set must not be touched

The winner's curse

Evaluate many candidates on the same validation set and pick the one with the best score. The best score is the *minimum of several noisy estimates*, and a minimum of noisy numbers is pulled below the true value of whatever it picked. The reported validation score of the winner is therefore optimistic, even though each candidate's score was honest.

Purpose. To explain why a third set is needed at all, and why that set can be used only once.

Impact. The more candidates you compare, and the noisier the validation estimate, the larger the optimism. Using the test set to choose anything turns it into a validation set, and the optimism returns.

The minimum of honest estimates is optimistic

Let $\widehat{R}_{\text{val}}(\hat{f}_h)$, $h \in \mathcal{C}$, be validation risks of models fitted without the validation rows, so each is honest: $\mathbb{E}[\widehat{R}_{\text{val}}(\hat{f}_h)] = R(\hat{f}_h)$. For any particular h_0, the minimum over all h is at most the value at h_0: $\min_h \widehat{R}_{\text{val}}(\hat{f}_h) \le \widehat{R}_{\text{val}}(\hat{f}_{h_0})$ for every sample. Take expectations: $\mathbb{E}[\min_h \widehat{R}_{\text{val}}(\hat{f}_h)] \le R(\hat{f}_{h_0})$. This holds for every h_0, so it holds for the best one:

$$\mathbb{E}\Big[\min_{h\in\mathcal{C}} \widehat{R}_{\text{val}}(\hat{f}_h)\Big] \;\le\; \min_{h\in\mathcal{C}} R(\hat{f}_h). \quad \text{[on your sheet]}$$

The inequality is strict as soon as the estimates are noisy and no single h wins in every sample. How large? For two candidates with equal true risk R and independent Gaussian estimation noise of standard deviation σ, $\mathbb{E}[\min] = R - \sigma/\sqrt{\pi} \approx R - 0.56\sigma$ [on your sheet]; for n candidates the optimism grows like $\sigma\sqrt{2\ln n}$ [on your sheet]: 1.18σ for $n = 2$, 2.15σ for $n = 10$, 3.0σ for $n = 100$, 3.7σ for $n = 1{,}000$. The dependence on n is slow (logarithmic), the dependence on the noise σ is linear: a small validation set hurts more than a long list of candidates.

Adaptive overfitting of a validation set

Setup. Labels that are pure coin flips (no signal at all), 1,000 rows, split 500/500.

Held fixed. The data; every "model" is a random rule that ignores x.

What we manipulate. The number n of random rules tried: 1, 10, 100, 1,000, 10,000. For each, keep the rule with the best validation accuracy.

What we measure. The best validation accuracy, and the accuracy of that same rule on the 500 test rows.

What the plot shows. Best validation accuracy rises with n: about 50%, 53%, 56%, 57%, 59%. Test accuracy of the winner stays at 50% for every n.

How to read it. Nothing was learned, yet the validation number "improved" steadily, because you selected the luckiest of n coin-flip scores. Any real development loop does a weaker version of this: every look at the validation score steers the next choice. The flat test line is what an untouched test set buys. If you now start choosing *among test scores*, the test line starts rising the same way and you have no honest number left.

Public leaderboards

In data-science competitions the public leaderboard is scored on a fixed subset. Teams that submit hundreds of times and keep whatever scores best on it routinely drop many places on the private leaderboard, which is scored on rows they never saw. The public set has become their validation set; the private set is the untouched test set. The same happens inside a company when "the test set" is re-run after every change: the reported number drifts upward while deployment performance does not.

How the test set gets touched without anyone noticing

- Running the test set "just to see" after each change, then keeping the change if the test number improved. That is selection on the test set.
- Choosing the decision threshold, or the metric to report, by looking at test results.
- Tuning on cross-validation over all rows, then reporting the cross-validation score as if it were a test score. The folds chose the hyperparameters; they are a validation estimate of the winner, hence optimistic.
- Early stopping on the test loss. The stopping epoch is a hyperparameter; it must be chosen on validation.
- "Fresh" test sets that share devices, sessions or time with the training rows.

Fix: freeze the test rows before development; use them once; if you must continue developing after seeing the test result, obtain new test data or use nested cross-validation (Section 4) and report the outer estimate.

Key insight. A validation score is honest for each candidate and optimistic for the winner; the optimism grows linearly with the noise of the estimate and only logarithmically with the number of candidates. The test set is the one set that chose nothing, so touch it once.

How the professor asks it

Q. *You tried 50 hyperparameter configurations on a validation set of 200 rows and the best scored 91%. Why should you expect less than 91% in deployment, and what two things would reduce the discrepancy?*

A. The 91% is the minimum validation error among 50 noisy estimates, and the expected

minimum of honest estimates lies below the best true risk; with 200 rows the estimation noise is about $\sqrt{0.09 \cdot 0.91/200} \approx 0.02$, and the optimism for 50 candidates is of the order of $\sigma\sqrt{2\ln 50} \approx 2.8\sigma \approx 0.06$, so a few points of optimism are plausible. Reduce it by (1) a larger validation set or cross-validation, which shrinks σ (the linear factor), and (2) a coarser or smaller search, which shrinks $\sqrt{2\ln n}$ (the logarithmic factor). To *know* the deployment number, evaluate the winner once on a test set that took no part in the 50 comparisons.

4 Cross-validation

K-fold cross-validation

Partition the development rows into K folds of equal size. For each fold j, fit the procedure on the other $K-1$ folds and evaluate it on fold j; average the K scores:

$$\widehat{R}_{\mathrm{CV}}(h) = \frac{1}{K}\sum_{j=1}^{K} \widehat{R}_{\mathrm{val}}^{(j)}(\hat{f}_h^{(-j)}). \quad \text{[on your sheet]}$$

Every row is validated exactly once and used for fitting $K-1$ times.

Purpose. To estimate the held-out risk of a *procedure* (with configuration h) when the data are too few to spare a large fixed validation set, and to compare configurations on all the data instead of one slice.

Impact. Replaces one noisy validation number by the average of K numbers computed on K different held-out slices, at the cost of K fits per configuration. What it estimates is the risk of "run this procedure on $N(K-1)/K$ rows", not the risk of any one of the K fitted models.

The cross-validation protocol

Setup. Development rows (everything except the frozen test set), a candidate set $\mathcal{C}$ of configurations, K (5 or 10 are standard).

Held fixed. The fold assignment, drawn once and reused for every configuration, so that all candidates are judged on the same slices. Grouped or temporal folds if deployment demands it.

What we manipulate. Which fold is held out (inner loop over j); which configuration is run (outer loop over h).

What we measure. $\widehat{R}_{\mathrm{val}}^{(j)}$ for each (h, j); their mean $\widehat{R}_{\mathrm{CV}}(h)$ and their spread across folds.

What the plot shows. $\widehat{R}_{\mathrm{CV}}(h)$ against h, with error bars from the fold-to-fold spread. The curve is the validation curve of Section 5, now with a measure of its noise.

How to read it. Pick $\hat{h}$ at the minimum, but treat configurations whose curves lie within the error bars as tied; among ties prefer the simpler (the "one standard error" habit). Then *refit the chosen configuration on all development rows*; the K fold models were only instruments. The score at $\hat{h}$ is still a validation score of a winner, hence optimistic; the test set gives the honest number.

What the cross-validation estimate is, and its cost

Fold j holds N/K rows that the model $\hat{f}_h^{(-j)}$ never saw, so $\widehat{R}_{\mathrm{val}}^{(j)}(\hat{f}_h^{(-j)})$ is an honest estimate of $R(\hat{f}_h^{(-j)})$, the risk of a model trained on $N(K-1)/K$ rows. Averaging the K of them estimates the *expected* risk of the procedure at that training size. With $K = N$ (leave-one-out) the training sets are almost the full data, so the estimate is almost unbiased for the full-data procedure, but the N fitted models are nearly identical, so the N scores are highly correlated and their average has high variance, and the cost is N fits. With $K = 5$ or 10 the training sets are 80–90% of the

data (slightly pessimistic, since the final model sees more), the folds are more different (lower variance of the average), and the cost is 5 or 10 fits per configuration: 20 configurations at $K = 5$ is 100 fits.

Spam filter with 600 labelled emails

A single 60/20/20 split leaves 120 validation emails: an accuracy difference of 2% is below its noise. Five-fold cross-validation on the 480 development emails validates every email once and gives five numbers per configuration; their average is far less noisy and their spread tells you whether "$\lambda = 0.1$ beats $\lambda = 1$" is real. After choosing λ, refit on all 480, then score the 120 frozen test emails once.

Cross-validation done wrong

- **Preprocessing or feature selection outside the folds.** Selecting the top features on all development rows, then cross-validating the classifier, lets every validation fold influence the representation it is scored on. With many features and few rows this alone can produce near-perfect cross-validation scores on random labels. Fix: the entire pipeline (scaler, imputer, selector, model) is refitted inside each fold on its training portion [on your sheet].
- **Random folds on grouped or temporal data.** Rows of the same patient, device or session land on both sides; a time series is shuffled. Fix: grouped folds (each group in one fold) or forward-chaining folds (train on the past, validate on the next block).
- **Reporting the cross-validation score of the winner as the final estimate.** It chose $\hat{h}$, so it is a validation score. Fix: a test set, or *nested* cross-validation: an outer loop whose held-out folds are never used for selection, and an inner cross-validation, inside each outer training portion, that chooses h. The outer average is honest for the whole procedure "select then fit".
- **Deploying one of the K fold models**, or averaging them, instead of refitting on all development rows. (Averaging them is a bagging-like ensemble, legitimate but a different method.)
- **Different fold assignments for different candidates.** Then the differences between candidates include the differences between slices.

Validation split versus cross-validation versus out-of-bag versus nested cross-validation

	What is held out	Cost	Honest for
Single validation set	one fixed slice	1 fit per candidate	each candidate; noisy if small
K-fold CV	each fold in turn	K fits per candidate	each candidate; less noisy; still optimistic for the winner
Out-of-bag (bagging)	the $\approx 36.8\%$ rows missing from each bootstrap sample	free (a by-product of bagging)	the fixed ensemble; same winner's-curse caveat if used to select
Nested CV	outer folds for estimation, inner folds for selection	$K_{\text{out}} \times K_{\text{in}}$ fits per candidate	the whole select-then-fit procedure

Key insight. Cross-validation changes which rows are held out, not the rule that held-out rows must have played no part in anything that was fitted or chosen. Everything data-dependent goes inside the fold.

How the professor asks it

Q. *With 300 rows and 5,000 features, a student ranks the features by correlation with the label on all 300 rows, keeps the best 20, and then runs 10-fold cross-validation of logistic regression on those 20, obtaining 97%. The labels are in fact unrelated to the features. Explain the 97%.*
A. With 5,000 features and 300 rows, by chance alone some features will be strongly correlated with the labels on this sample; ranking on all rows picks exactly those lucky features. Every cross-validation fold then validates on rows that already took part in choosing the features, so the "held-out" rows are not held out from the selection step: leakage of the validation labels into the representation. Running the selection inside each fold, on the 270 training rows only, would pick different lucky features each time and the validation folds would score near 50%.

Q. *Why does leave-one-out cross-validation have high variance although each of its N fitted models uses almost all the data?*
A. Its N training sets differ by one row, so the N fitted models are nearly identical and their N held-out losses are strongly correlated; the average of correlated quantities does not shrink its variance like $1/N$ (the same mechanism as the ensemble variance formula $\sigma^2(\rho + (1-\rho)/M)$ [on your sheet], with ρ close to one). Five- or ten-fold folds are more different from each other, so their average is more stable, at the price of a slight pessimism because each model sees 80–90% of the rows.

5 Hyperparameter tuning

Parameters, hyperparameters, and the selection loop

A *parameter* θ is set by the training algorithm from the training rows (weights, coefficients, split thresholds, leaf values). A *hyperparameter* h is a choice the training algorithm cannot make for itself because training error would always push it to maximal capacity: λ, k, degree, depth, number of trees, learning rate, number of epochs, m_{try}, the feature subset, the decision threshold. The selection loop is

$$\hat{\theta}_h \in \arg\min_{\theta} \widehat{R}_{\text{train}}(\theta; h), \qquad \hat{h} \in \arg\min_{h \in \mathcal{C}} \widehat{R}_{\text{val}}(\hat{f}_h), \quad \text{[on your sheet]}$$

inner problem on training rows, outer problem on validation rows (or cross-validation).

Purpose. To choose the capacity, the regularisation and the stopping point from data without letting the training objective decide, since the training objective is biased toward flexibility.

Impact. Each hyperparameter moves the model along the validation curve: too little capacity on the left (both errors high), too much on the right (training low, validation high).

Why hyperparameters cannot be chosen on the training set

Regularised empirical risk minimisation $\hat{\theta}_\lambda \in \arg\min_\theta [\widehat{R}_{\text{train}}(\theta) + \lambda\Omega(\theta)]$ [on your sheet] is equivalent to minimising $\widehat{R}_{\text{train}}$ subject to $\Omega(\theta) \le c$ with $c = \Omega(\hat{\theta}_\lambda)$ [on your sheet]; a larger λ gives a smaller c and a smaller family $\mathcal{H}_c$. The families are nested, $c_1 < c_2 \Rightarrow \mathcal{H}_{c_1} \subseteq \mathcal{H}_{c_2}$, so the minimum training risk over $\mathcal{H}_{c_2}$ is at most that over $\mathcal{H}_{c_1}$ [on your sheet]. Hence training risk is minimised at $\lambda = 0$: choosing λ on training data always answers "no regularisation". The same argument with degree, depth, $1/k$ or the number of epochs shows that every capacity hyperparameter is pushed to its maximum by training error. That is the whole reason validation exists.

The validation curve: sweep one hyperparameter

Setup. Training and validation rows (or folds); a model with one knob (say ridge λ, or kNN's k, or tree depth).

Held fixed. All other hyperparameters, the preprocessing (fitted on training rows), the rows, the metric. Standardise before penalising, otherwise λ penalises a feature in millimetres a thousand times more than the same feature in metres.

What we manipulate. The knob on a grid; for λ a logarithmic grid $10^{-3}, 10^{-2}, \ldots, 10^{2}$, because the useful range spans orders of magnitude; for k odd integers $1, 3, 5, \ldots$; for depth $1, 2, \ldots, 12$.

What we measure. Training and validation risk at every grid point.

What the plot shows. Training risk decreasing monotonically in capacity (so *increasing* in λ and in k); validation risk U-shaped.

How to read it. The minimum of the validation curve is $\hat{h}$. A flat valley means the choice does not matter much; a sharp valley means it does. If the minimum is at the edge of the grid, extend the grid. Remember the direction of each knob: capacity grows with degree, depth, epochs, trees in boosting, and with *smaller* λ and *smaller* k.

Ridge on the four-point example

With $\sum x^2 = 14$, $\sum xy = 19$, $N = 4$, the ridge slope is $\hat{w}_\lambda = 19/(14 + 4\lambda)$ [on your sheet]: 1.357 at $\lambda = 0$, 1.056 at $\lambda = 1$, 0.352 at $\lambda = 10$. Training error is smallest at $\lambda = 0$ by the nesting argument; whether $\lambda = 1$ is better for new points can only be seen on validation points. The same four points fitted by polynomials of degree 0, 1, 3 have training risks 1.25, 0.45, 0 and validation errors at the fifth point $(4, 5)$ of 6.25, 0.25, 100: the training column points to degree 3, the validation column to degree 1.

Early stopping: the training curve over epochs

Setup. An iterative learner (gradient descent on a network, or boosting rounds), training and validation rows.

Held fixed. Everything but the number of iterations.

What we manipulate. The epoch t (or boosting round m), by simply continuing to train and recording a checkpoint at each epoch.

What we measure. Training loss and validation loss after each epoch.

What the plot shows. Training loss falls throughout; validation loss falls, reaches a minimum at t^*, then rises.

How to read it. Keep the checkpoint $\hat{\theta}_{t^*}$ with the smallest validation loss [on your sheet], not the last one. A *patience* p continues p epochs past the current best before stopping, to avoid stopping on a wobble. The number of epochs is a capacity knob exactly like λ; stopping on the test loss is tuning on the test set.

Coupled hyperparameters: learning rate, rounds and depth in boosting

Setup. Gradient-boosted trees, training and validation rows.

Held fixed. The data and the loss.

What we manipulate. Learning rate $\eta \in \{0.3, 0.1, 0.03\}$ and depth $\in \{2, 4, 6\}$ on a grid; for each pair, the number of rounds M is chosen by early stopping on validation.

What we measure. The validation loss at the selected round, for each (η, depth).

What the plot shows. One validation curve over rounds per (η, depth). Halving η moves the minimum to roughly twice as many rounds; deeper trees reach the minimum in fewer rounds.

How to read it. Never tune one of the three alone: the best M depends on η and depth. Compare configurations by their validation loss *at their own selected round*. Smaller η usually ends a little lower at a higher cost.

Tuning mistakes

- Choosing λ, k, depth or epochs on the training set (always maximal capacity, proof above), or on the test set (Section 3).
- A fine grid on a tiny validation set: the valley is noise; two configurations within one standard error are tied.
- Forgetting to standardise before an L_2 or L_1 penalty, or penalising the intercept.
- Reading the kNN curve backwards: capacity *increases* as k decreases.
- Keeping the last epoch instead of the best checkpoint.
- After cross-validation, deploying a fold model instead of refitting $\hat{h}$ on all development rows.
- Tuning the decision threshold on training data or on test data; it is a hyperparameter and goes on validation.

The knobs, their direction, and what they trade

Knob	Capacity grows when	What is traded	Chosen on
polynomial degree, tree depth, network width	it increases	approximation error down, estimation error up	validation curve
k in kNN	k decreases	smoother average versus local sensitivity	validation curve
λ (ridge, lasso)	λ decreases	bias from shrinkage versus variance	log grid on validation
epochs / boosting rounds	they increase	fit of recurring patterns then of noise	early stopping on validation
number of bagged trees M	does not grow capacity	stability only; OOB plateau	OOB or validation
m_{try} in a forest	larger values make trees stronger and more alike	tree strength versus correlation	OOB or validation

Key insight. A hyperparameter is any knob that training error would turn to maximal flexibility; so it is turned on data that did not fit the parameters, and its best value is read at the bottom of a U.

How the professor asks it

Q. *Prove that the training error of ridge regression is minimised at $\lambda = 0$, and state what this implies for how λ must be chosen.*
A. The penalised solution $\hat{\theta}_\lambda$ also solves the constrained problem $\min \widehat{R}_{\text{train}}$ s.t. $\Omega(\theta) \le c_\lambda$ with $c_\lambda = \Omega(\hat{\theta}_\lambda)$: if some feasible θ had smaller training risk, then $\widehat{R}(\theta) + \lambda\Omega(\theta) \le \widehat{R}(\theta) + \lambda c_\lambda < \widehat{R}(\hat{\theta}_\lambda) + \lambda c_\lambda$ would contradict optimality of $\hat{\theta}_\lambda$. The feasible sets shrink as λ grows, and a minimum over a smaller set is at least as large, so $\widehat{R}_{\text{train}}(\hat{\theta}_\lambda)$ is nondecreasing in λ and smallest at $\lambda = 0$. Therefore the training set would always choose no regularisation; λ must be chosen on validation rows or by cross-validation, and the test set is kept for the final number.

Q. *Your boosting model is best at 400 rounds with learning rate 0.1. A colleague switches to*

learning rate 0.01 and keeps 400 rounds, and reports worse validation loss. What happened and what should be done?
A. With a ten times smaller step, 400 rounds apply about one tenth of the total correction, so the model is underfitted: the validation curve over rounds has not reached its minimum. Rounds, learning rate and depth are coupled; the number of rounds must be re-selected by early stopping for the new rate, which will land near 4,000 rounds, and the two configurations are then compared at their own selected rounds. The smaller rate may end slightly lower, at ten times the cost.

6 Learning curves: vary the amount of data

Learning curve

A learning curve plots training and validation performance against the number of training rows N used, with everything else fixed. [on your sheet]

Purpose. To answer "is more data worth collecting?" and to tell bias from variance: the plateau height is bias (plus noise), the gap is variance.

Impact. It is the only experiment that distinguishes "the family cannot express the relation" from "the family could, but the sample is too small": the first does not improve with N, the second does.

The learning-curve protocol

Setup. Development rows and a fixed configuration h (already tuned, or tuned at each size).

Held fixed. The configuration, the validation rows (or folds), the metric, the preprocessing rule.

What we manipulate. The number of training rows N: subsample $10\%, 20\%, \ldots, 100\%$ of the training set (nested subsamples, so that each larger set contains the smaller).

What we measure. At each N: training risk of the fitted model on its own N rows, and validation risk on the fixed validation rows. Repeat with several random subsamples at each N and average.

What the plot shows. Training risk *rises* with N (more rows are harder to fit perfectly); validation risk *falls* with N; the two approach each other.

How to read it. Three shapes. (1) Both curves high and already flat, close together: bias; more data will not help; change the representation or increase capacity. (2) Training low, validation high, wide gap that narrows slowly: variance; regularise, reduce capacity, or collect more representative data (the gap will close with N). (3) Both still falling and converging: data-limited; more rows are likely to help. A validation curve that *worsens* over calendar time is a fourth case, distribution shift, not visible on a learning curve.

Why the two curves bracket the same limit

Fix the family $\mathcal{H}$ with best member $f^*_{\mathcal{H}}$. For the fitted model on N rows, the sandwich of Section 2 holds at every N:

$$\mathbb{E}[\widehat{R}_{\text{train}}(\hat{f}_N)] \;\le\; R(f^*_{\mathcal{H}}) \;\le\; \mathbb{E}[R(\hat{f}_N)].$$

As N grows, the empirical risk of every member of a reasonable family approaches its population risk uniformly, so the fitted model approaches $f^*_{\mathcal{H}}$ and both outer terms approach $R(f^*_{\mathcal{H}})$: the training curve rises to it from below, the validation curve falls to it from above. The common limit is the *approximation error* of the family plus the irreducible noise; it does not depend on N and cannot be bought with data. The gap between the curves at finite N is the *estimation error*; it shrinks with N and grows with capacity. In the bias–variance language at a fixed x, $\mathbb{E}(Y - \hat{f}(x))^2 = \sigma^2 + \text{Bias}^2 + \text{Var}$ [on your sheet]: the plateau is $\sigma^2 + \text{Bias}^2$, the gap is mostly Var.

Should the security team label 20,000 more flows?

The team has 10,000 labelled flows. They train a random forest on 1,000, 2,000, ..., 8,000 and validate on the same 2,000. Validation recall goes $0.71, 0.78, 0.83, 0.86, 0.87, 0.875, 0.878, 0.88$ while training recall stays near 1.0: a large gap that is closing slowly and a validation curve that has almost flattened. Doubling the data would move recall from 0.88 to perhaps 0.89; the remaining gap is variance that regularisation (larger leaves, fewer features per split) addresses more cheaply, and the plateau says the features carry no more information. Labelling 20,000 more flows is not the best use of the budget. Had the validation curve still been rising steeply at 8,000, the answer would be yes.

Learning-curve mistakes

- Confusing the three horizontal axes: capacity (validation curve), epochs (training curve), data size (learning curve). Each has its own shape and its own diagnosis.
- Re-tuning nothing when N changes: the best λ or depth depends on N; at small N a tuned model is more regularised. Either tune at each size or state that the configuration was fixed.
- Using non-nested random subsamples, so that noise between subsamples hides the trend; or a single run per size.
- Believing more data removes bias. It removes variance; the plateau is set by the family and the features.
- Reading "errors close and low" as "done": close and low on a random split can still be a shortcut; the protocol question of Section 2 comes first.

Reading the curves: the diagnostic table

Observed pattern	Likely issue	Response
training and validation both high, flat in N	underfitting (bias)	richer representation, more capacity, less regularisation
training low, validation high, gap narrows slowly with N	high variance	regularise, reduce capacity, more representative data, ensembles
both decreasing and converging	data-limited	more representative examples will help
validation fine on the split, degrades over calendar time	distribution shift	temporal evaluation, retraining schedule, monitoring

[on your sheet]

Key insight. Data buys variance reduction, not bias reduction: the learning curve's gap closes with N, its plateau does not move.

How the professor asks it

Q. *A learning curve shows training error at 2% and validation error at 15% at every data size from 1,000 to 50,000 rows, with no narrowing. Which diagnosis does the table suggest, and why might the table be wrong here?*

A. The table says high variance (large gap). But variance shrinks with N, and a gap that does not narrow across a fiftyfold increase in data is not sampling variance: it is a systematic difference between the training rows and the validation rows. Typical causes are a shortcut present in the training collection and absent from the validation protocol (a grouped or temporal validation set), duplicates in the training rows, or label noise concentrated in one subset. The response is

not more data or regularisation but an audit of the split and the features.

Q. *Explain, using the sandwich inequality, why the training curve of a learning curve rises with N while the validation curve falls.*
A. At every N, $\mathbb{E}[\widehat{R}_{\text{train}}(\hat{f}_N)] \le R(f^*_{\mathcal{H}}) \le \mathbb{E}[R(\hat{f}_N)]$. With few rows the minimiser can fit the sample's accidents, so the training risk is far below $R(f^*_{\mathcal{H}})$ and the true risk far above it; with many rows the empirical risk of every member of the family is close to its population risk, so the fitted model is close to $f^*_{\mathcal{H}}$ and both terms approach $R(f^*_{\mathcal{H}})$. The lower term therefore rises toward the limit and the upper term falls toward it. The limit is the approximation error plus noise, which N cannot change.

7 Bias–variance experiments

Bias and variance of a procedure

At a fixed input x, with $f^*(x) = \mathbb{E}[Y \mid X = x]$ and $\bar{f}(x) = \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(x)]$ the average prediction over training sets,

$$\text{Bias}(x) = \bar{f}(x) - f^*(x), \qquad \text{Var}(x) = \mathbb{E}_{\mathcal{D}}[(\hat{f}_{\mathcal{D}}(x) - \bar{f}(x))^2], \quad \text{[on your sheet]}$$

and $\mathbb{E}_{\mathcal{D},Y|x}[(Y - \hat{f}_{\mathcal{D}}(x))^2] = \sigma^2(x) + \text{Bias}(x)^2 + \text{Var}(x)$ [on your sheet]. These are properties of the *procedure* (family plus algorithm plus N), not of one fitted model: one model has no variance.

Purpose. To split the error at x into the part the family cannot fix (bias), the part the sample causes (variance), and the part nothing can fix (noise).

Impact. Capacity moves bias down and variance up; data moves variance down; regularisation and averaging trade a little bias for a lot of variance.

The decomposition, line by line

Write $Y = f^*(x) + \varepsilon$ with $\mathbb{E}[\varepsilon \mid x] = 0$, $\text{Var}(\varepsilon \mid x) = \sigma^2(x)$, and the training set independent of the test pair.
Step 1, split off the noise. $Y - \hat{f}_{\mathcal{D}}(x) = \varepsilon + (f^*(x) - \hat{f}_{\mathcal{D}}(x))$. Square: $\varepsilon^2 + 2\varepsilon(f^* - \hat{f}_{\mathcal{D}}) + (f^* - \hat{f}_{\mathcal{D}})^2$. Expectation: $\mathbb{E}[\varepsilon^2] = \sigma^2$; the cross term is $\mathbb{E}[\varepsilon] \cdot \mathbb{E}_{\mathcal{D}}[f^* - \hat{f}_{\mathcal{D}}] = 0$ because ε is independent of $\mathcal{D}$ and has mean zero.
Step 2, split around the average prediction. $f^* - \hat{f}_{\mathcal{D}} = (f^* - \bar{f}) + (\bar{f} - \hat{f}_{\mathcal{D}})$. The first bracket is the constant $-\text{Bias}(x)$; the second has $\mathbb{E}_{\mathcal{D}}$-mean zero by definition of $\bar{f}$. Square and take $\mathbb{E}_{\mathcal{D}}$: $\text{Bias}^2 + 0 + \text{Var}$.
Both steps are the same move: add and subtract a mean, square, and let the cross term die because a mean-zero quantity multiplies a constant.

Simulating bias and variance

Setup. A world you control: $f^*(x) = \sin(2\pi x)$, noise $\sigma = 0.3$, $N = 25$ training points per set, a grid of 50 test inputs.

Held fixed. The truth, the noise, N, the test grid.

What we manipulate. Two things, in two separate runs: (a) the training set, drawn 500 times (this is the repetition that defines $\mathbb{E}_{\mathcal{D}}$); (b) the capacity: polynomial degree $0, 1, 3, 5, 9$, or k in kNN.

What we measure. For each capacity and each test input: the 500 predictions; their mean $\bar{f}(x)$; $\text{Bias}^2(x) = (\bar{f}(x) - f^*(x))^2$; $\text{Var}(x)$ as their spread; average both over the grid.

What the plot shows. Bias^2 falling with capacity, variance rising, their sum plus σ^2 U-shaped

and matching the measured test error.

How to read it. The U of the validation curve is the sum of a falling and a rising term. Degree 0 is all bias; degree 9 on 25 points is all variance. Now repeat run (b) with $N = 250$: the variance curve drops everywhere, the bias curve does not move, and the minimum of the U shifts to a higher degree. That is the whole story of "more data allows a bigger model".

Bias and variance computed exactly: estimate a mean

No input; $Y = \mu + \varepsilon$, $\mathrm{Var}(\varepsilon) = \sigma^2 = 1$, $N = 4$ draws. Three procedures. *Sample mean*: unbiased, variance $\sigma^2/N = 0.25$. *Ignore the data, predict 0*: bias $-\mu$, variance 0. *Shrunk mean*, half the sample mean: bias $-\mu/2$, variance $0.25/4 = 0.0625$. Bias^2+variance: for $\mu = 1$: 0.25, 1, 0.3125; for $\mu = 0.2$: 0.25, 0.04, 0.0725. When the truth is far from the shrinkage target the unbiased procedure wins; when it is near, the biased ones win by trading a little bias for a large variance reduction. Ridge regression is this trade with a dial: $\hat{w}_\lambda$ is the least-squares slope times a factor in $(0, 1]$.

Variance reduction by averaging: the ensemble-size curve

Setup. Bootstrap samples of one training set, a deep tree per sample, a validation set.

Held fixed. The data, the tree settings.

What we manipulate. The number of trees M: $1, 5, 10, 25, 50, 100, 200$; and, in a second run, m_{try} (all features versus $\sqrt{d}$).

What we measure. Validation error of the average of the first M trees; optionally, the variance of the ensemble prediction at a few fixed inputs across bootstrap replicates.

What the plot shows. Error falling steeply then flattening. The curve with $m_{\text{try}} = \sqrt{d}$ flattens *lower*.

How to read it. The floor is $\rho\sigma^2$ in $\mathrm{Var}(\bar{e}) = \sigma^2(\rho + (1-\rho)/M)$ [on your sheet]: with $\rho = 0.2$ and $M = 25$, $0.232\sigma^2$, and never below $0.2\sigma^2$ however many trees. Feature randomisation lowers ρ, which lowers the floor; that is why the forest beats bagging. Averaging does nothing for bias: if every tree lacks the same feature, every error points the same way and $\rho \approx 1$.

Bias–variance mistakes

- Speaking of "the variance of this model". Variance is over training sets; one fitted model is one draw. The experiment that measures it must refit many times.
- Expecting more data to reduce bias, or regularisation to reduce variance for free: regularisation adds bias (shrinkage) and removes variance; whether the sum falls depends on where the truth is (the $\mu = 1$ versus $\mu = 0.2$ table).
- Treating "low bias, low variance at moderate capacity" as achievable in general: at the bottom of the U both terms are positive, just small.
- Forgetting that the decomposition is for squared error at a fixed x with training noise independent of test noise; for classification the "bias" and "variance" of a 0–1 loss do not add this way.
- Assuming the U always holds: very large models can show double descent, with test error peaking near the interpolation threshold (parameters $\approx N$) and falling again beyond [on your sheet]. The classical U is the first model, not a law.

What each lever does to bias and variance

Lever	Bias	Variance	Note
more capacity	down	up	U-shaped sum
more data N	unchanged	down	plateau unchanged
regularisation λ, shrinkage	up	down	worth it when the truth is near the target
early stopping	up slightly	down	capacity through training time
bagging, forests	unchanged	down to the floor $\rho\sigma^2$	shared errors survive
boosting	down	up with rounds	early stopping on rounds
better features	down	often down	attacks the plateau

Key insight. Bias is what repeating the experiment never averages away; variance is what a different sample would change; and the experiment that separates them is "refit many times from fresh samples and look at the cloud of predictions at one x".

How the professor asks it

Q. *A 1-nearest-neighbour regressor and a 50-nearest-neighbour regressor are both fitted on the same 200 points. Describe the bias–variance experiment that compares them and predict its outcome.*
A. Draw many training sets of 200 points from the same world, fit both regressors on each, record both predictions at a fixed x, and compute the spread of each cloud (variance) and the distance of each cloud's centre from $f^*(x)$ (bias). The 1-NN prediction is one noisy neighbour's y, so its cloud is as wide as the noise, Var $\approx \sigma^2$, and its centre is close to $f^*(x)$: low bias, high variance. The 50-NN prediction averages 50 neighbours, so its spread is about $\sigma^2/50$, but the neighbours reach far from x and their average follows a smoothed version of f^*: higher bias, low variance. On the sheet: Bias $= \frac{1}{k}\sum_{j\le k} f(x^{(j)}) - f(x)$, Var $= \sigma^2/k$ [on your sheet]. Which total is smaller depends on how curved f^* is relative to the noise; the validation curve over k finds the balance.

Q. *Give a counterexample to "averaging many models always reduces error".*
A. Take M copies of the same fitted model, or M models that all lack the same essential feature. Their errors at any x are identical ($\rho = 1$), so $\mathrm{Var}(\bar{e}) = \sigma^2(1+0) = \sigma^2$: the average has the error of one model. More generally the floor $\rho\sigma^2$ is untouched by M; averaging removes only the part of the error that differs between models, never shared bias.

8 The complete protocol, start to finish

The experiment everyone runs, as twelve steps

1. **State the deployment question** (new cases of what kind: future? new devices? same population?). This decides the split rule.
2. **Audit the data**: duplicates, conflicting labels, missing values, class imbalance, features known only after the outcome.
3. **Freeze the test set** according to the split rule (random, stratified, grouped, temporal). Write down its row identifiers. Do not look at it again.
4. **Fix the metric from the costs**: accuracy hides minority failure; use recall, precision, specificity, balanced accuracy, F_1 [on your sheet] as the costs dictate; on 10,000 flows with 50 attacks, TP = 40, FN = 10, FP = 90 gives accuracy 0.99 and precision 0.31.
5. **Fit baselines** (majority class, a shallow tree, logistic regression, kNN) under the same protocol; anything you build must beat them.

6. **Build the pipeline** so that every data-dependent step (scaler, imputer, selector, model) is fitted on training rows only.
7. **Tune** each hyperparameter on validation rows or by cross-validation on a log or integer grid; read the U; treat near-ties as ties; choose the number of epochs or rounds by early stopping; choose the decision threshold on validation.
8. **Diagnose** with learning curves (data-limited? bias? variance?) and, where instability is suspected, with repeated fits over seeds or bootstrap samples.
9. **Probe shortcuts**: shuffle the labels (performance must collapse to chance), drop metadata columns (performance must not collapse), evaluate under the deployment-like protocol.
10. **Refit the chosen configuration on all development rows** (training plus validation).
11. **Evaluate once on the test set**; report the number with its uncertainty; do not go back.
12. **Monitor in deployment**: a degrading metric over calendar time is distribution shift; schedule retraining and keep a time-aware evaluation.

The ten mistakes of this document

1. Grading a model on the rows that chose it (training rows for θ, validation rows for h).
2. Fitting the scaler, imputer or feature selector on all rows before splitting, or outside the cross-validation folds.
3. A random split when deployment is in the future (temporal) or on new entities (grouped).
4. Looking at the test set more than once, including early stopping or threshold choice on it.
5. Reporting the winner's validation or cross-validation score as the final estimate.
6. Choosing λ, k, depth or epochs on training error (always maximal capacity).
7. Confusing the three curves: capacity axis, epoch axis, data-size axis.
8. Expecting more data to remove bias, or an ensemble to remove shared error.
9. Tuning boosting's rate, rounds and depth separately.
10. Comparing candidates on different fold assignments, on a tiny validation set, or deploying a fold model instead of refitting.

The experiments in ten lines

1. Every experiment: hold all fixed, vary one knob, measure risk on fitting rows and on unseen rows, read two curves.
2. Fixed model: held-out risk is honest. Fitted model: training risk is optimistic, $\mathbb{E}\widehat{R}_{\text{train}}(\hat{f}) \le R(f^*_{\mathcal{H}}) \le \mathbb{E}R(\hat{f})$.
3. Three sets, three jobs: train fits θ, validation chooses h, test reports once. The split rule comes from the deployment question.
4. Leakage is tomorrow's information today: preprocessing on all rows, duplicates, groups across sets, post-outcome features, shuffled time.
5. Winner's curse: the minimum of honest estimates is optimistic; it grows with noise σ linearly and with the number of candidates like $\sqrt{2\ln n}$.
6. Cross-validation holds out each fold in turn; the whole pipeline is refitted inside each fold; refit on all development rows afterwards; nested CV when you need an honest number without a test set.
7. Hyperparameters are knobs training error would turn to maximal capacity; choose them at the bottom of the validation U; epochs by early stopping; knobs in boosting are coupled.
8. Learning curves: training rises, validation falls, both converge to approximation error plus noise; gap is variance, plateau is bias.
9. Bias–variance: refit on many samples, look at the cloud at one x; squared error $= \sigma^2 + \text{Bias}^2 + \text{Var}$; capacity trades them, data lowers variance, averaging lowers variance to the floor $\rho\sigma^2$.
10. The protocol: question, audit, freeze test, metric from costs, baselines, pipeline inside folds, tune on validation, diagnose with curves, probe shortcuts, refit, test once, monitor.

Understanding ML, Chapter 1: What Does It Mean to Learn?

After reading: you can say what learning is, when to write rules and when to learn, which paradigm a task belongs to and why, what the four ingredients of any learning system are, why finite data never determine the rule, how a shortcut is born and how to catch it, and why cross-entropy is the loss of classification

Fundamentals of Machine Learning, midterm

2026-11-03

What this chapter is about, in one paragraph

A machine can be given an ability in two ways: a human writes the procedure, or the machine is given *experience* and a *capacity to learn* and finds the procedure itself. The second route exists because people can do many things (read a handwritten 3, spot a wolf) that they cannot describe. Experience reaches the machine as examples, observations or interactions, and the kind of feedback that reaches the learner decides whether the problem is supervised, unsupervised, self-supervised or reinforcement learning. Any learning system can be described by four ingredients: the experience, the model, the learning algorithm and the evaluation criterion. The thing we actually want is *generalization*, good behaviour on situations never seen, and the chapter proves why the training data can never guarantee it (finitely many examples agree with infinitely many rules) and shows the typical failure, the *shortcut*: a cue that predicts the labels in the collection but not in the world. The chapter closes with the one inequality (Gibbs) that explains why cross-entropy will be the training criterion for classification. What is manipulated throughout: the experience (which examples, how diverse, which labels), the representation (which numbers the model sees), and the rules by which success is judged.

Contents

1 Learning from experience **2**

2 Two routes to machine ability **3**

3 What counts as experience? The three paradigms **5**

4 Supervised learning: examples, labels, outputs **8**

5 Representations: what is presented to the learner **11**

6 Four ingredients of a learning system **14**

7 The challenge of generalization **16**

8 A connection to information theory **20**

Coverage of the textbook chapter **23**

1 Learning from experience

Learning, capacity to learn, experience

Learning is the process through which experience changes a learner and develops what it can do. Two things are needed: **experience** (observation, practice, interaction, instruction, feedback) and a **capacity to learn** (a stone exposed to a thousand piano lessons learns nothing). For animals the capacity comes from the brain; for a machine it comes from design choices. The chapter's picture is

$$\text{Capacity to Learn} + \text{Experience} \xrightarrow{\text{Learning}} \text{Acquired Abilities.}$$ [not on your sheet: know it]

The abilities we care about are four overlapping activities, a *vocabulary*, not a taxonomy: **recognise** (what is it?), **predict** (what may happen?), **reason** (what follows?), **act** (what should I do?). Language draws on all four at once.

Purpose. The definition separates the two things a designer controls: what the system experiences, and what in the system can change in response.

Impact. Change the experience and the same learner acquires a different ability; remove the capacity to change and no amount of experience helps. Everything later in the course is a way of engineering one of the two.

One task, four abilities: a radiologist

Reading a chest X-ray, a radiologist *recognises* anatomical structures and abnormal shadows, *predicts* how a lesion may evolve, *reasons* from symptoms and image to a differential diagnosis, and *acts* by ordering a test or a treatment. A machine-learning system may take on any subset: a classifier flagging suspicious regions is recognition; a model of tumour growth is prediction; a treatment-recommendation system is decision and action. Keeping the four apart lets you state precisely what the system is supposed to do, which is the first question of any project.

Using the four abilities as a classification scheme

Students write "this task is recognition, therefore it is supervised learning" or treat the four words as exhaustive categories of intelligence. The notes say explicitly that they are *not* an exhaustive classification. They are words for describing what an ability does; the learning paradigm (next section) is decided by the feedback, not by which of the four verbs fits.

Key insight. Learning is a change in the learner caused by experience; a machine learns only if something in it can change and something in its experience tells it which change is good.

How the professor asks it

Q. *Define learning in the sense of the course and explain why "exposure to data" is not by itself learning. Give one example of a system that has experience but no capacity to learn.*
A. Learning is the process by which experience changes a learner and develops what it can do. Exposure alone is not learning because the second ingredient, a capacity to change in response to the experience, may be absent: the stone that attends a thousand piano lessons is exposed to the same experience as the student and acquires nothing. A fixed rule-based spam filter that receives millions of emails but whose rules never change has experience and no capacity to learn; its error rate on new spam does not move. The consequence for design is that a learning system

must specify *what can change* (the model's parameters, possibly its representation) and *what information drives the change* (the evaluation signal).

Q. *A hospital wants "an AI for radiology". Using the four abilities, show why the request is underspecified and rewrite it as two distinct, well-posed tasks.*
A. The request names no ability. Reading an X-ray involves recognising structures, predicting evolution, reasoning to a diagnosis and acting on it, and a system can take on any subset; each subset needs different experience and a different evaluation. Task 1 (recognition): flag regions of a chest X-ray that look abnormal, trained on images with marked regions, judged by how many abnormalities are flagged and how many false flags are raised. Task 2 (prediction): estimate the probability that a lesion grows within six months, trained on pairs of scans six months apart, judged on later patients. The two tasks share the images and nothing else; conflating them makes the evaluation criterion impossible to state.

2 Two routes to machine ability

Route 1: encode human knowledge

Humans learn first, then transfer the result to the machine as explicit instructions:

$$\text{Human Knowledge} \longrightarrow \text{Explicit Rules or Procedures} \longrightarrow \text{Computer.}$$ [not on your sheet: know it]

Pascal's seventeenth-century Pascaline embodied arithmetic in a mechanism; a program embodies a tax law, merge sort, or "hash the password and compare". Early AI pushed the route into expert domains: an **expert system** such as MYCIN stored knowledge elicited from specialists as explicit facts and `IF ...THEN ...` rules and could reason well in a restricted domain. As systems grew, extracting, expressing and maintaining the knowledge became the hard part: the **knowledge acquisition bottleneck**.

Purpose. Exact, auditable behaviour when a reliable procedure is known.

Impact. No data are needed and no approximation is introduced; but the system knows only what was written, and cannot improve from what it observes.

Route 2: make the machine learn

When useful experience is available and a complete procedure is hard to specify, give the machine the experience and a capacity to learn, and let it acquire the ability itself. The question changes from "what rules should humans write?" to "how can we give a machine the capacity to learn from experience?". Designing computational methods that make this possible is **machine learning**. The capacity to learn is provided by design choices: what the system can represent, what can change through experience, what information guides the changes, and what counts as good performance. Machine learning *still involves programming*: humans build the learner, the data pipeline and the decision logic; only the task-specific behaviour is acquired from data.

Purpose. Abilities we can recognise but cannot describe.

Impact. Behaviour becomes approximately correct and data-dependent: it improves with better experience and degrades when the world drifts away from the experience.

The rules-versus-examples experiment on handwritten 3s

Setup. A 20×20 binary image; decide whether it shows a 3. Route 1 writes rules: (R1) two bumps opening left, i.e. the rightmost black pixel of the top half and of the bottom half both lie in the right third; (R2) the middle row has black pixels only in the right half (the waist); (R3) between 40 and 90 black pixels; (R4) no black pixel touches the left border. Route 2 collects a

few thousand labelled 3s and non-3s and lets a learner find regularities.

Held fixed. The task (is it a 3?), the image size, the set of test images.

What we manipulate. The way the ability is produced (rules versus examples), and the *perturbations* applied to the digit: shift 4 pixels right, write it narrowly, rotate by 15°, use a thick stroke.

What we measure. For each perturbation, which rules still hold; for the learner, the error on perturbed test images, and the cost of handling a new kind of variation (editing rules versus adding examples).

What the plot shows. A table of perturbations against rules: shift breaks R2 (the waist crosses the centre column), narrow writing breaks R1 (bumps no longer reach the right third), rotation breaks R2 (the waist leaves the middle row), a thick stroke breaks R3 (the pixel count doubles). Every fix breeds a new failure: "right third" becomes "right third of the bounding box", corrupted by a stray dot; "middle row" becomes "row of minimum width between the bumps", which also matches 5s and 8s; the pixel range widens until it excludes nothing.

How to read it. Rules pile up with thresholds nobody knows how to set and interactions nobody can reason about, while the learner absorbs each variation from examples that already contain it. The experiment reveals the essential difference: explicit programming requires us to *know and state* the procedure; learning requires only that we can *recognise correct behaviour on examples*.

Programs that should stay programs, and one that should not

Income tax: the rule is written in law; a program applies it exactly; a learned model would approximate the law from past returns and be wrong on rare cases, and nobody wants an approximately correct tax bill. *Sorting*: merge sort is proved correct for every input. *Password check*: "hash and compare" must be right every time, which learning never guarantees. By contrast, an industrial fault-diagnosis expert system built from two hours a week with an engineer runs into three separate difficulties: *obtaining* the knowledge (the engineer "just knows" that a rattle means a loose coupling and cannot state it as a test), *representing* it (judgements are graded, interacting, full of exceptions; hundreds of rules contradict each other in unanticipated corners), and *unfamiliar situations* (a new generation of machines shows faults the engineer never saw; the rules are silent and cannot improve). Learning attacks all three: patterns come from logged sensor readings and repair records, graded effects are numbers, and the model is retrained on the new machines' data. Expert knowledge still decides which measurements to record, sanity-checks the model, and covers rare catastrophic faults with no data.

Writing rules versus learning from experience

	Route 1: rules	Route 2: learning
What is transferred	the *result* of human learning, as a procedure	the *experience*, plus a capacity to learn
Requirement	someone can state the complete procedure	someone can recognise correct behaviour on examples, and examples exist
What is manipulated	the rule base	the experience, the model family, the objective
Behaviour	exact, only on anticipated cases	approximate, adapts to the cases in the data
Failure mode	knowledge acquisition bottleneck; silent on unfamiliar cases	wrong or narrow experience teaches the wrong lesson confidently
Prefer when	procedure known, exactness required (tax, sorting, crypto, safety interlocks)	ability without describable procedure, many examples (digits, speech, fraud)

Most real systems mix both: a learned model produces a prediction and an explicit program decides what to do with it under fixed business rules.

"Machine learning replaces programming" and "expert knowledge is obsolete"

Both are false and both cost points. Learning replaces the *task-specific rules*; the learner, the pipeline and the decision logic are still programmed. Expert knowledge decides what to measure, checks whether the model's behaviour is physically plausible, and covers cases for which no data exist. The tempting error is to answer "learn it" to every task; the three questions to ask are: can a competent person write the complete procedure (then program it); can we recognise correct behaviour on examples and collect many (then learn it); what does a mistake cost (learning is approximately correct; when that is unacceptable, keep the program or wrap the learned part in explicit safeguards).

Key insight. Having an ability is not the same as having an explicit procedure that describes it; the whole field rests on this gap, and human vision is its largest example.

How the professor asks it

Q. *At what point in the handwritten-digit thought experiment would you stop adding rules and start giving the computer examples? What does the experiment reveal about the difference between the two routes? (Exercises ch. 1, Q1(d))*
A. When the cost of writing and maintaining rules exceeds the cost of collecting labelled examples, which happens as soon as each new batch of handwriting exposes new exceptions and each rule change has unpredictable side effects on the others. Mechanism: rules must anticipate every variation (shift, scale, slant, stroke width) explicitly, with thresholds nobody can set; examples already contain the variations, and the learner absorbs them. The experiment reveals that the two routes have different *requirements*: programming needs the procedure to be known and stated, learning needs only that correct behaviour be recognisable on examples. Consequence: route 1 fails exactly on the abilities humans find easiest, because those are the ones we perform without a describable procedure.

Q. *Give a counterexample to the claim "if a task is important and we have a lot of data, we should learn it from the data".*
A. Income tax, or a password check. The procedure is completely known (written in law; "hash and compare"), so a program applies it exactly; a learned model would only approximate it from past cases and be wrong on rare ones, and "approximately correct" is unacceptable there. Mechanism: learning gives behaviour that is correct on average over situations resembling the data, never a guarantee on every input. Consequence: the decision is made by three questions (is the procedure known; can correct behaviour be recognised on examples; what does a mistake cost), not by the importance of the task or the volume of data.

3 What counts as experience? The three paradigms

Examples, observations, interactions

Experience reaches a machine in three forms. **Examples** pair an input with a desired output (an image with its digit, a house with its sale price): *direct guidance* about what the output should be. **Observations** are data without a supplied answer (images, text, audio, sensor logs): they contain *regularities* that can be exploited without anyone labelling them. **Interactions** arise when an agent acts in an environment and observes what follows: the learner *partly shapes its own future experience*, because its actions change what happens next.

Purpose. To know, before any algorithm is chosen, what information the learner will receive; that decides what kind of learning is even possible.

Impact. The same raw data (X-rays, network flows) can be presented as any of the three forms, and the form changes which questions can be answered and how success is judged.

The three learning settings

Supervised learning: experience consists of examples with desired outputs. **Unsupervised learning**: the learner searches for useful structure in observations without desired outputs; **self-supervised learning** also learns from observations but *constructs* a learning signal from them (hide a part, predict it). **Reinforcement learning**: experience comes through interaction; consequences and rewards provide information about the quality of behaviour. These are broad settings, not mutually exclusive kinds of system; one system can use several.

Purpose. A paradigm names *what information about "doing well" reaches the learner*: a correct answer per example, nothing external, an answer manufactured from the data itself, or a reward earned through action.

Impact. Supervision is the most informative signal and the most expensive to obtain; observations are cheap and plentiful but carry no answer; rewards are delayed and evaluative ("that was good", not "this was the right answer") and the learner's own actions bias what it sees next.

Same data, four paradigms: the hospital experiment

Setup. One collection of chest X-rays; for some patients a diagnosis was recorded. Run four projects on it.

Held fixed. The images themselves.

What we manipulate. What is presented to the learner and what feedback it gets: (1) image plus recorded diagnosis, feedback is agreement with the diagnosis; (2) images only, feedback is an internal criterion such as within-group similarity; (3) images only, but a region is hidden and the model must reconstruct it, feedback is the match with the hidden pixels; (4) a robotic device positions the sensor and receives a reward for good positioning and a penalty for unsafe movements.

What we measure. For (1) accuracy against diagnoses on new patients; for (2) the quality of the grouping by the internal criterion and by whether a clinician finds the groups meaningful; for (3) reconstruction error on hidden regions of new images; for (4) the reward collected on new positioning episodes.

What the plot shows. There is no single curve; the point is a table with one row per project and columns "experience", "feedback", "paradigm": supervised, unsupervised, self-supervised, reinforcement.

How to read it. The paradigms differ not by the data type (all four use X-rays) but by what information about doing well reaches the learner. Only in (4) does the learner influence its own future experience. "Unsupervised" does not mean "no objective": project (2) still needs a criterion for what a good grouping is; what is absent is a human-supplied answer per example.

Same records, three questions; one cybersecurity task per paradigm

A university has ten years of student records. *Will each student graduate?* Supervised, categorical output: classification. *What will the final grade be?* Supervised, numerical output: regression. *Which students have similar trajectories?* No desired output: unsupervised. The collection supports several problems because the *problem* is defined by the question asked, not by the data. In security: supervised, classify flows as benign or attack with the label of each training flow as the signal; self-supervised, predict a masked field of a flow from the rest, the hidden part

is the signal and no human labelled anything; reinforcement, an agent chooses block, rate-limit or nothing in a simulated network, the signal is a reward (attack contained minus disruption caused) and its actions change the traffic it sees next.

Supervised versus unsupervised versus self-supervised versus reinforcement

	Supervised	Unsupervised	Self-supervised	Reinforcement
Experience	examples (x, y)	observations x	observations x	actions and consequences
Signal	the desired output per example	an internal criterion (similarity, density)	a target built from the data itself (the hidden part)	a reward, delayed and evaluative
Who shapes the experience	the data collector	the data collector	the designer of the pretext task	partly the learner's own actions
Typical output	a prediction for a new x	a structure (groups, factors)	a representation, or a generator	a policy from states to actions
Failure mode	bad or narrow labels	meaningless groups under the wrong criterion	pretext task that does not transfer	reward hacking; exploring unsafe actions

Naming the paradigm from the data type

"It uses images, so it is supervised" or "it uses logs, so it is unsupervised". The paradigm is decided by the feedback that reaches the learner: four projects on the same X-rays belong to four paradigms. A related slip: calling self-supervision "supervised because there is a target". There is a target, but it was manufactured from the observation itself; nobody labelled anything, which is exactly why it scales to the whole internet.

Key insight. Ask what information about "doing well" reaches the learner; that single question names the paradigm.

How the professor asks it

Q. *A hospital holds X-rays, some with recorded diagnoses. Classify the following four projects into learning settings and justify each: predict pneumonia on a new X-ray; group thousands of unlabelled X-rays by visual similarity; hide a region of each X-ray and predict it; teach a robot to position the sensor with rewards. (Exercises ch. 1, Q5)*

A. (1) Supervised: experience is images paired with diagnoses and the feedback is agreement with the recorded diagnosis. (2) Unsupervised: images only; the only feedback is an internal criterion such as within-group similarity, no external answer per image. (3) Self-supervised: images only, but each image supplies its own target (the hidden region), so a supervised-looking problem is built without labels. (4) Reinforcement: experience is the device's own actions and their consequences; the feedback is a delayed, evaluative reward, not a correct answer, and only here does the learner shape what it sees next. The data are identical in all four; the paradigm is decided by what information about doing well reaches the learner.

Q. *Explain why "unsupervised" does not mean "no objective", with an example.*

A. Unsupervised means no human supplies the desired answer for each example; it does not mean the learner has nothing to optimise. Grouping X-rays still requires a criterion for a good grouping (for instance, images within a group should be more similar to each other than to images in other groups), and the learner judges structure by that internal criterion. Change the criterion and the groups change, so the designer still chooses an objective. Consequence: an unsupervised result

is only as meaningful as its criterion, and "the algorithm found these clusters" is not evidence that the clusters mean anything clinically.

Q. *Why is reinforcement learning the only setting in which the learner influences its own future experience, and what difficulty does that create?*
A. In supervised and unsupervised learning the data are collected independently of the learner; in reinforcement learning the agent's actions change what happens next, so the experience it accumulates depends on the policy it currently has. The difficulty is twofold: the reward is delayed and evaluative, so the learner is never told which action was right, only roughly how good a sequence was; and a poor early policy visits poor situations, so the learner can fail to discover better behaviour (and may explore unsafe actions, which is why the sensor-positioning robot is penalised for unsafe movements). Consequence: data collection and learning cannot be separated as they can in the other settings.

4 Supervised learning: examples, labels, outputs

Supervised example, training dataset

A **supervised example** is a pair (x, y) of an **input** x and a **desired output** y; a **training dataset** is a finite collection $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{N}$ [on your sheet]. The desired output is called the **target**, the **label** or the **ground truth**. The learner's goal is a model that produces useful outputs for *new* inputs, not for the N it has seen.

Purpose. The pair is the unit of direct guidance: it says, for one input, what the right answer was.

Impact. Every property of the pairs (how many, how varied, how the y were obtained, how old they are) limits what can be learned; no algorithm compensates for bad pairs.

Where labels come from, and why "ground truth" is a strong phrase

Labels come from three imperfect sources. **Measurement**: temperature, electricity consumption, a recorded sale price; instrument error and recording mistakes. **Annotation**: identifying an object in an image, interpreting a scan; experts disagree, and labels depend on conventions fixed when the task was defined (is "dog" correct for a wolf-dog hybrid?). **Delayed outcome**: whether a loan is repaid, whether a machine fails within a month; known only after time has passed, hence only for old inputs, in a world that may have changed since.

Purpose. To treat label quality as an explicit limit on achievable accuracy.

Impact. A model trained on labels that are wrong 10% of the time cannot be expected to agree with the (noisy) labels more than about 90% of the time, and its errors will partly mirror the annotators' errors. A reported 99% accuracy on labels that are themselves 90% reliable should raise suspicion, not admiration.

The training cycle, as a procedure

Setup. A model with adjustable parameters, a training dataset of pairs.

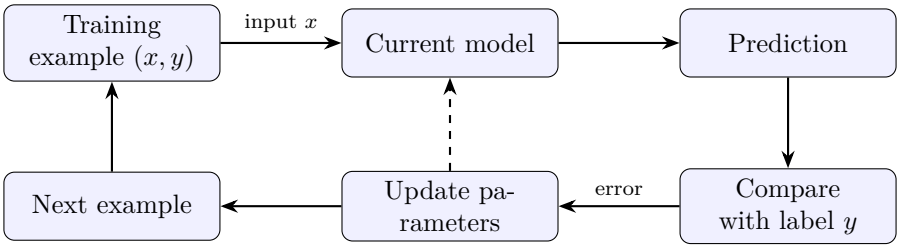
Held fixed. The dataset, the form of the model, the rule that compares a prediction with a label.

What we manipulate. The model's parameters, one update at a time.

What we measure. The disagreement between the prediction and the label on the current example; aggregated over the data, the training error.

What the plot shows. Training error against the number of updates: typically falls, fast at first and then slowly.

How to read it. The cycle is: receive an input, produce a prediction, compare with the desired output, use the comparison to change the model, move to the next example, repeat across the available experience. This computational process is **training**. Chapter 2 turns the informal cycle into a mathematical problem (a loss, an empirical risk, a minimisation); the falling curve says only that the model fits what it has seen, which, as Section 7 shows, is not the goal.



Phishing detection: designing the experience before any algorithm

Where do examples come from? Reported phishing pages (security feeds, user reports) and legitimate pages (popular sites, the organisation's own traffic, and the real login pages that phishing imitates, which are the hard negatives). *How varied?* Many languages, many impersonated brands, pages built with different kits, crude and sophisticated attacks; otherwise the model learns "phishing = badly spelled English page imitating one bank". *How are correct outcomes established?* Analyst confirmation or confirmation by the impersonated organisation; both imperfect, and over-eager reporters mislabel some legitimate pages. *How fast does the world change?* Monthly; labels from three years ago describe attacks nobody uses, and today's attacks are absent. A million pages from one feed, one language and one year is a very large dataset that teaches a narrow, outdated notion of phishing with great confidence. Volume does not substitute for coverage.

Regression and classification

A supervised problem is **regression** when the desired output is numerical (a real number or a vector) and **classification** when it is a category from a finite set. *The output decides the type, not the input.* Richer outputs exist (object detection returns boxes with labels, segmentation a class per pixel, translation a sentence); they are still supervised, with structured outputs.

Purpose. The output type decides which losses make sense, which model outputs are needed, and how to evaluate.

Impact. The same medical image gives regression (tumour size in millimetres) or classification (benign or malignant); the same weather data gives regression (millimetres of rain) or classification (will it rain?).

Prediction, decision, action

A **prediction** expresses what the model says about an unknown output. A **decision** uses the available information together with a *criterion* to select a response. An **action** is what is carried out when the decision calls for something to be done.

Purpose. To separate what the data say from what should be done about it; the second depends on costs the data do not contain.

Impact. A model predicts a 12% probability of disease; a protocol recommends testing above 10% (decision); the test is scheduled (action). Change the threshold to 20% and the same prediction leads to a different decision. The three coincide only when all errors cost the same; they separate as soon as a missed disease costs more than an unnecessary test (Chapter 2 makes this exact).

Regression versus classification; prediction versus decision

	Regression	Classification
Output	a number or vector (price in MAD, millimetres, hours, megawatts)	one of a finite set of classes (digit, spam or not, species)
What "close" means	numerical distance between y and the prediction	right class or wrong class; possibly a probability over classes
Decided by	the output, never the input	the output, never the input
Trap	"regression" does not mean linear regression	"classification" does not mean two classes
	Prediction	**Decision**
Object	what the model says (a number, a class, a probability)	a response chosen by applying a criterion (threshold, costs, protocol)
Changes when	the data or the model change	the costs or the protocol change, with the same prediction

A label is not truth; a probability is not a decision

Three slips. (1) Trusting an accuracy figure without asking how the labels were produced: measured, annotated or observed later, each with its own error, and label error caps what any model can reach. (2) Reading "regression" as "linear" or "classification" as "binary". (3) Writing "the model says yes" when the model says 0.6: the model has made a prediction, and the decision needs a threshold that encodes costs; two users with different costs should decide differently on the same 0.6.

Key insight. Supervised learning is only as good as its pairs: where the labels came from, how varied the inputs are, and how old the pairs are limit what can be learned before any algorithm is chosen.

How the professor asks it

Q. *Network flows are labelled benign or attack from incident metadata; the system scores a new flow and decides whether to alert. Identify the physical object, the representation x, the target y, the model output and the final action, and explain why the output and the action are different objects. (Tutorial 1, Q1(a) and (c))*

A. Physical object: the completed flow, the actual packet exchange between two endpoints. Representation x: the feature vector computed when the flow completes (duration, packets and bytes per direction, protocol, destination port, flags). Target $y \in \{0 = \text{benign}, 1 = \text{attack}\}$ from the ground-truthing process, so it describes the episode the flow belonged to, not its appearance. Model output: a score or a posterior estimate $\hat{p}(y = 1 \mid x)$. Action: alert or not, possibly a third action, send to an analyst. The output is a *prediction*, a number summarising what the features say; the alert is a *decision*, a rule applied to that number which also needs the costs of a false alarm and a miss. The same 0.3 should alert where a miss is far costlier than a false alarm and not where alert fatigue dominates; keeping them apart lets you change the threshold without retraining.

Q. *Some attack flows and some benign flows have exactly the same observed features. Explain why no classifier using these features can separate them, and whether the labels are therefore wrong. (Tutorial 1, Q1(b))*

A. It cannot, and the labels can still be right. A classifier is a function of x alone, so it gives both flows the same answer and is wrong on one of them whatever it does; the best possible rule predicts the majority class at that x and still loses the minority share. The labels are correct because "attack" is defined by episode membership: an ordinary-looking reconnaissance request is correctly labelled attack. What is missing is information in x (the host's other flows, the time

context), not correctness in y. Consequence: the fix is a richer representation, not relabelling, and no model beats this floor on the given features.

5 Representations: what is presented to the learner

Representation, feature engineering, representation learning

A learner never sees a house or a photograph; it sees numbers. A **representation** is a transformation of the original input into a form that makes useful structure easier for a learning model to exploit. The form of experience says what information *exists*; the representation says what information the model *gets*. Two pipelines:

$$\text{Raw Input} \longrightarrow \text{Human-Designed Features} \longrightarrow \text{Learned Predictor},$$
$$\text{Raw Input} \longrightarrow \text{Learned Representation} \longrightarrow \text{Learned Predictor}.$$[not on your sheet: know it]

Feature engineering (first pipeline): humans decide what to present (floor area, location, age for a house; edges, corners, textures for vision; mel-frequency cepstral coefficients for speech) and the model learns how to use it. **Representation learning** (second pipeline): the machine discovers the transformation from experience; the idea is broader and older than deep learning, which suits it because successive layers build higher-level features (parts, shapes) from lower-level ones (edges, local patterns). Humans still choose the architecture, the method, the experience and the objective.

Purpose. To control which information actually reaches the model; the choice can decide whether learning succeeds at all.

Impact. With few examples, a good engineered representation makes a problem learnable that is hopeless on raw input; with enormous data, a learned representation can surpass the engineered one because the designer forgot features that matter.

Two representations of the same email

Setup. A spam filter. Representation (a): the raw sequence of characters. Representation (b): a vector of counts (how often each of 10,000 common words appears, whether the sender is in the address book, the number of links, the number of exclamation marks).

Held fixed. The emails, the labels, the learning algorithm.

What we manipulate. The representation, and the number of training examples (a few thousand, then billions).

What we measure. Error on new emails.

What the plot shows. Error against training-set size, one curve per representation: with a few thousand examples (b) is learnable and (a) is hopeless; the (a) curve only comes down with billions of examples and a model able to build its own representation, and can then cross below (b).

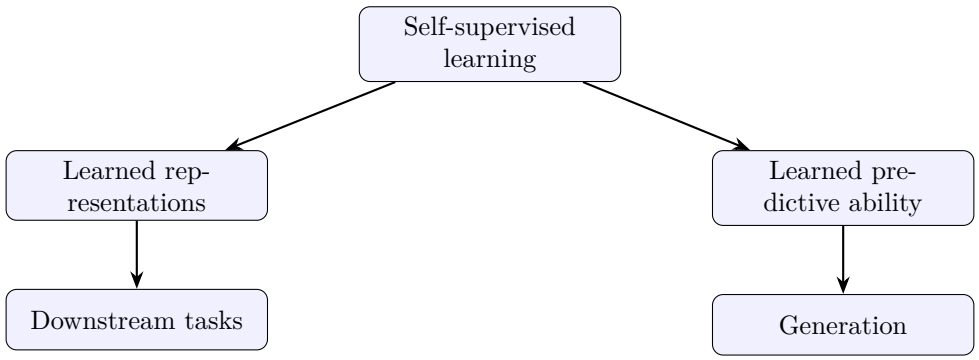
How to read it. Representation (b) makes "many links, no known sender, the word *prize*" directly visible as a few coordinates; (a) hides the same facts behind thousands of characters whose positions vary from message to message. The crossing shows the two regimes: engineered features win when data are scarce; learned representations win when data are abundant, because the designer of (b) forgot features that matter.

Self-supervised learning and generation

In **self-supervised learning** a learning problem is constructed from the observations themselves: part of an observation is used to predict another part. Hide a word in "The train to _____ leaves at nine"; mask a square of a photograph and reconstruct it; remove half a second of speech and predict it. Nobody labelled anything; the data are their own supervision. Two uses of what is learned: the *representation* can be adapted with a small labelled dataset to classification or detection (*downstream tasks*); or the *predictive ability* is useful directly. An **autoregressive language model** learns to predict the next token from the previous ones; once it can predict, it can **generate**: predict a next token, append it, predict again.

Purpose. To learn from unlabelled observations, which exist in vastly larger quantities than labelled examples.

Impact. The pretext task gives the model a reason to discover regularities; the quality of the representation depends on whether the pretext task requires the structure that the downstream task needs.



The two branches are two *uses* of what was learned, not two kinds of model: a generative language model also learns internal representations.

A next-token model computed by hand, and what it generates

Corpus: `the cat sat on the mat . the cat ate the rat .` The simplest next-token model looks only at the previous word and estimates probabilities by counting. After `the` the next word is `cat` twice, `mat` once, `rat` once, four occurrences in all, so

$$\mathbb{P}(\texttt{cat} \mid \texttt{the}) = \tfrac{2}{4}, \qquad \mathbb{P}(\texttt{mat} \mid \texttt{the}) = \tfrac{1}{4}, \qquad \mathbb{P}(\texttt{rat} \mid \texttt{the}) = \tfrac{1}{4}.$$ [not on your sheet: know it]

After `cat`: `sat` and `ate`, each $\frac{1}{2}$. After `sat`: `on` with probability 1; after `on`: `the`; after `mat` or `rat`: the full stop. Each number is a count divided by the number of times the context word appeared; no label was written, the corpus itself says what followed what.

Generation. Start from `the`; sample from $(\frac{1}{2}, \frac{1}{4}, \frac{1}{4})$, say `cat`; sample from $(\frac{1}{2}, \frac{1}{2})$, say `ate`; after `ate` the only continuation is `the`; and so on. One possible output is `the cat ate the mat .`, a sentence that never appears in the corpus: the model recombined regularities it found.

Probability of a whole sentence. Multiply the transitions along the way. For `the cat sat on the mat .` started at `the`: $\frac{1}{2} \cdot \frac{1}{2} \cdot 1 \cdot 1 \cdot \frac{1}{4} \cdot 1 = \frac{1}{16}$. The unseen `the cat ate the mat .` gets exactly the same $\frac{1}{16}$: the model learned transitions, not sentences. Large language models do the same with thousands of context tokens and billions of parameters in place of a count table; the principle, self-supervised next-token prediction followed by repeated sampling, is this one.

Feature engineering versus representation learning versus self-supervision

	Feature engineering	Representation learning	Self-supervision
Who chooses the features	humans, from domain knowledge	the machine, from data	the machine, from unlabelled data via a pretext task
Needs labels	for the predictor only	yes, for the whole pipeline (unless self-supervised)	no; targets come from the data
Strength	works with little data; interpretable	finds features the designer forgot	uses observations that exist in huge quantities
Failure	designer forgets what matters	needs much data; opaque	pretext task may not transfer to the downstream task

Treating self-supervision as "supervised" or as "unsupervised" without saying why

Both labels are half right, and the professor wants the mechanism: the learning signal is a target, so training looks supervised, but the target was manufactured from the observation (the hidden word, the masked region), so no human labelled anything, which is what unsupervised means. The point to state is *where the signal comes from*. A second slip: "representation learning = deep learning". Deep learning suits it, but the idea predates it and representations can be learned by many methods.

Key insight. The representation decides what the model can see; a learner cannot use information that its input does not carry, however much data it is given.

How the professor asks it

Q. *Explain why the choice of representation can decide whether learning succeeds, using two representations of the same email, and say in which regime each one wins.*
A. A learner can exploit only the regularities visible in its input. The count-vector representation exposes "many links, no known sender, the word *prize*" as a few coordinates, so a few thousand examples suffice to find the rule; the raw character sequence hides the same facts behind thousands of characters whose positions vary from message to message, so no small dataset reveals them. With billions of emails and a model that builds its own representation, the raw input can surpass the counts because the designer of the count vector forgot features that matter. Consequence: engineered features win when data are scarce, learned representations win when data are abundant; the representation is a design choice, not a detail.

Q. *In the corpus* `the cat sat on the mat . the cat ate the rat .`, *compute the probability that the bigram model, started at* `the`, *generates* `the cat ate the mat .`, *and explain why a sentence absent from the corpus can have the same probability as one present in it.*
A. Multiply the transitions: $\mathbb{P}(\texttt{cat} \mid \texttt{the})\,\mathbb{P}(\texttt{ate} \mid \texttt{cat})\,\mathbb{P}(\texttt{the} \mid \texttt{ate})\,\mathbb{P}(\texttt{mat} \mid \texttt{the})\,\mathbb{P}(\texttt{.} \mid \texttt{mat}) = \frac{1}{2}\cdot\frac{1}{2}\cdot 1\cdot\frac{1}{4}\cdot 1 = \frac{1}{16}$. The corpus sentence `the cat sat on the mat .` also gets $\frac{1}{2}\cdot\frac{1}{2}\cdot 1\cdot 1\cdot\frac{1}{4}\cdot 1 = \frac{1}{16}$. The model learned *what follows what*, one word at a time, not whole sentences; any sequence built from seen transitions is as probable as the product of those transitions, whether or not it occurred. Consequence: generation is recombination of learned regularities, which is both its power (novel output) and its risk (fluent nonsense).

Q. *Give one self-supervised task for network traffic data and say what the learned representation could be used for afterwards.*
A. Hide one field of a flow record (say the destination port, or the byte count in one direction) and train a model to predict it from the remaining fields and the neighbouring flows of the same host. The signal is the hidden field, so no analyst labels anything. To predict it, the

model must learn what normal combinations of protocol, timing and volume look like, which is a representation of "typical traffic". Afterwards a small labelled set of attack and benign flows adapts the representation to classification, and a flow that the model reconstructs badly is a candidate anomaly. Consequence: labels, the scarce resource in security, are needed only for the last step.

6 Four ingredients of a learning system

Experience, model, learning algorithm, evaluation criterion

Almost any learning system can be examined through four questions.

Experience	What information is available?
Model	In what form can acquired knowledge be represented, and what does it output?
Learning algorithm	How does experience change the model?
Evaluation criterion	How is success judged?

This begins to answer "what gives a machine the capacity to learn?": the model determines what can be represented; the learning algorithm is the mechanism through which experience changes it; the evaluation criterion indicates which changes are useful; the experience determines what evidence exists in the first place. Representation learning adds that the internal representation can itself be shaped by experience.

Purpose. To describe, criticise and improve any application without naming an algorithm.

Impact. Each ingredient is responsible for a different failure: bad experience, a model that cannot express the relationship, an algorithm that cannot find it, or a criterion that rewards the wrong thing. Diagnosing a failing system means asking which ingredient is at fault.

Four systems, four ingredients

	Experience	Model	Learning algorithm	Evaluation
Bigram text model	a corpus	a table of next-word probabilities	count and normalise	probability assigned to held-out text
Digit classifier	labelled images	a function from pixels to 10 probabilities	adjust parameters to reduce error on examples	accuracy on new images
Fraud detector	past transactions with confirmed outcomes	a function from transaction features to a fraud probability	the same, with rare positives weighted up	cost of missed fraud plus cost of blocked customers
Warehouse robot	its own actions in simulation	a policy from sensor readings to motor commands	reinforcement from rewards	task completion and safety in the real warehouse

Each row can be discussed without naming an algorithm; that is the point of the checklist.

Designing an application from scratch: the irrigation pump

Setup. Predict, from a farm's sensor data, whether an irrigation pump will fail in the next week.

Held fixed. The question (failure within seven days) and the deployment (the maintenance team ranks pumps weekly).

What we manipulate. The four ingredients, written in the recommended order: (1) *evaluation criterion*: on pumps and months not used during development, how many failures were flagged in advance and how many unnecessary inspections were triggered, converted into money; (2) *experience*: two years of hourly readings (current, vibration, temperature, flow) from 300 pumps, with the maintenance log giving each failure and its date; each week of readings becomes an input and the label is whether a failure occurred in the following week; (3) *model output*: a probability of failure within seven days, so pumps can be ranked; (4) *learning algorithm*: its signal is the disagreement between predicted probability and what happened, aggregated over many pump-weeks, with missed failures weighted far above false alarms because failures are rare.

What we measure. The criterion in (1), on held-out pumps and later months.

What the plot shows. Not a curve but a checklist; the professor's question is "map each part to an ingredient".

How to read it. Start with the criterion (what would make the client say "it works"?), then the experience (what is logged, what can be labelled, what changes over time), then the model output (number, class, probability, ranking, sentence?), and only then the algorithm, which has the most textbooks and the least influence on whether the project makes sense.

Starting from the algorithm

The tempting answer to "design a system for X" is "use a random forest" or "use a neural network". It earns nothing because it says nothing about what the system will see, what it outputs, or how anyone will know it works. The fix is the order criterion, experience, output, algorithm. A second slip is to confuse the model's *output* (a probability of failure) with the *decision* (inspect the top ten pumps): the output is the model ingredient, the inspection policy is a criterion applied to it.

Key insight. Any learning system is experience plus model plus algorithm plus criterion; the capacity to learn lives in the last three and the evidence lives in the first.

How the professor asks it

Q. *Describe an application not used in the chapter without naming any algorithm, and map each part of your description to one of the four ingredients. (Exercises ch. 1, Q9)*
A. Hourly electricity demand of a city for the next day. (a) Experience: years of hourly demand with weather, calendar and events. (b) Model output: a number for each of the 24 hours, possibly an upper quantile since under-supply costs more than over-supply. (c) Signal for improvement: the gap between predicted and realised demand on past days, with the asymmetric cost built in. (d) Success: error on *later* days not used for training, weighted by that cost, against the baseline "same as last week". Mapping: (a) experience, (b) model, (c) learning algorithm, (d) evaluation criterion. Design in the order (d), (a), (b), (c).

Q. *Explain why the evaluation criterion should be written before the learning algorithm is chosen, with an example where choosing the algorithm first leads to a useless system.*
A. The criterion states what the client means by "it works"; the algorithm is a means of producing a model that scores well on that criterion, so without the criterion there is nothing to optimise and no way to compare candidates. Example: a fraud team picks a high-accuracy classifier first; with 0.5% fraud, the rule "never fraud" already scores 99.5% accuracy and the chosen model is tuned to match it, blocking nobody. Had the criterion been written first (cost of missed fraud plus cost of blocked customers, on later transactions), the team would have weighted rare positives up and judged the model on the money it saves. Consequence: the algorithm has the most textbooks and the least influence on whether the project makes sense.

Q. *A learned system performs badly. Name, for each of the four ingredients, one way that*

ingredient alone could be the cause.
A. Experience: the examples are narrow, mislabelled or outdated, so the regularities learned do not hold at deployment (phishing from one feed). Model: the family cannot express the relationship (a straight-line model for a curved effect), so no amount of data helps. Learning algorithm: it fails to find a good solution of its own objective (stops too early, gets stuck), so the model is worse than the family allows. Evaluation criterion: it rewards the wrong thing (accuracy on 0.5% fraud), so a useless model looks excellent. Consequence: diagnosis means isolating the ingredient, because the fixes (collect, enlarge the family, optimise better, change the metric) are entirely different.

7 The challenge of generalization

Generalization

Experience is always finite (no digit recogniser sees every handwriting, no medical system every patient, no robot every situation) and the deployed system meets *new* inputs. **Generalization** is the ability of a learned system to perform well in new situations that were not encountered during learning. The central challenge of machine learning is

$$\text{Finite Experience} \xrightarrow{\text{Learning}} \text{Ability Useful in New Situations.}$$ [not on your sheet: know it]

Purpose. It is the actual goal; performance on the training examples is a side effect, and a dangerous one to optimise for.

Impact. Chapter 2 will give it two numbers, the training risk $\widehat{R}$ (computable) and the population risk R (wanted); Chapter 3 studies their gap. Here the point is qualitative: training performance is not evidence of generalization.

Finite data are consistent with infinitely many rules

Proposition. Let $\mathcal{X}$ be an infinite input space, $\mathcal{Y}$ have at least two elements, and $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{N}$ be any finite dataset. Then infinitely many functions $f : \mathcal{X} \to \mathcal{Y}$ agree with the data, and two of them may disagree on *every* input not in the data. [not on your sheet: know it]
Proof, step by step. Let $S = \{x^{(1)}, \dots, x^{(N)}\}$, a finite set, so $\mathcal{X} \setminus S$ is infinite (an infinite set minus a finite one stays infinite). Fix two distinct outputs $y_0 \neq y_1$ in $\mathcal{Y}$. For each subset $T \subseteq \mathcal{X} \setminus S$ define

$$f_T(x) = \begin{cases} y^{(i)} & \text{if } x = x^{(i)} \in S, \\ y_1 & \text{if } x \in T, \\ y_0 & \text{otherwise.} \end{cases}$$

Every f_T agrees with the data, because on S it returns the recorded label. Distinct subsets $T \neq T'$ give distinct functions, because they differ on any x belonging to one of them and not the other. An infinite set has infinitely many subsets, so there are infinitely many data-consistent functions. Finally $T = \mathcal{X} \setminus S$ and $T' = \emptyset$ give two data-consistent functions that disagree on every input outside the data. □
What it means. Off the training inputs, the data say nothing at all. Whatever a learned system does there comes from its *model* and its *learning algorithm*, that is, from assumptions built in by the designer. Generalization is never a property of the data alone; it is a property of the match between those assumptions and the world.

The perfect memoriser

"Look up x in the training set and return its label; otherwise return class 0." Zero error on every training example; nothing learned. On a continuous input space a new x is never in the table, so the rule always answers 0, and with two balanced classes it is right half the time: a coin toss. A learner evaluated only on its training data cannot tell this lookup table from a genuinely useful model. This is why performance on previously encountered examples gives only part of the evidence about what has been learned.

The shortcut experiment: wolves and snow

Setup. A system distinguishes photographs of wolves from dogs. In the collection, most wolf photographs contain snow and most dog photographs do not. Say 1000 wolves of which 900 in snow, 1000 dogs of which 100 in snow.

Held fixed. The learner, the labels, the task.

What we manipulate. The *conditions* of the test images: first the developers' held-out images (same collection, same correlation), then deployment images where users photograph dogs in snow and wolves in summer (say 30% snow in both classes), then two probe images, a husky in snow and a wolf on green grass.

What we measure. Accuracy under each condition, for the learner and for the explicit rule "white background ⇒ wolf".

What the plot shows. Accuracy against condition. The snow rule scores $(900 + 900)/2000 = 90\%$ on the collection and the learner scores similarly on the developers' held-out split (it shares the correlation). At deployment with 30% snow in both classes the snow rule scores $0.5 \cdot 0.3 + 0.5 \cdot 0.7 = 50\%$, a coin toss, and the learner's accuracy collapses with it. On the two probes a system that learned wolves gets both right; a system that learned snow gets both wrong.

How to read it. (1) *Initial success*: the proposition above guarantees that alternative rules agree with the data; "snow ⇒ wolf" is one, and far easier to detect than wolf anatomy, so the learner adopted it. (2) *Failure*: the correlation was a property of the *collection*, not of wolves, so the rule is systematically wrong once the collection's conditions change. (3) *Diagnosis*: decouple the cue from the label and look. Nothing here opens the model; the reasoning uses only the definition of generalization.

Quantity, diversity, representativeness

Three different properties of a collection of experience. **Quantity**: how many examples. **Diversity**: how much of the space of situations they cover. **Representativeness**: whether they are distributed like the situations the deployed system will meet. A learner finds regularities in whatever it is given; a million near-repeats of one situation contain one situation's regularities, which will not hold elsewhere.

Purpose. To separate "more data" from "better experience".

Impact. More data lowers the error of estimating regularities *present* in the collection and does nothing about regularities *absent* from it; generalization depends on the second.

Three stories about experience

Birds. Dataset A: one million photographs from one camera in one park. Dataset B: fifty thousand by many people in many countries, seasons, weathers, backgrounds and angles. For a system used by the public, B: A's images share lighting, background, altitude and season, so the regularities a learner finds in A (this park's foliage, this camera's colour balance) will not hold elsewhere, exactly as with the snow. A would be preferable if the system is deployed *in that park with that camera*, where A is perfectly representative, or if the species are so similar that

sheer numbers are needed and the deployment conditions match. *The robot in the wrong world.* A warehouse robot trained only in a simulator with flat floors, identical friction and constant lighting fails in reality. If only the simulator can be changed, vary what varies in reality *and affects the decisions*: floor slope and bumps, box weights and friction, lighting, sensor noise, worker behaviour, wheel slippage. A policy trained across the range cannot rely on any one constant. Making the simulator visually realistic would not help by itself: if the floor is still flat, the robot still learns that wheels never slip. *The advertisement.* "Our AI has learned every possible situation from millions of examples." This confuses situations that were *observed* with situations that *may occur*. Millions of examples are a finite sample; the situations a deployed system faces are not a fixed list, include ones nobody recorded, and the world changes after collection. The honest claim is good performance on seen situations and an *expectation* that the regularities carry over; whether they do is generalization, and no amount of data makes the question disappear.

Shortcut versus target leakage

Target leakage: a feature that would *not be available at decision time*, or that is a consequence of the label (an incident ticket number, "host quarantined afterwards", "number of analyst comments"). It predicts the label almost perfectly in the dataset and does not exist for a new input. Fix: remove the field. **Shortcut**: a feature that *is* available at decision time but whose association with the label is an artefact of how the data were collected (snow, hour of day, hospital tag, camera). The feature means something else at deployment. Fix: collect data where the association is broken and test on them; dropping the feature is one option, not the only one.

Purpose. The two look identical in a table of accuracies (both inflate it) and need different repairs.

Impact. Leakage disappears when the field is removed; a shortcut survives feature removal if another collection artefact remains, and is only exposed by data gathered under different circumstances.

Testing for generalization without opening the model

Setup. Any fitted model, suspected of relying on collection-specific cues.

Held fixed. The model (no retraining in steps 1 to 3).

What we manipulate. The test conditions: (1) *hold out*: evaluate on examples the learner never saw (Chapter 2 makes this validation and test sets); (2) *break the shortcuts*: construct test inputs where suspected spurious cues are decoupled from the label (dog in snow, wolf on grass, attack by day); (3) *shift the conditions*: evaluate on data from a different camera, hospital, country or year; (4) *ablate*: retrain without the suspected feature and compare.

What we measure. Error under each condition; the *drop* between the original held-out error and the shifted or shortcut-breaking error.

What the plot shows. A bar per condition. A model that learned the task shows small drops; a model that learned the collection shows a large drop under (2) and (3). Step (4) alone is suggestive, not conclusive: if the validation split comes from the same collection, the shortcut holds on both sides of the split and the ablation measures how much the model leans on the feature, not whether the feature is stable.

How to read it. A large drop reveals reliance on regularities that held in the experience but not in the world. Also list the situations absent from the experience: the model's behaviour there is an assumption, not a learned fact.

Memoriser versus shortcut learner versus genuine learner

	Memoriser	Shortcut learner	Genuine learner
Training error	zero	near zero	small
Held-out, same collection	bad (coin toss off the table)	good (the correlation is still there)	good
Shifted or shortcut-breaking test	bad	bad, systematically wrong	good
What it learned	the table	a collection artefact	a regularity of the world
Caught by	any held-out set	only decoupled or shifted data	nothing to catch

"One million is more than fifty thousand" and "zero training error is an achievement"

Quantity is not coverage: a billion photographs of one park are one situation. Zero training error is compatible with having learned nothing (the memoriser) or the wrong thing (the snow detector); it is a reason to look at new data, not a result. And calling a shortcut "leakage" loses the mark: leakage is unavailable at decision time and is fixed by removal; a shortcut is available, spuriously associated, and fixed by breaking the association in the data and testing on it.

Key insight. Off the training inputs the data say nothing; what a learner does there comes from its built-in assumptions, and a shortcut is what happens when the assumption "cues that worked in the collection work in the world" is false.

How the professor asks it

Q. *A wolf-versus-dog classifier performs extremely well for its developers and badly after deployment; most training wolves stood in snow. Explain the initial success, the failure, and a diagnosis, without assuming anything about the model's internals. (Exercises ch. 1, Q4)*
A. Initial success: "white background ⇒ wolf" agrees with almost all training examples (finite data admit many consistent rules) and is far simpler to detect than wolf anatomy, so the learner adopted it; the developers' held-out images came from the same collection and shared the correlation, so it scored well there too. Failure: users photograph dogs in snow and wolves in summer; the correlation was a property of the collection, not of wolves, so the rule is now systematically wrong. Diagnosis: two photographs, a husky in snow and a wolf on green grass; a system that learned wolves gets both right, one that learned snow gets both wrong. Only the definition of generalization was used: good on seen situations, poor on new ones, because a regularity held in the experience and not in the world.

Q. *Most attacks in the training data occurred at night and most benign flows by day. Explain how "hour of day" becomes a shortcut, why this is not target leakage, and what you would do about it. (Tutorial 1, Q1(e))*
A. In the collected data the hour predicts the label well, so the model can use it instead of learning what attack traffic looks like; at deployment a daytime attack is missed and night-time backups raise alerts. It is a shortcut, not leakage: the hour *is* available when a new flow is scored and is not caused by the label; the association was created by the collection (a red-team campaign run overnight) and is not stable. Leakage would be a field like the incident ticket number, absent for a new flow and fixed by removal. For the shortcut, collect or construct data in which the hour varies independently of the label (attacks by day, benign traffic by night), test on them, and consider removing the hour only if nothing stable is left in it.

Q. *Removing "hour of day" raises validation error from* 0.04 *to* 0.21. *On a later week with attacks in office hours, the full model scores* 0.35 *and the model without the hour* 0.08. *Interpret both results and explain why the ablation alone could not have raised the alarm.*

A. The ablation shows the model leans on the hour far more than on any flow property, which is suspicious (the hour describes the capture, not the flow) but not proof: a stable feature would also hurt when removed. The later week settles it: 0.35 against 0.08 is a shortcut costing 0.27 at deployment while it gained 0.17 on the original split. The ablation could not see this because the validation set came from the same collection, where "night ⇒ attack" held on both sides of the split; it measured reliance, not stability. Only data collected under different circumstances separate a stable regularity from an artefact.

8 A connection to information theory

Entropy, cross-entropy, Kullback–Leibler divergence

For distributions p and q on the same C outcomes, with $q_c > 0$ wherever $p_c > 0$ and the convention $0 \log 0 = 0$:

$$H(p) = -\sum_c p_c \log p_c \text{ [on your sheet]}, \qquad H(p,q) = -\sum_c p_c \log q_c \text{ [on your sheet]},$$

$$\mathrm{KL}(p \,\|\, q) = \sum_c p_c \log \frac{p_c}{q_c} \text{ [on your sheet]}.$$

$H(p)$, the **entropy**: how uncertain the truth is; $-\log p_c$ is the surprise of outcome c, and $H(p)$ is the average surprise when you believe the truth and the truth happens. $H(p,q)$, the **cross-entropy**: the average surprise when you believed q and the truth was drawn from p. $\mathrm{KL}(p\|q)$, the **divergence**: the extra surprise caused by believing q instead of p. Expanding the divergence gives the identity

$$H(p,q) = H(p) + \mathrm{KL}(p \,\|\, q) \text{ [on your sheet]}.$$

Base-2 logarithms give bits, natural logarithms (customary in machine learning) give nats; the choice multiplies everything by a constant.

Purpose. A numerical way to say how good a set of predicted probabilities is, and a vocabulary for how much information a representation keeps.

Impact. Cross-entropy will be the training criterion for classification (Chapter 2); the identity says its floor is $H(p)$, set by the problem, and the part a model can reduce is the divergence.

Gibbs' inequality: $\mathrm{KL}(p\|q) \geq 0$ with equality iff $p = q$

Step 1, a one-line inequality about logarithms. For every $t > 0$, $\ln t \leq t - 1$ with equality iff $t = 1$ [on your sheet]. Proof: $g(t) = t - 1 - \ln t$ has $g'(t) = 1 - 1/t$, negative on $(0,1)$ and positive on $(1,\infty)$, so g has its unique minimum at $t = 1$, where $g(1) = 0$; hence $g \geq 0$. Check: $t = 0.5$ gives $\ln 0.5 = -0.693 \leq -0.5$; $t = 2$ gives $0.693 \leq 1$.
Step 2, apply it term by term and average under p. Let $S = \{c : p_c > 0\}$. Then

$$-\mathrm{KL}(p\|q) = \sum_{c\in S} p_c \ln \frac{q_c}{p_c} \;\leq\; \sum_{c\in S} p_c\Big(\frac{q_c}{p_c} - 1\Big) = \sum_{c\in S} q_c - \sum_{c\in S} p_c = \sum_{c\in S} q_c - 1 \;\leq\; 0,$$

where the first inequality is Step 1 with $t = q_c/p_c$, the next equality cancels p_c against $1/p_c$, the sum of p_c over S is 1, and $\sum_{c\in S} q_c \leq 1$ because q is a distribution. Hence $\mathrm{KL}(p\|q) \geq 0$. Changing the base multiplies by a positive constant, so the sign is unaffected.
Step 3, equality. Equality needs equality in Step 1 for every $c \in S$, i.e. $q_c = p_c$ wherever $p_c > 0$, and $\sum_{c\in S} q_c = 1$, which forces $q_c = 0$ wherever $p_c = 0$. So $q = p$. Conversely $p = q$ gives every term $\log 1 = 0$. □
Consequences. $H(p,q) = H(p) + \mathrm{KL}(p\|q) \geq H(p)$ with equality iff $q = p$: among all predictions q, the expected surprise is smallest when q is the truth. With q uniform, $\log C - H(p) =$

$\mathrm{KL}(p\|\mathrm{unif}) \geq 0$, so the uniform distribution has the largest entropy on C outcomes, $\log C$. The whole theorem is the tangent-line inequality $\ln t \leq t - 1$ applied to the ratios q_c/p_c and averaged under p; "bound the log by its tangent, then average" recurs throughout machine learning.

The calibration experiment: what cross-entropy rewards

Setup. A binary outcome whose true distribution is $p = (0.7, 0.3)$ (say 70% of flows at this x are benign). A forecaster announces q.

Held fixed. The truth p.

What we manipulate. The announced q: the truth, hedging, overconfidence in the right direction, the wrong direction, extreme overconfidence.

What we measure. The average surprise $H(p, q)$ over many draws from p, and the divergence $\mathrm{KL}(p\|q) = H(p, q) - H(p)$, in bits.

What the plot shows. $H(p, q)$ against q_1: a U-shaped curve with its minimum $H(p) = 0.881$ bits at $q_1 = 0.7$, rising to $+\infty$ at both ends.

Prediction q	$H(p, q)$	$\mathrm{KL}(p\|q)$	Reading
$(0.7, 0.3)$	0.881	0	the truth: minimum possible
$(0.5, 0.5)$	1.000	0.119	hedging costs 0.119 bits
$(0.9, 0.1)$	1.103	0.222	overconfident in the right direction: worse than hedging
$(0.3, 0.7)$	1.370	0.489	wrong direction
$(0.99, 0.01)$	2.003	1.122	the 0.3 mass pays $\log_2 100 = 6.64$ bits each time

How to read it. The floor 0.881 is a property of the *problem*: no prediction beats it because the outcome is genuinely uncertain. Overconfidence is punished sharply: $(0.9, 0.1)$ favours the right class and still scores worse than the uninformative $(0.5, 0.5)$. Minimising cross-entropy therefore trains *calibrated* probabilities, not merely the right argmax. In nats the same table reads 0.611, 0.693, 0.765, 0.950, 1.389.

Why this matters for learning

Cross-entropy as a criterion. A classifier assigns probabilities to several classes; cross-entropy compares them with what actually occurs: high probability on the correct class costs little, low probability costs much. For a *one-hot* truth ($p_y = 1$) it reduces to $-\log q_y$ [on your sheet], the penalty for the probability given to the observed class: probabilities 0.9, 0.5, 0.1 on the true class cost 0.105, 0.693, 2.303 nats. By Gibbs, the expected penalty is minimised exactly when the predicted probabilities equal the true conditional probabilities, which is why a familiar information quantity became the workhorse loss of classification. *Representations and information.* Raw observations contain many kinds of information, only some useful for the purpose; an informative representation retains what is relevant about the output and discards the rest.

Entropy versus cross-entropy versus divergence

	$H(p)$	$H(p, q)$	$\mathrm{KL}(p\|q)$
Depends on	the truth only	truth and prediction	truth and prediction
Measures	uncertainty of the outcome	average surprise of a believer in q	extra surprise caused by believing q
Minimum	0 for a certain outcome; $\max \log C$ for uniform	$H(p)$, at $q = p$	0, at $q = p$
Symmetric	not applicable	no	no: $\mathrm{KL}(p\|q) \neq \mathrm{KL}(q\|p)$ in general
Role in learning	the irreducible floor	the training loss	the part the model can reduce

Three slips with entropies

(1) Quoting KL with the arguments swapped: it is not symmetric; check the order. (2) Predicting probability exactly 0 for an outcome that can happen: $q_c = 0$ with $p_c > 0$ gives $H(p, q) = +\infty$. (3) Thinking that the argmax is all that matters: cross-entropy penalises overconfidence, so $(0.9, 0.1)$ loses to $(0.5, 0.5)$ when the truth is $(0.7, 0.3)$; a model trained with it is pushed towards calibrated probabilities, which Chapter 2 and Chapter 4 rely on.

Key insight. $H(p, q) = H(p) + \mathrm{KL}(p\|q)$ with $\mathrm{KL} \geq 0$: the expected surprise of a prediction is the problem's own uncertainty plus a penalty that vanishes only when the prediction is the truth.

How the professor asks it

Q. *State Gibbs' inequality, prove it, and explain what it implies about minimising cross-entropy.*
A. $\mathrm{KL}(p\|q) \geq 0$ with equality iff $q = p$. Proof: $\ln t \leq t - 1$ for $t > 0$ (the function $t - 1 - \ln t$ has derivative $1 - 1/t$, so its minimum is 0 at $t = 1$); apply it to $t = q_c/p_c$ and average under p: $-\mathrm{KL} = \sum_c p_c \ln(q_c/p_c) \leq \sum_c (q_c - p_c) = \sum_c q_c - 1 \leq 0$; equality forces $q_c = p_c$ wherever $p_c > 0$ and the remaining q_c to be 0. Since $H(p, q) = H(p) + \mathrm{KL}(p\|q)$, cross-entropy is minimised exactly at $q = p$ with minimum value $H(p)$: minimising expected cross-entropy pushes a classifier towards the true conditional probabilities, which is why it is the training criterion for classification.

Q. *The truth is* $(0.5, 0.5)$ *and a model predicts* $(0.99, 0.01)$. *Compute the cross-entropy in bits, compare with the honest prediction, and say what happens for the prediction* $(1, 0)$.
A. $H(p, q) = -0.5 \log_2 0.99 - 0.5 \log_2 0.01 = 0.5(0.0145) + 0.5(6.644) \approx 3.33$ bits, against $H(p) = 1$ bit for $q = p$: overconfidence more than triples the penalty, because the half of the draws that land on the second outcome each pay $\log_2 100$ bits. For $(1, 0)$, $\log_2 0 = -\infty$ and $H(p, q) = +\infty$: never predict exactly 0 for something that can happen. Consequence: a loss that explodes on confident mistakes trains models to hedge in proportion to their real uncertainty.

Q. *Explain why* $H(p)$ *is a floor that no model can beat, and what that means for a reported error on a problem with genuinely uncertain outcomes.*
A. By Gibbs, $H(p, q) = H(p) + \mathrm{KL}(p\|q) \geq H(p)$ for every q, so the smallest achievable expected surprise is the entropy of the truth, reached only by predicting the truth itself. The floor is a property of the problem (how random the outcome is given the input), not of the learner. Consequence: on a problem where 30% of flows at a given x are attacks, a cross-entropy of 0.611 nats is perfect, not mediocre; a model reporting far below the floor on its training data has memorised noise, and a claim of near-zero loss on an inherently uncertain outcome should be disbelieved.

Coverage of the textbook chapter

Textbook item	Where here
Learning, informal; four abilities; radiologist example	Section 1
Route 1, Pascaline, expert systems, MYCIN, bottleneck; programs that should stay programs; knowledge engineer's year; the "3" thought experiment; ability $\neq$ procedure; route 2; deciding between routes; gotcha	Section 2
Three forms of experience; three settings; hospital four projects; same records three questions; "unsupervised" gotcha	Section 3
Supervised example and dataset; where labels come from; phishing; training cycle figure; regression and classification; ten tasks; structured outputs; prediction, decision, action; screening system; gotchas	Section 4
Feature engineering; two representations of an email; representation definition and pipelines; self-supervision; three tasks; bigram example and generation; two-branch figure	Section 5
Four ingredients; four systems table; irrigation pump; writing the ingredients in order	Section 6
Generalization; proposition and proof; memoriser; wolves and snow; quantity versus diversity; robot; advertisement; testing without opening the model; gotchas	Section 7
Entropy, cross-entropy, KL, identity; Gibbs with proof; numbers table; why it matters; computing with entropies; classic questions	Section 8
Notes on the lecture notes (stray citation marker, "ground truth", four abilities not a taxonomy, generalization deferred, information theory recalled without proof)	the warnings of Sections 1, 4 and 8

The ten mistakes of this chapter

1. Answering "learn it" to every task: tax, sorting and password checks have a known procedure and need exactness.
2. "Machine learning replaces programming" or "expert knowledge is obsolete": only the task-specific rules are replaced; experts choose what to measure and cover cases with no data.
3. Naming the paradigm from the data type: four X-ray projects, four paradigms; the feedback decides.
4. "Unsupervised means no objective": there is always a criterion, only no human answer per example.
5. Trusting "ground truth": labels are measured, annotated or observed later, and their error caps the achievable accuracy.
6. "Regression = linear", "classification = binary": the words name output types.
7. Reading a probability as a decision: the decision needs costs; the same 0.6 decides differently under different costs.
8. "A million examples beat fifty thousand": quantity is not diversity or representativeness;

a million photos of one park are one situation.

9. Zero training error as an achievement, and confusing a shortcut (available, spurious; fix by breaking the association) with leakage (unavailable at decision time; fix by removal).
10. Entropy slips: swapped KL arguments, probability exactly 0, and forgetting that cross-entropy punishes overconfidence, so it trains calibrated probabilities rather than just the right argmax.

Chapter 1 in ten lines

1. Learning = capacity to learn + experience → ability; abilities: recognise, predict, reason, act (vocabulary, not taxonomy).
2. Route 1 writes the rules (procedure known, exactness required); route 2 learns from experience (correct behaviour recognisable, procedure not describable). Ability ≠ procedure.
3. Expert systems hit the knowledge acquisition bottleneck: obtaining, representing, unfamiliar situations; learning attacks all three, experts still matter.
4. Experience is examples, observations or interactions; the feedback that reaches the learner names the paradigm: supervised, unsupervised or self-supervised, reinforcement.
5. Supervised pairs (x, y) with labels from measurement, annotation or delayed outcomes, each imperfect; the output type decides regression versus classification; prediction, decision and action are three objects.
6. The representation decides what the model sees: engineered features win with little data, learned representations with much; self-supervision manufactures targets from the data; next-token prediction sampled forward is generation.
7. Four ingredients: experience, model, learning algorithm, evaluation criterion; design in the order criterion, experience, output, algorithm.
8. Generalization is the goal; finite data agree with infinitely many rules, so off-sample behaviour comes from the designer's assumptions.
9. Shortcuts: a collection artefact (snow, hour of day) fits the sample and fails in the world; catch it with decoupled or shifted test data; shortcut ≠ leakage.
10. $H(p, q) = H(p) + \mathrm{KL}(p\|q)$ with $\mathrm{KL} \geq 0$ (Gibbs: $\ln t \leq t - 1$, then average): cross-entropy is minimised by the truth, has floor $H(p)$, punishes overconfidence, and trains calibrated probabilities.

Understanding ML, Chapter 2: From Representations to Decisions

After this chapter you can explain what a loss decides, why the Bayes rule is the target of every learner, how kNN imitates it and why it dies in high dimension, what ERM promises and does not promise, why cross-entropy trains calibrated probabilities, and why a number computed on data that chose the model is never a performance estimate

Fundamentals of Machine Learning, midterm

2026-11-03

What this chapter is about, in one paragraph

A learner receives numbers, not objects, so somebody first decides how to describe an object and how to encode the description: that is the *representation*, and it bounds everything that follows. If we knew the whole world, that is the joint distribution of inputs and outputs, the best decision at each input would be a one-line rule: pick the action with the smallest average damage, where "damage" is the *loss* we chose. Different losses pick different summaries of the same uncertainty (mean, median, quantile, most probable class, a cost threshold), so "the best prediction" has no meaning until the loss is named. We never know the world; we have a finite sample. The first idea is to imitate the optimal rule locally, by looking at the nearest examples (kNN), which works until the number of features makes "nearby" meaningless. The second idea is to restrict ourselves to a *family* of rules with adjustable parameters and let the sample pick one member by minimising the average loss on the sample (ERM). That average is honest for a rule chosen in advance and flattering for the rule the sample chose, which is why a model's own training score is never evidence of quality, why the training score can only reward more flexibility, and why we keep validation data to choose settings and test data, touched once, to report performance. Throughout, what we *manipulate* is: the representation, the loss, the neighbourhood size k, the family and its hyperparameters, and which subset of the data is allowed to influence which decision.

Contents

1 From objects to numbers: the representation **2**

2 Deciding when the world is known: the Bayes rule **3**

3 From the known distribution to finite data: nearest neighbours **7**

4 Parameterised models: families of rules **11**

5 Learning the parameters from data: ERM and cross-entropy **13**

6 Evaluating a learned model: train, validation, test **17**

7 The learning problem in one picture **20**

1 From objects to numbers: the representation

Input space, feature vector, one-hot encoding

A learning algorithm can only receive a list of d numbers. Two human choices turn an object into such a list: **describe** it (select which properties to record: area, rooms, age, neighbourhood) and **encode** the description as a vector

$$x = [x_1, \dots, x_d]^\top \in \mathcal{X} \subseteq \mathbb{R}^d \quad \text{[on your sheet]}$$

$\mathcal{X}$ is the **input space**, d the number of **features**, x_j feature j. A numerical property is one component. A categorical property with K possible values becomes K binary components with a single 1 in the position of the value (**one-hot encoding**). A house of 145 m^2, 4 rooms, 12 years, in neighbourhood A of $\{A, B, C\}$ is $x = [145, 4, 12, 1, 0, 0]^\top \in \mathbb{R}^6$.

Purpose. Give the learner the information the decision needs, in a form where "similar inputs" means "similar objects".

Impact. The representation fixes the ceiling: no later stage can recover information that was not encoded, and every distance-based method (kNN, clustering, anything with a scale) inherits whatever geometry the encoding invented.

Three descriptions of the same emails

Setup. One labelled collection of emails (spam or not). Build three representations of each email: (1) its length in characters, $d = 1$; (2) counts of the 10,000 most frequent words, $d = 10{,}000$; (3) the same counts plus 500 columns of random noise, $d = 10{,}500$.

Held fixed. The emails, the labels, the learner (say logistic regression), the split, the metric.

What we manipulate. Only the representation.

What we measure. Validation error of the fitted model for each representation, and how it changes as the training set grows from 500 to 50,000 emails.

What the plot shows. Three curves of error against training size. Curve (1) is flat and high: length cannot separate spam. Curve (2) drops fast. Curve (3) ends at the same level as (2) but needs many more emails to get there.

How to read it. Description (1) lacks information: no amount of data fixes it. Description (3) has the same information as (2) but makes the learner spend data discovering that 500 coordinates are noise. More features are not automatically better; *informative* features are.

Neighbourhoods coded as 1, 2, 3

A real-estate team codes neighbourhoods A, B, C as $1, 2, 3$ to save columns. Their kNN price model then treats a house in A as at distance 1 from a house in B and at distance 2 from one in C: the code invented "B is halfway between A and C". With one-hot coding every pair of neighbourhoods is at the same distance $\sqrt{2}$ (for instance $\|(1,0,0) - (0,1,0)\| = \sqrt{1+1}$), which says "different, and equally so". Same for units: a feature in metres and the same feature in millimetres carry the same information but weigh a thousand times differently in any distance.

Integer codes for categories; forgetting units

People code categories as integers because it is one column instead of K. The learner then reads an order and a spacing that do not exist, so neighbours are chosen for the wrong reason and

the validation error is worse than the data allow. Fix: one-hot encode unless the categories are genuinely ordered (small, medium, large). Second mistake: leaving features in raw units; see the scaling experiment in Section 3.

One-hot versus integer coding

	One-hot	Integer code
columns	K	1
distances between categories	all equal ($\sqrt{2}$)	$1, 2, \ldots$: an invented order
when right	unordered categories	genuinely ordered levels
failure	none for distance methods; many columns	fake geometry, wrong neighbours

Key insight. Object → chosen features → vector: both arrows are human decisions, and "similar" means whatever those decisions make it mean.

How the professor asks it

Q. *A colleague represents a network flow by (duration in ms, protocol coded as TCP= 1, UDP= 2, ICMP= 3). Explain what a nearest-neighbour detector will do with the protocol column and how you would fix it.*
A. The detector will treat UDP as "between" TCP and ICMP and ICMP as twice as far from TCP as UDP is: distances that have no meaning. Mechanism: Euclidean distance subtracts codes, so the arbitrary numbering becomes geometry. Consequence: flows are matched partly on an accident of coding. Fix: one-hot encode the protocol (three binary columns, every pair at distance $\sqrt{2}$), then scale the duration column so that one protocol difference and one typical duration difference count comparably, with the scale chosen on validation data.

Q. *Give an example where adding features makes the learned model worse, and explain the mechanism.*
A. Append 500 columns of random noise to a good word-count representation. The information is unchanged, but a flexible learner can fit the training labels partly through the noise columns, so the fitted rule changes from sample to sample; the validation error rises until the training set is large enough for the learner to discover that those columns carry nothing. The representation decides how much data a given error level costs.

2 Deciding when the world is known: the Bayes rule

Conditional distribution, action, decision loss, conditional risk, Bayes rule

Write X for the input and Y for the output regarded as random; $\mathbb{P}(X, Y)$ is the joint distribution (how likely each pair is in the world). After seeing $X = x$, the uncertainty that *remains* is the conditional distribution $\mathbb{P}(Y \mid X = x)$: two houses with the same description can still sell at different prices because something was not recorded. An **action** $a \in \mathcal{A}$ is what we do (a predicted number, a declared class, "raise the alarm"). The **decision loss** $\ell_{\text{dec}}(y, a)$ is the damage of doing a when the truth is y. The **conditional risk** of a at x is its average damage under the remaining uncertainty, and the **Bayes decision rule** picks the action with the smallest conditional risk:

$$R(a \mid x) = \mathbb{E}[\ell_{\text{dec}}(Y, a) \mid X = x], \qquad f^*(x) \in \arg\min_{a \in \mathcal{A}} R(a \mid x) \quad \text{[on your sheet]}$$

"Bayes" here means "optimal under a known probability distribution"; nothing is assumed about

priors on parameters.

Purpose. Define the target that every learning method tries to approximate, and make explicit that the target depends on the loss.

Impact. Change the loss and the optimal action changes, with the same world and the same probabilities. Change the probabilities (better information) and the actions change too; the rule itself never changes.

Pointwise optimality is global optimality

Claim: for any decision rule f, $\mathbb{E}[\ell_{\text{dec}}(Y, f^*(X))] \le \mathbb{E}[\ell_{\text{dec}}(Y, f(X))]$; the left side is the **Bayes risk**, the floor nobody beats.

1. Tower property: $\mathbb{E}[\ell_{\text{dec}}(Y, f(X))] = \mathbb{E}\big[\mathbb{E}[\ell_{\text{dec}}(Y, f(X)) \mid X]\big]$. Averaging over all pairs equals "average over Y at each fixed x, then average over x".
2. At a fixed x, $f(x)$ is one particular action, so by definition of a minimiser $\mathbb{E}[\ell_{\text{dec}}(Y, f(x)) \mid X = x] \ge \min_a \mathbb{E}[\ell_{\text{dec}}(Y, a) \mid X = x] = \mathbb{E}[\ell_{\text{dec}}(Y, f^*(x)) \mid X = x]$.
3. An inequality that holds at every x survives averaging over x. Done.

Why it matters: we may think about one input at a time. Doing the best thing at every x is the best thing overall.

Regression: which number to report

Squared loss selects the conditional mean

Let $m(x) = \mathbb{E}[Y \mid X = x]$. Claim: $\mathbb{E}[(Y - a)^2 \mid X = x] = \text{Var}(Y \mid X = x) + (m(x) - a)^2$ [on your sheet], so the best a is $m(x)$.

1. Add and subtract the mean: $Y - a = (Y - m(x)) + (m(x) - a)$.
2. Square: $(Y - a)^2 = (Y - m(x))^2 + 2(Y - m(x))(m(x) - a) + (m(x) - a)^2$.
3. Take the conditional expectation. First term: $\text{Var}(Y \mid X = x)$ by definition. Middle term: $(m(x) - a)$ is a constant and $\mathbb{E}[Y - m(x) \mid X = x] = 0$, so it vanishes. Last term: a constant.
4. The variance does not depend on a; $(m(x) - a)^2 \ge 0$ with equality only at $a = m(x)$.

Reading: average squared error = unavoidable spread + your own offset squared. The spread is the problem's; the offset is yours. The move "add and subtract a mean, square, watch the cross term die" is reused in the bias–variance decomposition of Chapter 3.

Absolute loss selects a median; asymmetric loss selects a quantile

A **median** m satisfies $\mathbb{P}(Y \le m \mid x) \ge \frac{1}{2}$ and $\mathbb{P}(Y \ge m \mid x) \ge \frac{1}{2}$. Take $a > m$ and compare the two losses for each possible y (everything conditional on x):

- $y \le m$: $|y - a| - |y - m| = a - m$;
- $y \ge a$: $|y - a| - |y - m| = -(a - m)$;
- $m < y < a$: $|y - a| - |y - m| = a + m - 2y > -(a - m)$.

So $|y - a| - |y - m| \ge (a - m)[\mathbf{1}(y \le m) - \mathbf{1}(y > m)]$. Take expectations: $\mathbb{E}|Y - a| - \mathbb{E}|Y - m| \ge (a - m)[2\mathbb{P}(Y \le m) - 1] \ge 0$ because $\mathbb{P}(Y \le m) \ge \frac{1}{2}$. The case $a < m$ is symmetric. No a beats a median. [not on your sheet: know it](the sheet only states the result)

For the **asymmetric** loss $\ell_\tau(y, a) = \tau(y - a)$ if $y \ge a$ (under-prediction costs τ per unit) and $(1-\tau)(a-y)$ if $y < a$, the same three cases give $\mathbb{E}\ell_\tau(Y, a) - \mathbb{E}\ell_\tau(Y, m) \ge (a-m)[\mathbb{P}(Y \le m) - \tau] \ge 0$

whenever m is a τ-quantile ($\mathbb{P}(Y \le m) \ge \tau$, $\mathbb{P}(Y \ge m) \ge 1-\tau$). With costs c_u per unit under and c_o per unit over, $\tau = c_u/(c_u + c_o)$ [on your sheet]. $\tau = \frac{1}{2}$ is the median.

A delivery promise: three losses, three answers, one distribution

A courier must promise a delivery time for a route whose actual time is 100 minutes with probability 0.5, 200 with probability 0.3 and 600 with probability 0.2 (occasional road closures).

- Squared loss: promise the mean, $0.5(100) + 0.3(200) + 0.2(600) = 230$. Check: risk at 230 is $0.5(130)^2 + 0.3(30)^2 + 0.2(370)^2 = 36{,}100$; at 200 it is 37,000. The mean wins, although 230 never happens.
- Absolute loss: $\mathbb{P}(Y \le 100) = 0.5$ and $\mathbb{P}(Y \le 200) = 0.8$, $\mathbb{P}(Y \ge 200) = 0.5$, so every value in $[100, 200]$ is a median; the expected absolute loss is 130 for all of them ($a = 100$: $0.3(100) + 0.2(500)$; $a = 200$: $0.5(100) + 0.2(400)$) and 148 at the mean 230. The rare 600 drags the mean but barely moves the median.
- Being late costs 9 times being early: $\tau = 0.9$, so promise the 0.9-quantile, 600. The company prefers to disappoint nobody.

Nothing about the world changed; only the consequences did.

Classification: which class to declare

Zero–one loss selects the most probable class; the Bayes error

With $\ell_{\text{dec}}(y, a) = \mathbf{1}(y \ne a)$, the conditional risk of declaring a is $\mathbb{E}[\mathbf{1}(Y \ne a) \mid X = x] = \mathbb{P}(Y \ne a \mid X = x) = 1 - \mathbb{P}(Y = a \mid X = x)$. Minimising $1 - \mathbb{P}(Y = a \mid x)$ is maximising $\mathbb{P}(Y = a \mid x)$:

$$f^*(x) = \arg\max_c \mathbb{P}(Y = c \mid X = x) \quad \text{[on your sheet]}$$

$$\text{conditional risk } 1 - \max_c \mathbb{P}(Y = c \mid X = x)$$

Averaged over x this is the **Bayes error**, a property of the *representation*, not of any learner: with class probabilities $(0.5, 0.3, 0.2)$ at some x, the best rule errs half the time there and nothing can do better unless x is enriched. "Predict the most likely class" is the Bayes rule *only* for zero–one loss.

Cost-sensitive threshold for a binary decision

Class 1 is the event to detect, $p(x) = \mathbb{P}(Y = 1 \mid X = x)$, a false positive costs C_{FP}, a false negative C_{FN}, correct decisions cost 0. Declaring positive is wrong exactly when $Y = 0$, declaring negative exactly when $Y = 1$:

$$R(\text{positive} \mid x) = C_{\text{FP}}\,[1 - p(x)], \qquad R(\text{negative} \mid x) = C_{\text{FN}}\,p(x).$$

Positive is cheaper iff $C_{\text{FP}}(1-p) < C_{\text{FN}}\,p \iff C_{\text{FP}} < (C_{\text{FP}} + C_{\text{FN}})\,p$, i.e.

$$\text{declare positive iff } p(x) > t = \frac{C_{\text{FP}}}{C_{\text{FP}} + C_{\text{FN}}} \quad \text{[on your sheet]}$$

Equal costs give $\frac{1}{2}$. C_{FP} sits on top: expensive misses (large C_{FN}) give a *small* threshold and more alarms. Intrusion detection with $C_{\text{FN}} = 10$, $C_{\text{FP}} = 1$: $t = 1/11 \approx 0.09$; at $p = 0.2$ alarming costs 0.8, silence costs 2: alarm, although an attack is unlikely.

A third action, review, and why one threshold stops being enough

Add "send to an analyst", which resolves the case at a fixed cost C_{R} whatever the truth: $R(\text{review} \mid x) = C_{\mathrm{R}}$, a horizontal line in the plot of conditional risk against p. Each action is optimal where its line is lowest:

$$\begin{aligned} \text{negative iff } C_{\mathrm{FN}}\, p \le C_{\mathrm{R}} &\iff p \le C_{\mathrm{R}}/C_{\mathrm{FN}}, \\ \text{positive iff } C_{\mathrm{FP}}(1-p) \le C_{\mathrm{R}} &\iff p \ge 1 - C_{\mathrm{R}}/C_{\mathrm{FP}}, \\ \text{review iff } C_{\mathrm{R}}/C_{\mathrm{FN}} < p &< 1 - C_{\mathrm{R}}/C_{\mathrm{FP}} \quad \text{[on your sheet]} \end{aligned}$$

The two sloped lines cross once at $p^* = C_{\mathrm{FP}}/(C_{\mathrm{FP}} + C_{\mathrm{FN}})$, where their common height is $C_{\mathrm{FP}}C_{\mathrm{FN}}/(C_{\mathrm{FP}} + C_{\mathrm{FN}})$; the flat review line cuts the lower envelope twice iff C_{R} is below that peak, so a review band exists iff $C_{\mathrm{R}} < C_{\mathrm{FP}}C_{\mathrm{FN}}/(C_{\mathrm{FP}} + C_{\mathrm{FN}})$ [on your sheet]. With $C_{\mathrm{FP}} = 2$, $C_{\mathrm{FN}} = 8$, $C_{\mathrm{R}} = 0.9$ (Tutorial 1, Q2): benign for $p < 0.1125$, attack for $p > 0.55$, review between; the peak is 1.6 at $p^* = 0.2$. Cheaper review widens the band; raising C_{R} to 1.6 closes it.

The threshold-versus-cost experiment

Setup. A validation set of 10,000 flows with labels and, for each flow, a probability $p(x)$ from a calibrated model. Costs $C_{\mathrm{FP}} = 1$ and $C_{\mathrm{FN}} \in \{1, 3, 10, 30, 100\}$.

Held fixed. The model and its probabilities, the validation flows, C_{FP}.

What we manipulate. The threshold t from 0 to 1 in steps of 0.01, and separately the cost ratio.

What we measure. For each t: the number of false positives and false negatives on the validation set and the total cost $C_{\mathrm{FP}} \cdot \mathrm{FP}(t) + C_{\mathrm{FN}} \cdot \mathrm{FN}(t)$.

What the plot shows. Total cost against t, one curve per cost ratio. Each curve is U-shaped; its minimum sits at $t \approx C_{\mathrm{FP}}/(C_{\mathrm{FP}} + C_{\mathrm{FN}})$: $0.5, 0.25, 0.09, 0.03, 0.01$. As C_{FN} grows the minimum slides left and the curve becomes steep on the right (missing attacks is what hurts).

How to read it. The probabilities were learned once; the *policy* is a dial on top of them. Changing costs moves the operating point, never the model. If the minimum is not near the formula's value, the probabilities are not calibrated (Chapter 4).

Triage: the most probable class loses

A patient is healthy, mildly ill or severely ill with probabilities $(0.5, 0.3, 0.2)$. Actions: discharge, monitor, treat. Costs (rows truth, columns action): healthy $(0, 1, 3)$, mild $(4, 0, 2)$, severe $(20, 6, 0)$. Conditional risks are dot products of the probability row with each column: discharge $0.5(0) + 0.3(4) + 0.2(20) = 5.2$; monitor $0.5(1) + 0 + 0.2(6) = 1.7$; treat $0.5(3) + 0.3(2) + 0 = 2.1$. Bayes decision: **monitor**, although "healthy" is the most probable state. The 20 in the corner (discharging a severe case) rules the calculation. Other real cases: card fraud with $C_{\mathrm{FP}} = 5$ (annoyed customer) and $C_{\mathrm{FN}} =$ the amount gives a threshold $5/(5 + A)$ that depends on x; a court sets the threshold far above $\frac{1}{2}$ ("beyond reasonable doubt"); medical screening sets it below.

Inverting the threshold; predicting the mode under every loss

The threshold is $C_{\mathrm{FP}}/(C_{\mathrm{FP}} + C_{\mathrm{FN}})$, not $C_{\mathrm{FN}}/(\cdot)$ and not $C_{\mathrm{FP}}/C_{\mathrm{FN}}$ (that is an odds). Check with the words: expensive misses must give *more* alarms, so a smaller threshold. Second mistake: "predict the most likely value" regardless of loss. Under squared loss the answer is the mean, which may be a value that never occurs; under absolute loss the median; under zero–one the mode. Third: the Bayes rule needs the *true* $\mathbb{P}(Y \mid X = x)$; with an estimate in its place, the decision inherits every error of the estimate, which is why calibration (Chapter 4) matters for thresholds and review bands.

The four losses and what each one selects

Loss	Optimal action	Sensitive to	Typical use
squared $(y-a)^2$	conditional mean	outliers (they pull the mean)	symmetric numeric errors
absolute $\lvert y-a\rvert$	conditional median	little; robust	robust point prediction
asymmetric, c_u under / c_o over	τ-quantile, $\tau = c_u/(c_u+c_o)$	the cost ratio	stock, delivery promises
zero–one $\mathbf{1}(y \neq a)$	most probable class	nothing but the ranking	equal-cost classification
cost table $C_{\mathrm{FP}}, C_{\mathrm{FN}}$	declare positive iff $p > C_{\mathrm{FP}}/(C_{\mathrm{FP}}+C_{\mathrm{FN}})$	the cost ratio	detection, screening

Key insight. Probabilities describe uncertainty, the loss describes consequences, and only together do they determine an action; "best prediction" has no meaning until the loss is specified.

How the professor asks it

Q. *A spam filter estimates $p(x) = 0.3$ for an email. Deleting a legitimate email costs $C_{\mathrm{FP}} = 7$, letting spam through costs $C_{\mathrm{FN}} = 3$. Derive the conditional risks of both actions and decide; then explain in one sentence why the decision would flip if the costs were swapped.*
A. Keep the email. Deleting is wrong with probability 0.7 and costs $7 \times 0.7 = 4.9$; keeping is wrong with probability 0.3 and costs $3 \times 0.3 = 0.9$. The threshold is $7/10 = 0.7$ and $0.3 < 0.7$. With costs swapped the threshold is $3/10 = 0.3$ and the email sits exactly at it: either action is optimal, and any p above 0.3 would be deleted. The probability did not change; the policy did.

Q. *Explain why the Bayes error is a property of the representation and not of the learning algorithm, and what a team should do when the Bayes error is too high for the application.*
A. At a fixed x the best possible rule declares the majority class and still loses the minority share $1 - \max_c \mathbb{P}(Y = c \mid x)$; averaged over x that is the Bayes error. No learner can beat it because it is computed from the true conditional probabilities, which no learner knows better than the world. Its size is set by how much of Y the recorded features explain. If it is too high, the fix is a richer x (more or better features, a different sensor), not a better algorithm or relabelling.

Q. *A delivery time is 20 minutes with probability 0.6 and 80 minutes with probability 0.4. What should be promised under squared loss, under absolute loss, and when a late delivery costs four times an early one? Explain each choice.*
A. Squared loss: the mean $0.6(20) + 0.4(80) = 44$, because squared error splits into variance plus squared offset from the mean. Absolute loss: a median; $\mathbb{P}(Y \leq 20) = 0.6 \geq \frac{1}{2}$ and $\mathbb{P}(Y \geq 20) = 1$, so 20 (any value in $[20, 20]$ only). Late costs 4, early costs 1: $\tau = 4/5 = 0.8$, and we need $\mathbb{P}(Y \leq m) \geq 0.8$, which first holds at $m = 80$: promise 80. Three promises from one distribution; the loss chose.

3 From the known distribution to finite data: nearest neighbours

Training dataset, the i.i.d. assumption, the plug-in idea

We never know $\mathbb{P}(X, Y)$. What we have is a **training dataset** $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{N}$ [on your sheet]: N rows, superscript (i) indexes examples, subscript j indexes features, so $x_j^{(i)}$ is feature j of example i. The standard assumption is that the rows are **independent and identically distributed** draws from $\mathbb{P}(X, Y)$: like N tickets drawn independently from one urn. It fails for consecutive days of a time series (dependent), for houses in one street (dependent), and when the market

changes (a different urn: distribution shift). The task of learning is to build, from the sample, a rule that approximates what we would want under the unknown distribution. The most direct way is the **plug-in idea**: estimate $\mathbb{P}(Y \mid X = x)$ or $\mathbb{E}[Y \mid X = x]$ from the sample and feed the estimate to the Bayes rule of Section 2 unchanged.

Purpose. Separate the two objects that students confuse: the true distribution (unknown, what we want) and the sample (observed, what we have).

Impact. Every guarantee of this chapter (honest estimates, convergence) rests on i.i.d.; grouped or temporal data break it and need a different split (Chapter 3).

k-nearest neighbours

If many examples had exactly the input x we would average their outputs; exact repeats are rare, so use the k examples whose representations are *closest*. With $N_k(x)$ the indices of the k nearest training inputs under a chosen distance,

$$\hat{f}_k(x) = \frac{1}{k} \sum_{i \in N_k(x)} y^{(i)}, \qquad \hat{p}_k(Y = c \mid x) = \frac{1}{k} \sum_{i \in N_k(x)} \mathbf{1}(y^{(i)} = c) \quad \text{[on your sheet]}$$

Regression averages the neighbours' values; classification reports the *fraction* of neighbours in each class, and under zero–one loss declares the majority (under costs, feed $\hat{p}_k$ to the threshold). kNN is **lazy**: nothing is fitted at training time; the data are the model. It has *no parameters* and three **hyperparameters**: k, the distance, and the feature scales. For $k = 1$ the input space is cut into **Voronoi cells**, one per training point, and the boundary between two points is their perpendicular bisector.

Purpose. Imitate the Bayes rule locally with no assumption on the shape of $\mathbb{P}(Y \mid X)$.

Impact. Small k follows individual points (low training error, high sensitivity to noise); large k averages over a wide region (smoother, can erase genuine local variation). Capacity *rises* as k *falls*; $k = N$ gives the global average everywhere.

kNN by hand: the decision flips with k

Six training points with class and value: 1:$(1, 1)$ A 10; 2:$(2, 1)$ A 12; 3:$(1, 3)$ B 20; 4:$(3, 3)$ B 22; 5:$(4, 1)$ A 14; 6:$(3, 2)$ B 18. Query $x = (2, 1.8)$. Squared Euclidean distances (same ordering as Euclidean, no roots): to 1: $1 + 0.64 = 1.64$; 2: 0.64; 3: 2.44; 4: 2.44; 5: 4.64; 6: $1 + 0.04 = 1.04$. Order of closeness $2, 6, 1, \{3, 4\}, 5$. $k = 1$: neighbour $\{2\}$, class A, value 12. $k = 3$: $\{2, 6, 1\}$, classes A, B, A, $\hat{p}(\text{A}) = 2/3$, class A, value $(12 + 18 + 10)/3 = 13.3$. $k = 5$: $\{2, 6, 1, 3, 4\}$, A, B, A, B, B, $\hat{p}(\text{B}) = 3/5$, class **B**, value 16.4. $k = 6$: a 3:3 tie, value 16, the global mean. The answer changed from A to B as k grew: that is what k manipulates.

Vary k and plot training and validation error

Setup. One labelled dataset, standardised features, a fixed random split into training and validation (or K folds).

Held fixed. The data, the split, the distance.

What we manipulate. $k \in \{1, 3, 5, 9, 15, 25, 51, 101, N\}$.

What we measure. For each k, the error of the kNN rule on the training points themselves and on the validation points.

What the plot shows. Two curves against k, with k increasing to the right. Training error starts at exactly 0 for $k = 1$ (every point is its own nearest neighbour) and rises with k. Validation error is high at $k = 1$ (each prediction trusts one noisy neighbour), falls to a minimum at moderate k, then rises again towards the error of "predict the global majority" at $k = N$. Read with the

capacity axis reversed: small k is high capacity.

How to read it. The k with the lowest *validation* error is the one to keep; training error would always pick $k = 1$. The gap between the curves at small k is overfitting; both curves high at large k is underfitting. With more data the minimum moves to larger k (more neighbours are genuinely close).

Distances: Minkowski, Euclidean, Manhattan, Hamming, scaled and weighted

For $x, z \in \mathbb{R}^d$ and $p \geq 1$, $d_p(x, z) = \left(\sum_j |x_j - z_j|^p\right)^{1/p}$ [on your sheet]: $p = 1$ Manhattan (sum of coordinate gaps, "along the streets"), $p = 2$ Euclidean (straight line), $p \to \infty$ the largest gap. For binary vectors the **Hamming** distance counts differing positions (10110 versus 11100: 2). The **scaled, weighted** Euclidean distance $d_w(x, z) = \sqrt{\sum_j w_j ((x_j - z_j)/s_j)^2}$ [on your sheet]divides each gap by a scale s_j (units, typical spread) and multiplies by a weight $w_j \geq 0$ ($w_j = 0$ ignores feature j). For $x = (1, 2)$, $z = (3, 5)$: gaps $2, 3$; $d_1 = 5$, $d_2 = \sqrt{13} \approx 3.61$, $d_\infty = 3$.

Purpose. Define "nearest". The distance *is* the inductive bias of kNN.

Impact. Scales and weights change who the neighbours are, hence the prediction, without touching the data; they are hyperparameters chosen on validation.

The scaling experiment: same customers, different neighbour

Setup. Two training customers described by (income in USD, age in years): $P = (30{,}000, 25)$ and $Q = (31{,}000, 60)$. Query: a 25-year-old earning 30,800.

Held fixed. The three people.

What we manipulate. The units: raw, then standardised with typical spreads $s_{\text{income}} = 1000$, $s_{\text{age}} = 10$.

What we measure. The nearest neighbour.

What the plot shows. Raw: distance to P is $\sqrt{800^2 + 0} = 800$, to Q is $\sqrt{200^2 + 35^2} \approx 203$: nearest is Q, a 60-year-old, because a few hundred dollars dwarf 35 years *in these units*. Standardised: $P \to (30.0, 2.5)$, $Q \to (31.0, 6.0)$, query $\to (30.8, 2.5)$; distance to P is 0.8, to Q is $\sqrt{0.04 + 12.25} \approx 3.5$: nearest is P, the customer of the same age.

How to read it. Nothing about the customers changed; the choice of units silently chose the answer. Standardise each feature with $z_j = (x_j - \mu_j)/\sigma_j$, with μ_j, σ_j computed on the *training* rows only and reused for every new input.

The curse of dimensionality, two ways

Grid count. Covering $[0, 1]^d$ with 10 positions per coordinate needs 10^d cells (each coordinate chosen independently among 10): 10^2 cells for $d = 2$, 10^{10} for $d = 10$. A sample that fills the grid in two dimensions is empty in ten.

Probability of a true neighbour. Put n uniform points in the unit ball of $\mathbb{R}^d$, query at the centre, "true neighbour" means within radius $r = 0.1$. A ball of radius r has volume r^d times the unit ball's (scaling each coordinate by r scales d-dimensional volume by r^d), so one point is a neighbour with probability 0.1^d; the count is $M_d \sim \text{Binomial}(n, 0.1^d)$, and independence gives

$$\mathbb{P}(M_d \geq 1) = 1 - (1 - 10^{-d})^n \approx 1 - e^{-n\,10^{-d}} \quad \text{[on your sheet]}$$

With $n = 1000$:

d	1	2	3	4	5	6	10
$\mathbb{P}(\geq 1$ neighbour)	≈ 1	0.99996	0.632	0.095	0.010	0.001	10^{-7}

$d = 3$: $1 - e^{-1}$; $d = 4$: $1 - e^{-0.1}$; expected count $n\,10^{-d}$ is 1 at $d = 3$ and 0.1 at $d = 4$. The sheet's other form, the radius needed for a 95% chance of one neighbour, $r_d = (1-0.05^{1/n})^{1/d} \to 1$ [on your sheet], says the same thing: in high dimension the "neighbourhood" is the whole ball. The calculation flatters kNN: the query sits at the centre (most volume is near the surface, where half the neighbourhood is empty), one neighbour is counted as enough (averaging label noise needs k), and every dimension is assumed relevant.

The curse experiment: dimension against the chance of a neighbour

Setup. Draw $n = 1000$ points uniformly in the unit ball of $\mathbb{R}^d$; query at the centre.

Held fixed. n, the radius $r = 0.1$ that defines "genuinely close".

What we manipulate. $d = 1, 2, \ldots, 10$; repeat each d 1000 times.

What we measure. The fraction of repetitions in which at least one point lies within r of the query; also the distance to the nearest point.

What the plot shows. The fraction against d: flat at 1 for $d \leq 2$, 0.63 at $d = 3$, 0.10 at $d = 4$, essentially 0 from $d = 6$. The nearest-point distance rises with d towards the radius of the whole ball.

How to read it. The "neighbours" kNN averages in high dimension are far away, so the local average is no longer an estimate of anything local. Local estimation cannot exploit many features without extra structure; that is the motivation for a shared parametric model (Section 4). Remedies that stay with kNN: select features, learn a representation, or reweight dimensions.

Intrusion detection with 80 flow statistics

A team describes each flow by 80 statistics and runs 5-NN on 50,000 flows. In 80 dimensions no stored flow is genuinely close to a new one, so the five "neighbours" merely share the dominant large-scale features (duration, bytes) and the detector degrades to a crude rule on those. Standardising helps, selecting the 8 features that matter helps more, and a model that shares parameters across all flows (Chapters 4 and 5) is what gets deployed. Cost also bites: 5-NN stores all N rows ($O(Nd)$ memory) and computes N distances per query ($O(Nd)$ per prediction), slow at line rate.

Choosing k or the scales on the training error, or on the test set

Tempting because the training error is available immediately. But the 1-NN training error is 0 by construction (each point is its own neighbour), so training error always picks $k = 1$ and says nothing about new flows. Tuning the scale w on the query you are being asked about, or on the test set, fits the metric to the evaluation. Produces: an optimistic error and a k that overfits. Fix: choose k, the distance and the scales on held-out validation data (or by cross-validation), then evaluate once on the test set. Second mistake: computing μ_j, σ_j on all rows before splitting; that leaks test statistics into the representation.

kNN versus a parametric model

	kNN	Parametric model f_θ
what is learned	nothing; the data are the model	θ, from all examples jointly
assumption	nearby inputs have similar outputs	the family $\mathcal{H}$ contains a good rule
what we manipulate	k, distance, scales	family, hyperparameters, θ
training cost	zero	an optimisation
prediction cost	$O(Nd)$ per query	usually $O(d)$
fails when	d large relative to N (curse), unscaled features	the family is wrong (bias)
prefer when	low d, plenty of data, irregular structure	high d, structure that generalises across inputs

Key insight. kNN is the Bayes rule estimated locally; "local" is defined by the distance you chose and stops existing when the dimension is large.

How the professor asks it

Q. *Tutorial 1, Q3: a query flow $x = (50\ ms, 10\ packets)$ has neighbours $x_A = (60, 100)$ labelled attack and $x_B = (400, 12)$ labelled benign. Which is nearest in raw units, which after scaling duration by 100 and packets by 20, and what does the disagreement teach?*
A. Raw: $d(x, x_A) = \sqrt{10^2 + 90^2} \approx 90.6$, $d(x, x_B) = \sqrt{350^2 + 2^2} \approx 350.0$: A is nearest, so attack. Scaled: $z = (0.5, 0.5)$, $z_A = (0.6, 5.0)$, $z_B = (4.0, 0.6)$; $d = \sqrt{0.01 + 20.25} \approx 4.50$ to A and $\sqrt{12.25 + 0.01} \approx 3.50$ to B: B is nearest, so benign. The flows did not change; the metric did, because milliseconds produce the large numbers in raw units. Consequence: scales are hyperparameters, chosen on validation flows, never on the query or the test set.

Q. *Exercises 2, Q6: with $n = 1000$ uniform points in the unit ball and "neighbour" meaning within radius 0.1, find the first dimension at which the probability of at least one neighbour drops below one half, and name three ways the calculation still flatters kNN.*
A. $d = 4$: $\mathbb{P} = 1 - (1 - 10^{-4})^{1000} \approx 1 - e^{-0.1} = 0.095$, against 0.632 at $d = 3$. Mechanism: the volume of the 0.1-ball is 10^{-d} of the unit ball, so each point is a neighbour with that probability. It flatters because the query sits at the centre (near the surface half the neighbourhood is outside the ball), one neighbour is counted as enough although averaging noise needs k, and all d dimensions are assumed relevant whereas irrelevant ones spend the radius on noise.

Q. *Why is the training error of 1-NN zero, and why does this not make 1-NN a good classifier?*
A. Each training point is its own nearest neighbour, so it receives its own label: zero error by construction. This measures memorisation, not the population risk; a 1-NN rule copies every mislabelled or noisy point into its Voronoi cell. If k were chosen by training error it would always be 1, which is exactly why k must be chosen on data the rule did not see.

4 Parameterised models: families of rules

Parameterised model, hypothesis space, nested families

A **parameterised model** is a function $f_\theta(x)$ whose behaviour is set by adjustable numbers $\theta \in \Theta$, the **parameters**. The set of all functions it can produce,

$$\mathcal{H} = \{f_\theta : \theta \in \Theta\} \quad \text{[on your sheet]}$$

is the **hypothesis space**. The model *form* fixes how parameters are shared across inputs; θ picks one member. Linear regression: $f_\theta(x) = w^\top x + b$, $\theta = (w, b)$; with $w = (2, -1)$, $b = 3$

and $x = (1, 4)$, $f_\theta(x) = 2 - 4 + 3 = 1$. Every member is a flat surface: no setting of the dials produces a curve, and that restriction is the **inductive bias** of linear regression. Polynomials of degree $\leq p$ in one variable: changing coefficients selects another polynomial of the same degree; changing p changes the *family*. Families are **nested** when $\mathcal{H}_1 \subseteq \mathcal{H}_2$ (degree $\leq p$ inside degree $\leq p+1$: set $\theta_{p+1} = 0$).

Purpose. Share structure across inputs so that all N examples inform the prediction at every x, which is what kNN could not do in high dimension.

Impact. Choosing $\mathcal{H}$ decides what can be expressed before any data are seen; a wrong family is an error that more data never removes.

Probability simplex, probabilistic classifier, softmax

A classifier for C classes estimates the vector of conditional probabilities $\hat{p}_\theta(x) = [\hat{p}_\theta(Y = 1 \mid x), \ldots, \hat{p}_\theta(Y = C \mid x)]^\top$ in the **probability simplex** $\Delta^{C-1} = \{p \in \mathbb{R}^C : p_c \geq 0,\ \sum_c p_c = 1\}$: all legitimate probability assignments. $(0.5, 0.3, 0.2)$ is in it; $(0.5, 0.5, 0.5)$ and $(1.2, -0.2, 0)$ are not. The usual construction computes C real **scores** $s_{\theta,c}(x)$ and maps them by the **softmax** $p_c = e^{s_c} / \sum_{c'} e^{s_{c'}}$ [on your sheet]: scores $(2, 0, -1)$ become $(7.39, 1, 0.37)/8.76 \approx (0.84, 0.11, 0.04)$. A decision rule of Section 2 then turns the vector into an action.

Purpose. Keep prediction (a probability vector) separate from decision (an action), so one model serves every cost structure.

Impact. The model outputs are what thresholds, review bands and prior corrections (Chapter 4) act on; if they are not calibrated, every policy built on them is off.

Parameters versus hyperparameters

Parameters are learned from the training examples by the learning algorithm. **Hyperparameters** are choices made around that estimation: some define the family (polynomial degree, number of layers), some control the procedure (number of neighbours k, step sizes, feature scales):

$$\text{model form} + \text{hyperparameters} \to \mathcal{H} \to \theta \to \text{one particular model.}$$

Model	Parameters (learned)	Hyperparameters (chosen)
kNN	none	k, distance, feature scales
linear regression	w, b	which features to include
polynomial regression	coefficients $\theta_0, \ldots, \theta_p$	degree p
neural network	all weights	layers, width, learning rate

Purpose. Separate what the data decide (inside a fixed configuration) from what we decide (which configuration), because the two need different evidence (Section 6).

Impact. The same quantity can be either: the degree is a hyperparameter if we fix it and fit coefficients, a parameter if a procedure searches over it with its own criterion. What makes it one or the other is the *procedure*, not the symbol.

Letting the training data choose a hyperparameter

kNN with k picked by training error returns $k = 1$ every time; a polynomial degree picked by training error returns the largest degree offered (Section 5). The mechanism is the same: the training score can only reward flexibility. Produces: a model that memorises. Fix: hyperparameters are chosen on validation data, parameters on training data.

Key insight. A hypothesis space is a bet placed before seeing data; parameters are what the data adjust inside that bet, hyperparameters are the bet itself.

How the professor asks it

Q. *Exercises 2, Q8: is the degree of a polynomial regression a parameter or a hyperparameter? Answer with the criterion that decides, and give a procedure under which the answer changes.*
A. It depends on the procedure. If we fix the degree and fit the coefficients by least squares on the training set, the degree is a hyperparameter: chosen by us, outside the fitting criterion, on validation data. If instead a procedure searches jointly over degree and coefficients with a single criterion that penalises degree (a complexity-penalised risk), the degree is being estimated from data by that criterion and is a parameter of that procedure. The dividing line is who chooses it and on which data, not what it is called.

Q. *Explain why a classifier that outputs scores needs a map to the simplex before any decision, and what goes wrong if scores are compared with a cost threshold directly.*
A. A cost threshold $C_{\mathrm{FP}}/(C_{\mathrm{FP}}+C_{\mathrm{FN}})$ is a statement about probabilities: it compares $C_{\mathrm{FP}}(1-p)$ with $C_{\mathrm{FN}}p$. Raw scores have no scale (adding a constant or doubling them changes nothing about the ranking), so comparing them with 0.2 is meaningless. Softmax (or the sigmoid) turns scores into nonnegative numbers summing to one; only then do the conditional risks of Section 2 make sense. In the binary case the equivalent operation on the score is to compare s with $\log \frac{t}{1-t}$ (Chapter 4).

5 Learning the parameters from data: ERM and cross-entropy

Two losses, two jobs; population risk, empirical risk, ERM

Two losses do two jobs: ℓ_{train} guides the *fitting* of the model to data, ℓ_{dec} guides the *action* taken from the fitted model's output. They may coincide (squared loss to fit and to judge a regression) or differ (cross-entropy to fit probabilities, zero–one or costs to decide). For parameters θ:

$$R(\theta) = \mathbb{E}_{(X,Y)\sim\mathbb{P}}\big[\ell_{\mathrm{train}}(Y, f_\theta(X))\big] \text{ (population risk)},$$

$$\widehat{R}_{\mathrm{train}}(\theta) = \frac{1}{N}\sum_{i=1}^{N} \ell_{\mathrm{train}}(y^{(i)}, f_\theta(x^{(i)})) \text{ (empirical risk)} \quad \text{[on your sheet]}$$

R is the average loss over every input the world will ever produce: what we want, unknowable. $\widehat{R}$ is the average over our N rows: computable (three examples with squared errors $1, 4, 0$ give $5/3$). **Empirical risk minimisation** picks $\hat{\theta} \in \arg\min_\theta \widehat{R}_{\mathrm{train}}(\theta)$ [on your sheet]; the hat says "computed from this sample", and another sample gives another $\hat{\theta}$. The best population parameters $\theta^*_{\mathcal{H}} \in \arg\min_\theta R(\theta)$ may still have risk above the Bayes risk if $\mathcal{H}$ does not contain the Bayes rule.

Purpose. Replace an average over the unknown world by an average over the sample, the only thing we can compute.

Impact. Everything that can go wrong in learning lives in the gap between R and $\widehat{R}$: the next two results say when the gap is harmless and when it is not.

Honest for a fixed model, optimistic after minimisation

Fixed model. Fix θ without looking at the data. Each term $\ell_{\mathrm{train}}(y^{(i)}, f_\theta(x^{(i)}))$ is an independent copy of the random variable $\ell_{\mathrm{train}}(Y, f_\theta(X))$ whose mean is $R(\theta)$; the expectation of an average of such terms is $R(\theta)$. So $\mathbb{E}[\widehat{R}_{\mathrm{train}}(\theta)] = R(\theta)$ and, by the law of large numbers, $\widehat{R}_{\mathrm{train}}(\theta) \to R(\theta)$ as $N \to \infty$. [not on your sheet: know it]
Fitted model. Let θ^* minimise R and $\hat{\theta}$ minimise $\widehat{R}_{\mathrm{train}}$. Then

$$\mathbb{E}[\widehat{R}_{\mathrm{train}}(\hat{\theta})] \le \min_\theta R(\theta) \le \mathbb{E}[R(\hat{\theta})] \quad \text{[on your sheet]}$$

1. Because $\hat{\theta}$ minimises $\widehat{R}_{\text{train}}$, for every sample $\widehat{R}_{\text{train}}(\hat{\theta}) \le \widehat{R}_{\text{train}}(\theta^*)$.
2. Take expectations; θ^* is fixed, so the right side has expectation $R(\theta^*) = \min_\theta R$ by the first part.
3. $R(\hat{\theta}) \ge \min_\theta R$ holds for every sample, hence in expectation.

Reading: the fitted model was chosen *because* it looks good on these rows; part of its score is real, part is luck that will not recur. The training score is at least as flattering as the truth, on average.

The honest-versus-optimistic experiment

Setup. A simulated world where you can draw as many samples as you like (say $Y = 2x + \varepsilon$). Pick a family (lines) and one fixed line θ_0 chosen *before* seeing any sample.

Held fixed. The world, the family, θ_0, the sample size $N = 20$.

What we manipulate. Repeat 1000 times: draw a fresh training sample; compute (a) the training risk of the fixed line θ_0, (b) the training risk of the least-squares line $\hat{\theta}$ fitted on that sample, (c) the true risk $R(\hat{\theta})$ of that fitted line (computable because the world is simulated, or from a huge fresh sample).

What we measure. The averages of (a), (b), (c) over the 1000 repetitions, and the histograms.

What the plot shows. Three histograms. (a) is centred on $R(\theta_0)$: honest. (b) is centred *below* $\min_\theta R$: optimistic. (c) is centred *above* $\min_\theta R$: the fitted line is a little worse than the best line, because it chased the sample's noise. The ordering (b) < best < (c) is the proposition, seen as data.

How to read it. The same number, "average loss on the training rows", is an honest estimate for a rule chosen in advance and a flattering one for the rule the rows chose. Shrinking N widens the gap; growing N closes it.

Bigger families fit at least as well; enough flexibility fits anything

Nested families. If $\mathcal{H}_1 \subseteq \mathcal{H}_2$ then $\min_{f \in \mathcal{H}_2} \widehat{R}_{\text{train}}(f) \le \min_{f \in \mathcal{H}_1} \widehat{R}_{\text{train}}(f)$ [on your sheet]: the minimum over a larger set is at most the minimum over a subset, because every candidate of $\mathcal{H}_1$ is still available in $\mathcal{H}_2$. Raising the polynomial degree never raises the best training risk.

Interpolation. Given N points with distinct inputs $x^{(1)}, \dots, x^{(N)} \in \mathbb{R}$, the Lagrange polynomial $L_i(x) = \prod_{j \ne i} \frac{x - x^{(j)}}{x^{(i)} - x^{(j)}}$ has degree $N - 1$, equals 1 at $x^{(i)}$ and 0 at every other $x^{(j)}$; so $f(x) = \sum_i y^{(i)} L_i(x)$ passes through all N points and has training risk 0 under any loss with $\ell(y, y) = 0$. [not on your sheet: know it]

Together: making the family more flexible can only lower the training risk, all the way to zero, even after performance on new inputs has started to deteriorate. Training risk is therefore a poor guide to *complexity*; parameters are estimated on training data inside a fixed configuration, configurations are compared on validation data.

Least squares by hand, and the nested families in action

Fit $f(x) = wx + b$ to $(0,1), (1,3), (2,2), (3,4)$ under squared loss. Setting $\partial \widehat{R}/\partial b = 0$ and $\partial \widehat{R}/\partial w = 0$ gives the normal equations $\sum y = w \sum x + Nb$ and $\sum xy = w \sum x^2 + b \sum x$, hence

$$w = \frac{N \sum xy - \sum x \sum y}{N \sum x^2 - (\sum x)^2}, \qquad b = \frac{\sum y - w \sum x}{N}. \quad \text{[not on your sheet: know it]}$$

Here $N = 4$, $\sum x = 6$, $\sum y = 10$, $\sum x^2 = 14$, $\sum xy = 19$: $w = (76 - 60)/(56 - 36) = 0.8$, $b = (10 - 4.8)/4 = 1.3$. Predictions $1.3, 2.1, 2.9, 3.7$; residuals $-0.3, 0.9, -0.9, 0.3$ (they sum to

zero, as $\partial\widehat{R}/\partial b = 0$ demands); training risk $(0.09 + 0.81 + 0.81 + 0.09)/4 = 0.45$. Degree 0: the mean 2.5, risk 1.25. Degree 3: a cubic through all four points, risk 0. The sequence $1.25, 0.45, 0$ is non-increasing, exactly as the nested-family result says, and it says nothing about which degree predicts a fifth point; Chapter 3 adds the point $(4, 5)$ and the cubic predicts 15.

Learning probabilities: why cross-entropy and not zero–one

Why the zero–one training risk cannot be optimised

The natural decision loss for classification is zero–one with decision $\arg\max_c \hat{p}_{\theta,c}(x)$. As a *training* loss, $\widehat{R}_{01}(\theta) = \frac{1}{N}\sum_i \mathbf{1}(y^{(i)} \neq \arg\max_c s_{\theta,c}(x^{(i)}))$ [on your sheet]is unusable: it ignores changes in confidence, it is piecewise constant in θ, and its gradient is 0 wherever it exists and undefined where a decision flips. One feature, threshold classifier $f_\theta(x) = \mathbf{1}(x > \theta)$, data $x = 1, 2, 3, 4$ with labels $0, 0, 1, 1$: $\widehat{R}_{01}$ is $\frac{2}{4}$ for $\theta < 1$, $\frac{1}{4}$ on $[1, 2)$, 0 on $[2, 3)$, $\frac{1}{4}$ on $[3, 4)$, $\frac{2}{4}$ beyond: a staircase. Between steps the slope is 0, so "move θ to decrease the loss" gets no information.

Purpose. Explain what a training loss must have: it must vary smoothly with the model's output and reward better probabilities, not just a correct argmax.

Cross-entropy training loss

With $\hat{p}_{\theta,c}(x)$ the model's probability for class c, the **cross-entropy** of an example is minus the log of the probability given to the class that actually occurred, and the empirical cross-entropy is its average:

$$\ell_{\mathrm{CE}}(y^{(i)}, \hat{p}_\theta(x^{(i)})) = -\log \hat{p}_{\theta,y^{(i)}}(x^{(i)}), \qquad \widehat{R}_{\mathrm{CE}}(\theta) = -\frac{1}{N}\sum_{i=1}^{N} \log \hat{p}_{\theta,y^{(i)}}(x^{(i)}) \quad \text{[on your sheet]}$$

Binary form with $\hat{p} = \hat{p}_\theta(Y = 1 \mid x)$ and $y \in \{0, 1\}$: $-y \log \hat{p} - (1 - y)\log(1 - \hat{p})$ [on your sheet]; only the term of the class that occurred survives. Natural logs: probability 0.9 on the true class costs 0.105; 0.6 costs 0.511; 0.5 costs 0.693; 0.1 costs 2.303; 0.01 costs 4.605. Three examples at $0.9, 0.6, 0.1$ give $\widehat{R}_{\mathrm{CE}} = (0.105 + 0.511 + 2.303)/3 \approx 0.973$: the one confident mistake dominates. Where it comes from: the one-hot vector of the observed label has cross-entropy $-\sum_c t_c \log \hat{p}_c = -\log \hat{p}_y$ with the model's vector.

Purpose. A smooth, graded training loss whose minimiser is the true conditional probability (next box), so the output can feed any decision rule afterwards.

Impact. Confident mistakes are punished hardest; hedging costs a little everywhere. Training with cross-entropy and deciding with costs is the standard division of labour.

Cross-entropy is minimised by the true probabilities, and equals maximum likelihood

Let $p_c(x) = \mathbb{P}(Y = c \mid X = x)$ be the truth. At a fixed x, $Y = c$ with probability $p_c(x)$, so the expected loss of the model's vector is

$$\mathbb{E}[-\log \hat{p}_{\theta,Y}(x) \mid X = x] = -\sum_c p_c(x) \log \hat{p}_{\theta,c}(x)$$
$$= H(p(x)) + \mathrm{KL}\left(p(x) \,\|\, \hat{p}_\theta(x)\right) \;\geq\; H(p(x)) \quad \text{[on your sheet]}$$

Steps: the first equality is the definition of expectation; the second is the identity cross-entropy = entropy + KL (add and subtract $\sum_c p_c \log p_c$); the inequality is Gibbs (KL ≥ 0, equality iff $\hat{p}_\theta(x) = p(x)$), itself from $\ln u \leq u - 1$. Averaging over X: $R_{\mathrm{CE}}(\theta) = H(Y \mid X) + \mathbb{E}_X[\mathrm{KL}(p(X) \| \hat{p}_\theta(X))]$. The first term is the problem's own noise, which no model removes; the second is *your* error, and training pushes only that down. Over all probability models the

minimiser outputs the true conditional probabilities: **calibration**. Inside a restricted family the minimiser is the member closest to the truth in averaged KL, which need not equal it.
Maximum likelihood. The conditional likelihood of the observed labels is $L(\theta) = \prod_i \hat{p}_{\theta,y^{(i)}}(x^{(i)})$ and $\log L(\theta) = \sum_i \log \hat{p}_{\theta,y^{(i)}}(x^{(i)}) = -N\widehat{R}_{\mathrm{CE}}(\theta)$ [on your sheet]; log is increasing and $-N < 0$, so maximising L and minimising $\widehat{R}_{\mathrm{CE}}$ have the same solutions.

Learning objective versus learning algorithm

ERM specifies an **objective**: which θ is preferred. A **learning algorithm** specifies how it is found: a closed form (least squares) or an iterative optimisation producing $\theta_0, \theta_1, \ldots$ (gradient methods, later in the course). The full chain separates learning, inference and decision:

$$\mathcal{D} \xrightarrow{\ell_{\text{train}}} f_{\hat{\theta}} \xrightarrow{\text{new } x} \text{inferred quantity } \hat{p}_{\hat{\theta}}(x) \xrightarrow{\ell_{\text{dec}}} \text{action.}$$

Purpose. Keep three questions apart: what is a good model (objective), how to find it (algorithm), what to do with its output (decision).

Impact. A bad algorithm can fail to reach the objective's minimiser (optimisation error, Chapter 3); a bad objective reaches the wrong target perfectly.

Reporting training risk; comparing models by accuracy when probabilities are what you use

Reporting the fitted model's training risk as its performance is the most common error in student reports: that number is optimistic by the proposition above. Fix: report a held-out number (Section 6). Second mistake: choosing between probabilistic models by accuracy and then deploying a threshold; accuracy is blind to calibration, cross-entropy is not. Third, a hand-computation slip: taking $-\log$ of the *largest* probability instead of the probability of the *true* class.

Decision loss versus training loss; zero–one versus cross-entropy

	Zero–one (a decision loss)	Cross-entropy (a training loss)
measures	whether the argmax was right	how much probability the true class got
in θ	piecewise constant, zero gradient	smooth, differentiable where $\hat{p} > 0$
minimiser in population	any rule with the Bayes argmax	the true conditional probabilities
sees confidence	no	yes: 0.9 beats 0.6, 0.1 is punished hard
can rank two models differently	yes: equal accuracy, different calibration	
role	judge the action; set with costs	fit the model; equals maximum likelihood

Key insight. ERM minimises what we can compute; its score is honest for a rule chosen in advance and flattering for the rule it chose, and cross-entropy is the training loss whose minimiser is the truth.

How the professor asks it

Q. *Exercises 2, Q4: two classifiers have the same accuracy on a validation set, but one has a much lower cross-entropy. Explain how this is possible, which one you would deploy with a cost-sensitive threshold, and why.*
A. Accuracy only counts whether the argmax was right; cross-entropy measures the probability placed on the true class. A model that says 0.51 on every correct case and 0.49 on every wrong

one can have the same accuracy as one that says 0.95 and 0.05, and far higher cross-entropy. Deploy the lower cross-entropy model: its expected cross-entropy is $H(Y \mid X)$ plus the KL from its probabilities to the truth, so lower means closer to the true posterior, which is what a threshold $C_{\text{FP}}/(C_{\text{FP}} + C_{\text{FN}})$, a review band or a prior correction consume. Accuracy would not have seen the difference.

Q. *Derive the normal equations for a line fitted by least squares, and explain what the condition* $\partial \widehat{R}/\partial b = 0$ *says about the residuals.*
A. $\widehat{R}(w,b) = \frac{1}{N}\sum_i (y^{(i)} - wx^{(i)} - b)^2$. $\partial \widehat{R}/\partial b = -\frac{2}{N}\sum_i (y^{(i)} - wx^{(i)} - b) = 0$ gives $\sum y = w\sum x + Nb$; $\partial \widehat{R}/\partial w = -\frac{2}{N}\sum_i x^{(i)}(y^{(i)} - wx^{(i)} - b) = 0$ gives $\sum xy = w\sum x^2 + b\sum x$. The first equation says the residuals $y^{(i)} - \hat{y}^{(i)}$ sum to zero: the fitted line passes through the centroid $(\bar{x}, \bar{y})$. On $(0,1),(1,3),(2,2),(3,4)$ this gives $w = 0.8$, $b = 1.3$, residuals $-0.3, 0.9, -0.9, 0.3$, which do sum to zero.

Q. *Why can the training risk never increase when a polynomial's degree is raised, and why does that make it useless for choosing the degree?*
A. Degree-p polynomials are a subset of degree-$(p+1)$ polynomials (set the new coefficient to 0), so the minimum over the larger family is at most the minimum over the smaller. Pushed far enough, degree $N-1$ passes through all N points and the risk is 0. The training curve is therefore monotone in the degree and reaches its minimum at the most flexible family regardless of how that family predicts new points; a criterion that always answers "more" decides nothing. The degree must be chosen on validation data, where the curve is U-shaped.

Q. *A model assigns probability* 0.8 *to class* 1 *and the true label is* 0. *Compute the binary cross-entropy and explain why it is larger than the loss of the same model on an example with true label* 1.
A. $-\log(1-0.8) = -\ln 0.2 = 1.609$ nats, against $-\ln 0.8 = 0.223$ when the label is 1. The loss is minus the log of the probability given to what happened; a confident probability on the wrong class leaves 0.2 for the truth, and $-\ln$ grows without bound as that probability shrinks. Confident mistakes are the expensive ones, which is what makes the loss push probabilities towards calibration.

6 Evaluating a learned model: train, validation, test

Training, validation and test sets

The data are partitioned into three subsets by *role*, not by size:

Subset	Role	What it is allowed to influence
training $\mathcal{D}_{\text{train}}$	learn the parameters θ through ℓ_{train}	θ only
validation $\mathcal{D}_{\text{val}}$	compare fitted configurations, select hyperparameters, the threshold	λ, k, degree, features, threshold
test $\mathcal{D}_{\text{test}}$	estimate the final performance, once, after every choice is fixed	nothing; it is consulted, never obeyed

A subset is informative about new data only until it is used to make a choice; then it has become "training" in disguise. Consulting the test set repeatedly turns it into a second validation set and makes its estimate optimistic by the mechanism of Section 5.

Purpose. Give each of the three decisions (parameters, hyperparameters, the reported number) evidence that was not already used to make it.

Impact. The reported test number is honest exactly to the extent that nothing was selected on it.

Held-out risk of a fixed model is honest; the best of several is not

If the model f was chosen without using the M held-out rows, drawn i.i.d., then $\frac{1}{M}\sum_i \ell(y^{(i)}, f(x^{(i)}))$ has expectation $R(f)$: identical to the fixed-model argument of Section 5, with the held-out rows in place of the training rows. [not on your sheet: know it]The catch is the word *fixed*. Select, among candidates $h \in \mathcal{C}$, the one with the lowest validation risk: for every h_0, $\min_h \widehat{R}_{\text{val}}(\hat{f}_h) \le \widehat{R}_{\text{val}}(\hat{f}_{h_0})$ pointwise; take expectations and then the minimum over h_0:

$$\mathbb{E}\big[\min_h \widehat{R}_{\text{val}}(\hat{f}_h)\big] \le \min_h R(\hat{f}_h) \quad \text{[on your sheet]}$$

with strict inequality as soon as the estimates are noisy (the winner's curse). The minimum of several noisy numbers is pulled below the truth. The test set restores honesty because nothing was selected on it.

Hyperparameter selection by validation

For each candidate λ learn the parameters on the training set, then compare the fitted models on the validation set and keep the best:

$$\hat{\theta}_\lambda = \arg\min_\theta \widehat{R}_{\text{train}}(\theta;\lambda), \qquad \hat{\lambda} = \arg\min_\lambda \widehat{R}_{\text{val}}(\hat{\theta}_\lambda;\lambda) \quad \text{[on your sheet]}$$

Two nested "pick the best" operations on two different datasets. Suppose polynomial degrees give training risks $1.25, 0.45, 0.40, 0.10, 0.00$ for degrees $0, 1, 2, 3, 5$ and validation risks $1.30, 0.50, 0.62, 1.90, 8.40$: training keeps falling (nested families), validation is smallest at degree 1; keep the line. The validation criterion may differ from ℓ_{train} when the application values something else (cost-weighted error, recall at a false-alarm budget).

Purpose. Choose complexity on evidence the fit did not see.

Impact. The selected validation score is itself optimistic (winner's curse); it is a selection device, not the number to report.

K-fold cross-validation

A single validation split leaves fewer rows for fitting and makes the choice depend on one accidental split. Divide the development data (training plus validation) into K folds; for each candidate λ and each fold j, train on the other $K-1$ folds and score on fold j; average:

$$\widehat{R}_{\text{CV}}(\lambda) = \frac{1}{K}\sum_{j=1}^{K} \widehat{R}^{(j)}_{\text{val}}(\lambda), \qquad \hat{\lambda} \in \arg\min_\lambda \widehat{R}_{\text{CV}}(\lambda) \quad \text{[on your sheet]}$$

Every development row validates exactly once and trains $K-1$ times. After selecting $\hat{\lambda}$, refit on all development data; the test set stays untouched. $K = N$ is leave-one-out. Cost: about K fits per candidate.

Purpose. Use limited data efficiently and average out the luck of one split.

Impact. Reduces the variance of the selection; does not remove the winner's curse, does not produce a final model by itself, and does not replace the test set.

The K-fold protocol, step by step

Setup. N rows of development data, a candidate set $\mathcal{C}$ (say $k \in \{1, 3, 5, 9, 15\}$ for kNN), $K = 5$.

Held fixed. The fold assignment, drawn once, randomly for exchangeable rows, by group or by time otherwise (see the Experiments document).

What we manipulate. The candidate λ and the held-out fold j.

What we measure. For each (λ, j): fit every data-dependent step (scaler, imputer, feature selection, the model) on the four training folds only; apply them to fold j; record the validation loss $\widehat{R}^{(j)}_{\text{val}}(\lambda)$. That is $|\mathcal{C}| \times K = 25$ fits.

What the plot shows. A table of K scores per candidate and their mean; a plot of $\widehat{R}_{\text{CV}}$ against λ with the spread across folds as error bars. With $K = 3$ and fold errors $0.40, 0.20, 0.60$ for $k = 1$ (mean 0.40) and $0.30, 0.30, 0.30$ for $k = 3$ (mean 0.30): choose $k = 3$. Fold 2 alone would have chosen $k = 1$.

How to read it. Pick the λ with the lowest mean, refit it on all N rows, evaluate once on the test set, report that. The error bars tell you whether two candidates are really different; a 10-row fold cannot separate 12% from 18%.

Fraud detection, the whole pipeline

Underlying problem: transactions X, fraud indicator Y; decision loss $C_{\text{FP}} = 5$ per blocked legitimate payment, $C_{\text{FN}} =$ the amount. Bayes rule: block iff $p(x) > 5/(5 + \text{amount})$; unknown, since p is. Data: two years of labelled transactions, split by *time* into development (first 20 months) and test (last 4). Family: a probabilistic classifier with hyperparameters (representation, model size). Training: minimise cross-entropy on $\mathcal{D}_{\text{train}}$ for each configuration. Validation: choose the configuration *and* the threshold by cost-weighted error on $\mathcal{D}_{\text{val}}$ (not by cross-entropy: the application cares about money). Test: apply the chosen policy to the last four months once and report the total cost. Deployment: the threshold stays adjustable because the policy is separate from the probabilities.

The five ways a reported number becomes a lie

(1) Reporting the training risk of the fitted model. (2) Reporting the validation minimum after selecting on it: the best of 500 noisy numbers is strongly optimistic. (3) Consulting the test set during development, which makes it a validation set. (4) Fitting the scaler, imputer, vocabulary or feature selection on all rows before splitting: test statistics leak into the representation. (5) Splitting rows at random when rows are grouped (near-duplicate flows of one attack, several images of one patient) or time-ordered: the test set then contains copies of the training set. Fix: decide the split before any result, by group or time when needed; fit every data-dependent step inside the training portion; select configuration and threshold on validation; touch the test set once; if you go back and change something, you need a new test set.

Training versus validation versus test

	Training	Validation	Test
decides	θ	λ, k, features, threshold	nothing
its risk is	optimistic for $\hat{\theta}$	honest per candidate, optimistic for the winner	honest, if used once
reused?	every fit	every comparison	never
replaced by	nothing	K-fold CV when data are scarce	a fresh set if you go back

Key insight. A number is a performance estimate only if the thing it measures was fixed before the number was computed.

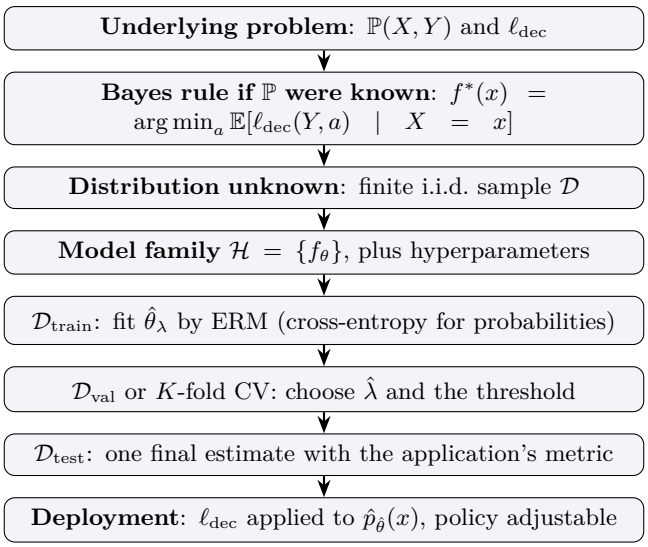
How the professor asks it

Q. *Exercises 2, Q10: a report standardises all rows before splitting, splits rows at random although each attack produced near-duplicate flows, tries* 500 *configurations and keeps the one with the best* test *accuracy, and thresholds at* 0.5 *although* $C_{\mathrm{FN}} = 100\, C_{\mathrm{FP}}$. *Give every reason the reported accuracy misleads and write the correct protocol.*
A. (1) Preprocessing leakage: test statistics shaped the inputs the model was trained on. (2) Duplicate leakage: the test set contains copies of training flows, so the score measures recognition of seen attacks. (3) Selection on the test set: the best of 500 noisy numbers is a validation score with a large winner's curse. (4) Accuracy at low prevalence hides missed attacks (the all-benign rule scores 0.99). (5) The Bayes threshold is $1/101 \approx 0.01$; at 0.5 every attack with posterior in $(0.01, 0.5)$ is missed. Protocol: deduplicate; split by episode and by time; fit scaling and selection inside each training fold; choose configuration and threshold on validation; evaluate once on the test set and report recall, precision and cost at the operating threshold.

Q. *Explain, using the optimism result, why the validation error of the chosen hyperparameter is not an estimate of the model's performance, and what is.*
A. The chosen $\hat{\lambda}$ minimises $\widehat{R}_{\mathrm{val}}$ over the candidates, so its validation score is the minimum of several noisy unbiased estimates, and $\mathbb{E}[\min_h \widehat{R}_{\mathrm{val}}(\hat{f}_h)] \leq \min_h R(\hat{f}_h)$: the minimum is pulled below the truth, more so with more candidates and noisier estimates. The validation set played the role of the training set and the candidates the role of Θ. The honest number is the test risk of the final model, computed once, after $\hat{\lambda}$ and the threshold are fixed.

Q. *Show with three folds that a single validation split can select the wrong k for kNN, and say what cross-validation does and does not fix.*
A. Fold errors $0.40, 0.20, 0.60$ for $k = 1$ (mean 0.40) and $0.30, 0.30, 0.30$ for $k = 3$ (mean 0.30). Validating on fold 2 alone picks $k = 1$; the average over folds picks $k = 3$. Cross-validation removes the dependence on one accidental split and uses every row for validation once. It does not remove the winner's curse (the selected 0.30 is still optimistic), it does not produce the final model (refit on all development data with $\hat{k}$), and it does not replace the untouched test set.

7 The learning problem in one picture



Experience is the finite sample; the hypothesis space says which rules may be considered; the loss measures the quality of outputs. Training risk guides parameter estimation, validation performance guides complexity, test performance estimates how the completed procedure behaves on new observations. Why a finite sample is evidence about future performance at all is the subject of Chapter 3.

How the professor asks it

Q. *Place the following in the picture and say which subset each is allowed to touch: the cost ratio, the number of neighbours k, the feature scales, the decision threshold, the reported recall.*
A. Cost ratio: part of the underlying problem (ℓ_{dec}), fixed by the application, touches no data. k and the feature scales: hyperparameters, chosen on validation or by cross-validation, never on training error (always $k = 1$) nor on the test set. Threshold: a policy on top of the probabilities, derived from the costs if the probabilities are calibrated, otherwise tuned on validation. Reported recall: computed once on the test set after everything above is frozen.

The ten mistakes of this chapter

1. Coding categories as integers, or leaving features in raw units, so that distance measures the coding instead of the object.
2. Predicting the most likely value under every loss; the mean, median, quantile and mode are answers to different losses.
3. Inverting the cost threshold: it is $C_{\text{FP}}/(C_{\text{FP}}+C_{\text{FN}})$; expensive misses give a *small* threshold and more alarms.
4. Treating the Bayes error as a learner's failure; it is the representation's, and the fix is a richer x.
5. Choosing k or the scales on training error (always $k = 1$) or on the query or test set.
6. Confusing the true distribution with the sample, and assuming i.i.d. for grouped or time-ordered rows.
7. Reporting the training risk of the fitted model; it is optimistic by construction.
8. Choosing complexity by training risk, which can only fall with flexibility, all the way to zero.
9. Comparing probabilistic models by accuracy and then deploying a threshold; accuracy is blind to calibration, cross-entropy is not.
10. Selecting on the test set, fitting the scaler before the split, or reusing the validation minimum as the reported number.

Chapter 2 in ten lines

1. Object $\to$ chosen features $\to$ vector in $\mathbb{R}^d$; both arrows are human choices; one-hot code categories; the representation bounds everything.
2. Uncertainty after seeing x is $\mathbb{P}(Y \mid X = x)$; the loss describes consequences; the Bayes rule minimises the conditional risk, pointwise and therefore globally.
3. Squared loss selects the conditional mean (add and subtract the mean, the cross term dies), absolute loss a median, asymmetric loss the τ-quantile with $\tau = c_u/(c_u + c_o)$.
4. Zero–one loss selects the most probable class and leaves the Bayes error; unequal costs give the threshold $C_{\text{FP}}/(C_{\text{FP}} + C_{\text{FN}})$; a fixed-cost review action adds a band $(C_{\text{R}}/C_{\text{FN}},\ 1 - C_{\text{R}}/C_{\text{FP}})$ iff $C_{\text{R}} < C_{\text{FP}}C_{\text{FN}}/(C_{\text{FP}} + C_{\text{FN}})$.
5. The sample is N i.i.d. rows from the unknown distribution; kNN plugs a local average into the Bayes rule; k, the distance and the scales are hyperparameters; capacity rises as k falls.
6. The curse: a point is within radius 0.1 with probability 10^{-d}, so $\mathbb{P}(\text{some neighbour}) = 1-(1-10^{-d})^n$ collapses at $d = 4$ for $n = 1000$; local methods need shared structure.
7. A parameterised family $\mathcal{H} = \{f_\theta\}$ shares structure across inputs; parameters are learned, hyperparameters chosen; the procedure decides which is which.

8. ERM minimises the empirical risk; honest for a fixed model, optimistic after fitting ($\mathbb{E}[\widehat{R}(\hat{\theta})] \leq \min R \leq \mathbb{E}[R(\hat{\theta})]$); larger families only lower training risk, down to zero at interpolation.
9. Cross-entropy $-\log \hat{p}_{\theta,y}(x)$ is smooth, equals negative log-likelihood, and its expectation is $H(Y \mid X) + \mathbb{E}[\mathrm{KL}]$, minimised by the true probabilities; zero–one is a staircase and cannot be trained on.
10. Training data learn θ, validation data (or K-fold CV) choose λ and the threshold, the test set gives one honest number; the best of several validation scores is optimistic; preprocessing fitted on all rows, duplicates across subsets and selection on the test set all inflate the estimate.

Coverage of the textbook chapter

Textbook item (Chapter 2)	Where in this document
Symbol table; input space, feature vector, one-hot; "why not $1, 2, 3$"; three email descriptions; gotchas on codes and units	Section 1
Conditional distribution; decision loss; Bayes rule and conditional risk; pointwise = global (proof)	Section 2, first two boxes
Squared loss → mean (proof); absolute → median (proof); asymmetric → quantile (proof); delivery example; loss table	Section 2, regression boxes and versus table
Zero–one → argmax, Bayes error; cost threshold (proof); detection, fraud, court, screening examples; triage cost table; technique; classics; gotchas	Section 2, classification boxes, example, warning, drill
Reject / review band (Essentials, Tutorial 1 Q2, on the sheet)	Section 2, review box
Training dataset, i.i.d.; true distribution vs sample; kNN definition; kNN by hand; Voronoi; distances; scaling example; standardise on training rows	Section 3
Grid count; probability of a neighbour (proof, table); computation; kNN technique; classics ($k = 6$, expected count, 1-NN error)	Section 3, math box, example, drill
Parameterised model, hypothesis space, inductive bias; parameters vs form; nested spaces; simplex, softmax; parameters vs hyperparameters table; lazy gotcha	Section 4
Two losses; population and empirical risk, ERM; honest (proof); optimistic (proof); nested (proof); interpolation (proof); least squares by hand; staircase example; cross-entropy definition and table; $H + \mathrm{KL}$ theorem (proof); MLE (proof); objective vs algorithm; technique; classics	Section 5
Three subsets; held-out honest (proof); selection optimism; hyperparameter selection with the degree table; K-fold CV with the fold table; splitting technique; gotchas	Section 6
The picture; fraud pipeline; notes on the slides (median not unique, centre-of-ball caveat, softmax not examinable from this lecture)	Section 7, Section 6 example, Sections 2–3

Understanding ML, Chapter 3: From Fitting to Generalization

Why training performance lies, what capacity, bias and variance actually are, what regularisation, feature selection, early stopping and the split protocols manipulate, and the experiments that reveal each one

Fundamentals of Machine Learning, midterm

2026-11-03

What this chapter is about, in one paragraph

Chapter 2 taught fitting: pick the parameters that make the training examples look good. This chapter is about the gap between looking good on the training examples and being good on the next case. It names the gap (generalisation gap), proves that training performance is optimistic, shows that the cause is capacity (a flexible family can fit the sample's accidents), splits the error at one input into noise, bias and variance, and then goes through every dial that controls effective capacity: penalties (L_2 shrinks, L_1 zeroes), early stopping, feature selection. It ends with the rules that make an evaluation honest: a split that matches deployment, metrics that expose minority failure, no leakage. Every method here is an answer to one question: *which of the regularities in my sample will still hold on the next case?* The separate Experiments document describes the protocols (split, cross-validation, learning curves, tuning) in full; here they are tied to the chapter's own propositions. Formulas marked [on your sheet] are on your two-page sheet.

Contents

1 Generalisation, the gap, and inductive bias **2**

2 Model capacity: underfitting and overfitting **4**

3 Bias and variance **6**

4 Controlling complexity: regularisation and early stopping **8**

5 Feature and model selection, and the overuse of validation **11**

6 Reliable evaluation: splits, metrics, leakage **13**

7 Diagnosis: learning curves, and Lab 1 **16**

1 Generalisation, the gap, and inductive bias

Empirical risk, population risk, generalisation gap

For a model f and a dataset $\mathcal{D}$ of N examples, the empirical risk is $\widehat{R}_{\mathcal{D}}(f) = \frac{1}{N}\sum_i \ell(y^{(i)}, f(x^{(i)}))$ and the population risk is $R(f) = \mathbb{E}[\ell(Y, f(X))]$ [on your sheet]. For the model $\hat{f}_{\mathcal{D}}$ fitted on $\mathcal{D}$, the *generalisation gap* is

$$G(\hat{f}_{\mathcal{D}}) = R(\hat{f}_{\mathcal{D}}) - \widehat{R}_{\mathcal{D}}(\hat{f}_{\mathcal{D}}). \quad \text{[on your sheet]}$$

Both terms refer to the *same* fitted model; only the source of the examples differs. A spam filter with 2% errors on its training emails and 9% on next month's emails has a gap of 0.07.

Purpose. To give a name and a sign to "the fit did not carry over".

Impact. Only $\widehat{R}_{\mathcal{D}}$ is observable. Everything in the chapter is about estimating R, or bounding the gap, without the data that chose the model.

Training risk of a fitted model is optimistic

Let $\hat{f}_{\mathcal{D}}$ minimise $\widehat{R}_{\mathcal{D}}$ over $\mathcal{H}$ and $f^*_{\mathcal{H}}$ minimise R over $\mathcal{H}$. Since $\hat{f}_{\mathcal{D}}$ is the minimiser, $\widehat{R}_{\mathcal{D}}(\hat{f}_{\mathcal{D}}) \le \widehat{R}_{\mathcal{D}}(f^*_{\mathcal{H}})$ for every sample. Take expectations over $\mathcal{D}$: $f^*_{\mathcal{H}}$ is fixed, so $\mathbb{E}[\widehat{R}_{\mathcal{D}}(f^*_{\mathcal{H}})] = R(f^*_{\mathcal{H}})$ (an average of i.i.d. losses is unbiased). And $R(\hat{f}_{\mathcal{D}}) \ge R(f^*_{\mathcal{H}})$ pointwise. Therefore

$$\mathbb{E}_{\mathcal{D}}[\widehat{R}_{\mathcal{D}}(\hat{f}_{\mathcal{D}})] \;\le\; \min_{f\in\mathcal{H}} R(f) \;\le\; \mathbb{E}_{\mathcal{D}}[R(\hat{f}_{\mathcal{D}})], \qquad \text{so} \qquad \mathbb{E}_{\mathcal{D}}[G(\hat{f}_{\mathcal{D}})] \ge 0. \quad \text{[on your sheet]}$$

The mechanism is that the data play two roles, $\mathcal{D} \to \hat{f}_{\mathcal{D}} \to \widehat{R}_{\mathcal{D}}(\hat{f}_{\mathcal{D}})$: they choose the model and then grade it, so the procedure can exploit accidental regularities of $\mathcal{D}$. A second sample $\mathcal{D}'$ would give another $\hat{f}_{\mathcal{D}'}$; that variability is part of learning, not a defect.

Seeing the gap: fit, then test on fresh cases

Setup. Any supervised dataset; a flexible family (a deep tree, a high-degree polynomial).

Held fixed. The family, the training rows.

What we manipulate. Nothing; the point is the two measurements.

What we measure. The risk on the training rows, and the risk on rows never used (held out before fitting, or next month's data).

What the plot shows. Two bars. The held-out bar is higher; the difference is the gap.

How to read it. A gap near zero: training performance transfers. A large gap: the training score was flattering. Low training loss is *necessary* evidence that optimisation worked and *incomplete* evidence that learning worked. That is the entire justification for validation and test sets.

Stable and spurious regularities

A training sample contains regularities of two kinds: those that describe the relationship you want (animal shape predicts "cow"), and those created by how the data were collected (the background predicts "cow" because cows were photographed on grass and camels on sand). Both fit every training example; training performance cannot tell them apart. Dataset composition is therefore *part of the learning method.*

Purpose. To explain how a model can have zero training loss and a large gap: it learned a cue that is perfect in the sample and absent at deployment.

Impact. Spurious cues appear wherever a nuisance variable co-varies with the label in the collection: a network segment or capture time predicting attacks, a scanner type identifying a diagnosis, a customer identifier predicting churn.

The shortcut probe

Setup. A classifier trained on the collection (cows on grass, camels on sand).

Held fixed. The fitted model.

What we manipulate. The nuisance factor alone: present the model with a cow on sand, a camel on grass, or, in intrusion detection, flows from a new network segment or a new week; or shuffle the labels and retrain; or delete the metadata columns and retrain.

What we measure. Accuracy on the probe cases versus on the original test split.

What the plot shows. A table: original split high; probe cases much lower; label-shuffled model at chance; metadata-free model close to the original if the shortcut was not used, far below it if it was.

How to read it. A drop on the probe is a shortcut. Shuffled labels must collapse to chance (if they do not, something leaks the label). Removing metadata must not collapse performance (if it does, the model was reading identifiers, not behaviour).

Weakening shortcuts in practice

Vary nuisance factors at collection time (several cameras, sites, seasons, devices); augment only with label-preserving, realistic transformations (flip a cat photo: fine; flip the digit 6: it becomes a 9); and evaluate under deployment-like conditions, because a random split hides a shortcut that is present in every part of the collection. In the wolves-and-snow story the probe was a wolf on grass; in security it is a new segment or a new month.

Inductive bias

A finite sample is compatible with infinitely many functions that agree on the sample and differ elsewhere. Learning therefore needs *assumptions that make some solutions more plausible than others off the sample*: the inductive bias. It enters through (1) the data and representation (which cues exist), (2) the hypothesis space (which relationships can be expressed), (3) regularisation (which fits are preferred), and (4) the optimisation algorithm (which of many minimisers is reached).

Purpose. To make clear that generalisation is never a property of the data alone; it succeeds when the assumptions remain true at deployment.

Impact. Every technique of this chapter chooses, strengthens or checks one of the four entries. "Bias" here means a preference among solutions; it is broader than the statistical bias of Section 3.

Inductive bias versus statistical bias

	Inductive bias	Statistical bias $\mathrm{Bias}(x)$
What it is	a preference among functions off the sample	average prediction minus truth, over training sets
Where it lives	data, family, regulariser, optimiser	a number at each x
Good or bad	necessary; wrong if it fails at deployment	a cost, traded against variance

Key insight. The sample chooses the model and then grades it; that is why the training score is optimistic. What decides the next case is not the data but the assumptions (inductive bias), and whether they still hold where the model is used.

How the professor asks it

Q. *Prove that the expected generalisation gap of an empirical risk minimiser is nonnegative, and say exactly where the assumption "$\hat{f}_{\mathcal{D}}$ minimises the empirical risk" is used.*
A. Claim: $\mathbb{E}_{\mathcal{D}}[\widehat{R}_{\mathcal{D}}(\hat{f}_{\mathcal{D}})] \leq \mathbb{E}_{\mathcal{D}}[R(\hat{f}_{\mathcal{D}})]$. Let $f^*_{\mathcal{H}}$ be the population minimiser in $\mathcal{H}$. Minimisation is used once: it gives $\widehat{R}_{\mathcal{D}}(\hat{f}_{\mathcal{D}}) \leq \widehat{R}_{\mathcal{D}}(f^*_{\mathcal{H}})$ on every sample. Taking expectations, the right side equals $R(f^*_{\mathcal{H}})$ because $f^*_{\mathcal{H}}$ is fixed and the loss average is unbiased for a fixed function. Finally $R(f^*_{\mathcal{H}}) \leq R(\hat{f}_{\mathcal{D}})$ pointwise because $f^*_{\mathcal{H}}$ is best in the family. Chain the three facts. Consequence: a model's own training score is not evidence about new cases.

Q. *A model reaches 100% on an image collection in which all cows stand on grass and all camels on sand. Explain why this number is uninformative and design the experiment that would reveal the problem.*
A. Two cues, shape and background, predict every training label, and training performance cannot separate them: both give zero loss. Only inputs where the cues disagree can. Experiment: collect or construct cows on sand and camels on grass (hold the animal fixed, manipulate the background), measure accuracy on them; a collapse shows the model used the background. Complement with a label-shuffle control (must fall to chance) and with a grouped or site-wise split (hold out whole locations). The fix is at collection time: vary the background, or augment with realistic label-preserving changes.

2 Model capacity: underfitting and overfitting

Capacity and nested families

The capacity of $\mathcal{H} = \{f_\theta\}$ is how varied its functions can be. It is set by structure (degree, depth, width), constraints, regularisation, the optimiser and the data, not by the parameter count alone. Families are *nested* when $\mathcal{H}_1 \subseteq \mathcal{H}_2 \subseteq \ldots$: every function of a smaller family is available in a larger one. Typical knobs: polynomial degree, tree depth, number of units, and k in kNN (capacity grows as k *shrinks*).

Purpose. To have one word for "how much the family can bend to the sample".

Impact. Training error can only fall with capacity: $\min_{\mathcal{H}_{m+1}} \widehat{R}_{\text{train}} \leq \min_{\mathcal{H}_m} \widehat{R}_{\text{train}}$ [on your sheet], because the larger family can reuse the smaller one's minimiser. Nothing of the sort holds for the population risk.

Underfitting and overfitting

An *underfitting* model is too restricted to capture important structure: high training *and* high validation error. An *overfitting* model is too sensitive to the particular sample: very low training error, substantially higher validation error. The distinction is not whether the model is complicated in isolation but whether its effective flexibility matches the available evidence: a family unstable on 50 points may be well constrained by 50,000.

The validation curve: sweep capacity

Setup. Training rows, validation rows (or folds), a nested family with a knob.

Held fixed. Rows, preprocessing (fitted on training rows), metric.

What we manipulate. Degree $0, 1, 2, \ldots$; or depth; or $k = 1, 3, 5, \ldots$ (plot against $1/k$ so capacity increases to the right).

What we measure. Training and validation risk at each knob value.

What the plot shows. Training risk decreasing monotonically; validation risk falling then rising: the U.

How to read it. Left of the minimum: both high, underfitting. Right: training low and validation rising, overfitting. The minimum is the capacity the data can support; with more data the minimum moves right. For kNN remember the direction: small k means small adaptive neighbourhoods (low training error, high sensitivity), large k averages broadly.

Three polynomials on four points, judged on a fifth

Points $(0,1),(1,3),(2,2),(3,4)$ fitted by degree 0, 1, 3 give training risks 1.25, 0.45, 0. A validation point $(4,5)$ arrives. Degree 0 (constant 2.5) predicts 2.5, error 6.25. Degree 1 ($0.8x+1.3$) predicts 4.5, error 0.25. Degree 3 (the interpolating cubic $1+5.5x-4.5x^2+x^3$, which passes through all four points) predicts $1+22-72+64=15$, error 100. The constant underfits; the cubic has the best training score and the worst validation score, because it bent itself around the noise of four points; the line, worse than the cubic on the training points, is the one to deploy.

Capacity mistakes

- Treating zero training error as an achievement. With enough capacity it is guaranteed (enough flexibility fits anything, Chapter 2); it is a warning to look at validation error.
- Reading the kNN direction backwards: *decreasing* k increases capacity.
- Judging capacity by parameter count: a heavily regularised million-parameter model can have less effective capacity than an unregularised cubic (Section 4).
- Comparing families by training error (always favours the larger family, proof above).

Underfitting versus overfitting

	Training error	Validation error	Fix
Underfitting	high	high, close to training	more capacity, better features, less regularisation
Overfitting	low	much higher than training	regularise, reduce capacity, more data, ensembles
Well matched	low	slightly above training	stop

Key insight. Capacity is what the training error rewards and the validation error punishes beyond a point; the U is the signature of a family that can fit both the signal and the sample's accidents.

How the professor asks it

Q. *Why does training error never increase when you go from polynomial degree 2 to degree 3, and why does this not imply that degree 3 generalises better?*
A. Degree-2 polynomials are degree-3 polynomials with the cubic coefficient zero, so the degree-2 minimiser is a candidate in the degree-3 family and the degree-3 minimum is at most its value: nested families, training error nonincreasing. The population risk has no such monotonicity: the extra coefficient can fit stable structure or accidental variation, and on four points the cubic fits all the noise (training risk 0, validation error 100 in the example). Which happened is decided by validation data, never by the training score.

3 Bias and variance

Bias and variance of a learning procedure

At a fixed input x, with $f^*(x) = \mathbb{E}[Y \mid X = x]$ and $\bar{f}(x) = \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(x)]$ the average prediction across training sets,

$$\text{Bias}(x) = \bar{f}(x) - f^*(x), \qquad \text{Var}(x) = \mathbb{E}_{\mathcal{D}}[(\hat{f}_{\mathcal{D}}(x) - \bar{f}(x))^2]. \quad \text{[on your sheet]}$$

Fix one house; train a thousand times on a thousand training sets; record the thousand predicted prices. Bias: is their average off from the true mean price in a consistent direction? Variance: how widely do they scatter around their own average? A model that always says "1 million" has zero variance and possibly large bias; a very flexible model may have tiny bias and huge variance.

Purpose. To turn "sensitive to the sample" into a number and separate it from "systematically wrong".

Impact. Restrictive families and poor features create bias, which more data does not remove. High capacity, small samples, irrelevant variables and unstable feature selection create variance, which more representative data reduces.

The bias–variance decomposition, with proof

Assume $Y = f^*(X) + \varepsilon$, $\mathbb{E}[\varepsilon \mid X] = 0$, $\text{Var}(\varepsilon \mid X = x) = \sigma^2(x)$, and the training sample independent of the test pair. Then at every x,

$$\mathbb{E}_{\mathcal{D},Y|x}[(Y - \hat{f}_{\mathcal{D}}(x))^2] = \sigma^2(x) + \text{Bias}(x)^2 + \text{Var}(x). \quad \text{[on your sheet]}$$

Step 1 (split off the noise). $Y - \hat{f}_{\mathcal{D}}(x) = \varepsilon + (f^*(x) - \hat{f}_{\mathcal{D}}(x))$. Square: $\varepsilon^2 + 2\varepsilon(f^* - \hat{f}_{\mathcal{D}}) + (f^* - \hat{f}_{\mathcal{D}})^2$. $\mathbb{E}[\varepsilon^2 \mid x] = \sigma^2(x)$. In the cross term ε is independent of $\mathcal{D}$ and has mean zero, so $\mathbb{E}[\varepsilon(f^* - \hat{f}_{\mathcal{D}})] = \mathbb{E}[\varepsilon]\,\mathbb{E}_{\mathcal{D}}[f^* - \hat{f}_{\mathcal{D}}] = 0$.
Step 2 (split around the average prediction). $f^* - \hat{f}_{\mathcal{D}} = (f^* - \bar{f}) + (\bar{f} - \hat{f}_{\mathcal{D}})$. The first bracket is the constant $-\text{Bias}(x)$, whose square is Bias^2; the second has $\mathbb{E}_{\mathcal{D}}$-mean zero by definition of $\bar{f}$, so the cross term $2(f^* - \bar{f})\mathbb{E}_{\mathcal{D}}[\bar{f} - \hat{f}_{\mathcal{D}}]$ vanishes; its square averages to $\text{Var}(x)$.
Twice the same move: add and subtract a mean, square, and the cross term dies because a mean-zero quantity multiplies a constant. The three pieces: irreducible noise (even f^* makes it), systematic error (repeating never averages it away), sample sensitivity (a different sample would change it).

Measuring bias and variance by repeated sampling

Setup. A simulated world with known f^* and noise; a procedure (family plus fitting rule plus N).

Held fixed. f^*, σ, N, a grid of test inputs.

What we manipulate. The training set: draw it 500 times and refit each time. In a second sweep, vary capacity (degree, k) and in a third, vary N.

What we measure. At each test x: the mean and spread of the 500 predictions; Bias^2 and Var averaged over the grid; the measured test error for comparison.

What the plot shows. Against capacity: bias^2 falling, variance rising, their sum plus σ^2 equal to the test error and U-shaped. Against N: variance falling, bias flat.

How to read it. Capacity trades bias for variance (a useful principle, not a law). Data buys variance only. The minimum of the U moves to higher capacity as N grows.

Bias and variance computed exactly

No input: $Y = \mu + \varepsilon$, $\mathrm{Var}(\varepsilon) = \sigma^2$, N draws. *Sample mean*: $\mathbb{E}[\hat{f}] = \mu$, bias 0, variance σ^2/N. *Ignore the data, predict 0*: bias $-\mu$, variance 0. *Shrunk mean* ($\frac{1}{2}$ of the sample mean): bias $-\mu/2$, variance $\sigma^2/(4N)$. With $\sigma^2 = 1$, $N = 4$: bias^2+variance is 0.25, 1, 0.3125 for $\mu = 1$ and 0.25, 0.04, 0.0725 for $\mu = 0.2$. When the truth is far from zero the unbiased procedure wins; when it is near zero both biased procedures beat it by trading a little systematic error for a large variance reduction. Regularisation is exactly this trade, made tunable.

Three sources of excess error, and double descent

Relative to the Bayes risk, a learned model's extra risk is *approximation error* (the family cannot express a good enough function), *estimation error* (finite data do not identify the best member) and *optimisation error* (the algorithm does not reach a good solution of its own objective). Capacity lowers the first and raises the second. *Double descent*: in modern overparameterised models the test error can peak at the interpolation threshold (parameters $\approx N$) and fall again beyond it [on your sheet]; it depends on regularisation, optimiser, data geometry and which interpolating solution is selected, and an analogue occurs over training time. The classical U remains the first model; validation behaviour must be examined, not assumed.

Bias–variance mistakes

- "The variance of this trained model." Variance is across training sets; one model is one draw.
- "Doubling the data halves the bias." It halves the variance in the example; bias comes from the family and does not move.
- "Moderate complexity gives low bias and low variance." At the bottom of the U both are positive, just small; zero bias and zero variance would mean the procedure returns f^* regardless of the sample.
- Using the decomposition for 0–1 loss or without independence of training and test noise.
- 1-NN: small bias, large variance (one noisy neighbour decides). Students often say the opposite.

Noise versus bias versus variance

	What it is	What changes it	Quick example
$\sigma^2(x)$	randomness of Y that x does not explain	better features only	two identical houses, different prices
Bias^2	average prediction off the truth	family, features, regularisation (up)	constant model on a curve
Var	spread across training sets	capacity (up), N (down), averaging (down)	1-NN, interpolating cubic

Key insight. Add and subtract the true function, then add and subtract the average prediction: squared error splits into noise, bias^2 and variance, and the three respond to different levers (features, family, data).

How the professor asks it

Q. *A procedure has* $\mathrm{Bias}(x) = 0.3$, $\mathrm{Var}(x) = 0.05$ *and noise* $\sigma^2 = 0.1$ *at* x. *Compute its expected squared error, state which part more data could reduce, and which part a richer family could reduce.*

A. $0.1 + 0.3^2 + 0.05 = 0.1 + 0.09 + 0.05 = 0.24$. More data acts on the variance 0.05 (it shrinks roughly like $1/N$ for stable procedures); a richer family acts on the bias 0.09 (it can move $\bar{f}$ closer to f^*) at the price of raising the variance; nothing acts on the noise 0.1 except new information in the features.

Q. *Can a learning procedure have zero bias and zero variance? Justify.*
A. Only if $\hat{f}_{\mathcal{D}}(x) = f^*(x)$ for every training set, i.e. the procedure returns the truth regardless of the sample, which means it already knew the answer. Any procedure that actually uses the data has either sample dependence (variance) or a restriction that keeps it from f^* (bias), usually both.

4 Controlling complexity: regularisation and early stopping

Regularised empirical risk minimisation

With a penalty $\Omega : \Theta \to [0, \infty)$ and strength $\lambda \geq 0$,

$$\hat{\theta}_\lambda \in \arg\min_\theta \left[\widehat{R}_{\text{train}}(\theta) + \lambda\Omega(\theta)\right], \quad \text{[on your sheet]}$$

$$\Omega_2 = \|\theta\|_2^2 = \sum_j \theta_j^2 \text{ (ridge)}, \qquad \Omega_1 = \|\theta\|_1 = \sum_j |\theta_j| \text{ (lasso)}. \quad \text{[on your sheet]}$$

"Fit the data *and* keep the parameters small." λ says how much a unit of penalty costs in units of training loss: $\lambda = 0$ is plain ERM; large λ makes any sizeable coefficient too expensive. With coefficients $(3, -4)$: $\Omega_2 = 25$, $\Omega_1 = 7$.

Purpose. To reduce effective capacity without removing parameters: the model keeps all its numbers but is no longer free to use them.

Impact. Larger λ means a smaller effective family (next box), more bias from shrinkage, less variance. Other dials with the same purpose: restricting the family, increasing k, selecting features, stopping early, collecting more relevant data.

A penalty is a soft constraint, and the families are nested

Let $\hat{\theta}_\lambda$ solve the penalised problem and put $c = \Omega(\hat{\theta}_\lambda)$. Claim: $\hat{\theta}_\lambda$ also solves $\min_\theta \widehat{R}_{\text{train}}(\theta)$ subject to $\Omega(\theta) \leq c$ [on your sheet]. Take any feasible θ ($\Omega(\theta) \leq c$). Optimality of $\hat{\theta}_\lambda$ for the penalised problem gives $\widehat{R}(\hat{\theta}_\lambda) + \lambda c \leq \widehat{R}(\theta) + \lambda\Omega(\theta) \leq \widehat{R}(\theta) + \lambda c$, hence $\widehat{R}(\hat{\theta}_\lambda) \leq \widehat{R}(\theta)$: no feasible point beats $\hat{\theta}_\lambda$, which is itself feasible. The constrained families $\mathcal{H}_c = \{f_\theta : \Omega(\theta) \leq c\}$ are nested in c, and $\lambda \uparrow$ corresponds to $c \downarrow$ (the exact map depends on the problem). Consequences: (i) regularised ERM is ERM over a smaller effective family, so capacity is about which functions can win the fitting contest, not how many numbers are stored; (ii) training risk is minimised at $\lambda = 0$ (nested families), so λ cannot be chosen on training data.

Ridge in one variable: smooth shrinkage

Model $f_w(x) = wx$, objective $J(w) = \frac{1}{N}\sum_i (y^{(i)} - wx^{(i)})^2 + \lambda w^2$, a convex quadratic. $J'(w) = -\frac{2}{N}\sum_i x^{(i)}(y^{(i)} - wx^{(i)}) + 2\lambda w = 0$ gives $w(\sum_i (x^{(i)})^2 + N\lambda) = \sum_i x^{(i)} y^{(i)}$, so

$$\hat{w}_\lambda = \frac{\sum_i x^{(i)} y^{(i)}}{\sum_i (x^{(i)})^2 + N\lambda} = \hat{w}_{\text{LS}} \cdot \frac{\sum_i (x^{(i)})^2}{\sum_i (x^{(i)})^2 + N\lambda}, \quad \text{[on your sheet]}$$

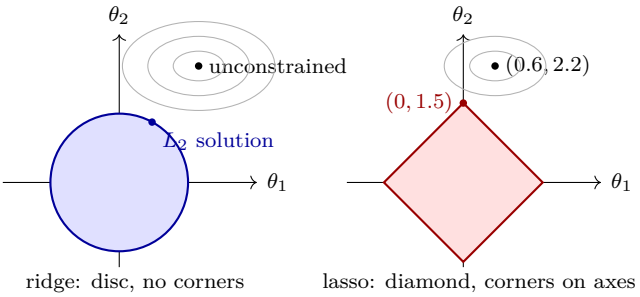
the least-squares slope times a factor in $(0, 1]$: shrunk toward 0, smoothly, never exactly 0 for finite λ. With $\sum x^2 = 14$, $\sum xy = 19$, $N = 4$: $19/14 = 1.357$ at $\lambda = 0$, $19/18 = 1.056$ at $\lambda = 1$, $19/54 = 0.352$ at $\lambda = 10$. As $\lambda \to \infty$ all penalised coefficients go to 0 and the model becomes the intercept alone.

Lasso in one variable: soft thresholding and exact zeros

Minimise $\frac{1}{2}(w-z)^2+\lambda|w|$ over $w\in\mathbb{R}$, where z is where the unpenalised fit would put the coefficient. Case $z>0$. For $w\geq 0$ the objective is $\frac{1}{2}(w-z)^2+\lambda w$, a parabola with unconstrained minimiser $w=z-\lambda$: if $z>\lambda$ this is positive and is the minimum on $[0,\infty)$; if $z\leq\lambda$ the parabola increases on $[0,\infty)$ and the minimum is at $w=0$. For $w<0$ the objective is $\frac{1}{2}(w-z)^2-\lambda w$, whose unconstrained minimiser $z+\lambda$ is positive, so it decreases toward 0 and no negative w beats $w=0$. Hence $\hat{w}=\max(z-\lambda,0)$ for $z>0$; by symmetry

$$\hat{w}=\operatorname{sign}(z)\max(|z|-\lambda,0), \quad \text{[on your sheet]}$$

and $\hat{w}=0$ exactly whenever $|z|\leq\lambda$. With $\lambda=0.5$: $z=2.2\to 1.7$; $z=0.4\to 0$; $z=-3\to -2.5$. With $\lambda=1.5$ and $z=1.2$: 0. Geometrically the L_1 ball $\|\theta\|_1\leq c$ is a diamond with corners on the axes and the optimum often lands on a corner where coordinates are exactly zero (the slides' $(0.6,2.2)\to(0,1.5)$: feature 1 dropped); the L_2 ball is a disc without corners, so ridge shrinks everything and keeps everything. L_1 therefore performs *embedded feature selection*; with correlated features the selected subset can be unstable, and in more than one dimension the lasso solution is not obtained by coordinate-wise soft thresholding, which is why the result is stated in one variable.



The regularisation path: sweep λ

Setup. Standardised features (fitted on training rows), ridge and lasso, training and validation rows.

Held fixed. Rows, features, metric; the intercept unpenalised.

What we manipulate. λ on a logarithmic grid $10^{-3},\ldots,10^{2}$.

What we measure. Every coefficient at every λ (the *path*), plus training and validation risk.

What the plot shows. Coefficient paths against $\log\lambda$: ridge paths shrink smoothly toward zero and never reach it; lasso paths hit zero one after another, so the number of nonzero coefficients steps down. Below them, the validation U.

How to read it. The minimum of the U gives $\hat{\lambda}$. On the lasso path, the order in which coefficients die is a feature ranking; with two near-identical features ridge splits the weight equally while lasso keeps one and may switch which one between folds (instability). Without standardisation a feature in millimetres is penalised a thousand times more than the same feature in metres, and the path is meaningless.

Credit scoring with 300 candidate variables

A bank has 300 features for 5,000 loans. Ridge keeps all 300 with small weights: good prediction, no story. Lasso at the validation-chosen λ keeps 18, which the risk team can audit and which are cheap to collect at application time. Among two near-duplicate income variables lasso kept one; which one changes between folds, so the team reports the pair, not the individual. The L_1 choice was driven by the belief that few variables matter and the need to know which; L_2 is the

choice when many variables each contribute a little.

Early stopping

Iterative optimisation produces $\hat{\theta}_0, \hat{\theta}_1, \ldots$. Early iterations learn strong recurring patterns; later ones fit exceptions and noise. Early stopping keeps the checkpoint with the smallest validation risk, $t^* = \arg\min_t \widehat{R}_{\text{val}}(\hat{\theta}_t)$ [on your sheet], with a *patience* p: continue p epochs without improvement before stopping, keeping the best checkpoint seen.

Purpose. Regularisation for free in any iterative method: training time controls effective capacity while the parameter count stays fixed.

Impact. The stopping epoch is a hyperparameter chosen on validation. Patience protects against stopping on a wobble; it can still miss a later decrease under epoch-wise double descent.

Early stopping: the training curve over epochs

Setup. An iterative learner, training and validation rows.

Held fixed. Everything except the number of epochs.

What we manipulate. The epoch t, by training on and saving a checkpoint each epoch.

What we measure. Training and validation loss after each epoch.

What the plot shows. Training loss falls throughout; validation loss falls, bottoms at t^*, rises.

How to read it. Roll back to $\hat{\theta}_{t^*}$, never keep the last epoch. Stopping on the test loss is tuning on the test set.

Regularisation mistakes

- Choosing λ on training data: training risk is minimised at $\lambda = 0$ (nested families), so the answer is always "no regularisation".
- Penalising unstandardised features, or penalising the intercept.
- Searching λ on a linear grid: the useful range spans orders of magnitude.
- Expecting ridge to select features (it never produces exact zeros) or lasso to be stable under correlated features (it is not).
- Keeping the last checkpoint instead of the best one; stopping without patience on a noisy validation curve.

L_2 versus L_1 versus early stopping versus smaller family

Dial	What it manipulates	Effect on coefficients	Use when
L_2 (ridge)	size of $\|\theta\|_2$	smooth shrinkage, none exactly zero	many features each contribute a little
L_1 (lasso)	size of $\|\theta\|_1$	soft threshold, exact zeros	few features matter and you want to know which
early stopping	number of iterations	the path stops before fitting noise	any iterative method; free
smaller family	degree, depth, width, larger k	fewer functions available	structure is known to be simple

Key insight. A penalty does not remove parameters; it removes functions from the contest. Ridge shrinks every coefficient by a factor, lasso subtracts λ and stops at zero, early stopping limits how far the optimiser travels; all three are chosen on validation because training error would set them to nothing.

How the professor asks it

Q. *Derive the one-variable ridge solution and explain in words why it can never set the coefficient exactly to zero, while the lasso can.*
A. Minimise $\frac{1}{N}\sum(y-wx)^2+\lambda w^2$: the derivative $-\frac{2}{N}\sum x(y-wx)+2\lambda w=0$ gives $\hat{w}_\lambda=\sum xy/(\sum x^2+N\lambda)$, the least-squares slope times $\sum x^2/(\sum x^2+N\lambda)\in(0,1]$. The factor is strictly positive for finite λ, so the coefficient keeps the sign and magnitude order of the unpenalised one, only scaled. The L_2 penalty's derivative $2\lambda w$ vanishes at $w=0$, so there is no force holding the coefficient at zero. The L_1 penalty has a kink at zero: its subgradient there is the whole interval $[-\lambda,\lambda]$, so whenever the data's pull $|z|$ is at most λ the kink absorbs it and $\hat{w}=0$ exactly (soft threshold $\operatorname{sign}(z)\max(|z|-\lambda,0)$).

Q. *Two identical features are included in a ridge model and in a lasso model. What does each do with them, and why does this matter for interpreting a lasso's selected subset?*
A. Ridge splits the weight equally (any other split has the same fit and a larger $\|\theta\|_2^2$, since a^2+b^2 is minimised at $a=b$ for fixed $a+b$). Lasso is indifferent among all splits with the same sum ($|a|+|b|$ is constant for same-sign splits), so it may keep either one and drop the other, and which one survives can change between folds or samples. A lasso's selected subset is therefore an unstable ranking under correlation; report groups of correlated features, or use the path across folds.

5 Feature and model selection, and the overuse of validation

Feature selection as a model choice

Choosing which inputs to use changes the representation. It can remove irrelevant variables (which increase estimation difficulty), reduce redundancy, improve interpretability, and exclude variables unavailable at decision time. It is a data-dependent choice and must be evaluated like any other: on rows that took no part in it.

Purpose. Fewer, relevant inputs mean less variance and a model you can explain.

Impact. With d features there are 2^d subsets [on your sheet]: exhaustive search costs 2^d fits, and $2^{30}=1{,}073{,}741{,}824$. Practical searches are filters and greedy procedures, each blind to something.

Filter versus forward versus backward versus embedded selection

Method	What it does	Cost	Blind to
Correlation filter	scores each variable alone, $s_j=\lvert\widehat{\mathrm{Corr}}(X_j,Y)\rvert$, keeps the top ones	d scores, no fits	nonlinear relations, interactions, redundancy between kept features
Forward selection	start empty, add the feature with the largest validation gain, repeat	about $d(d+1)/2$ fits [on your sheet]	combinations blocked by an early greedy choice
Backward elimination	start full, remove the least useful, repeat	same order; needs the full model to be fittable	same; impossible when $d>N$
Embedded (L_1)	the penalty zeroes coefficients during fitting	one path	unstable under correlated features

Useless alone, perfect together

X_1, X_2 independent fair ±1 coins and $Y=X_1X_2$. Each feature has correlation 0 with Y (knowing X_1 says nothing about the sign of the product), so a correlation filter discards both; together they determine Y exactly. Filters look at one variable at a time; interactions are invisible to

them. Weak marginal correlation never proves a feature useless jointly with others.

Feature selection inside versus outside the folds

Setup. 300 rows, 5,000 features, labels generated independently of the features (no signal).

Held fixed. The data, 10-fold cross-validation of logistic regression on 20 features.

What we manipulate. Where the selection step runs: (a) rank features on all 300 rows, keep the top 20, then cross-validate; (b) inside each fold, rank on that fold's 270 training rows only.

What we measure. Cross-validation accuracy in both arrangements.

What the plot shows. (a) reports high accuracy (often above 90%); (b) reports about 50%.

How to read it. In (a) the validation rows helped choose the representation they are scored on: leakage. All data-dependent scoring and subset search belong *inside* the training pipeline and are repeated within every fold [on your sheet].

The three subsets and the best-of-many problem

Let h be an entire configuration (features, family, λ, k, threshold). Training fits $\hat{f}_h$; validation selects $\hat{h} \in \arg\min_{h \in \mathcal{C}} \widehat{R}_{\text{val}}(\hat{f}_h)$ [on your sheet]; the test set stays outside the loop. Each validation score is honest for its own candidate; the *minimum* over candidates is optimistic.

The best of many validation scores is optimistic

If each $\widehat{R}_{\text{val}}(\hat{f}_h)$ is unbiased for $R(\hat{f}_h)$ (models fitted without the validation rows), then for every h_0, $\min_h \widehat{R}_{\text{val}}(\hat{f}_h) \le \widehat{R}_{\text{val}}(\hat{f}_{h_0})$ pointwise; take expectations and then the minimum over h_0 on the right:

$$\mathbb{E}\Big[\min_{h \in \mathcal{C}} \widehat{R}_{\text{val}}(\hat{f}_h)\Big] \le \min_{h \in \mathcal{C}} R(\hat{f}_h), \quad \text{[on your sheet]}$$

strict as soon as the estimates are noisy and no single h wins in every sample. Try a hundred configurations and report the lowest score: that is the minimum of a hundred noisy numbers, pulled downward. Every reported validation result also steers the next design choice, so development gradually adapts to the validation set. The test set protects against this accumulated adaptation, and only if its result is not then used to continue development, at which point it has become another validation set. For the magnitude ($\sigma/\sqrt{\pi}$ for two equal candidates, $\sigma\sqrt{2\ln n}$ for n) see the Experiments document.

K-fold cross-validation for configurations

Partition the development data into K folds; for configuration h and fold j fit $\hat{f}_h^{(-j)}$ without fold j and evaluate on it; $\widehat{R}_{\text{CV}}(h) = \frac{1}{K}\sum_j \widehat{R}_{\text{val}}^{(j)}(\hat{f}_h^{(-j)})$, $\hat{h} \in \arg\min_h \widehat{R}_{\text{CV}}(h)$ [on your sheet]. Each observation validates once and trains $K-1$ times. It uses limited data efficiently at K fits per configuration; the estimate still varies with the sample; random folds assume exchangeable rows (grouped or temporal data need adapted folds); and it does not replace a final untouched test set.

Selection mistakes

- Scoring or selecting features on all rows, then cross-validating: leakage, inflated estimate.
- Trusting a correlation filter to find interacting features (the XOR example).
- Reporting the best validation (or cross-validation) score as the expected performance.
- Continuing to develop after looking at the test score.

- Using random folds on grouped or temporal data.

Key insight. Selection is part of training: whatever looks at the labels to choose features, hyperparameters or thresholds must be redone inside each fold, and the winner's score is optimistic by construction, which is why one untouched test set exists.

How the professor asks it

Q. *Explain why the minimum of 100 honest validation estimates is not an honest estimate of the winner's risk. Give the one-line inequality.*
A. Each estimate scatters around its own true risk; picking the minimum picks, in part, the luckiest scatter. Formally $\min_h \widehat{R}_{\text{val}}(\hat{f}_h) \le \widehat{R}_{\text{val}}(\hat{f}_{h_0})$ for any fixed h_0, so $\mathbb{E}[\min_h \widehat{R}_{\text{val}}(\hat{f}_h)] \le \min_{h_0} R(\hat{f}_{h_0})$: the expected best score lies below the best true risk, with strict inequality whenever the estimates are noisy and no single configuration always wins. The winner's true risk is at least $\min_h R(\hat{f}_h)$, so the reported score underestimates it. Only rows that took no part in the selection can estimate it honestly.

Q. *Why must feature selection be repeated inside every cross-validation fold rather than done once before cross-validation?*
A. Because selection reads the labels. If it reads the labels of the rows that will later be a validation fold, that fold's labels have influenced the representation on which the fold is then scored, so the fold is no longer unseen; on label-free data this produces high cross-validation accuracy from nothing. Repeating the selection inside each fold, on the training portion only, restores the condition that the validation rows played no part in anything that was chosen.

6 Reliable evaluation: splits, metrics, leakage

The split must match deployment

Ask first: what kind of *new* case will the model face? Then hold out exactly that kind. *Random*: each observation assigned independently (credible only when deployment cases are independent draws from the same population). *Stratified*: random but preserving class proportions in every subset, stabilising rare-class counts; it must not override dependence (records from one device must not be scattered to match proportions). *Grouped*: every group (user, device, patient, hospital) entirely in one subset, so the learner cannot use identity and is evaluated on unseen groups. *Temporal*: train on the past, validate and test on later observations, preserving the direction of prediction and revealing deterioration as behaviour or attacks change. [on your sheet]

Purpose. To make the held-out number describe what will happen at deployment.

Impact. Random, grouped and temporal splits of the same data give different numbers; only the one that mirrors deployment is predictive.

Three protocols, one model

Setup. Flows from 50 devices over 6 months; one model.

Held fixed. The model, the features, the metric.

What we manipulate. The split rule: random; grouped by device (hold out 10 devices); temporal (train months 1–4, test months 5–6, during which a new attack tool appeared).

What we measure. Test accuracy under each.

What the plot shows. Random 99.5% (the model learned "device 17 is the compromised laptop"); grouped 96% (it must recognise behaviour); temporal 90% (new tool).

How to read it. Only the third number describes next month. "Will this customer churn next

quarter" is temporal (features before, outcome after); "does this model work on a hospital it has never seen" is grouped by hospital.

Metrics from the confusion matrix

With the rare class (attacks) positive, count TP, FN, FP, TN. [on your sheet]

$$\text{Recall} = \frac{\text{TP}}{\text{TP}+\text{FN}}, \qquad \text{Precision} = \frac{\text{TP}}{\text{TP}+\text{FP}}, \qquad \text{Specificity} = \frac{\text{TN}}{\text{TN}+\text{FP}},$$
$$\text{BalancedAcc} = \frac{\text{Recall}+\text{Specificity}}{2}, \qquad F_1 = \frac{2PR}{P+R}.$$

Recall: of the real attacks, what fraction did we catch? Precision: of the alarms, what fraction were real? Specificity: of the benign flows, what fraction were left alone? Balanced accuracy: average of the two per-class rates, immune to imbalance. F_1: harmonic mean of precision and recall, high only when both are, ignores true negatives.

Purpose. Accuracy hides minority failure: with a positive fraction π, the classifier that always says "negative" has accuracy $1-\pi$ and recall 0 [on your sheet]; at $\pi = 0.01$ that is 99% accuracy and no attack ever detected (its balanced accuracy, 0.5, is the number that exposes it).

Impact. The metric and the threshold must reflect the costs of missed attacks, false alarms and alert volume, exactly as the cost-sensitive threshold of Chapter 2.

A detector on 10,000 flows with 50 attacks

TP $= 40$, FN $= 10$, FP $= 90$, TN $= 9860$. Accuracy $9900/10000 = 0.990$; recall $40/50 = 0.80$; precision $40/130 = 0.31$; specificity $9860/9950 = 0.991$; balanced accuracy $(0.80 + 0.991)/2 = 0.895$; $F_1 = 2(0.31)(0.80)/(0.31 + 0.80) = 0.44$. Accuracy says 99%; precision says two alarms in three are false. Which matters depends on whether an analyst must investigate each alarm. Another case: TP $= 30$, FN $= 70$, FP $= 0$ gives precision 1.0, recall 0.3, $F_1 = 0.6/1.3 \approx 0.46$.

F_1 sits between the worse rate and the average

For $P, R \in (0, 1]$: $\min(P, R) \le F_1 \le \frac{P+R}{2}$, with equality on the right iff $P = R$. Lower bound: if $P \le R$ then $F_1 = P \cdot \frac{2R}{P+R} \ge P$ since $2R \ge P + R$. Upper bound: $\frac{P+R}{2} - \frac{2PR}{P+R} = \frac{(P+R)^2-4PR}{2(P+R)} = \frac{(P-R)^2}{2(P+R)} \ge 0$. So F_1 is dragged toward the *worse* of the two: 0.31 and 0.80 give 0.44, far below the average 0.56. That is the point of a harmonic mean: a great recall cannot hide a poor precision.

Threshold sweep on validation

Setup. A probabilistic classifier, validation rows, costs $C_{\text{FP}}, C_{\text{FN}}$.

Held fixed. The fitted model, the validation rows.

What we manipulate. The decision threshold t from 0 to 1.

What we measure. Recall, precision, false alarms per 100,000 benign flows, expected cost at each t.

What the plot shows. Recall falling and precision rising with t; expected cost U-shaped with minimum near the cost threshold $t = C_{\text{FP}}/(C_{\text{FP}} + C_{\text{FN}})$ when the probabilities are calibrated.

How to read it. The threshold is a hyperparameter: choose it on validation, report on test. Lab 1 does exactly this, reporting test recall, false-positive rate and false alarms per 100,000 benign flows.

Leakage

Leakage is any use, during development, of information that will not be available when a new decision is made [on your sheet]: duplicates or near-duplicates in several subsets; the same device, session or patient in training and test; predictors created after the outcome or after a human investigation ("ticket closed as intrusion", "number of analyst comments"); preprocessing fitted with validation or test observations; hyperparameters repeatedly chosen on test results. It creates an unrealistically favourable estimate.

Purpose. A checklist of the ways tomorrow's information gets into today's model.

Impact. The fix for preprocessing: the training set estimates $\hat{\mu}_j = \frac{1}{N_{\text{tr}}} \sum_{i \in \text{tr}} x_j^{(i)}$ and $\hat{\sigma}_j^2$, and the *same* values are reused unchanged on validation, test and deployment [on your sheet]; likewise imputation, feature selection, dimensionality reduction, resampling; in cross-validation each fold fits them on its own training portion. Leaking pipeline: all data $\rightarrow$ estimate $\mu, \sigma \rightarrow$ split. Valid pipeline: split $\rightarrow$ estimate on training $\rightarrow$ reuse.

Evaluation mistakes

- Random split for future events or new entities; stratifying across groups.
- Accuracy on imbalanced data; a metric chosen after seeing the results.
- A feature that predicts perfectly because it was recorded after the outcome.
- Scaler, imputer or selector fitted on all rows; a time series shuffled before splitting.
- Threshold tuned on training or test rows.

Random versus stratified versus grouped versus temporal

Split	Deployment question it answers	What it holds out	Fails when
random	new independent cases from the same population	rows	rows are dependent (groups, time)
stratified	same, with a rare class	rows, class proportions kept	it scatters groups to match proportions
grouped	new users, devices, patients, sites	whole groups	deployment is in the future too
temporal	the future	later periods	deployment is on new entities too (combine)

Key insight. Hold out the kind of novelty you will meet, measure what the costs care about, and let nothing from the held-out rows or from the future touch the pipeline.

How the professor asks it

Q. *A detector reports 99% accuracy on a dataset with 1% attacks. Explain what this number can and cannot mean, compute the balanced accuracy of the trivial classifier, and name the two metrics you would ask for.*

A. With $\pi = 0.01$, the classifier that always says "benign" has accuracy $1 - \pi = 0.99$ and recall 0: it detects nothing. So 99% accuracy is consistent with a detector that never fires. Its specificity is 1 and its recall 0, so its balanced accuracy is $(0 + 1)/2 = 0.5$, the number that exposes it. Ask for recall (fraction of attacks caught) and precision or false alarms per 100,000 benign flows (what each alarm costs an analyst); F_1 summarises the two but ignores true negatives.

Q. *Give an example of a feature that would yield near-perfect test performance and be useless in deployment, and explain the mechanism.*

A. "Number of analyst comments on the event": comments are written during the investigation that follows an intrusion, so the feature is a consequence of the label recorded after the fact. In the historical table it separates the classes almost perfectly; at decision time, before any investigation, its value is zero for every new event. This is leakage from the future; the rule for catching it is to ask, for each feature, at what moment its value becomes known relative to the moment of the decision.

7 Diagnosis: learning curves, and Lab 1

Learning curves

Vary the amount of training data and plot training and validation performance against it [on your sheet]. Patterns: both errors high and flat, underfitting (improve representation or capacity); training low and validation high, high variance (regularise, reduce capacity, more representative data); errors close and both still decreasing as N grows, data-limited (more representative examples will help); test performance degrading over calendar time, distribution shift (update data, time-aware evaluation). "Close and decreasing" means a small gap and both improving: not overfitting strongly, limited by evidence. Curves are diagnostic evidence, not proofs; read them with domain knowledge and error analysis. The full protocol and the sandwich argument for the two curves' shapes are in the Experiments document.

Lab 1 at a glance, mapped to the chapter

The intrusion-detection lab audits a labelled-flow dataset (imbalance, missing or non-finite values, duplicates, conflicting labels, redundancy); pre-declares random-row, grouped and chronological train/validation/test protocols (the split section); fits baselines (majority class, depth-3 tree, logistic regression, kNN) under each protocol; probes metadata shortcuts and shuffled labels (the shortcut probe); selects the decision threshold on validation (selection section); and reports test recall, false-positive rate and false alarms per 100,000 benign flows (the metrics). Every idea of this chapter appears there.

How the professor asks it

Q. *A learning curve shows both curves high, flat and close together from 2,000 to 40,000 rows. What do you conclude and what would you change?*
A. The procedure is not limited by data (more rows changed nothing) nor by variance (the gap is small): it is limited by bias, meaning the family or the representation cannot express the relationship. Add informative features, use a more flexible family, or reduce regularisation; collecting more of the same rows will not move the plateau.

Coverage of the textbook chapter

Textbook item (Chapter 3)	Where in this document
Definition: empirical risk, population risk, generalisation gap	§1 concept box
Proposition: training risk of a fitted model is optimistic	§1 math box, drill
Example: cows on grass, camels on sand; technique: weakening shortcuts	§1 shortcut probe, example
Definition: inductive bias	§1 concept box, versus box
Definition: capacity, nested families; proposition: training error favours capacity	§2
Definition: underfitting, overfitting; example: three polynomials; kNN direction; gotcha	§2
Definition: bias and variance; theorem and proof; example with numbers	§3
Remark: double descent; definition: three sources of excess error; classic questions	§3 concept box, drills
Definition: regularised ERM; theorem: soft constraint; ridge 1-D; lasso soft threshold; geometry figure	§4
Choosing λ; definition: early stopping; technique: regularisation in practice; classics	§4
Proposition: 2^d subsets; definition: filters and greedy searches; XOR example; leakage gotcha	§5
Three subsets; proposition: best of many validation scores; definition: K-fold CV	§5
Splits (random, stratified, grouped, temporal); intrusion example	§6
Proposition: accuracy hides minority failure; definition: metrics; table; proposition on F_1	§6
Definition: leakage; technique: preprocessing inside the pipeline; classics	§6
Definition: learning curves; diagnostic table; Lab 1	§7 and the Experiments document

The ten mistakes of this chapter

1. Reading a training score as evidence about new cases.
2. Not probing for shortcuts (new site, new period, shuffled labels, metadata removed).
3. Comparing families or choosing λ, k, depth, epochs by training error.
4. Talking about the bias or variance of one fitted model; expecting data to remove bias.
5. Getting the kNN direction backwards; treating zero training error as success.
6. Penalising unstandardised features or the intercept; linear grids for λ; last epoch instead of best.
7. Expecting ridge to select or lasso to be stable under correlation.
8. Selecting features on all rows before cross-validation; reporting the winner's validation score as final.
9. A random split for the future or for new entities; accuracy on imbalanced data.
10. Features known only after the outcome; preprocessing fitted on all rows; thresholds tuned on test.

Chapter 3 in ten lines

1. Gap $G = R(\hat{f}) - \widehat{R}_{\mathcal{D}}(\hat{f})$; $\mathbb{E}[\widehat{R}_{\mathcal{D}}(\hat{f}_{\mathcal{D}})] \leq \min_{\mathcal{H}} R \leq \mathbb{E}[R(\hat{f}_{\mathcal{D}})]$: the sample chooses and grades.
2. Spurious cues fit the sample as well as stable ones; only new contexts separate them; inductive bias (data, family, regulariser, optimiser) decides off the sample.
3. Capacity: training error nonincreasing in nested families; validation U; underfitting both high, overfitting low/high; kNN capacity grows as k shrinks.
4. Squared error $= \sigma^2 + \text{Bias}^2 + \text{Var}$ by adding and subtracting f^* then $\bar{f}$; capacity trades them, data lowers variance only.
5. Regularised ERM = ERM over the nested family $\Omega \leq c$; training risk minimal at $\lambda = 0$, so λ is chosen on validation.
6. Ridge: factor $\sum x^2/(\sum x^2 + N\lambda)$, never zero. Lasso: $\text{sign}(z)\max(|z| - \lambda, 0)$, exact zeros, diamond corners,

unstable under correlation.

7. Early stopping: keep $\arg\min_t \widehat{R}_{\text{val}}(\hat{\theta}_t)$ with patience; training time is capacity.
8. 2^d subsets; filters miss interactions (XOR); greedy searches block; selection inside every fold or it leaks.
9. Best of many validation scores is optimistic; CV averages K honest fold scores; the test set is used once.
10. Split from the deployment question (random, stratified, grouped, temporal); metrics from the costs (recall, precision, specificity, balanced accuracy, F_1); no leakage; learning curves for bias/variance/data-limited.

Understanding ML, Chapter 4: Discriminative and Generative Classification

After reading this you can say, for any classifier, what it models, what it assumes, what knob you are turning, what experiment reveals the difference, and what goes wrong when the assumption is false

Fundamentals of Machine Learning, midterm

2026-11-03

What this chapter is about, in one paragraph

Chapter 2 settled what to do once the posterior $p(y = c \mid x)$ is known: pick the class with the largest posterior, or compare it with a cost threshold. Chapter 4 is about the only hard part, *getting the posterior out of labelled data*. There are exactly two ways. The **discriminative** route writes a parametric family $p_\theta(y \mid x)$ (a score, squashed by a sigmoid or a softmax) and fits it to predict the labels; it learns *where the label changes* and never looks at how the inputs are distributed. The **generative** route writes a story of how a labelled example is produced (choose a class from the prior π_c, then draw features from the class-conditional $p_\theta(x \mid y = c)$), fits that story, and reverses it with Bayes' rule; it learns *what each class looks like*. Everything else in the chapter is a consequence of that choice: which assumptions you make (Gaussian clouds, shared or separate covariance, independent features), how many numbers you estimate, how much data you need, what the boundary looks like, what happens when the class frequencies change between training and deployment, and whether the model can say "I have never seen anything like this". The quantities we manipulate in the experiments are the amount of labelled data N, the covariance assumption, the smoothing constants λ and α, the class prior, the decision threshold, and the familiarity of the input.

Contents

1 The decision is known, the posterior is not: two routes **2**

2 The discriminative route: scores, sigmoid, softmax, cross-entropy **4**

3 The generative route: a story, reversed by Bayes' rule **8**

4 Gaussian class models: LDA and QDA **9**

5 Naive Bayes: independence given the class **13**

6 Comparing the routes: the data-size experiment **17**

7 Calibration, class imbalance and prior shift **18**

8 Unfamiliar inputs, combined systems, language models **21**

Coverage of the textbook chapter **23**

1 The decision is known, the posterior is not: two routes

Posterior, prior, class-conditional, and the decision that uses them

Let $Y \in \{1, \ldots, C\}$ be the class and $X \in \mathbb{R}^d$ the feature vector.

- The **posterior** $p(y = c \mid x)$: probability of class c *once* x *has been seen.*
- The **prior** $\pi_c = p(y = c)$: how frequent class c is *before* any input is seen.
- The **class-conditional** $p(x \mid y = c)$: the distribution of inputs that class c produces (a density for continuous x, a mass function for discrete x).

Given the posterior, the decision is a one-liner: under zero–one loss $f^*(x) = \arg\max_c p(y = c \mid x)$ [on your sheet]; with costs, alert iff $p(y = 1 \mid x) > C_{\text{FP}}/(C_{\text{FP}} + C_{\text{FN}})$ [on your sheet]. The posterior is unknown; it must be *estimated* from $\mathcal{D}_N = \{(x^{(i)}, y^{(i)})\}_{i=1}^N$.

Purpose. The whole chapter is the estimation problem; the decision rule is inherited from Chapter 2.

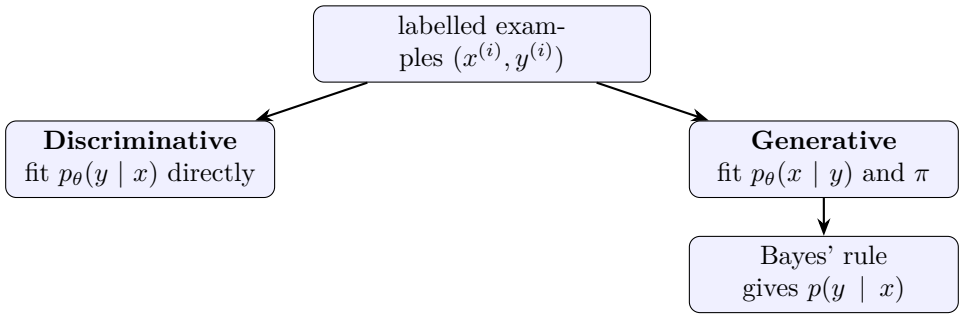
Impact. A better posterior estimate improves *every* policy built on it; a different cost threshold changes the action without changing the estimate.

Discriminative and generative classifiers

- A **discriminative** model directly approximates the posterior with a parametric family $p_\theta(y = c \mid x)$. Training produces $p_{\hat{\theta}}(y = c \mid x)$. It learns *where the label changes* as a function of the features.
- A **generative** classifier models the prior $p(y = c)$ *and* the class-conditional $p(x \mid y = c)$, then obtains the posterior by Bayes' rule. It learns *what each class looks like* and asks which class explains the observed input best.

Purpose. Same target (a posterior), two opposite directions of modelling.

Impact. Discriminative: fewer assumptions, nothing about the inputs, needs more data. Generative: more structure, works with little data if the structure is right, carries a persistent error if it is wrong, and gains two abilities the other route lacks (plausibility of an input, handling missing features).



Where each route is used

- *Spam filtering.* Naive Bayes (generative) models token frequencies of spam and of legitimate mail; cheap, estimated from counts, still a baseline or a first triage layer.
- *Fraud and intrusion screening.* Class-conditional models score how *compatible* an event is with normal behaviour and with known attacks; they serve classification and anomaly triage at once. Modern systems add discriminative and behavioural signals.
- *Image classification.* Deep discriminative networks dominate: modelling the full density of

images per class is far harder than learning where the label changes.

"Generative" is about the *direction* of modelling, not the task: a class-conditional model can infer the class of an observed x or, in principle, draw a new x for a chosen class. Classical generative classifiers and modern generative AI share the direction; families and scales differ enormously.

Discriminative versus generative, the one table to remember

Question	Discriminative	Generative
Modelled quantity	posterior $p_\theta(y \mid x)$ or a boundary	prior π_c and class model $p_\theta(x \mid y = c)$
Assumptions on the input	usually fewer	usually stronger (a family for each class)
What is fitted	one conditional objective (labels given inputs)	class counts, then one model per class from that class's examples
What it can do	rank the known classes	rank, plus judge plausibility, sample inputs, marginalise missing features
Failure mode	underdetermined with little data; confident about unfamiliar inputs	persistent bias when the family is wrong; double counting under false independence
When to prefer	many labels, raw high-dimensional input, prediction is the only goal	few labels, credible class model, need for a reject option or prior correction

Key insight. Both routes end in a softmax of class scores. A discriminative model fits the scores directly; a generative model builds them as $\log \pi_c + \log p_\theta(x \mid y = c)$. Everything that separates the routes is where the scores come from and what had to be assumed to get them.

How the professor asks it

Q. *Explain why a posterior estimate $\hat{p}(y = 1 \mid x)$ and the alert decision are not the same object, and which of the two changes when (a) the costs change, (b) the model is retrained.*
A. They are different objects: the posterior is a number in $(0, 1)$ produced by the model; the decision is an action obtained by comparing that number with a threshold fixed by costs. (a) Changing $C_{\text{FP}}, C_{\text{FN}}$ moves the threshold $t = C_{\text{FP}}/(C_{\text{FP}} + C_{\text{FN}})$ [on your sheet] and so the action, without touching the model; the same 0.35 is "benign" at $t = 0.5$ and "attack" at $t = 0.2$. (b) Retraining changes the posterior, not the costs. One probability model supports several policies; no policy repairs a bad probability.

Q. *A security team says "attacks are only 1% of traffic, so a flow should almost never be classified as an attack". Give the generative argument against this, with the numbers $\pi = (0.99, 0.01)$, $p(x \mid \textit{benign}) = 0.001$, $p(x \mid \textit{attack}) = 0.20$.*
A. The prior is only one factor of the posterior odds; the likelihood ratio is the other. Prior-weighted likelihoods: $0.99 \times 0.001 = 0.00099$ and $0.01 \times 0.20 = 0.00200$; posterior of attack $0.002/0.00299 = 0.67$. In odds: prior odds $1/99$ times likelihood ratio 200 equals $2.02 > 1$. The prior settles the matter only when the input carries no evidence (likelihood ratio 1); here the flow is 200 times more compatible with the attack model, which beats the 99-to-1 prior.

2 The discriminative route: scores, sigmoid, softmax, cross-entropy

Score (logit), sigmoid, logistic regression

A discriminative model usually first produces a real-valued **score** $s_{\theta,c}(x)$ per class, then turns scores into probabilities, then into an action:

$$\{s_{\hat{\theta},c}(x)\} \longrightarrow \{p_{\hat{\theta}}(y = c \mid x)\} \longrightarrow \hat{y}(x).$$

The score model fixes how inputs are *ranked*; the probability map quantifies *relative* uncertainty among classes; the decision loss picks the *action*. For two classes one score suffices and the **sigmoid** converts it, $p_\theta(y = 1 \mid x) = \sigma(s_\theta(x)) = 1/(1 + e^{-s_\theta(x)})$ [on your sheet]. **Logistic regression** is the linear score $s_\theta(x) = w^\top x + b$. Sigmoid facts you must be able to use: $0 < \sigma < 1$; $\sigma(-s) = 1 - \sigma(s)$ [on your sheet]; strictly increasing; $\sigma(s) > t \iff s > \log \frac{t}{1-t}$ [on your sheet]; $\sigma' = \sigma(1 - \sigma)$ [on your sheet]. Values: $\sigma(0) = 0.5$, $\sigma(0.5) = 0.62$, $\sigma(2) = 0.88$, $\sigma(-2) = 0.12$, $\sigma(5) = 0.993$.

Purpose. Turn an unbounded number, convenient to model (a linear function, a network output), into a probability.

Impact. A score of 0 means "no idea"; large positive, "confidently class 1". With $w = (1, -2)$, $b = 0.5$, $x = (2, 1)$: $s = 2 - 2 + 0.5 = 0.5$, $p = \sigma(0.5) \approx 0.62$. One unit more of x_j adds w_j to the log-odds, i.e. multiplies the odds by e^{w_j}, whatever the other features are.

The log-odds are the score; the boundary is a hyperplane; costs only slide it

Step 1 (log-odds). From $p = \sigma(s)$ and $1 - \sigma(s) = \sigma(-s)$: $\dfrac{p}{1-p} = \dfrac{1/(1+e^{-s})}{e^{-s}/(1+e^{-s})} = e^s$, hence $\log \dfrac{p}{1-p} = s_\theta(x) = w^\top x + b$ [on your sheet]. Logistic regression is the model "log-odds linear in x".

Step 2 (equal costs). Zero–one Bayes decision: declare 1 iff $p > \frac{1}{2}$. Since σ is increasing and $\sigma(0) = \frac{1}{2}$, this is $s > 0$. For a linear score, $\{x : w^\top x + b = 0\}$ is a hyperplane.

Step 3 (unequal costs). The cost threshold on the probability is $t = C_{\text{FP}}/(C_{\text{FP}} + C_{\text{FN}})$ [on your sheet]. Invert the sigmoid: $\sigma(s) > t \iff 1 + e^{-s} < 1/t \iff e^{-s} < (1-t)/t \iff s > \log \frac{t}{1-t}$. With $\frac{t}{1-t} = \frac{C_{\text{FP}}}{C_{\text{FN}}}$ the rule is

$$\hat{y} = 1 \iff s_\theta(x) > \log \frac{C_{\text{FP}}}{C_{\text{FN}}} \quad \text{[on your sheet]}.$$

Reading. The boundary moves to a *parallel* hyperplane $w^\top x + b = \log(C_{\text{FP}}/C_{\text{FN}})$. The score family fixes the geometry (linear score, flat boundary; curved score g_θ, curved surface $g_\theta(x) = 0$); the costs only translate it along the score axis. Cyberattack with $C_{\text{FN}} = 10$, $C_{\text{FP}} = 1$: threshold $s > \log 0.1 = -2.30$, i.e. alarm already when the score is mildly negative (probability above $\sigma(-2.30) = 0.09$). Expensive misses lower the threshold; expensive false alarms raise it.

Sliding the threshold: one model, many policies

Setup. Fit one logistic regression on intrusion flows (training rows). Keep a validation set with labels.

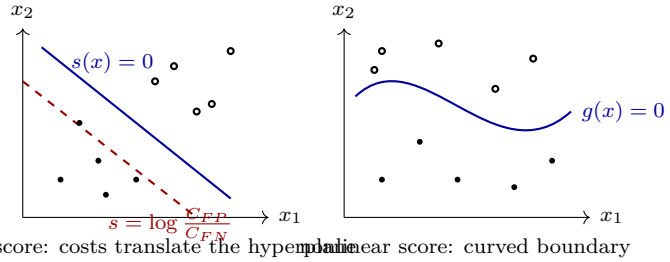
Held fixed. The fitted w, b; the validation rows.

What we manipulate. The decision threshold on the score, $\tau = \log(C_{\text{FP}}/C_{\text{FN}})$, from -4 to $+4$ (equivalently the cost ratio from 1:55 to 55:1).

What we measure. At each τ: recall of attacks, false-positive rate, number of alarms per 100,000 benign flows.

What the plot shows. Recall rises and the false-positive rate rises as τ falls; the curve of (false-positive rate, recall) is the ROC curve of *this* model. The model's ranking never changes; only the cut point moves.

How to read it. If two thresholds give very different recall at the same false-positive rate, you misread the plot: one model has one curve. To get a *better* curve you must change the model (features, family), not the threshold. Pick τ on validation from the costs, then report on the untouched test set.



linear score: costs translate the hyperplane nonlinear score: curved boundary

Softmax for C classes

$p_\theta(y = c \mid x) = \text{softmax}(s_\theta(x))_c = e^{s_{\theta,c}(x)} / \sum_{k=1}^{C} e^{s_{\theta,k}(x)}$ [on your sheet]. Exponentiate to make every score positive, divide by the total so the classes compete for probability one.

Purpose. The C-class version of the sigmoid.

Impact. Three facts. (i) The ordering of probabilities is the ordering of scores, so $\arg\max_c p = \arg\max_c s_c$. (ii) **Shift invariance**: adding the same constant a to every score multiplies numerator and denominator by e^a; nothing changes, so only $C - 1$ score *contrasts* are identifiable (and by hand you subtract the largest score before exponentiating, which avoids huge numbers). (iii) With $C = 2$, $\frac{e^{s_1}}{e^{s_1}+e^{s_2}} = \frac{1}{1+e^{s_2-s_1}} = \sigma(s_1 - s_2)$ [on your sheet]: softmax is the sigmoid of the score difference. Scores $(2, 0, -1)$: exponentials $(7.39, 1.00, 0.37)$, total 8.76, probabilities $(0.84, 0.11, 0.04)$. Implementations keep C scores rather than $C - 1$ so that every class plays the same role and one output layer serves any C; a reference class removes the redundancy when coefficients must be interpreted.

Cross-entropy, and why it is maximum likelihood

For example i, $\ell_{\text{CE}}^{(i)}(\theta) = -\sum_c \mathbf{1}(y^{(i)} = c) \log p_\theta(y = c \mid x^{(i)}) = -\log p_\theta(y^{(i)} \mid x^{(i)})$ [on your sheet]: minus the log of the probability given to the *true* class. True-class probabilities $0.9, 0.5, 0.1$ cost $0.105, 0.693, 2.303$ (natural logs): confident mistakes are punished hardest.

Purpose. The training loss for any probabilistic classifier, from logistic regression to deep networks.

Impact. Minimising the empirical cross-entropy is *exactly* maximising the conditional likelihood of the labels given the inputs; the model is never asked to explain the inputs.

Cross-entropy equals negative conditional log-likelihood

Assume the labels are conditionally independent given their inputs. The conditional likelihood is the product $L_{\text{cond}}(\theta) = \prod_{i=1}^{N} p_\theta(y^{(i)} \mid x^{(i)})$ (independence turns a joint probability into a product). Take $-\log$: the product becomes a sum,

$$-\log L_{\text{cond}}(\theta) = -\sum_{i=1}^{N} \log p_\theta(y^{(i)} \mid x^{(i)}) = N\, \widehat{R}_{\text{CE}}(\theta) \quad \text{[on your sheet]}.$$

Since $-\log$ is decreasing, maximising L_{cond} and minimising $\widehat{R}_{\text{CE}}$ have the same solutions. Three examples with true-class probabilities $0.9, 0.5, 0.1$: likelihood $0.9 \times 0.5 \times 0.1 = 0.045$; negative log $0.105 + 0.693 + 2.303 = 3.10$; empirical cross-entropy $3.10/3 = 1.03$. "Conditional" is the whole point: only the labels given the inputs are explained. For logistic regression the gradient is $\frac{1}{N}\sum_i(\hat{p}^{(i)} - y^{(i)})x^{(i)}$ [on your sheet], the average of error times input, and the objective is convex: no local minima.

Deep discriminative classifiers

A feed-forward network computes $h^{(0)} = x$, $h^{(\ell)} = \varphi(W_\ell h^{(\ell-1)} + b_\ell)$ for $\ell = 1, \ldots, L-1$ (multiply by a matrix, add a vector, bend with an elementwise nonlinearity φ), then scores $s_\theta(x) = W_L h^{(L-1)} + b_L$ and softmax. Cross-entropy trains all layers end to end.

Purpose. Let the score be a *curved* function of x so the boundary can curve.

Impact. Logistic regression is the case $L = 1$ (no hidden layer). The learned representation is optimised for predicting the label, not for describing the inputs; that is why the next concept applies to every discriminative model, deep or not.

What the discriminative objective does not provide

A discriminative classifier focuses on predicting Y from X. Consequences: it need not learn whether an input x is *plausible*; a large $\max_c p_{\hat{\theta}}(y = c \mid x)$ does not mean x resembles the training data; it has no mechanism for sampling inputs or marginalising missing features. Softmax probabilities are *relative* comparisons among the modelled classes: if all classes explain an unfamiliar input poorly, one of them can still get posterior near one.

Purpose. Know the limit of the route before trusting its confidence.

Impact. Confidence is not familiarity. This is the motivation for the reject option of Section 8.

The toaster experiment: out-of-distribution confidence

Setup. Train a three-class network (cat, dog, horse) with cross-entropy on photographs of those animals.

Held fixed. The trained network; the softmax output layer.

What we manipulate. The *familiarity* of the test input: animal photos from the test split, then inputs from outside the training classes (a toaster, random noise, a blank image).

What we measure. $\max_c \hat{p}(y = c \mid x)$, the top softmax probability, on each group.

What the plot shows. Histograms of top probability: high for familiar inputs (as expected) and *also* high for many unfamiliar ones. The toaster gets scores $(4.0, 0.5, -1.0)$, softmax $(0.96, 0.03, 0.01)$: "cat, 96%".

How to read it. The 0.96 measures how much better "cat" explains the toaster than "dog" and "horse" do, which is all a softmax can measure; nothing in the objective asked "is this even an animal?". A high top probability therefore separates the *known classes*, not familiar from unfamiliar. The experiment that fixes this adds a likelihood check (Section 8).

Reading $\max_c p(y = c \mid x)$ as "probability that the input is normal"

Tempting because the number is near one and looks like certainty. It inflates perceived reliability on novel inputs: a new attack mechanism far from every known class still receives a confident known-class label, so the detector reports high confidence precisely where it knows least. Fix: a reject option based on the class likelihood $\log p(x \mid y = c^*)$, which only a generative model provides, or a separate anomaly model.

Class imbalance treated as a bug of the discriminative route

If training and deployment share the same distribution, an imbalanced training prior is *not* an error: the fitted posterior should reflect the prevalence, and the 0.5 contour moves toward the positives (with 5% positives it sits deep in the positive region). The real difficulty is *too few minority examples* to locate the boundary; weighting or resampling helps in that small-sample regime, has limited benefit with enough minority data, and distorts the probabilities (Section 7). Fix: diagnose "few positives" (estimation) versus "prevalence changes at deployment" (shift) before touching the data.

Score versus probability versus decision

- **Score** s: unbounded, fixes the ranking and the boundary geometry; compare it with $\log(C_{\mathrm{FP}}/C_{\mathrm{FN}})$ to decide, you never need σ.
- **Probability** $\sigma(s)$ or softmax: in $(0,1)$, relative to the modelled classes; log-odds back to score: $s = \log \frac{p}{1-p}$ ($p = 0.9$ gives $s = 2.20$).
- **Decision**: the action after a cost threshold; changing costs slides the boundary, changing the family bends it.

Key insight. The score family decides the *shape* of the boundary, the costs decide its *position*, and the softmax only compares the classes it knows.

How the professor asks it

Q. *A logistic regression for intrusion detection has $w = (1.5, -0.8)$, $b = -1$. The cost of a missed attack is ten times that of a false alarm. Derive the decision rule in terms of the score, state the geometric effect on the boundary, and decide the flow $x = (1, 1)$.*
A. Rule: alert iff $s > \log(C_{\mathrm{FP}}/C_{\mathrm{FN}}) = \log(1/10) = -2.30$. Derivation: the probability threshold is $t = 1/(1+10) = 1/11$ and $\sigma(s) > t \iff s > \log \frac{t}{1-t} = \log \frac{1}{10}$. Geometry: the boundary $w^\top x + b = -2.30$ is a hyperplane parallel to the equal-cost one, shifted toward the benign side, so more flows are flagged. Flow: $s = 1.5 - 0.8 - 1 = -0.3 > -2.30$, so alert, although $\sigma(-0.3) = 0.43 < 0.5$ would say "benign" at equal costs.

Q. *Why does adding the same constant to all C scores leave the softmax unchanged, and what does this say about how many scores are really needed?*
A. Unchanged because $e^{s_k + a} = e^a e^{s_k}$ for every k, so e^a factors out of numerator and denominator and cancels. Consequently only the $C - 1$ differences $s_c - s_C$ matter: with $C = 2$ the softmax is $\sigma(s_1 - s_2)$, one score. Implementations keep C outputs for symmetry and vectorisation; by hand, subtract the largest score first to avoid overflow, it changes nothing.

Q. *Explain why minimising cross-entropy is maximum likelihood, and why the word "conditional" is essential to the comparison with the generative route.*
A. Because $-\log \prod_i p_\theta(y^{(i)} \mid x^{(i)}) = \sum_i -\log p_\theta(y^{(i)} \mid x^{(i)}) = N \widehat{R}_{\mathrm{CE}}$ under conditional independence of the labels given the inputs; $-\log$ is decreasing, so the minimisers coincide. "Conditional" because the likelihood explains the labels given the inputs only; the generative route maximises the *joint* likelihood $\prod_i \pi_{y^{(i)}} p_\theta(x^{(i)} \mid y^{(i)})$, which also has to explain the inputs. That is the extra structure (and the extra assumptions) of the generative route in one word.

3 The generative route: a story, reversed by Bayes' rule

Generative classifier

An observation is imagined to be produced in two steps: first choose a class, then generate features from that class,

$$Y \sim \text{Categorical}(\pi), \qquad X \mid (Y = c) \sim p_\theta(\cdot \mid y = c), \qquad p_\phi(x, y = c) = \pi_c \, p_\theta(x \mid y = c),$$

with parameters $\phi = (\pi, \theta)$. The joint factorises as prior times class-conditional.

Purpose. Describe each class, then compare descriptions.

Impact. The same fitted story can classify (infer y from x), judge plausibility ($p(x \mid y)$ small for every class), fill in missing features (marginalise), and generate.

Bayes' rule for classification, and its odds form

By the definition of conditional probability, $p(y = c \mid x) = p(x, y = c)/p(x)$. Numerator: $\pi_c \, p_\theta(x \mid y = c)$ by the factorisation. Denominator: the marginal $p(x) = \sum_k p(x, y = k) = \sum_k \pi_k \, p_\theta(x \mid y = k)$, a sum over the exhaustive, exclusive classes. Hence

$$p_\phi(y = c \mid x) = \frac{\pi_c \, p_\theta(x \mid y = c)}{\sum_{k=1}^{C} \pi_k \, p_\theta(x \mid y = k)} \quad \text{[on your sheet]}.$$

The denominator does not depend on c, so the arg max is decided by the **prior-weighted likelihood** $\pi_c \, p_\theta(x \mid y = c)$: "how common is the class" times "how well does it explain this x". For two classes divide the two numerators; the denominator cancels:

$$\underbrace{\frac{p(y = 1 \mid x)}{p(y = 0 \mid x)}}_{\text{posterior odds}} = \underbrace{\frac{\pi_1}{\pi_0}}_{\text{prior odds}} \times \underbrace{\frac{p(x \mid y = 1)}{p(x \mid y = 0)}}_{\text{likelihood ratio}} \quad \text{[not on your sheet: know it]}.$$

Rare-attack numbers: benign prior 0.99, likelihood 0.001; attack prior 0.01, likelihood 0.20. Prior-weighted: 0.00099 versus 0.00200; posterior of attack $0.002/0.00299 = 0.67$. Attacks stay rare in the population, yet this flow is more compatible with the attack model, and the posterior says so.

The generative likelihood splits by class; the MLE prior is the class frequency

For independent labelled observations, $L(\phi) = \prod_i p_\phi(x^{(i)}, y^{(i)})$. Take logs and substitute the factorisation:

$$\begin{aligned} -\log L(\phi) &= -\sum_{i=1}^{N} \left[\log \pi_{y^{(i)}} + \log p_\theta(x^{(i)} \mid y^{(i)}) \right] \\ &= \underbrace{-\sum_{c=1}^{C} N_c \log \pi_c}_{\text{prior part}} \; \underbrace{-\sum_{c=1}^{C} \sum_{i:\, y^{(i)} = c} \log p_\theta(x^{(i)} \mid y = c)}_{\text{one term per class}}, \end{aligned}$$

grouping the prior terms by class (label c occurs N_c times) and the class-conditional terms likewise. The first group depends only on π, the second only on θ, and within it class c's parameters appear only in class c's sum. **Consequence:** the prior is fitted from the counts alone, and *each class model is fitted from that class's examples alone*; nothing is estimated jointly.

Prior. Write $q_c = N_c/N$. Then $\sum_c N_c \log \pi_c = N \sum_c q_c \log \pi_c = -N \, H(q, \pi)$, minus N times the cross-entropy between the empirical class distribution and the model prior. By Gibbs' inequality

$H(q, \pi) \geq H(q)$ with equality iff $\pi = q$ [on your sheet], so the prior part is maximised at $\hat{\pi}_c = N_c/N$ [on your sheet]. Generative training asks each example to be explained *twice*: its class through π and its features through $p_\theta(x \mid y)$.

Two kinds of error of a fitted generative posterior

Shape approximation: the family $p_\theta(x \mid y)$ may not contain the truth; this error does *not* vanish with more data. *Finite-sample estimation*: within the family, $\hat{\phi}$ differs from the best parameter because N is finite; this error shrinks with N.

Purpose. Separate what more data can fix from what only a better model can fix.

Impact. The learning-curve experiment of Section 6 is built on exactly this split.

Confusing the prior with the posterior, or counting the prior per feature

"Attacks are 1%, so this is benign" uses the prior as if it were the posterior; the likelihood ratio can overturn it by a factor of hundreds. In hand computations with several features the prior is counted *once* in the score, never once per feature (a common slip in Naive Bayes). Fix: write the score as $\log \pi_c + \log p(x \mid y = c)$ and only then add per-feature terms.

Key insight. Posterior odds equal prior odds times likelihood ratio: the prior is one factor, the evidence the other, and the generative route makes both explicit.

How the professor asks it

Q. *Show that under the generative model the prior and the class-conditional parameters can be estimated separately, and state what each is estimated from.*
A. Claim: the negative log-likelihood is a sum of a term depending only on π and, for each class c, a term depending only on class c's parameters. Mechanism: $\log p_\phi(x, y) = \log \pi_y + \log p_\theta(x \mid y)$, so summing over i and grouping by label gives $-\sum_c N_c \log \pi_c - \sum_c \sum_{i:y^{(i)}=c} \log p_\theta(x^{(i)} \mid y = c)$. Minimising a sum of terms with disjoint variables is minimising each term. So $\hat{\pi}$ comes from the class counts (and equals N_c/N by Gibbs), and the model of class c from the examples of class c only. Consequence: adding examples of one class never changes another class's fitted model, only the priors.

Q. *Why is the MLE of the prior exactly the class frequency? Give the one-line argument.*
A. Because $\sum_c N_c \log \pi_c = -NH(q, \pi)$ with $q_c = N_c/N$, and the cross-entropy $H(q, \pi) = H(q) + \mathrm{KL}(q \| \pi)$ [on your sheet] is minimised, by Gibbs' inequality, exactly when $\pi = q$. So maximising the prior part means matching the model prior to the observed class proportions.

4 Gaussian class models: LDA and QDA

Multivariate Gaussian class model

$p_\theta(x \mid y = c) = \mathcal{N}(x; \mu_c, \Sigma_c) = (2\pi)^{-d/2} |\Sigma_c|^{-1/2} \exp[-\frac{1}{2}(x - \mu_c)^\top \Sigma_c^{-1} (x - \mu_c)]$ [on your sheet]. μ_c is the centre of class c's cloud; Σ_c its shape: a multiple of I gives circular contours, unequal diagonal entries stretch into an axis-aligned ellipse, off-diagonal entries rotate it (correlated features). The quadratic form is the **squared Mahalanobis distance**, "how many standard deviations from the centre in the direction of x". The prefactor with $|\Sigma_c|$ makes the density integrate to one: a diffuse class spreads its probability thin.

Purpose. The simplest "world" for continuous features.

Impact. The covariance assumption you make becomes the *shape of the boundary* you get.

Closed-form maximum likelihood, one dimension in full

By the split of the likelihood, fix one class and drop c; its $n = N_c$ examples are $x^{(1)}, \dots, x^{(n)}$. The negative log-likelihood is

$$J(\mu, \sigma^2) = \frac{n}{2} \log(2\pi\sigma^2) + \frac{1}{2\sigma^2} \sum_{i=1}^{n} (x^{(i)} - \mu)^2.$$

Step 1 (mean). For fixed σ^2, $\partial J/\partial\mu = -\frac{1}{\sigma^2} \sum_i (x^{(i)} - \mu) = 0$ gives $\mu = \frac{1}{n} \sum_i x^{(i)}$; J is a convex quadratic in μ, so this is the minimum. **Step 2 (variance).** With $\mu = \hat{\mu}$ and $S = \sum_i (x^{(i)} - \hat{\mu})^2$, put $v = \sigma^2$: $\partial J/\partial v = \frac{n}{2v} - \frac{S}{2v^2} = 0$ gives $v = S/n$; the derivative is negative below and positive above, so it is the minimum. In d dimensions the same two steps give the class mean and the average outer product of the centred examples:

$$\hat{\pi}_c = \frac{N_c}{N}, \qquad \hat{\mu}_c = \frac{1}{N_c} \sum_{i:\, y^{(i)}=c} x^{(i)}, \qquad \widehat{\Sigma}_c = \frac{1}{N_c} \sum_{i:\, y^{(i)}=c} (x^{(i)} - \hat{\mu}_c)(x^{(i)} - \hat{\mu}_c)^\top \quad \text{[on your sheet]}.$$

N_c versus $N_c - 1$. N_c is the maximum-likelihood denominator; $N_c - 1$ gives the unbiased sample covariance. Immaterial for large N_c, but they are different estimators: say which one you use.

One dimension by hand: posterior and boundary

Class 0: $\{0, 1, 2, 0, 1, 2\}$; class 1: $\{3, 4, 5\}$. Counts: $\hat{\pi}_0 = 6/9 = 2/3$, $\hat{\pi}_1 = 1/3$. Means: $\hat{\mu}_0 = 6/6 = 1$, $\hat{\mu}_1 = 12/3 = 4$. Variances (divide by N_c): $\hat{\sigma}_0^2 = (1 + 0 + 1 + 1 + 0 + 1)/6 = 2/3$, $\hat{\sigma}_1^2 = (1 + 0 + 1)/3 = 2/3$.

Posterior at $x = 2$. Equal variances, so the prefactors $1/\sqrt{2\pi \cdot 2/3}$ are common and drop. Class 0: $\exp[-(2-1)^2/(2 \cdot \frac{2}{3})] = e^{-0.75} = 0.472$; class 1: $\exp[-(2-4)^2/(2 \cdot \frac{2}{3})] = e^{-3} = 0.050$. Prior-weighted: $\frac{2}{3}(0.472) = 0.315$, $\frac{1}{3}(0.050) = 0.017$. Normalise: $p(y = 0 \mid x = 2) = 0.315/0.332 = 0.950$. Both the distance (one unit from class 0's centre, two from class 1's) and the larger prior push toward class 0.

Boundary. With equal variances the log prior-weighted likelihoods are $\log \hat{\pi}_c - (x - \hat{\mu}_c)^2/(2 \cdot \frac{2}{3})$. Set them equal: $\frac{3}{4}[(x-4)^2 - (x-1)^2] = \log \frac{1}{3} - \log \frac{2}{3} = -\log 2$, i.e. $\frac{3}{4}(15 - 6x) = -0.693$, so $x = (15 + 0.924)/6 = 2.65$. With equal priors the boundary would be the midpoint 2.5; the larger prior of class 0 pushes it 0.15 toward class 1. This is LDA in one dimension: a single threshold.

Generative class score; LDA; QDA

Define $s_{\hat{\phi},c}(x) = \log \hat{\pi}_c + \log p_{\hat{\theta}}(x \mid y = c)$. By Bayes' rule, $\log p_{\hat{\phi}}(y = c \mid x) = s_{\hat{\phi},c}(x) - \log \sum_k \exp[s_{\hat{\phi},k}(x)]$: the fitted posterior is the *softmax of the generative scores* [on your sheet], and prediction uses the scores alone.

- **LDA** (Linear Discriminant Analysis): every class shares one covariance $\widehat{\Sigma}$. Scores are affine in x: $s_c(x) = x^\top \widehat{\Sigma}^{-1} \hat{\mu}_c - \frac{1}{2} \hat{\mu}_c^\top \widehat{\Sigma}^{-1} \hat{\mu}_c + \log \hat{\pi}_c$ [on your sheet]. Boundaries are hyperplanes.
- **QDA** (Quadratic Discriminant Analysis): each class has its own $\widehat{\Sigma}_c$. Scores are quadratic: $s_c(x) = -\frac{1}{2}(x - \hat{\mu}_c)^\top \widehat{\Sigma}_c^{-1} (x - \hat{\mu}_c) - \frac{1}{2} \log |\widehat{\Sigma}_c| + \log \hat{\pi}_c$ [on your sheet]. Boundaries are conic sections.

Purpose. Trade the number of estimated numbers against the flexibility of the boundary.

Impact. LDA estimates $d(d+1)/2$ covariance entries once; QDA estimates them C times. The word *linear* in LDA describes the induced boundary, not the Gaussian density.

Why a shared covariance cancels the quadratic term

Take the log of the Gaussian density: $\log p(x \mid y = c) = -\frac{d}{2}\log(2\pi) - \frac{1}{2}\log|\widehat{\Sigma}| - \frac{1}{2}(x-\hat{\mu}_c)^\top \widehat{\Sigma}^{-1}(x-\hat{\mu}_c)$. With a shared covariance the first two terms are the same for every class. Expand the quadratic form, using symmetry of $\widehat{\Sigma}^{-1}$ to merge the two cross terms:

$$(x-\hat{\mu}_c)^\top \widehat{\Sigma}^{-1}(x-\hat{\mu}_c) = x^\top \widehat{\Sigma}^{-1}x - 2x^\top \widehat{\Sigma}^{-1}\hat{\mu}_c + \hat{\mu}_c^\top \widehat{\Sigma}^{-1}\hat{\mu}_c.$$

The only term quadratic in x, $x^\top \widehat{\Sigma}^{-1}x$, does not depend on c and drops out of every comparison between classes. What remains, times $-\frac{1}{2}$, plus $\log \hat{\pi}_c$, is the LDA score, affine in x; two affine functions are equal on a hyperplane. For two classes the boundary is $w^\top x + b = 0$ with $w = \Sigma^{-1}(\mu_1 - \mu_0)$ and $b = -\frac{1}{2}(\mu_1^\top \Sigma^{-1}\mu_1 - \mu_0^\top \Sigma^{-1}\mu_0) + \log\frac{\pi_1}{\pi_0}$ [on your sheet]. With separate covariances, $\log|\widehat{\Sigma}_c|$ and the whole quadratic form depend on c; equality of two scores is a quadratic equation in x (QDA).

Shared versus separate covariance: watch the boundary bend

Setup. Two-dimensional flow features (log duration, log bytes), two classes. Fit LDA and QDA on the same training rows.

Held fixed. Training rows, features, priors (the empirical ones).

What we manipulate. The covariance assumption only: one shared $\widehat{\Sigma}$ (LDA) versus two class-specific $\widehat{\Sigma}_c$ (QDA). Second run: shrink N_c of the attack class from 500 to 50 to 5.

What we measure. The fitted boundary drawn on the plane; validation error; whether $\widehat{\Sigma}_c$ is invertible.

What the plot shows. LDA: a straight line between the two class centres, tilted by $\widehat{\Sigma}^{-1}$. QDA: an ellipse, parabola or hyperbola; when one class is much more diffuse, the tighter class gets a closed region surrounded by the diffuse one. As N_c falls, QDA's boundary for the small class swings wildly from sample to sample; at $N_c = 5 < d+1$ its covariance is singular and no score can be computed.

How to read it. The curvature of the boundary is the visible fingerprint of the assumption. If LDA and QDA give similar validation error, the extra $d(d+1)/2$ parameters per class bought nothing and LDA is the safer choice. If QDA is better and stable, the clouds really do have different shapes.

Parameter count, singular covariance, shrinkage

A symmetric $d \times d$ matrix has $d + d(d-1)/2 = d(d+1)/2$ distinct entries (diagonal plus strictly upper triangle) [on your sheet]: 5050 for $d = 100$, per class under QDA. If a class has fewer than $d+1$ examples its covariance estimate has rank at most $N_c - 1$: singular, not invertible, the score cannot be computed; with slightly more it is unstable. **Shrinkage**: $\widehat{\Sigma}_\lambda = (1-\lambda)\widehat{\Sigma} + \lambda\alpha I$, $0 \le \lambda \le 1$, $\alpha > 0$ a reference variance (e.g. the average diagonal entry) [not on your sheet: know it]. $\lambda = 0$ keeps the empirical matrix; $\lambda = 1$ removes all correlations and gives every direction the same variance; for any $\lambda > 0$ the result is positive definite (positive semidefinite plus a positive multiple of I), hence invertible. λ is chosen on validation.

Purpose. Make the Gaussian route usable when N_c is small relative to d.

Impact. Blends the noisy data-specific matrix with a stable featureless one; pushed to $\lambda \to 1$ and then dropping correlations altogether leads to Naive Bayes.

Shrinkage sweep

Setup. QDA on $d = 20$ flow features, attack class with $N_c = 30$.

Held fixed. The data, the means, the priors.

What we manipulate. $\lambda \in \{0, 0.01, 0.1, 0.3, 0.6, 1\}$ in $\widehat{\Sigma}_\lambda$.

What we measure. Condition number of $\widehat{\Sigma}_\lambda$; training and validation log-loss.

What the plot shows. At $\lambda = 0$ the matrix is near-singular (huge condition number) and validation loss is terrible while training loss is excellent (the class fits its own 30 points). Validation loss falls, reaches a minimum at a moderate λ, then rises slowly toward the spherical model at $\lambda = 1$.

How to read it. The usual U-curve of Chapter 3, with λ as the capacity knob: it shrinks the *effective* number of covariance parameters without changing the model family. Pick the minimum on validation, never on training.

Credit scoring with few defaults

A bank has 40 features and 120 defaulted loans among 20,000. QDA needs $40 \cdot 41/2 = 820$ covariance numbers for the default class from 120 examples: unstable or singular. LDA pools the two classes for one covariance estimated from all 20,000 rows, at the price of assuming the default cloud has the same shape as the non-default cloud. Shrunk QDA sits in between. The usual outcome: LDA or shrunk QDA beats raw QDA on validation, and the "more flexible" model loses because it had to estimate what it could not estimate.

Choosing QDA because it is "more flexible"

Flexibility is only useful if it can be estimated. QDA multiplies the covariance parameters by C; with few examples per class the estimate is noisy (large variance in the Chapter-3 sense) or singular. It inflates training performance and deflates validation performance. Fix: compare LDA, shrunk QDA and QDA on validation; count parameters against N_c before fitting.

Logistic regression versus LDA versus QDA versus Gaussian Naive Bayes

	Logistic regression	LDA	QDA	Gaussian NB
Route	discriminative	generative	generative	generative
Modelled quantity	$p(y \mid x)$, log-odds linear	π_c, $\mathcal{N}(\mu_c, \Sigma)$ shared Σ	π_c, $\mathcal{N}(\mu_c, \Sigma_c)$	π_c, independent 1-D Gaussians per feature
Boundary	hyperplane	hyperplane	quadratic	quadratic (axis-aligned, a diagonal QDA)
Parameters	$(C-1)(d+1)$	$(C-1) + Cd + \frac{d(d+1)}{2}$	$(C-1) + Cd + C\frac{d(d+1)}{2}$	$(C-1) + 2Cd$
Data needed	more	less	most of the three generative	least
Fails when	few labels in high d; needs iterative fitting	class shapes differ; non-Gaussian clouds	$N_c < d+1$ (singular); non-Gaussian	correlated features double-count evidence

Same linear boundary for logistic regression and LDA, obtained by different commitments: LDA assumes Gaussian classes with shared covariance and gets $w = \Sigma^{-1}(\mu_1 - \mu_0)$ in closed form; logistic regression assumes only "log-odds linear" and fits w by iterative maximum likelihood.

Key insight. LDA versus QDA is a decision about how many numbers to estimate; the geometry of the boundary is the visible fingerprint of the assumption you made about the class clouds.

How the professor asks it

Q. *Two classes with equal priors, $\mu_0 = 0$, $\mu_1 = 4$, shared $\sigma^2 = 1$. Where is the boundary? Now set $\pi_0 = 0.9$, $\pi_1 = 0.1$ and recompute. Explain the direction of the move.*
A. Equal priors: the midpoint $x = 2$ (equal variances, equal priors, the log scores differ only in the squared distances). With priors 0.9/0.1: solve $-\frac{1}{2}x^2 + \log 0.9 = -\frac{1}{2}(x-4)^2 + \log 0.1$, i.e. $-\frac{1}{2}x^2 + \frac{1}{2}(x-4)^2 = \log 0.1 - \log 0.9 = -\log 9$; the left side is $-4x + 8$, so $4x - 8 = \log 9 = 2.197$ and $x = 2.55$. The boundary moves *away* from the frequent class: a point must be further toward class 1's centre before the evidence overcomes the 9:1 prior.

Q. *With $d = 10$ features and $C = 3$ classes, how many covariance parameters do QDA and LDA estimate? A class has $N_c = 5$ examples: what goes wrong and what are the two remedies?*
A. A 10×10 symmetric matrix has $10 \cdot 11/2 = 55$ entries. QDA: $3 \times 55 = 165$; LDA: 55. With $N_c = 5$ the centred examples span at most 4 directions, so $\widehat{\Sigma}_c$ has rank $\leq 4 < 10$: singular, the score needs $\widehat{\Sigma}_c^{-1}$ and cannot be computed. Remedies: shrink, $\widehat{\Sigma}_\lambda = (1-\lambda)\widehat{\Sigma}_c + \lambda\alpha I$, invertible for any $\lambda > 0$; or impose structure, shared covariance (LDA) or a diagonal one (Gaussian Naive Bayes, $2d = 20$ numbers per class).

Q. *Derive why LDA's boundary is linear although the Gaussian density is not, and name the single term whose cancellation is responsible.*
A. The log density is a constant minus $\frac{1}{2}\log|\Sigma|$ minus $\frac{1}{2}(x-\mu_c)^\top\Sigma^{-1}(x-\mu_c)$. Expanding the quadratic form gives $x^\top\Sigma^{-1}x - 2x^\top\Sigma^{-1}\mu_c + \mu_c^\top\Sigma^{-1}\mu_c$. The term $x^\top\Sigma^{-1}x$ is the only one quadratic in x, and with a *shared* Σ it is identical for every class, as is $\log|\Sigma|$; both cancel when two class scores are compared. What remains is affine in x, so the set where two scores tie is a hyperplane. If the covariances differ, neither term cancels and the tie set is quadratic (QDA).

5 Naive Bayes: independence given the class

Naive Bayes

Assume the features are **conditionally independent given the class**: $p_\theta(x \mid y = c) = \prod_{j=1}^{d} p_\theta(x_j \mid y = c)$ [on your sheet]. Taking logs the score becomes *additive*, $s_c(x) = \log\hat{\pi}_c + \sum_j \log p_{\hat{\theta}}(x_j \mid y = c)$ [on your sheet]: a ledger that starts with the prior evidence and adds one line per feature, each from a *one-dimensional* model of that feature within class c.

Purpose. Replace one hopeless d-dimensional density estimate by d easy one-dimensional ones.

Impact. Stable with little data; each feature may get its own family (Gaussian for a measurement, Gamma or log-normal for a skewed positive quantity, Poisson for a count, histogram for an irregular one). Two facts make it far more useful than the assumption suggests: a *misspecified density can still rank classes correctly* (getting the largest score right is weaker than getting every density right), and the one-dimensional fits are cheap. The main risk: correlated features *repeat the same evidence*; the classifier may still rank well while its probabilities become too extreme.

The duplicated-feature experiment: ranking survives, calibration dies

Setup. Gaussian Naive Bayes on flow data with features (duration, packets, bytes).

Held fixed. Training and validation rows; the prior; the decision threshold 0.5.

What we manipulate. Add a copy of the "bytes" column (or a feature that is 0.98-correlated with it, such as bytes per packet times packets) once, twice, five times.

What we measure. Validation accuracy; a reliability diagram (among flows given probability

u, the observed fraction of attacks); the average of $\max_c \hat{p}(y = c \mid x)$.

What the plot shows. Accuracy barely moves: the ranking is driven by the same information. The reliability curve bends into an S: predictions of 0.7 turn into 0.9, then 0.99, with the observed fraction unchanged. The mean top probability climbs toward 1.

How to read it. Each copy adds the same log-likelihood term again, so the evidence is counted k times and the log-odds are multiplied by k. The model becomes overconfident, which matters for cost thresholds, soft voting (Chapter 5) and any probability-based policy. Fix: remove redundant features or use a model that sees correlations (LDA, logistic regression).

Gaussian Naive Bayes and variance smoothing

For numerical features, $x_j \mid (y = c) \sim \mathcal{N}(\hat{\mu}_{cj}, \hat{\sigma}^2_{cj})$ with the one-dimensional MLEs feature by feature. Two problems: a fitted variance of 0 (all class-c values of feature j coincide) makes the density undefined; a merely tiny variance lets one feature's narrow density dominate the whole score. **Variance smoothing**: $\tilde{\sigma}^2_{cj} = \hat{\sigma}^2_{cj} + \lambda s^2_j$ with $\lambda > 0$ and s^2_j a reference scale such as the overall training variance of feature j [not on your sheet: know it]. Small λ prevents instability; if its value changes predictions, select it on validation.

Purpose. Keep one feature from vetoing the class.

Impact. Example: class c has $\hat{\sigma}^2 = 1$ for feature 1 and 10^{-6} for feature 2 (every training example of class c had the same port number). A test point whose feature 2 differs by 0.01 contributes $-\frac{1}{2}(0.01)^2/10^{-6} = -50$ to the score, dwarfing anything feature 1 can say. With $\lambda s^2_2 = 0.01$ added, the contribution is $-\frac{1}{2}(10^{-4})/0.01 \approx -0.005$: feature 2 still counts, but no longer vetoes.

Naive Bayes variants, by feature type

Variant	Feature model	Typical use
Gaussian	continuous Gaussian per feature	measurements, flow statistics
Bernoulli	binary presence/absence	indicators (has link, from known sender)
Multinomial	nonnegative event or token counts	documents, event counts
Hybrid	distribution chosen per feature type	mixed tabular data

A hybrid model multiplies contributions from different feature-specific distributions under *one* class prior; the prior appears once in the score, never once per feature.

Multinomial Naive Bayes for text

Fix a vocabulary of V words; a document is its count vector $x = (x_1, \ldots, x_V)$. For class c let $\theta_{cj} = p(\text{token} = j \mid y = c)$, $\sum_j \theta_{cj} = 1$. Ignoring the multinomial coefficient (same for every class, cancels in Bayes' rule), $p_\theta(x \mid y = c) \propto \prod_{j=1}^{V} \theta_{cj}^{x_j}$ [on your sheet] with $0^0 = 1$. Class c is a bag of words with proportions $\theta_{c\cdot}$; a document is produced by drawing words from the bag. "free free prize" with vocabulary (free, win, prize, meeting, project, report) has $x = (2, 0, 1, 0, 0, 0)$ and likelihood $\propto \theta^2_{c,\text{free}}\theta_{c,\text{prize}}$: the same word twice is twice the evidence; absent words contribute the factor 1; word order is ignored.

Purpose. The spam-filter model.

Impact. Everything is a count; estimation is counting; the only subtlety is a count of zero.

Zero counts, additive smoothing, and the "free meeting" numbers

Training counts, seven tokens per class:

	free	win	prize	meeting	project	report
spam	3	2	2	0	0	0
ham	0	0	0	3	2	2

Without smoothing $\hat{\theta}_{cj} = n_{cj}/7$ and "free meeting" gives $p(x \mid \text{spam}) \propto \frac{3}{7} \cdot \frac{0}{7} = 0$, $p(x \mid \text{ham}) \propto \frac{0}{7} \cdot \frac{3}{7} = 0$: one unseen word zeroes an entire product; here both vanish and the posterior is 0/0. In general a single word never seen in a class vetoes that class regardless of the other evidence.
Additive smoothing [on your sheet]:

$$\hat{\theta}_{cj} = \frac{n_{cj} + \alpha}{\sum_{\ell} n_{c\ell} + \alpha V}, \qquad p(x_j = v \mid y = c) = \frac{n_{cjv} + \alpha}{n_{cj} + \alpha K_j} \text{ for a categorical feature with } K_j \text{ levels.}$$

Properties, each one line: numerators are $\geq \alpha > 0$ and summing $n_{cj} + \alpha$ over j gives exactly the denominator, so the estimates are positive and sum to one; as $\alpha \to 0$ they tend to the raw frequencies; dividing numerator and denominator by α, as $\alpha \to \infty$ they tend to $1/V$ (uniform). The estimate is what you get by adding α imaginary occurrences of every word to every class.
With $\alpha = 1$**,** $V = 6$ (Laplace): spam counts $(4, 3, 3, 1, 1, 1)$ over $7 + 6 = 13$, ham $(1, 1, 1, 4, 3, 3)$ over 13. Equal priors $\frac{1}{2}$:

- "free meeting": spam $\frac{1}{2} \cdot \frac{4}{13} \cdot \frac{1}{13} = \frac{2}{169}$, ham $\frac{1}{2} \cdot \frac{1}{13} \cdot \frac{4}{13} = \frac{2}{169}$; posterior $\frac{1}{2}$, the evidence balances exactly.
- "free prize": spam $\frac{1}{2} \cdot \frac{4}{13} \cdot \frac{3}{13} = \frac{12}{338}$, ham $\frac{1}{2} \cdot \frac{1}{13} \cdot \frac{1}{13} = \frac{1}{338}$; $p(\text{spam} \mid x) = 12/13 = 0.92$.
- "free free prize": spam $\frac{1}{2}(\frac{4}{13})^2 \frac{3}{13} = \frac{48}{4394}$, ham $\frac{1}{2}(\frac{1}{13})^3 = \frac{1}{4394}$; $p(\text{spam} \mid x) = 48/49 = 0.98$. The repeated word counts twice, as the multinomial says it should.

The α sweep

Setup. Multinomial Naive Bayes on 2,000 labelled emails, vocabulary $V = 10{,}000$.

Held fixed. Training and validation emails, the vocabulary, the priors.

What we manipulate. $\alpha \in \{0, 10^{-3}, 10^{-2}, 0.1, 1, 10, 100\}$.

What we measure. Validation log-loss and accuracy; the fraction of validation emails containing at least one word unseen in one class.

What the plot shows. At $\alpha = 0$ log-loss is infinite (zero probabilities) and accuracy is erratic: every unseen word vetoes a class. Log-loss drops steeply for tiny α, bottoms out around α between 0.1 and 1, then rises as α pulls every $\hat{\theta}_{cj}$ toward $1/V$ and the classes stop being distinguishable (at $\alpha = 100$ with counts $(5, 0, 3, 2)$, $V = 4$, the estimates are $(0.256, 0.244, 0.251, 0.249)$, nearly uniform).

How to read it. α is a capacity knob in disguise: small α trusts the counts (high variance, vetoes), large α trusts the uniform prior (high bias). Choose it on validation. The unseen-word fraction tells you *why* $\alpha = 0$ fails: it is a property of the vocabulary size against the training size, not of the classifier.

Unseen categories

A categorical value that appears at validation or test time but never in training (a new port number) must have a defined representation: create an *other* level during preprocessing, count it among the K_j levels, map every unseen value to it. Smoothing then gives it a small positive probability instead of a crash.

Purpose. Make the pipeline robust to deployment novelty at the feature level.

Impact. Without it, the model cannot even compute a score for a new value.

Spam at day one versus spam at scale

200 labelled emails, 10,000-word vocabulary. Multinomial Naive Bayes estimates $2 \times 10{,}000$ smoothed word probabilities from simple counts and works reasonably from day one. Logistic regression on the same counts must locate a 10,000-dimensional hyperplane from 200 points: badly underdetermined, it overfits. With 2 million labelled emails the picture flips: logistic regression (or a network) learns interactions such as "*free* is spammy unless *invoice* is nearby", which the independence assumption forbids Naive Bayes from ever representing.

The three hand-computation slips

(1) Counting the prior once per feature instead of once per class, which inflates the prior's weight by a factor d. (2) Smoothing after normalising instead of before: add α to every count and αV to the total, then divide. (3) Multiplying raw probabilities for many features and underflowing or mis-multiplying: work in logs when there are more than three or four features, compare unnormalised scores first, normalise only if a posterior is asked for.

Gaussian NB versus LDA, and the hidden QDA

- Gaussian NB with class-specific per-feature variances is a *diagonal QDA*: scores quadratic in x, axis-aligned ellipses, $2d$ numbers per class. So its boundary can curve even though each feature is Gaussian.
- LDA with a diagonal shared covariance is a diagonal LDA: linear boundary, d variances in total.
- Full LDA sees correlations and never double-counts; NB ignores them and may, but needs far less data.

Key insight. Naive Bayes trades one strong assumption for stability; its ranking often survives the assumption being false, its probabilities do not.

How the professor asks it

Q. *Two features (has link, from known sender). Spam rates* $(0.9, 0.1)$*, ham rates* $(0.2, 0.8)$*, priors* 0.5 *each. Classify an email that has a link and comes from a known sender, and give the posterior. Then explain in one sentence what Bernoulli Naive Bayes assumed to let you multiply.*
A. Ham, with posterior 0.64. Spam: $0.5 \times 0.9 \times 0.1 = 0.045$; ham: $0.5 \times 0.2 \times 0.8 = 0.08$; normalise, $0.08/0.125 = 0.64$. The multiplication assumes that, within each class, having a link and coming from a known sender are independent events, so the class-conditional probability of the pair is the product of the two marginals.

Q. *Vocabulary* $V = 4$*, class counts* $(5, 0, 3, 2)$*. Give the smoothed probabilities for* $\alpha = 1$ *and say what happens as* $\alpha \to \infty$ *and why that is a bias, not an error.*
A. $\alpha = 1$: $(6, 1, 4, 3)/14$. As $\alpha \to \infty$ every estimate tends to $1/4$ regardless of the counts ($\alpha = 100$ already gives $(0.256, 0.244, 0.251, 0.249)$): the class models become identical and the classifier returns the prior. It is bias in the Chapter-3 sense: a systematic pull toward uniform that no amount of data in the counts can undo at that α; the correct response is to choose α on validation, where the pull is traded against the zero-count variance.

Q. *Why can a Gaussian Naive Bayes boundary be curved when every feature is Gaussian? Which assumption would make it straight?*
A. Because each class has its own variance per feature, the score $\log \pi_c - \sum_j [(x_j - \mu_{cj})^2/(2\sigma_{cj}^2) + \frac{1}{2}\log \sigma_{cj}^2]$ is quadratic in x with class-dependent coefficients: a diagonal QDA, so the tie set is quadratic. Sharing the variances across classes ($\sigma_{cj}^2 = \sigma_j^2$) makes the x_j^2 terms cancel, as in LDA, and the boundary becomes a hyperplane.

6 Comparing the routes: the data-size experiment

What decides between the routes

Generative structure can *reduce estimation error*: a restricted description of each class has few parameters and is fitted reliably from little data, provided the assumptions are approximately right. A wrong class-conditional family creates *approximation bias* that does not vanish with more data. A flexible discriminative model may reach lower error eventually because it spends no capacity on aspects of X irrelevant to Y.

Purpose. Turn "which is better" into "which assumptions are credible, and what is the deployment objective".

Impact. The expected picture, error versus N: generative lower at small N, discriminative lower at large N. Not a law: the crossing point, and whether it exists, depends on the distribution, the two families, regularisation and optimisation.

Learning curves of Naive Bayes versus logistic regression as N grows

Setup. Spam data, vocabulary V fixed, a fixed test set of 20,000 emails never used for anything else.

Held fixed. The features, the test set, the two model families, the smoothing α and the regularisation λ (each chosen once on validation at a moderate N).

What we manipulate. The number of labelled training emails $N \in \{50, 200, 1{,}000, 10{,}000, 100{,}000, 2\,\text{million}\}$, on a log axis; for each N repeat with 10 random training subsets to see variability.

What we measure. Test error of each model at each N, mean and spread across the 10 subsets.

What the plot shows. Two decreasing curves. Naive Bayes starts low and flattens early at a floor (its approximation bias: the independence assumption cannot represent interactions). Logistic regression starts high, with wide spread (underdetermined), falls steadily and crosses below Naive Bayes somewhere between 10^3 and 10^5, then keeps improving slowly.

How to read it. Where the curves cross is *the* fact to report: left of it, the generative assumptions are worth more than their bias; right of it, data has replaced assumptions. A flat generative curve with a gap to the discriminative one is approximation error (a better class model, not more data, is the fix). Wide spread at small N is estimation error (variance). If the curves never cross, either the assumptions are nearly true or N never got large enough.

Settling the choice by slogan

"Generative is for small data", "discriminative is always better": both are tendencies, not theorems. People choose by the label and skip the validation. What it produces: a Naive Bayes stuck at its bias floor when millions of labels were available, or a 10,000-dimensional logistic regression fitted to 200 rows. Fix: ask the three questions (credible class model? prediction with many labels? raw high-dimensional input?) and then *compare under deployment-like validation*, same split, same rows.

The three model-choice questions

1. *Can a credible class-conditional model $p_\theta(x \mid y)$ be stated?* If yes, its structure may reduce the labelled data required.
2. *Is the goal accurate prediction with many labelled examples?* A flexible discriminative model avoids unnecessary assumptions on the inputs.
3. *Is the raw input complex and high-dimensional?* Learn a representation first, then compare

simple classification heads; that is more credible than modelling the raw density.

Deployment-like validation decides between credible alternatives; the labels "generative" and "discriminative" do not replace empirical comparison.

Key insight. The crossing point of the two learning curves is where data has bought enough to replace assumptions; left of it structure wins, right of it flexibility wins.

How the professor asks it

Q. *Describe an experiment that shows the generative route beating the discriminative one at small N and losing at large N, and say which curve shape corresponds to approximation error and which to estimation error.*
A. Plot test error against the number of labelled training examples on a log axis for Naive Bayes and logistic regression, same features, same fixed test set, repeated over random training subsets. Generative: lower at small N, flattening at a floor; discriminative: higher and widely spread at small N, falling and crossing below. The flat floor that more data does not lower is approximation error (the family cannot express the truth); the spread across subsets that shrinks as N grows is estimation error (variance). The crossing exists only if the generative assumptions are wrong enough to leave a floor above what the discriminative model can reach.

Q. *A colleague says a discriminative model "cannot be worse than a generative one on the same features because it makes fewer assumptions". Give the counter-argument.*
A. Fewer assumptions mean more to estimate from the same data: with 200 emails and 10,000 features, logistic regression must locate a hyperplane from far too few points and overfits, while multinomial Naive Bayes estimates $2V$ smoothed frequencies from counts and works. Assumptions are a form of prior information; when they are approximately true they lower variance more than they add bias. The claim is only right asymptotically, when data has made the assumptions unnecessary.

7 Calibration, class imbalance and prior shift

Calibration

A probabilistic classifier is **calibrated** if, among cases assigned probability u, approximately a fraction u are positive. Calibration concerns the *interpretation* of probabilities; a decision threshold from costs concerns the *action*. Separate questions.

Purpose. Know whether "0.8" means eight out of ten.

Impact. Every formula that uses the probability as a probability (cost threshold, prior correction, soft voting) silently assumes calibration. Naive Bayes with correlated features is a standard example of a well-ranking, badly-calibrated model.

Class imbalance versus class-prior shift

Imbalance: one class is rare, and training and deployment share the same distribution. Not an error; the posterior *should* reflect the prevalence. The only problem is finite minority data: with few positives the minority distribution or the boundary is poorly estimated, and weighting or resampling may help estimation. Otherwise its benefit is limited and the probabilities may need recalibration. **Prior shift**: between training (t) and deployment (d) the class-conditionals are unchanged but the priors differ, $p_d(x \mid y = c) = p_t(x \mid y = c)$, $\pi_{d,c} \neq \pi_{t,c}$ [not on your sheet: know it]. Example: attacks are 20% of the training data but 5% of deployment traffic.

Purpose. Two situations constantly confused; one needs no correction, the other needs one even

with abundant data.

Impact. Imbalance is an estimation issue; shift is a modelling mismatch that the formulas below repair exactly.

Generative correction: swap the priors

At deployment Bayes' rule uses π_d and $p_d(x \mid y)$; under prior shift the latter equals $p_t(x \mid y)$, which is what the fitted $p_{\hat{\theta}}(x \mid y)$ estimates. Substituting,

$$p_d(y = c \mid x) = \frac{\pi_{d,c}\, p_{\hat{\theta}}(x \mid y = c)}{\sum_k \pi_{d,k}\, p_{\hat{\theta}}(x \mid y = k)} \quad \text{[not on your sheet: know it]}.$$

No class-conditional model is refitted: the prior is an explicit, separately estimated component of the generative model, so you replace it.

Discriminative correction: reweight the odds

Let $r_t(x) = p_t(y = 1 \mid x)$, $r_d(x) = p_d(y = 1 \mid x)$, $\pi_t = p_t(y = 1)$, $\pi_d = p_d(y = 1)$. **Step 1 (odds form).** At training, $\dfrac{r_t}{1 - r_t} = \dfrac{\pi_t\, p(x \mid y = 1)}{(1 - \pi_t)\, p(x \mid y = 0)}$ (divide the two Bayes numerators; the denominator cancels). Likewise at deployment with π_d. **Step 2 (shared likelihood ratio).** Under prior shift $p(x \mid y = 1)/p(x \mid y = 0)$ is the same at both times. Solve the training odds for it and substitute into the deployment odds:

$$\frac{r_d}{1 - r_d} = \frac{\pi_d/(1 - \pi_d)}{\pi_t/(1 - \pi_t)} \cdot \frac{r_t}{1 - r_t}.$$

Step 3 (back to a probability). If odds $= A/B$ then probability $= A/(A+B)$. With $A = \frac{\pi_d}{\pi_t} r_t$ and $B = \frac{1-\pi_d}{1-\pi_t}(1 - r_t)$,

$$r_d(x) = \frac{(\pi_d/\pi_t)\, r_t(x)}{(\pi_d/\pi_t)\, r_t(x) + \frac{1-\pi_d}{1-\pi_t}\,[1 - r_t(x)]} \quad \text{[not on your sheet: know it]}.$$

Reading. Multiply the training probability of class 1 by "how much rarer class 1 became", the probability of class 0 by "how much commoner class 0 became", renormalise. Attacks 20% → 5%: $\pi_d/\pi_t = 0.25$, $(1 - \pi_d)/(1 - \pi_t) = 0.95/0.80 = 1.1875$. A flow with $r_t = 0.5$: $A = 0.125$, $B = 0.59375$, $r_d = 0.125/0.71875 = 0.17$. A training coin flip is a 17% attack probability in deployment. Other values: $r_t = 0.2 \to 0.05$, $r_t = 0.8 \to 0.46$, $r_t = 0.9 \to 0.65$. The correction assumes r_t is a *calibrated* training posterior.

The prior-shift experiment: train at 20%, deploy at 5%

Setup. Build a training set with 20% attacks and a deployment-like test set with 5% attacks by subsampling attacks from the same pool (so the class-conditionals are identical by construction). Fit Gaussian Naive Bayes and logistic regression on the training set.

Held fixed. The fitted class-conditionals and the fitted logistic model; the test rows; the cost threshold.

What we manipulate. Whether the correction is applied: (a) none; (b) generative: replace $\hat{\pi} = (0.8, 0.2)$ by $(0.95, 0.05)$; (c) discriminative: reweight the odds with $\pi_d/\pi_t = 0.25$ and 1.1875.

What we measure. On the test set: log-loss, reliability diagram, false-positive rate and recall at the cost threshold, number of alarms.

What the plot shows. Uncorrected models are overconfident about attacks (reliability curve below the diagonal: predicted 0.5, observed 0.17) and raise about four times too many alarms.

After correction the reliability curve returns to the diagonal, log-loss falls, alarms drop to the level the costs intended, recall falls slightly.

How to read it. Accuracy may barely move, which is why people miss the problem: the *ranking* is unchanged by a monotone odds reweighting; what changes is every decision that reads the probability as a probability. Also note that the correction is exact for the generative model and only as good as the calibration of r_t for the discriminative one.

Fraud after a marketing campaign

A fraud model was trained on a month where 2% of transactions were fraudulent. A campaign triples legitimate volume; fraud stays constant, so the deployment prior is about 0.7%. The transaction patterns of fraud and of legitimate purchases did not change: pure prior shift. Without correction, the analysts see the same number of alarms per fraud but three times as many alarms per day and a precision that collapsed. Swapping the prior (generative) or reweighting the odds (discriminative) restores the intended precision; retraining is unnecessary and would waste the month's labels.

Resampling the training set and then trusting the probabilities

Oversampling the minority or undersampling the majority *changes the effective prior*: a model trained on a balanced resample outputs posteriors for $\pi = 0.5$, not for the real prevalence. The probabilities are inflated for the rare class by the odds factor $\frac{0.5/0.5}{\pi/(1-\pi)}$; alarm counts and precision are wrong. Tempting because balanced data "looks fair" and accuracy on the resample looks good. Fix: either keep the natural prior with enough minority data, or resample and correct back with the odds formula (π_t = resampled prior, π_d = true prior); and always apply the prior correction *before* the cost threshold, never after.

Imbalance versus shift; calibration versus threshold

	Imbalance	Prior shift
What is true	same distribution at training and deployment, one class rare	class-conditionals equal, priors differ
What is manipulated	the number of minority examples (weights, resampling)	the prior, at prediction time
Needs a correction?	no, with enough minority data; weighting helps only estimation	yes, always, even with abundant data
What goes wrong otherwise	noisy boundary, poor minority recall	systematically miscalibrated probabilities, wrong alarm volume

Calibration is about whether 0.8 means eight in ten (a property of the model); the **threshold** is about which action to take at 0.8 (a property of the costs). Correct the prior first, then threshold.

Key insight. Imbalance is an estimation problem and shift is a modelling mismatch; the generative route corrects shift by replacing one component, the discriminative route only by reweighting odds it must first have calibrated.

How the professor asks it

Q. *A detector was trained with* 20% *attacks; at deployment* 5% *of flows are attacks and attack behaviour is unchanged. A flow receives training posterior* 0.5. *Derive the deployment posterior and interpret the result. What must be true of the training posterior for this to be exact?*
A. Deployment posterior 0.17. Odds reweighting: posterior odds equal prior odds times the likelihood ratio, and the likelihood ratio is the same at both times under prior shift, so the

deployment odds are the training odds times $\frac{\pi_d/(1-\pi_d)}{\pi_t/(1-\pi_t)} = \frac{0.05/0.95}{0.20/0.80} = 0.2105$. Training odds 1; deployment odds 0.2105; probability $0.2105/1.2105 = 0.17$. Equivalently $A = 0.25 \times 0.5 = 0.125$, $B = 1.1875 \times 0.5 = 0.594$, $A/(A+B) = 0.17$. Interpretation: a training coin flip is an unlikely attack in deployment because attacks are four times rarer there. Exactness requires r_t to be calibrated: the formula transforms a true training posterior, not an arbitrary score.

Q. *Explain why oversampling the attack class to* 50% *and then using the model's probabilities with a cost threshold gives the wrong number of alarms, and how to fix it without retraining.*
A. The resampled training set has prior 0.5, so the fitted posterior is $p(y = 1 \mid x)$ under $\pi_t = 0.5$; at deployment the prior is the true π_d, so every attack probability is inflated by the odds factor $\frac{\pi_d/(1-\pi_d)}{1}$ being ignored. The cost threshold then fires far too often. Fix: apply the discriminative correction with $\pi_t = 0.5$ and π_d the true prevalence (multiply class-1 probability by $\pi_d/0.5$, class-0 by $(1-\pi_d)/0.5$, renormalise), then threshold. No refit is needed; the ranking was never the problem.

Q. *Give one situation where class imbalance should be left alone and one where it should be addressed, and say what "addressed" changes.*
A. Leave alone: 1% attacks but 50,000 labelled attack flows, same distribution at deployment; the posterior correctly reflects the prevalence and the boundary is well estimated. Address: 1% attacks with 40 labelled attack flows; the minority distribution is too poorly estimated to locate the boundary, so weighting or resampling reduces estimation variance. "Addressed" changes the effective prior and therefore the calibration, which must then be corrected back; it does not change what the true posterior is.

8 Unfamiliar inputs, combined systems, language models

Reject option via a compatibility check

Choose the most probable known class $c^* = \arg\max_c p_{\hat{\phi}}(y = c \mid x)$, then compare the fitted class likelihood with a class-specific threshold:

$$\hat{y} = \begin{cases} c^* & \text{if } \log p_{\hat{\theta}}(x \mid y = c^*) \geq \tau_{c^*}, \\ \text{reject} & \text{otherwise} \end{cases} \quad \text{[not on your sheet: know it].}$$

Two questions instead of one: *which known class fits best* (the posterior) and *does even the best class fit well enough* (the likelihood against a threshold). Only a generative model can answer the second, because only it has a notion of what inputs of class c look like.

Purpose. Let the system say "I do not recognise this".

Impact. A new attack mechanism far from every known class gets rejected or sent to review instead of a confident wrong label. Caveats: τ_c must be validated on data that reproduce the intended novelty (attack mechanisms excluded from training); a low likelihood is evidence of incompatibility, not a guarantee of detection; high-dimensional likelihoods behave counterintuitively and a misspecified class model can give low likelihood to valid cases.

Held-out-mechanism experiment for the reject threshold

Setup. Flow data with six attack families. Train a generative classifier on five families plus benign.

Held fixed. The trained class models and priors; the posterior-based decision.

What we manipulate. The threshold τ_c on $\log p(x \mid y = c^*)$, swept from very low (never reject) to high (reject most).

What we measure. On validation: rejection rate on known-family flows (false rejects) and on

the sixth, held-out family (true rejects); the known-class accuracy of the non-rejected flows.

What the plot shows. A trade-off curve: as τ rises, true rejects of the novel family climb toward one while false rejects of known flows climb more slowly; accuracy on the accepted flows rises slightly because the uncertain ones are gone.

How to read it. The sixth family is the novelty the deployment will face; a threshold tuned without it would be tuned to the wrong question. Compare with the softmax-only system, whose top probability on the novel family is high and gives no handle at all. The threshold is a hyperparameter: pick it on validation, report on an untouched test family if you have one.

Combined systems

A discriminative model ranks the known classes, $p_{\hat{\theta}_d}(y = c \mid x)$. A generative or anomaly model computes $\log p_{\hat{\theta}_g}(x \mid y = c)$ for the predicted class. The final policy accepts the known class, requests human review, or rejects the input as unfamiliar. The discriminative model is good at the ranking; the generative model is the plausibility check it lacks.

Purpose. Get the best of both routes.

Impact. Three outcomes (accept, review, reject) instead of a forced choice among known classes.

Generative language models used for classification

An autoregressive language model is trained generatively by factorising a token sequence, $p_\theta(w_{1:T}) = \prod_{t=1}^{T} p_\theta(w_t \mid w_{<t})$, with next-token prediction as the pretraining objective. A *sequence-classification head* instead models $p_\psi(y \mid w_{1:T})$ and minimises the discriminative objective $-\sum_i \log p_\psi(y^{(i)} \mid w^{(i)}_{1:T_i})$: one label for the entire sequence, not one per token. Alternatively a prompt asks the model to generate or score label tokens directly.

Purpose. See both routes inside one modern system.

Impact. Generative pretraining learns the representation; a discriminative objective learns the label; the two words describe the direction of each objective, not the architecture.

Lab 2 at a glance

The lab compares the routes on cybersecurity flow data: Gaussian, multinomial and hybrid Naive Bayes for numerical, service and mixed feature sets, against logistic regression and a decision tree; then limited training data (20% of each class), deployment prior shift ($\pi_d = 0.05$, corrected by swapping priors for the generative models and by odds reweighting for the discriminative ones), and multiclass classification with and without destination-port information. Every model uses the same fixed stratified split and the same rows, so that differences reflect the models rather than the data.

Validating the reject threshold on the known classes only

Tempting because that is the data at hand. It produces a threshold that rejects a comfortable fraction of known flows and tells you nothing about novel ones; at deployment the new attack family sails through with a confident label. Fix: hold out an entire mechanism or family for validation of τ, the grouped-split idea of Chapter 3 applied to classes.

Softmax confidence versus likelihood check

- **Top softmax probability**: relative fit among the known classes; normalised to one, so it cannot be low for every class at once; available from any classifier; high on novel inputs.
- **Class likelihood** $\log p(x \mid y = c^*)$: absolute compatibility of the input with the best class; can be low for all classes; needs a generative or anomaly model; thresholded with a

validated τ_c.

- **Use**: softmax to rank, likelihood to decide whether to trust the ranking at all.

Key insight. Softmax answers "which of my classes is best"; the likelihood answers "is any of them any good"; a reject option needs the second question and only a generative model can ask it.

How the professor asks it

Q. *A three-class network outputs* $(0.96, 0.03, 0.01)$ *on an input that belongs to none of its classes. Explain why this is not a bug of the training, and what you would add to the system to catch it.*
A. Not a bug: the cross-entropy objective only ever asked the network to explain the labels given the inputs, so its softmax compares the three known classes and nothing else; 0.96 says "cat explains this far better than dog or horse", which can be true of a toaster. The number measures relative, not absolute, fit. Add a compatibility check: a generative or anomaly model giving $\log p(x \mid y = c^*)$, compared with a threshold validated on held-out novel inputs; reject or send to review when the best class fits badly.

Q. *Why can the reject option (the compatibility check) not be implemented with a discriminative classifier alone?*
A. Because the check needs $p(x \mid y = c)$, a model of what inputs of class c look like, and a discriminative classifier never estimates any distribution over x: it models $p(y \mid x)$ only, and its outputs are normalised across classes, so "all classes fit badly" is invisible to it. Thresholding the top softmax probability is not a substitute, since that probability can be near one on unfamiliar inputs.

Coverage of the textbook chapter

- **Section 1:** symbol table; two routes; decision known, posterior not; definition of discriminative and generative; where each route shows up; "generative" is a direction, not a task.
- **Section 2:** scores, sigmoid, logistic regression; sigmoid properties; theorem on decision boundaries and the cost shift; softmax and its facts; cross-entropy equals conditional maximum likelihood; deep classifiers; what the objective does not provide; confident about nonsense; the imbalance remark; the technique box on scores and probabilities.
- **Section 3:** generative story; Bayes' rule and the rare-attack table; the likelihood splits by class; the MLE prior via Gibbs; shape error versus finite-sample error.
- **Section 4:** Gaussian model; Gaussian MLE with N_c versus $N_c - 1$; the one-dimensional example (posterior 0.950, boundary 2.65); generative score; LDA theorem; QDA; covariance parameter count; shrinkage; the by-hand technique; the classics (boundary at 2 and 2.55; 165 and 55 parameters; singular covariance).
- **Section 5:** Naive Bayes; Gaussian NB and variance smoothing; the variants table; multinomial NB; zero counts; additive smoothing and its properties; "free meeting", "free prize", "free free prize"; unseen categories; the by-hand technique; the classics (smoothing with $\alpha = 1$ and 100, Bernoulli 0.64, curved NB boundary).
- **Section 6:** the comparison table; the interpretation; 200 versus 2 million emails; the three model-choice questions.
- **Section 7:** calibration; imbalance; prior shift; generative correction; discriminative correction with the $0.5 \to 0.17$ numbers; the gotchas (imbalance is not shift, resampling changes the prior, correction before threshold).
- **Section 8:** confidence is not familiarity; the reject option; combined systems; language models; Lab 2.

The ten mistakes of this chapter

1. Reading $\max_c p(y = c \mid x)$ as "this input is normal". Softmax compares known classes only.
2. Using the prior as if it were the posterior ("attacks are 1%, so benign"). Posterior odds are prior odds times the likelihood ratio.
3. Believing costs bend the boundary. They slide it along the score axis; only the score family bends it.
4. Counting the prior once per feature in a Naive Bayes hand computation. Once per class.
5. Smoothing after normalising, or not at all. Add α to every count and αV to the total, then divide; $\alpha = 0$ lets one unseen word veto a class.
6. Picking QDA "because it is more flexible" with N_c close to d. Singular or unstable covariance; count $d(d+1)/2$ per class first; shrink or share.
7. Saying LDA is linear because the Gaussian is linear. It is linear because the shared $x^\top \Sigma^{-1} x$ cancels between classes.
8. Treating imbalance and prior shift as the same problem. Imbalance with enough data needs nothing; shift needs a correction even with abundant data.
9. Resampling to 50/50 and then trusting the probabilities or alarm counts. The effective prior changed; correct the odds back, then threshold.
10. Settling generative versus discriminative by slogan instead of by the learning-curve experiment under deployment-like validation.

Chapter 4 in ten lines

1. The decision rule is known from Chapter 2; the posterior must be learned. Two routes: model $p(y \mid x)$ directly, or model $p(x \mid y)$ and π and apply Bayes' rule.
2. Discriminative: score, sigmoid or softmax, cross-entropy. Log-odds equal the score; the boundary is $s = \log(C_{\mathrm{FP}}/C_{\mathrm{FN}})$; costs translate it, the score family shapes it.
3. Cross-entropy minimisation is conditional maximum likelihood: the labels given the inputs, never the inputs.
4. Generative: joint $= \pi_c\, p(x \mid y = c)$; posterior odds = prior odds $\times$ likelihood ratio; the likelihood splits by class; the MLE prior is the class frequency (Gibbs).
5. Gaussian classes: closed-form means and covariances (denominator N_c); LDA shares Σ and gets hyperplanes because $x^\top \Sigma^{-1} x$ cancels; QDA gets conics; a covariance costs $d(d+1)/2$; shrink when N_c is small.
6. Naive Bayes: independence given the class, additive score, one 1-D model per feature; ranking survives misspecification, calibration does not; smooth variances (λs_j^2) and counts (α).
7. Generative wins at small N if its assumptions are credible, discriminative wins at large N; the crossing of the two learning curves is the evidence; a flat floor is approximation error.
8. Imbalance is an estimation issue; prior shift is a modelling mismatch: swap priors (generative) or reweight the odds by π_d/π_t and $(1-\pi_d)/(1-\pi_t)$ (discriminative, needs calibration); resampling changes the effective prior.
9. Calibration is about what 0.8 means; the threshold is about what to do at 0.8. Correct the prior first, then threshold.
10. Confidence is not familiarity: add a likelihood-based reject option validated on held-out novelty; combine a discriminative ranker with a generative plausibility check; language models pair generative pretraining with a discriminative head.

Understanding ML, Chapter 5: Learning by Combining Models

After reading this you can explain why averaging helps and when it cannot, what bagging, random forests, boosting, XGBoost, stacking and mixtures of experts each manipulate, which experiment reveals each effect, and what goes wrong in practice

Fundamentals of Machine Learning, midterm

2026-11-03

What this chapter is about, in one paragraph

Chapters 3 and 4 fitted *one* model and worried about its bias, its variance and its posterior. This chapter fits *many* models and combines them. The reason is the variance term of Chapter 3: a flexible model such as a deep tree reflects the particular sample it was trained on, so a different sample would give a different fit. Averaging several fits cancels the part of the error that differs between them and keeps the part they share. Every method here is a recipe for producing competent models whose errors are as different as possible, plus a rule for combining them. *Voting* combines different algorithms on the same data. *Bagging* gives the same algorithm different bootstrap samples; *random forests* additionally hide features at each split to decorrelate the trees. *Boosting* does something else: it builds models one after another, each one fitted to what the current ensemble still gets wrong, so it attacks the remaining loss (mostly bias) rather than the variance; *XGBoost* is boosting with one regularised second-order objective used for both the tree shape and the leaf values. *Stacking* learns the combination rule from data, and a *mixture of experts* learns a combination that depends on the input. The knobs you manipulate are the number of models M, the correlation between their errors ρ (through bootstrap samples and m_{try}), the tree size, and for boosting the learning rate η, the depth and the number of rounds, with λ and γ as regularisers. Everything you need to compute is on your sheet; what the exam tests is whether you know what each knob does, which experiment shows it, and what breaks.

Contents

1 Why combine models at all **2**

2 Averaging, voting, and how much variance an average removes **4**

3 Decision trees, the base learner **6**

4 Bagging and out-of-bag evaluation **8**

5 Random forests **11**

6 Feature importance **13**

7 Boosting for regression **14**

8 Boosting for binary classification **17**

9 XGBoost: one objective for the tree shape and the leaf values **19**

10 Multi-class boosting, AdaBoost and the libraries **21**

11 Bagging versus boosting, and when ensembles do not help **23**

12 Stacking **24**

13 Mixture of experts **25**

14 Where ensembles are used, and Lab 3 **26**

1 Why combine models at all

Ensemble; competence and diversity

An *ensemble* is a set of M fitted models $\hat{f}_1, \ldots, \hat{f}_M$ whose predictions are combined into one prediction $\hat{f}_{\text{ens}}(x)$ by a vote, an average or a sum of corrections. Two conditions make the combination better than its members. *Competence*: each member must be better than noise, because averaging random guesses gives a random guess. *Diversity*: the members must make *different* errors, because an error that every member makes survives any average. Three identical copies of one model are not an ensemble.

Purpose. Reduce the sample-to-sample variability of a flexible learner (bagging, forests), or reduce the remaining loss step by step (boosting), or let the data decide how to weight different model families (stacking, mixture of experts).

Impact. The prediction at a fixed input becomes more stable (its variance over training samples falls), the decision boundary becomes smoother, and the cost is computation, memory and interpretability.

Instability of one fitted model

Setup. A dataset of N labelled flows (benign or attack), one learning algorithm (say a decision tree grown to purity).

Held fixed. The algorithm and all its settings; the input x_0 at which we look.

What we manipulate. The training sample: draw $B = 50$ different random subsamples (or bootstrap samples) of the data and refit the tree on each.

What we measure. The 50 decision boundaries, and the 50 predicted probabilities at x_0.

What the plot shows. Overlay the 50 boundaries on the two-feature scatter plot: a deep tree gives 50 visibly different staircases; a logistic regression gives 50 nearly identical lines. A histogram of the 50 predictions at x_0 is wide for the tree, narrow for the line.

How to read it. The width of that histogram *is* the variance term $\text{Var}(x_0)$ of Chapter 3. A learner whose boundary jumps around is a candidate for averaging. A learner whose boundary barely moves (large bias, small variance) will not profit from bagging, and only boosting or a richer model can help it.

Three annotators and one ox

At a 1906 livestock fair 787 visitors guessed the weight of an ox; individual guesses were far off, their mean was within one pound of the truth (Galton). The same logic is used to *label* security data: three annotators look at a network event, two say malicious and one says benign; the majority label is kept and strong disagreements go to an expert. Replace annotators by fitted models and you have an ensemble. The annotators are useful only because each knows something and they do not all share the same blind

spot; if all three were trained by the same person with the same checklist, their majority would repeat that checklist's mistakes.

Expecting an ensemble to fix a bias

Tempting because "more models" feels like "more power". But averaging only cancels fluctuations; if every member is missing the same input (no destination port in the features), or every member is the same architecture fitted to the same data, their errors point the same way and the average inherits them. The validation error of the ensemble then sits exactly where the single model's was, and people conclude "ensembles do not work here" when the real problem is a missing feature or a shift between training and deployment. Fix: diagnose the error source first (Section 11).

The three families at a glance

Family	How diversity is obtained	How combined	What is manipulated
Voting / averaging	different algorithms, settings, features or initialisations, usually on the *same* data	fixed vote or average	which models are in the pool
Bagging / random forest	randomised *rows* (bootstrap) and randomised *candidate features* at each split	fixed vote or average	M, tree size, m_{try}
Boosting	each new model targets the *current errors* of the ensemble	successive additive corrections	M, η, depth, λ, γ
Stacking	any pool of base models	a *learned* global weighting	the meta-model
Mixture of experts	experts trained jointly with a gate	a *learned, input-dependent* weighting	gate and routing

Key insight. An ensemble is a device for manufacturing *different* errors and then cancelling them; shared errors are immune to it.

How the professor asks it

Q. *Explain why three copies of the same logistic regression, trained on the same data, form an ensemble that is no better than one copy, and name the two conditions a useful ensemble must satisfy.*
A. No gain, because the three members make *identical* errors, and an average of identical errors is that error. An ensemble helps only when (i) each member is competent, better than chance, and (ii) their errors differ, so that at a given input some members err high and others low and the average cancels part of the fluctuation. Formally the variance of the average is $\sigma^2(\rho + (1-\rho)/M)$ [on your sheet]; identical members have $\rho = 1$ and the variance is σ^2 whatever M is. Consequence: diversity must be engineered (bootstrap rows, random features, residual targets), it does not come from copying.

Q. *A colleague says: "our intrusion detector misses every attack that comes through the new VPN gateway; let us bag it." What is the issue, and what would you do instead?*
A. Bagging will not help, because the error is *shared*: no feature distinguishes VPN traffic, so every bootstrap tree misses the same attacks and their errors have $\rho \approx 1$ on those cases. Averaging reduces variance, not a systematic blind spot. What to do: add the missing information (gateway identity, VPN flags), check for a distribution shift between training traffic and the new gateway's traffic with a temporal or grouped evaluation, and only then consider an ensemble for the remaining instability.

2 Averaging, voting, and how much variance an average removes

Averaging, hard voting, soft voting

For regression the ensemble prediction is the plain average $\hat{f}_{\text{ens}}(x) = \frac{1}{M}\sum_m \hat{f}_m(x)$. For classification, *hard voting* lets each model output a class and keeps the most frequent class; *soft voting* averages the models' posterior estimates, $\hat{p}_{\text{ens}}(y = c \mid x) = \frac{1}{M}\sum_m \hat{p}_m(y = c \mid x)$ [on your sheet], and takes the largest average. Hard voting counts heads; soft voting pools *confidence*: a model saying 0.49 and one saying 0.05 cast the same hard vote for class 0 but contribute very differently to a soft vote.

Purpose. Turn several predictions into one, either robustly (hard) or using the margin information (soft).

Impact. Soft voting uses strictly more information, the distance of each probability from the threshold, but only helps if those probabilities are comparable and reasonably calibrated under the same class prior; hard voting caps every member at one vote and is therefore robust to one overconfident member at the price of discarding margins.

Three flow classifiers, one flow

model	$\hat{p}(y = 0 \mid x)$	$\hat{p}(y = 1 \mid x)$	hard vote
A	0.51	0.49	0
B	0.52	0.48	0
C	0.05	0.95	1

Hard vote: class 0, two to one. Soft vote: average of the attack probabilities $(0.49 + 0.48 + 0.95)/3 = 0.64 > 0.5$, class 1. The soft vote used the fact that A and B were coin flips while C was nearly certain. Whether that is the right call depends on whether C's 0.95 deserves belief. If C is a model that says 0.95 whenever it leans towards attack, it will decide every soft vote alone, and the fix is to calibrate it, to weight the models on validation data (stacking), or to fall back to hard voting.

The variance of an average of correlated errors

Fix an input x and let $e_m = \hat{f}_m(x) - f(x)$ be the error of model m, a random variable over training samples. If every error has variance σ^2 and every pair has correlation ρ, the error of the average has

$$\operatorname{Var}(\bar{e}) = \sigma^2\Big(\rho + \frac{1-\rho}{M}\Big) \quad \text{[on your sheet]}$$

σ^2 is how wrong one member is on its own; ρ is how much of the error *direction* the members share; M is how many you average.

Purpose. It tells you exactly what averaging can and cannot do: it removes the uncorrelated part $(1-\rho)\sigma^2$ at rate $1/M$ and leaves the floor $\rho\sigma^2$ untouched.

Impact. More models push the variance down towards $\rho\sigma^2$ and then stop helping; lowering ρ (bootstrap rows, random features) lowers the floor itself; the formula says nothing about bias.

Deriving $\operatorname{Var}(\bar{e}) = \sigma^2(\rho + (1-\rho)/M)$

Line 1. The variance of a sum is the sum of all variances plus all pairwise covariances:

$$\operatorname{Var}\Big(\sum_{m=1}^{M} e_m\Big) = \sum_m \operatorname{Var}(e_m) + \sum_{m \neq m'} \operatorname{Cov}(e_m, e_{m'}).$$

Line 2. There are M variance terms, each σ^2, and $M(M-1)$ ordered pairs, each with covariance $\rho\sigma^2$ (correlation times the two standard deviations): the sum is $M\sigma^2 + M(M-1)\rho\sigma^2$.

Line 3. The average divides the sum by M, so its variance divides by M^2:

$$\mathrm{Var}(\bar{e}) = \frac{M\sigma^2 + M(M-1)\rho\sigma^2}{M^2} = \frac{\sigma^2}{M} + \frac{(M-1)\rho\sigma^2}{M}.$$

Line 4. Write $(M-1)/M = 1 - 1/M$: $\mathrm{Var}(\bar{e}) = \sigma^2(\frac{1}{M} + \rho - \frac{\rho}{M}) = \sigma^2(\rho + \frac{1-\rho}{M})$.
Why the floor. The M diagonal terms are divided by M^2 and vanish like $1/M$; the $M(M-1) \approx M^2$ cross terms are also divided by M^2 and do *not* vanish. The shared part of the error survives because there are as many cross terms as there are pairs. Special cases: $\rho = 0$ gives σ^2/M; $\rho = 1$ gives σ^2; $M \to \infty$ gives $\rho\sigma^2$.

Variance reduction against M for several correlations

Setup. Simulate: at a fixed x, draw M errors with common variance $\sigma^2 = 1$ and pairwise correlation ρ (or, with real data, fit M models on M bootstrap samples, repeat the whole thing B times, and compute the variance of the averaged prediction across the B repetitions).

Held fixed. σ^2, the input x, the base learner.

What we manipulate. $M \in \{1, 5, 10, 25, 100, 1000\}$ and $\rho \in \{0, 0.2, 0.5, 0.9\}$.

What we measure. The variance of the averaged error, in units of σ^2.

What the plot shows. One curve per ρ, M on a log axis. Recomputed values:

ρ	$M = 1$	5	10	25	100	1000
0	1	0.20	0.10	0.04	0.01	0.001
0.2	1	0.36	0.28	0.232	0.208	0.201
0.5	1	0.60	0.55	0.52	0.505	0.500
0.9	1	0.92	0.91	0.904	0.901	0.900

How to read it. Every curve falls steeply at first and flattens at $\rho\sigma^2$. With $\rho = 0.2$ and $M = 25$ the variance is $0.232\sigma^2$, not the $0.04\sigma^2$ that independence would give; going from 25 to 1000 trees buys only $0.031\sigma^2$. With $\rho = 0.9$ ten models buy almost nothing. The lesson: once the curve is flat, more models are wasted; the only remaining lever is ρ.

Reading σ^2/M when the errors are correlated

People quote "averaging M models divides the variance by M". That is the $\rho = 0$ case, and fitted models trained on overlapping data are never independent. The mistake produces an over-optimistic expectation of how many trees are needed and a mis-diagnosis when adding trees stops helping ("the implementation must be wrong"). Also: the formula is about variance only; averaging does not touch bias, so if every model is missing the same feature, ρ is close to 1 and the ensemble repeats the error. Fix: plot OOB or validation error against M and stop at the plateau; attack the plateau by decorrelating.

Hard voting versus soft voting

	Hard voting	Soft voting
combines	predicted classes	posterior estimates
uses	only which side of the threshold each model is on	the margin (distance from the threshold)
assumption	none beyond competence	probabilities comparable and reasonably calibrated, same prior
failure mode	discards a confident, correct minority	one overconfident model dominates every vote
prefer when	members are miscalibrated or heterogeneous	members are calibrated (e.g. bagged trees of one kind)

Key insight. Averaging buys $(1-\rho)\sigma^2$ and nothing more; every ensemble trick is a way to lower ρ without raising σ^2 too much.

How the professor asks it

Q. *Ten regressors each have error variance $\sigma^2 = 4$ at some input. Compute the variance of their average for $\rho = 0$, 0.3 and 0.9, the limit as $M \to \infty$ at $\rho = 0.3$, and the smallest M that brings the variance to 1.6 or below at $\rho = 0.3$.*
A. Use $\text{Var} = 4(\rho + (1-\rho)/10)$. $\rho = 0$: 0.4. $\rho = 0.3$: $4(0.3 + 0.07) = 1.48$. $\rho = 0.9$: $4(0.9 + 0.01) = 3.64$, barely better than one model. Limit at $\rho = 0.3$: $4\rho = 1.2$. $4(0.3 + 0.7/M) \le 1.6 \iff 0.7/M \le 0.1 \iff M \ge 7$. At $\rho = 0.9$ ten models are nearly useless, so the only productive move is to lower ρ, which is what bootstrap samples and random feature subsets do.

Q. *Give a situation in which hard voting beats soft voting, and one in which soft voting beats hard voting, and say what each rule throws away.*
A. Hard wins when one member is overconfident: two calibrated models at 0.55 and 0.60 for attack and an overconfident model at 0.02 give a soft average 0.39 (benign, wrong if the two are right) but a hard vote 2 to 1 for attack. Soft wins when confidence is deserved: two models at 0.51 for attack (wrong) and one calibrated model at 0.01 give hard 2 to 1 for attack, soft $(0.51 + 0.51 + 0.01)/3 = 0.343$, benign and right. Hard throws away the margins; soft throws away robustness to miscalibration. Four cases on a slide are not a validation set, so the choice is made by validating the two rules.

Q. *Explain why an ensemble of* 1000 *bagged trees can have a validation error almost identical to one of* 50 *bagged trees.*
A. Because the variance of the average has a floor: $\sigma^2(\rho + (1-\rho)/M)$ tends to $\rho\sigma^2$, and by $M = 50$ the term $(1-\rho)\sigma^2/M$ is already small compared with $\rho\sigma^2$. With $\rho = 0.5$, $M = 50$ gives $0.51\sigma^2$ and $M = 1000$ gives $0.5005\sigma^2$: the OOB curve is flat. More trees stabilise, they do not add capacity. To go lower, decorrelate the trees (random forest) or change the error source (boosting, better features).

3 Decision trees, the base learner

Decision tree

A tree with J leaves partitions the feature space into disjoint axis-aligned boxes $S_1, \dots, S_J$ and predicts one constant per box, $T(x) = \sum_j \theta_j \mathbf{1}[x \in S_j]$ [on your sheet]; exactly one indicator is 1, so $T(x)$ is "the constant of the box x falls in". Each box is reached by binary tests $x_j \le t$. The *depth* of a leaf is the number of tests on its path; maximum depth and minimum leaf size are the usual size controls. The ideal fit would choose boxes and constants jointly, with a penalty λJ on the number of leaves; this search is infeasible, so two simplifications are made: for fixed boxes the best constant per leaf has a closed form, and the boxes are chosen *greedily* one split at a time.

Purpose. A flexible, fast, scale-free learner that captures nonlinearity and interactions (a test on x_1 followed by a test on x_2 *is* an interaction), needing no feature scaling because only the order of values matters.

Impact. Depth up: more boxes, lower training error, higher variance. Minimum leaf size up: coarser boxes, smoother, lower variance. A single deep tree is a high-variance learner, which is exactly why ensembles use it.

The optimal leaf constant

For fixed regions the training loss splits by leaf, because each observation lies in exactly one box:

$$\sum_{i=1}^{N} \ell(y^{(i)}, T(x^{(i)})) = \sum_{j=1}^{J} \sum_{i:\, x^{(i)} \in S_j} \ell(y^{(i)}, \theta_j),$$

so each θ_j is chosen on its own leaf. *Squared error*: $\frac{d}{d\theta} \sum_i (y^{(i)} - \theta)^2 = -2 \sum_i (y^{(i)} - \theta) = 0$ gives

θ = the leaf mean (the finite-sample version of "squared loss selects the conditional mean"). *Absolute error*: the derivative is $\sum_i \text{sign}(\theta - y^{(i)})$, which changes sign when as many points lie below θ as above, so θ = the leaf median. *Zero–one loss*: predict the class that is wrong on the fewest points, the majority class. [on your sheet](leaf: mean, median, majority).

Impurity, Gini, greedy split

The *impurity* $I(S)$ measures how mixed a node is. For classification the Gini impurity is $I(S) = 1 - \sum_c \hat{p}_c(S)^2$ [on your sheet], where $\hat{p}_c(S)$ is the fraction of class c in the node; $\sum_c \hat{p}_c^2$ is the probability that two observations drawn from the node share a class, so I is the probability they differ: 0 for a pure node, 0.5 for two balanced classes. (Entropy $-\sum_c \hat{p}_c \log_2 \hat{p}_c$ is the other common choice [on your sheet].) For regression the within-node squared error plays the same role. The greedy rule picks the feature and threshold with the largest *size-weighted* impurity reduction $I(S) - \frac{|S_L|}{|S|} I(S_L) - \frac{|S_R|}{|S|} I(S_R)$ [on your sheet], then recurses in each child until a stopping rule.

Purpose. Decide where to cut, one cut at a time, without searching over all trees.

Impact. Greedy means best *immediate* reduction: a pure but tiny child counts little because of the size weights, and an early split that looks best now can block a better combination later (no look-ahead).

One split by hand: eight flows sorted by packet count

x	1	2	3	4	5	6	7	8
y	0	0	0	1	0	1	1	1

Parent: four of each class, $I = 1 - 0.25 - 0.25 = 0.50$. Candidate $x \le 3.5$: left $\{0,0,0\}$, $I = 0$; right $\{1,0,1,1,1\}$, $I = 1 - 0.8^2 - 0.2^2 = 0.32$; weighted $\frac{3}{8}(0) + \frac{5}{8}(0.32) = 0.20$; reduction 0.30. Candidate $x \le 4.5$: left $\{0,0,0,1\}$ and right $\{0,1,1,1\}$ both have $I = 1 - 0.75^2 - 0.25^2 = 0.375$; weighted 0.375; reduction 0.125. Greedy chooses $x \le 3.5$: a pure left leaf and the larger immediate reduction, even though $x \le 4.5$ splits the data in half.

Hierarchical lock-in: perturb the data, watch the tree

Setup. The eight flows above, a tree grown to depth 2.

Held fixed. The algorithm, the depth.

What we manipulate. Change one label (flip y at $x = 5$ from 0 to 1) and regrow.

What we measure. The root split and everything below it.

What the plot shows. Before: root $x \le 3.5$, then a sub-split on the right. After the flip, the right child becomes $\{1,1,1,1,1\}$, pure, and the whole lower structure disappears; flip a different point and the root itself may move to $x \le 4.5$, rebuilding every box below.

How to read it. One early decision commits everything underneath it; deeper splits can only refine the sub-region they sit in. That is *hierarchical lock-in*, and it is the mechanism behind a tree's high variance. Bagging repairs it by averaging many trees that locked in differently; boosting repairs it by letting each new shallow tree adjust the boundary anywhere.

Trusting a single deep tree, or scaling features for it

A deep tree with zero training error is reported as "the model"; on validation it is far worse, and a tiny change of the data gives a different tree with a different story. Tempting because trees are readable. The fix is the rest of this chapter: average many (bagging, forest) or sum many shallow ones (boosting). A second, harmless-but-telling mistake: standardising features "for the tree". Trees only use the order of the values, so scaling changes nothing; needing to scale is a sign one is thinking of kNN or a penalised linear model.

Tree versus logistic regression as a base learner

	Deep tree	Logistic regression
boundary	axis-aligned staircase, captures interactions	one hyperplane
bias / variance	low bias, high variance	higher bias, low variance
scaling needed	no	yes for penalties and optimisation
what averaging does	removes much of the variance: ideal bagging member	almost nothing: boundaries already agree

Key insight. A tree is a piecewise-constant function chosen greedily; it is flexible and cheap, and its instability is the raw material that ensembles turn into accuracy.

How the professor asks it

Q. *Prove that, for squared-error loss and fixed leaves, the best constant in each leaf is the mean of the targets in that leaf, and state what it is for absolute and zero–one loss.*
A. The training loss separates by leaf since each point is in one leaf, so minimise $\sum_{i \in S_j} (y^{(i)} - \theta)^2$ alone; its derivative $-2\sum_i (y^{(i)} - \theta)$ vanishes at $\theta = \frac{1}{|S_j|}\sum_i y^{(i)}$, and the objective is a convex parabola, so this is the minimum. Absolute loss: the derivative $\sum_i \operatorname{sign}(\theta - y^{(i)})$ changes sign at the median. Zero–one: the majority class. Consequence: a regression tree's leaf outputs are leaf means, which is why boosting's leaf outputs are mean residuals.

Q. *Node with* 10 *flows,* 5 *attacks and* 5 *benign. A split sends* 4 *flows left (all benign) and* 6 *right (*5 *attacks,* 1 *benign). Compute the Gini reduction and explain why the reduction is size-weighted.*
A. Parent $I = 0.5$. Left $I = 0$. Right $I = 1 - (5/6)^2 - (1/6)^2 = 10/36 = 0.278$. Weighted children $0.4 \cdot 0 + 0.6 \cdot 0.278 = 0.167$; reduction $0.5 - 0.167 = 0.333$. Weighting by child size prevents a split that isolates one pure observation from looking as good as a split that purifies half the data: purity on 5% of the rows is worth 5% of the credit.

Q. *Why does the greedy rule not guarantee the best tree of a given depth? Give the shape of a counterexample.*
A. Greedy chooses the split with the best immediate reduction and never revises it. XOR-like data: two features where neither single split reduces impurity at all ($Y = X_1 X_2$ with ± 1 coins) but a split on X_1 followed by a split on X_2 in each child is perfect. The greedy rule sees two useless first splits, picks one at random or stops, and misses the depth-2 tree that is perfect. Consequence: single trees can be far from optimal; ensembles partly compensate by trying many different first splits.

4 Bagging and out-of-bag evaluation

Bootstrap sample; bagging

A *bootstrap sample* $\mathcal{D}^*$ of a training set of size N is N draws from it *with replacement*; $\text{count}(i)$ is how many times observation i was drawn, and the observations with $\text{count}(i) = 0$ are *out of bag* for that sample. *Bagging* (bootstrap aggregating) draws M bootstrap samples, fits the same base learner to each, and averages the predictions or the probabilities, $\hat{f}_{\text{bag}}(x) = \frac{1}{M}\sum_m \hat{f}_m(x)$.

Purpose. Give the *same* learner *different* training sets without discarding data: splitting $\mathcal{D}$ into M disjoint parts would leave each model N/M points, too few.

Impact. Each model sees a perturbed copy of the whole dataset (some points repeated, about a third missing), so the fitted models differ, ρ drops below 1, and the average is more stable than any member; the bias of the base learner is unchanged.

One bootstrap sample of eight flows

From $\{1,\dots,8\}$ eight draws with replacement might give $\{2,5,2,8,1,5,7,2\}$:

observation i	1	2	3	4	5	6	7	8
count(i)	1	3	0	0	2	0	1	1

Flow 2 is seen three times and 5 twice; flows 3, 4 and 6 are out of bag and never influence this tree. A tree fitted to these counts weights flow 2 three times as heavily as flow 1, which is why it differs from a tree fitted to another sample.

The out-of-bag fraction $(1-1/N)^N \to e^{-1}$

Line 1. One draw picks observation i with probability $1/N$, so misses it with probability $1-1/N$. **Line 2.** The N draws are independent, so i is missed by all of them with probability $(1-1/N)^N$ [on your sheet]. **Line 3.** $\lim_{N\to\infty}(1+x/N)^N = e^x$ with $x=-1$ gives $e^{-1}\approx 0.368$. Recomputed: $N=8$: $(7/8)^8 = 0.344$; $N=20$: 0.358; $N=100$: 0.366; $N=1000$: 0.3677. **Consequence.** A large bootstrap sample contains about 63.2% of the distinct observations and leaves 36.8% out, a *different* third for every model. Expected number of models that never saw observation i: $0.368M$.

Out-of-bag prediction and OOB error

For training observation i, let $\mathcal{M}_i = \{m : i \notin \mathcal{D}_m^*\}$ be the models whose bootstrap sample omitted i, about $0.368M$ of them. The OOB prediction uses only those models, $\hat{f}_{\text{OOB}}(x^{(i)}) = \frac{1}{|\mathcal{M}_i|}\sum_{m\in\mathcal{M}_i}\hat{f}_m(x^{(i)})$ [on your sheet](average of probabilities, or a vote, for classification). The *OOB error* is the loss of these predictions over all training observations.

Purpose. A validation estimate *without holding anything out*: for each point, the models that vote on it never saw it, so the prediction is a genuine unseen-data prediction.

Impact. It is free, uses all the data for training, and is a slight underestimate of the full ensemble's quality (only about a third of the trees vote for each point). It estimates the error of a fixed ensemble under the training distribution; it is not a test set and says nothing about deployment shift.

OOB error against the number of trees

Setup. The flow dataset, bagged trees of a fixed size.

Held fixed. The base learner and its size setting ξ (depth, minimum leaf); the data.

What we manipulate. M from 1 to 500, adding trees one at a time and recomputing the OOB error after each.

What we measure. OOB error (and, on a separate held-out validation set, the validation error, to compare).

What the plot shows. A noisy, steeply falling curve that flattens: with $M=1$ almost no point has an OOB vote, so the first values are erratic; by $M\approx 50$ to 200 the curve is flat. The validation curve has the same shape, slightly below the OOB curve.

How to read it. The plateau height is the floor $\rho\sigma^2$ expressed as an error; its position tells you how many trees are enough; beyond it trees cost time and memory but change nothing. The gap between OOB and validation is the OOB pessimism (each OOB prediction uses $0.368M$ trees, not M). If validation lies far *above* OOB, the validation data differ from the training data: a shift, not an OOB defect.

Tuning the tree size at ensemble level

Setup. Bagged trees.

Held fixed. M large enough to be on the plateau (say 300); the data.

What we manipulate. One shared size setting ξ for *all* trees: minimum leaf size $\in\{1,5,20,100\}$ or maximum depth $\in\{2,4,8,\infty\}$.

What we measure. OOB error of the *complete* ensemble for each ξ.

What the plot shows. Typically a U: very small leaves (deep trees) give the best or near-best OOB error because averaging removes their variance; very large leaves underfit. With noisy labels or little data the minimum moves towards larger leaves.

How to read it. The setting is chosen for the *average*, not for any single tree. Tuning each tree separately would optimise the wrong target (one tree's accuracy) and push every tree towards the same strong, correlated fit.

Treating the OOB error as a test result, or tuning on it a hundred times

The OOB error is an internal validation estimate. Two misuses. (i) Reporting it as the final performance: it is tied to the training distribution and cannot detect that deployment traffic differs; keep a temporal or grouped test set untouched. (ii) Selecting tree size, m_{try}, feature set and threshold by repeated OOB comparisons, then reporting the best OOB value: the minimum of many noisy estimates is optimistically biased (the winner's curse of Chapter 3), exactly like reusing a validation set. The number reported is inflated; the fix is a final untouched test set.

OOB versus out-of-fold versus validation set versus test set

	Where it comes from	What it estimates	Limits
OOB	bootstrap by-product inside a bagged ensemble	error of the fixed ensemble under the training distribution	bagging only; slightly pessimistic; overfits if reused for selection
out-of-fold (CV)	explicit K-fold split, works for any learner	error of the *procedure*, averaged over folds	K fits per configuration; folds must respect groups and time
validation set	held-out part of development data	error of one fitted model, used to select configurations	worn out by repeated selection
test set	untouched until the end	performance of the final chosen model	used once; must match deployment conditions

Key insight. Bootstrap sampling manufactures diversity and, as a by-product, a free held-out third of the data for every tree; neither replaces the untouched test set.

How the professor asks it

Q. *Show that a given observation is absent from a bootstrap sample of size N with probability $(1-1/N)^N$, give its limit, and explain what it means for out-of-bag evaluation.*
A. Each of the N independent draws misses the observation with probability $1-1/N$, so all N miss it with probability $(1-1/N)^N \to e^{-1} = 0.368$. So each tree is trained on about 63% of the distinct rows and about 37% of the rows are a held-out set for that tree. For observation i about $0.368M$ trees never saw it; their averaged prediction at $x^{(i)}$ is an honest unseen-data prediction, and averaging the loss over all i gives the OOB error without any explicit split.

Q. *$N = 1000$ flows, $M = 500$ bagged trees. How many distinct flows does a tree see on average, how many trees are OOB for a given flow, and what is the probability that some flow has no OOB prediction at all for $M = 500$ and for $M = 10$?*
A. Distinct per tree $\approx 0.632 \times 1000 = 632$. OOB trees per flow $\approx 0.368 \times 500 = 184$. No OOB prediction means the flow was in-bag for every tree: $0.632^{500} \approx 10^{-100}$, impossible; for $M = 10$, $0.632^{10} = 0.010$, so about 10 of the 1000 flows would have no OOB vote and the OOB error with ten trees is both noisy and incomplete. Consequence: OOB estimates need a reasonably large M.

Q. *Why is the tree-size setting of a bagged ensemble tuned on the OOB error of the whole ensemble rather than tree by tree?*
A. Because the quantity to optimise is the error of the average, not of a member. One tree tuned alone picks a deep, strong fit on all the data; every tree tuned that way converges to the same fit, raising ρ

and recreating the correlated ensemble that bagging was meant to avoid. Procedure: one shared ξ for all $\mathcal{D}_m^* \to \widehat{T}_m(x;\xi)$, fit a complete ensemble per candidate ξ, compare OOB errors, keep the best, then confirm on an untouched test set.

5 Random forests

Random forest; m_{try}

A random forest is bagging plus one extra randomisation: at *every node* of every tree, only m_{try} of the d features, drawn uniformly at random, are offered as split candidates; the best split among those is taken; a new subset is drawn at every node. Common starting values: $m_{\text{try}} \approx \sqrt{d}$ for classification, $d/3$ for regression. $m_{\text{try}} = d$ recovers bagged trees. Aggregation is by averaging probabilities or voting.

Purpose. Bagged trees can remain too similar: a very strong feature wins the root of almost every bootstrap tree, so the trees have similar shapes and correlated errors (large ρ) and more trees stop helping early. Hiding features forces trees to start differently.

Impact. Small m_{try}: at many nodes the strongest feature is not on offer, so alternatives are used, trees differ, ρ falls; each tree is a little weaker (σ^2 rises a little). The strong feature is not lost, it is available at a fraction m_{try}/d of nodes and is used deeper down. Too small m_{try} gives poor splits (pure noise candidates). More trees M stabilise the forest but do not deepen it.

Probability that a feature is offered

A particular feature is among m_{try} candidates drawn without replacement from d with probability m_{try}/d [on your sheet](by symmetry, or $\binom{d-1}{m_{\text{try}}-1}/\binom{d}{m_{\text{try}}} = m_{\text{try}}/d$; with $d = 25$, $m_{\text{try}} = 5$: $\binom{24}{4}/\binom{25}{5} = 0.2$). It is absent from the root of one tree with probability $1 - m_{\text{try}}/d = 0.8$, and, roots being drawn independently, from all $M = 10$ roots with probability $0.8^{10} = 0.107$ [on your sheet]. Expected roots that offer it: $10 \times 0.2 = 2$, whereas bagging would offer it at all ten and use it at most of them.

The m_{try} sweep: single-tree error against forest error

Setup. Flow data with $d = 25$ features, forests of $M = 300$ trees.

Held fixed. M, leaf size, the bootstrap seeds, the data.

What we manipulate. $m_{\text{try}} \in \{1, 2, 5, 10, 25\}$.

What we measure. Two curves: the average OOB error of the *individual* trees (strength) and the OOB error of the *forest*.

What the plot shows. The single-tree curve is best at $m_{\text{try}} = 25$ (every tree sees the strongest feature) and worsens as m_{try} falls. The forest curve is U-shaped: it improves as m_{try} falls from 25 to around $\sqrt{d} = 5$, because ρ drops faster than σ^2 rises, then worsens at $m_{\text{try}} = 1$ where splits are chosen among pure-noise candidates.

How to read it. The two curves move in opposite directions over the useful range: *weaker trees, stronger forest.* This is $\sigma^2(\rho + (1-\rho)/M)$ in action: with M large the forest variance is essentially $\rho\sigma^2$, and a modest rise in σ^2 is paid back by a large fall in ρ. If signal is carried by very few features (say 5 of 400), $\sqrt{d} = 20$ candidates see no informative feature in $0.9875^{20} = 78\%$ of splits and the minimum moves to larger m_{try}; $\sqrt{d}$ is a starting point, not a law.

Intrusion detection with one dominant feature

In flow data the destination port is often the strongest single feature (port 22 brute force, port 445 worms). Bagged trees all put "port = 22?" at the root and share the mistakes that a port-based rule makes on attacks that use common ports. A forest with $m_{\text{try}} = 5$ of 25 offers the port at only a fifth of the roots; the other trees start with duration, packet rate or byte ratios and catch attacks the port rule misses.

Validation: the forest's false-negative rate on the "attacks on port 443" subgroup drops although each tree alone is slightly worse overall.

Tuning each tree, or adding trees to add capacity

Two confusions. (i) "My trees are weak, let me give them all features": $m_{\text{try}} = d$ makes each tree stronger and the forest worse, because the forest's error is the error of the average. (ii) "The forest underfits, add trees": M only stabilises; once the OOB curve has plateaued, 1000 trees give the same fit as 200. Capacity lives in tree depth, leaf size and m_{try}. The inflated number here is the budget, not an error estimate: hours of training for nothing.

Bagged trees versus random forest

	Bagged trees	Random forest
randomisation	rows (bootstrap)	rows and candidate features at each node
dominant feature	at the root of nearly every tree	at a fraction m_{try}/d of nodes
single tree	stronger, σ^2 small	slightly weaker, σ^2 larger
correlation ρ	large	small
forest variance $\approx \rho\sigma^2$	higher	lower for moderate m_{try}
controls	M, tree size	M, tree size, m_{try}

Key insight. A random forest deliberately weakens each tree to decorrelate the trees; the target is the average's error, and ρ matters more than σ^2 once M is large.

How the professor asks it

Q. *$d = 25$, $m_{\text{try}} = 5$, $M = 10$. What is the probability that x_1 is available at a given split, and that it is unavailable at the roots of all ten trees? Why can hiding x_1 from most roots lower each tree's accuracy and yet improve the forest?*
A. Available with probability $5/25 = 0.2$; unavailable at one root with 0.8, at all ten with $0.8^{10} = 0.107$; about two roots use it. Each tree is weaker (σ^2 up a little) but trees that all start with x_1 make correlated mistakes, and the floor $\rho\sigma^2$ cannot be averaged away; trees that start differently partition the space differently, so ρ falls and the forest variance $\sigma^2(\rho + (1-\rho)/M)$ falls with it. Weaker trees, stronger forest.

Q. *How would you choose m_{try} and the tree size for a forest, and why not tune them separately for each tree?*
A. Start at $m_{\text{try}} \approx \sqrt{d}$ and one shared leaf size; for each candidate pair on a small grid fit a complete forest and compute its OOB error (or a validation error); keep the best; raise M until the OOB error plateaus; keep a test set untouched because repeated OOB selection is optimistic. Not per tree, because the objective is the forest's error: a tree tuned alone picks $m_{\text{try}} = d$ and a deep fit, every tree does the same, and the correlated bagged forest is back.

Q. *A forest's OOB error has been flat from $M = 150$ onwards. The team proposes $M = 2000$ "to be safe". Evaluate.*
A. Useless and costly. The OOB curve measures $\sigma^2(\rho + (1-\rho)/M)$ as an error; past the plateau the $(1-\rho)/M$ term is negligible and only the floor $\rho\sigma^2$ remains, which M cannot touch. Prediction time and memory grow linearly in M for no accuracy. To lower the floor change m_{try}, the leaf size, or the features; to lower bias change the learner (boosting).

6 Feature importance

Mean decrease in impurity (MDI)

Each split at a node t holding N_t of the N training observations earns a *credit* equal to its impurity reduction weighted by the share of data it acts on, $\Delta_t = \frac{N_t}{N}\left[I(S_t) - \frac{N_L}{N_t} I(S_L) - \frac{N_R}{N_t} I(S_R)\right]$ [on your sheet], credited to the feature used at t. MDI_j sums the credits of all splits on x_j and averages over trees [on your sheet], usually normalised so the MDI values sum to 1: "this feature produced 75% of all the impurity the forest removed".

Purpose. A free, training-time summary of which features the forest *used*.

Impact. Biased towards features with many possible split points (continuous features, high-cardinality categoricals), which get more chances to find *some* reduction by signal or by noise; measures the training fit, not generalisation.

Credits and forest-level MDI

Root with 10 flows (5 attacks, 5 benign), Gini 0.5; the test "destination port = 22?" sends 4 benign flows left (Gini 0) and 6 flows right (5 attacks, 1 benign, Gini $1 - (5/6)^2 - (1/6)^2 = 0.278$). Weighted children $0.4(0) + 0.6(0.278) = 0.167$; with $N_t/N = 1$, $\Delta_t = 0.5 - 0.167 = 0.333$, credited to destination port. Two trees: tree 1 credits port 0.18, duration 0.06; tree 2 credits port 0.12, packets 0.04. Averages over the two trees (absent feature contributes 0): port 0.15, duration 0.03, packets 0.02, total 0.20; normalised MDI = 0.75, 0.15, 0.10.

Permutation importance

For a fitted forest and a feature j: compute the OOB (or validation) error; shuffle the values of x_j among the OOB observations, keeping every other column and the labels fixed; predict again. The importance is the increase in error, $\mathrm{PI}_j = \mathrm{err}(\text{shuffled } x_j) - \mathrm{err}$ [on your sheet]. No tree is retrained; in a forest each tree uses its own OOB sample and the increases are averaged.

Purpose. Measure how much the *fitted model relies* on x_j when predicting, on data it did not train on.

Impact. Shuffling destroys the information in x_j (its link with the label and the other columns) while preserving its marginal distribution, so the trees still receive plausible values. If the error barely moves the forest was not relying on x_j; if it jumps, it was.

The permutation protocol, with its control

Setup. A fitted forest, 100 OOB flows, original OOB error rate 0.10 (10 errors).

Held fixed. The fitted trees (never retrained), the labels, all columns but one.

What we manipulate. Which column is shuffled: destination port, flow duration, total packets; repeat each shuffle $R = 20$ times with different permutations and average, because one shuffle is noisy.

What we measure. Error after shuffling. Recomputed: port 18 errors (0.18, increase 0.08); duration 13 (0.13, +0.03); packets 11 (0.11, +0.01).

What the plot shows. A bar per feature with an error bar from the R repetitions. A useful *control*: add a column of pure noise; its bar should sit at 0 within the error bar, and any real feature whose bar is not clearly above the noise bar is not demonstrably used.

How to read it. Destination port matters most *for this fitted forest*: breaking it adds eight errors. The protocol says nothing about a feature the forest never learned to use because a correlated twin carried the same information.

Reading importance as causality, and missing masked features

Two readings that neither score supports. (i) *Masking*: if x_a and x_b carry the same information, shuffling x_a alone hurts little because trees that use x_b still have it; both look unimportant although the information

they share is essential. Fix: permute them jointly, or drop a column and refit. (ii) *Causality*: a high score means the fitted model relies on the feature; it may be a proxy (destination port stands in for the attacked service), a consequence recorded after the fact ("ticket closed as intrusion": leakage), or a collection artefact. Intervening on it would change nothing. A third slip: ranking features by MDI and believing the continuous ones are "more important" when they merely had more split points.

MDI versus permutation importance

	MDI	Permutation
computed from	training-time split credits	OOB or validation predictions after shuffling
cost	free	one pass per feature per repetition
measures	impurity removed while fitting	reliance of the fitted model in prediction
bias	favours many split points, high cardinality	masked by correlated features
causal?	no	no

Key insight. Importance scores describe the fitted model's reliance, not the world's mechanism; correlated twins hide each other and MDI rewards features with many thresholds.

How the professor asks it

Q. *Two credits in one tree: the root ($N_t/N = 1$) drops impurity from* 0.50 *to* 0.45*; a deeper node holding* 30% *of the rows drops it from* 0.48 *to* 0.08*. Which split earns more MDI credit, and what does this say about depth?*
A. Root: $1 \times (0.50 - 0.45) = 0.05$. Deeper: $0.30 \times (0.48 - 0.08) = 0.12$. The deeper split earns more: credit is share of data times reduction, so a big drop on fewer rows can beat a small drop on all rows. Depth alone does not decide credit; the product does.

Q. *After shuffling, the error rises by* 0.08 *for* x_a *and* 0.02 *for* x_b*; you know* x_a *and* x_b *are highly correlated. What can and cannot be concluded?*
A. Can: the fitted forest leans on x_a more than on x_b in prediction. Cannot: that x_b is unimportant, because when x_a is shuffled the trees that use x_b keep the shared information, and vice versa, so both bars underestimate the shared contribution. Also cannot: that x_a causes attacks. To see the shared part, permute both jointly or drop one and refit. Consequence: use importance for debugging and feature auditing, not for causal claims.

Q. *Why must permutation importance be computed on OOB or validation rows rather than on in-bag training rows?*
A. On in-bag rows a deep tree has memorised the labels, so its error is near zero before shuffling and the increase measures how memorisation breaks, not how the model generalises using x_j. OOB or validation rows give an unseen-data error before and after, so the increase measures reliance in prediction. Using training rows inflates every importance, most for features used deep in the trees.

7 Boosting for regression

Additive boosted model; learning rate

Boosting keeps the observations fixed and builds models *sequentially*: a first learner makes a baseline prediction, the *residuals* $r^{(i)} = y^{(i)} - \hat{y}^{(i)}$ are computed, a second learner is trained exclusively to predict those residuals, and so on. After m rounds, $F_m(x) = F_{m-1}(x) + \eta\, T_m(x)$ with $F_0 = \bar{y}$ [on your sheet]; unrolled, $F_M = F_0 + \eta \sum_m T_m$, a constant plus a scaled *sum* of trees. $\eta \in (0, 1]$ is the learning rate or shrinkage: $\eta = 1$ applies the full correction, $\eta = 0.1$ a tenth of it.

Purpose. Reduce the remaining training loss one step at a time, each tree acting only where

the current ensemble is still wrong.

Impact. Compare bagging's $\frac{1}{M}\sum_m \hat{f}_m$: there every model predicts the whole target and the average smooths them (variance down); here every tree predicts a *correction* and the corrections are summed (loss, mostly bias, down). Smaller η means smaller steps, more rounds for the same fit, usually slightly better generalisation.

Squared error: minimising the loss over the new tree is fitting the residuals

With $L = \frac{1}{2}\sum_i \left(y^{(i)} - F_m(x^{(i)})\right)^2$ and $F_m = F_{m-1} + \eta T_m$,

$$L = \tfrac{1}{2}\sum_i \Big(\underbrace{y^{(i)} - F_{m-1}(x^{(i)})}_{r_m^{(i)}} - \eta T_m(x^{(i)})\Big)^2.$$

At round m everything already built is frozen, so $r_m^{(i)}$ is a fixed number; the expression is the least-squares objective of a regression whose *targets* are the residuals and whose *predictor* is ηT_m. In practice T_m is fitted to r_m and η is applied in the update. The general recipe replaces the residual by the *pseudo-residual* $r_m^{(i)} = -\partial\ell/\partial F\big|_{F_{m-1}(x^{(i)})}$ [on your sheet], the direction in which changing the prediction at $x^{(i)}$ decreases the loss fastest; for $\ell = \frac{1}{2}(y-F)^2$ it is $y - F$, so "fit the residuals" and "fit the negative gradient" coincide. For other losses they differ, and the leaf outputs are then chosen to minimise the actual loss (next section).

Gradient boosting for regression, the algorithm

(1) $F_0(x) = \arg\min_w \sum_i \ell(y^{(i)}, w)$, the mean for squared error. (2) For $m = 1, \ldots, M$: (a) pseudo-residuals $r_m^{(i)} = -\partial\ell/\partial F$ at $F_{m-1}(x^{(i)})$; (b) fit a regression tree T_m to the targets $r_m^{(i)}$, giving leaves $S_{1,m}, \ldots, S_{Q_m,m}$; (c) in each leaf output the mean residual $w_{q,m} = \frac{1}{|S_{q,m}|}\sum_{i\in S_{q,m}} r_m^{(i)}$; (d) $F_m = F_{m-1} + \eta T_m$.

Purpose. A loss-agnostic way to add corrections: the tree learns *where* to correct, the leaf values *how much*.

Impact. Training loss decreases monotonically for ever; validation loss decreases and then rises once new trees fit noise.

One round by hand, $\eta = 0.1$

ID	x	y	F_0	$r_1 = y - F_0$	F_1	$r_2 = y - F_1$
1	1.2	10	20	-10	$20 - 0.1\cdot 10 = 19$	-9
2	4.5	20	20	0	$20 + 0.1\cdot 5 = 20.5$	-0.5
3	7.1	30	20	$+10$	20.5	9.5

$F_0 = \text{mean}(10, 20, 30) = 20$. Tree T_1 splits at $x < 2.85$: leaf $\{1\}$ with weight -10, leaf $\{2, 3\}$ with weight $(0 + 10)/2 = 5$. The update moves ID 3 from 20 to 20.5, closer to 30. Squared error $\frac{1}{2}\sum r^2$: 100 before, 85.75 after one shrunken step; the next tree is fitted to $(-9, -0.5, 9.5)$. With $\eta = 1$ the same tree would jump to $F_1 = (10, 25, 25)$ in one go: faster, and more willing to chase noise.

Boosting rounds: training against validation loss, two learning rates

Setup. Regression on house prices or flow byte counts; gradient-boosted trees of depth 3.

Held fixed. Depth, the data split, the loss.

What we manipulate. The number of rounds M from 1 to 2000, for $\eta = 1$ and $\eta = 0.1$.

What we measure. Training loss and validation loss after every round.

What the plot shows. Four curves. Both training curves fall monotonically for ever. Each validation curve falls, reaches a minimum, and rises: for $\eta = 1$ the minimum comes early (tens of rounds) and the

rise is steep; for $\eta = 0.1$ the minimum comes roughly ten times later, is slightly lower, and the rise is gentle.

How to read it. The validation minimum is the early-stopping point M^*. Halving η roughly doubles M^* (the coupling $\eta \propto 1/M$); the smaller rate wins a little accuracy for ten times the computation. The training curve is useless for choosing M: it never turns up.

Depth, rate and rounds interact

Setup. Same data.

Held fixed. Early stopping on validation for every configuration.

What we manipulate. A grid over depth $\in \{1, 2, 4, 8\}$ and $\eta \in \{0.3, 0.1, 0.03\}$; for each cell M is chosen by early stopping.

What we measure. Validation loss at the selected round, and M^*.

What the plot shows. A heat map of validation loss over (depth, η), with M^* printed in each cell: deeper trees reach their minimum in fewer rounds; depth 1 (stumps) can only model additive effects and plateaus higher when interactions matter; depth 4 to 6 usually wins; very deep trees overfit early.

How to read it. The three knobs are coupled and must be tuned together: depth limits the order of interactions a tree can express, η sets the step size, M the number of steps. Comparing configurations at a *fixed* M would be unfair to small η.

Choosing M by training loss, or tuning η and M separately

Training loss keeps falling, so "more rounds" always looks better on it; the validation curve is the only one that turns. Reporting the training curve produces a model that has memorised the noise. Second slip: fixing $M = 100$ and comparing $\eta = 0.3$ with $\eta = 0.01$; the small rate has not converged and looks bad, although at its own M^* it would win. Third: expecting an OOB error from boosting; there is none, the rounds are sequential and every tree sees the full (reweighted) data, so you must hold out a validation set or cross-validate.

One deep tree versus many shallow boosted trees

One deep tree	Boosted shallow trees
fits the whole rule at once	tree 1 isolates the loudest pattern; later trees target leftovers
hierarchical lock-in: early splits force rigid boundaries, deep splits only refine their sub-region	global adaptability: each new tree may adjust the boundary anywhere in feature space
a poor early split cannot be revised and amplifies noise	shrinkage η enforces gradual convergence
low bias, high variance in one shot	each tree high bias, low variance; the sum lowers bias while η and early stopping keep variance in check

Key insight. Boosting is a sum of shrunken corrections, each fitted to the negative gradient of the loss; it buys bias reduction with sequential steps, and its only brake is the validation curve.

How the professor asks it

Q. *Derive that, for squared-error loss, choosing the next tree to minimise the ensemble loss is the same as fitting the tree to the current residuals.*

A. Substitute $F_m = F_{m-1} + \eta T_m$ into $L = \frac{1}{2}\sum_i (y^{(i)} - F_m(x^{(i)}))^2$ and group $y^{(i)} - F_{m-1}(x^{(i)}) = r_m^{(i)}$, a fixed number because F_{m-1} is already fitted: $L = \frac{1}{2}\sum_i (r_m^{(i)} - \eta T_m(x^{(i)}))^2$, the least-squares objective with targets r_m and predictor ηT_m. The negative gradient of $\frac{1}{2}(y - F)^2$ in F is $y - F$, so the residual is also the pseudo-residual; that is why the general algorithm says "fit the negative gradient".

Q. $y = (0, 2, 4, 6)$ *at* $x = (1, 2, 3, 4)$, $F_0 = \bar{y}$, *one stump splitting at* $x \le 2.5$, $\eta = 0.5$. *Compute* F_1, *the new residuals, and the squared error before and after.*
A. $F_0 = 3$, residuals $(-3, -1, 1, 3)$. Left leaf mean $(-3 - 1)/2 = -2$, right $(1 + 3)/2 = 2$. $F_1 = 3 + 0.5 \cdot (-2, -2, 2, 2) = (2, 2, 4, 4)$; residuals $(-2, 0, 0, 2)$. Squared error $\sum r^2$: $9 + 1 + 1 + 9 = 20$ before, $4 + 0 + 0 + 4 = 8$ after. With $\eta = 1$ it would be $(1, 1, 5, 5)$ and residuals $(-1, 1, -1, 1)$, error 4: the full step fits more now and leaves a residual pattern a stump cannot fix, which is the trade shrinkage makes.

Q. *Explain why the learning rate, the number of rounds and the tree depth cannot be tuned one at a time, and describe the procedure you would use.*
A. They are coupled: halving η roughly doubles the number of rounds needed to reach the same fit, and deeper trees reach the validation minimum in fewer rounds, so the best M depends on the other two. Procedure: grid or random search over (depth, η); for each pair run boosting with early stopping on validation (or cross-validation) to pick M^*; compare configurations by their validation loss at their own M^*; confirm the winner on an untouched test set.

8 Boosting for binary classification

Boosting the logit

Probabilities live in $(0, 1)$ and cannot be summed with tree outputs ($0.9+0.4$ is not a probability). So the ensemble produces an unconstrained score $F^{(i)}$ and maps it through the sigmoid, $\hat{p}^{(i)} = \sigma(F^{(i)})$ [on your sheet]; trees are added to F, and $\hat{p}$ is recomputed. The loss is the binary cross-entropy written in the logit, $\ell = -[y \ln \hat{p} + (1 - y) \ln(1 - \hat{p})]$. This is the discriminative route of Chapter 4 with the score $s_\theta(x)$ now a sum of trees instead of $w^\top x + b$.

Purpose. Keep every intermediate ensemble a legal probability model while adding corrections freely.

Impact. The gradient and the curvature of the loss in F are $g = \hat{p} - y$ and $h = \hat{p}(1 - \hat{p})$ [on your sheet]; the pseudo-residual $-g = y - \hat{p}$ is "how far the probability is from the label".

$g = \hat{p} - y$ and $h = \hat{p}(1 - \hat{p})$, line by line

Line 1. $\sigma'(F) = \sigma(F)(1 - \sigma(F))$: with $\sigma = (1 + e^{-F})^{-1}$, $\sigma' = e^{-F}(1 + e^{-F})^{-2} = \sigma \cdot \frac{e^{-F}}{1+e^{-F}} = \sigma(1-\sigma)$. **Line 2.** $\partial \ell / \partial \hat{p} = -\frac{y}{\hat{p}} + \frac{1-y}{1-\hat{p}}$. **Line 3 (chain rule).** $g = \frac{\partial \ell}{\partial \hat{p}} \frac{d\hat{p}}{dF} = (-\frac{y}{\hat{p}} + \frac{1-y}{1-\hat{p}})\hat{p}(1-\hat{p}) = -y(1 - \hat{p}) + (1 - y)\hat{p} = -y + y\hat{p} + \hat{p} - y\hat{p} = \hat{p} - y$. The marks are in this cancellation. **Line 4.** $h = \partial(\hat{p} - y)/\partial F = d\hat{p}/dF = \hat{p}(1-\hat{p})$. **Reading.** h is the variance of a Bernoulli with parameter $\hat{p}$: it peaks at 0.25 when $\hat{p} = 0.5$ (unsure, the loss curves most) and vanishes when $\hat{p} \to 0$ or 1 (confident, the loss is flat in F). For $y = 1$ and a very negative F (confidently wrong), $-g \to 1$ is at its largest while $h \to 0$: a large residual with almost no curvature. Keep this picture.

Gradient boosting for classification and the line-search bottleneck

At round m: (1) pseudo-residuals $r_m^{(i)} = y^{(i)} - \hat{p}_{m-1}^{(i)}$; (2) grow a regression tree on them by squared-error splits (*the split search determines the regions*); (3) with the regions fixed, choose each leaf's output by a *line search* on the true loss, $w_{q,m} = \arg\min_w \sum_{i \in S_{q,m}} \ell(y^{(i)}, F_{m-1}^{(i)} + w)$ (*the line search determines the outputs*); (4) $F_m = F_{m-1} + \eta T_m$.

Purpose. Choose leaf values that minimise the actual log loss, not the squared error of the pseudo-residuals.

Impact. For squared error step 3 was "mean residual"; for log loss the leaf objective is a sum of log-sigmoids in w with no closed form, so it must be minimised numerically leaf by leaf, round after round: the *line-search bottleneck*.

The Newton leaf value, with and without λ

Expand the loss of observation i to second order around its current logit: $\ell(F+w) \approx \ell(F) + gw + \frac{1}{2}hw^2$ [on your sheet]. Summing over a leaf and adding an L_2 penalty $\frac{\lambda}{2}w^2$ gives a quadratic in the single number w, $\sum_i \ell^{(i)} + G_q w + \frac{1}{2}(H_q+\lambda)w^2$ with $G_q = \sum_{i\in q} g^{(i)}$, $H_q = \sum_{i\in q} h^{(i)}$; its leading coefficient is positive since $h > 0$. Derivative $G_q + (H_q+\lambda)w = 0$ gives

$$w_q^* = -\frac{G_q}{H_q+\lambda} = \frac{\sum_{i\in q}(y^{(i)} - \hat{p}^{(i)})}{\sum_{i\in q}\hat{p}^{(i)}(1-\hat{p}^{(i)}) + \lambda} \quad \text{[on your sheet]}$$

and $\lambda = 0$ is the plain Newton step "total residual over total curvature". In a leaf of unsure observations ($h \approx 0.25$ each) the step is moderate; in a leaf of confidently wrong observations ($h \approx 0$) the denominator collapses and the step *explodes*. λ is a floor on the denominator that caps the step, and it matters most exactly where $\sum h$ is smallest.

Two leaves, one safe and one explosive

Leaf A holds $(y, \hat{p}) = (1, 0.2), (1, 0.4), (0, 0.8)$. $g = \hat{p} - y = -0.8, -0.6, +0.8$, $G = -0.6$; $h = \hat{p}(1-\hat{p}) = 0.16, 0.24, 0.16$, $H = 0.56$. Newton value $-G/H = 0.6/0.56 = 1.071$; with $\lambda = 1$, $0.6/1.56 = 0.385$; with $\eta = 0.1$ the logit moves by 0.0385. The positive sign raises all three probabilities, helping the two positives more than it hurts the negative.
Leaf B holds two confidently wrong positives $(1, 0.01), (1, 0.02)$: $G = -1.97$, $H = 0.0099 + 0.0196 = 0.0295$. Newton value $1.97/0.0295 = 66.8$: a logit jump of 67 from two observations, pure overfitting. With $\lambda = 1$: $1.97/1.03 = 1.91$; with $\lambda = 0.1$: 15.2. Minimum child weight (reject a leaf whose H is below a threshold) would refuse to create leaf B at all.

Confusing the gradient with the residual, and ignoring the Hessian

The slides use both $r = y - \hat{p}$ and $g = \hat{p} - y$; they are consistent ($r = -g$) but a sign slip flips the direction of every update. Always ask "gradient or negative gradient?" before reading a formula. Second: treating the Newton target $-g/h$ of a confidently wrong point ($\approx 1/\hat{p}$, huge) as its influence; its weight h is tiny and the product $h \cdot (-g/h) = y - \hat{p}$ is bounded by 1, so the point contributes at most 1 to the numerator and nothing to the denominator. Third: running unregularised Newton steps on small leaves; the outputs explode and the validation log loss diverges while training loss looks wonderful.

Regression boosting versus classification boosting

	Squared error	Log loss
what is boosted	the prediction itself	the logit F, with $\hat{p} = \sigma(F)$
pseudo-residual	$y - F$	$y - \hat{p}$
curvature h	1 for every point	$\hat{p}(1-\hat{p})$, between 0 and 0.25
leaf output	mean residual (closed form)	line search, or Newton $\sum(-g)/(\sum h + \lambda)$
risk	none special	exploding steps where $\sum h \approx 0$

Key insight. In logit space the residual is $y - \hat{p}$ and the curvature is $\hat{p}(1-\hat{p})$; the Newton step divides one by the other, which is why confident leaves need λ.

How the professor asks it

Q. *Prove $g = \hat{p} - y$ and $h = \hat{p}(1-\hat{p})$ from $\ell = -y\ln\hat{p} - (1-y)\ln(1-\hat{p})$ with $\hat{p} = \sigma(F)$, and say why the Newton leaf objective is convex.*
A. Four lines: $\sigma' = \sigma(1-\sigma)$; $\partial\ell/\partial\hat{p} = -y/\hat{p} + (1-y)/(1-\hat{p})$; multiply by $\hat{p}(1-\hat{p})$ and simplify $-y + y\hat{p} + \hat{p} - y\hat{p} = \hat{p} - y$; differentiate again, $h = d\hat{p}/dF = \hat{p}(1-\hat{p})$. Since $h > 0$ for $\hat{p} \in (0,1)$, the quadratic $G_q w + \frac{1}{2}(H_q+\lambda)w^2$ has a positive leading coefficient, so it has a unique minimiser $-G_q/(H_q+\lambda)$.

Q. *One observation with $y = 1$ has $\hat{p}$ very close to 0. Its Newton target $-g/h$ is enormous. What happens to h, and why must the two be read together?*
A. $-g = 1 - \hat{p} \to 1$, $h = \hat{p}(1-\hat{p}) \to 0$, target $-g/h = 1/\hat{p} \to \infty$: the quadratic model says the loss is flat here, so a unit residual asks for a huge step. But its weight in the leaf objective is the same $h \to 0$, so what enters $w^* = \sum(-g)/(\sum h + \lambda)$ is $h \cdot (-g/h) = 1 - \hat{p} \leq 1$ in the numerator and almost nothing in the denominator. Target alone says "thousands", weight alone says "irrelevant"; the truth is a bounded pull in the right direction. A leaf made *only* of such points has $\sum h \approx 0$: that is what λ and minimum child weight are for.

Q. *Leaf $(1, 0.3), (1, 0.6), (0, 0.2)$. Compute g, h, G, H, the leaf value for $\lambda = 0$ and $\lambda = 1$, and the loss removed at $\lambda = 1$.*
A. $g = -0.7, -0.4, +0.2$, $G = -0.9$; $h = 0.21, 0.24, 0.16$, $H = 0.61$. $w^* = 0.9/0.61 = 1.475$ at $\lambda = 0$, $0.9/1.61 = 0.559$ at $\lambda = 1$. Loss removed $\frac{1}{2}G^2/(H+\lambda) = \frac{1}{2}(0.81)/1.61 = 0.252$. The regularised step is less than half the raw step because H is small compared with $\lambda = 1$; λ bites hardest in low-curvature leaves.

9 XGBoost: one objective for the tree shape and the leaf values

The XGBoost tree objective

A proposed tree with leaves $S_1, \ldots, S_Q$ and outputs w_q is scored by

$$\mathcal{L}_m = \sum_q \sum_{i \in S_q} [g^{(i)} w_q + \tfrac{1}{2} h^{(i)} w_q^2] + \gamma Q + \frac{\lambda}{2} \sum_q w_q^2 \quad \text{[on your sheet]}$$

Three parts: the second-order Taylor approximation of the loss change produced by the tree (the constant $\sum_i \ell^{(i)}$ is dropped), a charge γ per leaf, and a charge $\frac{\lambda}{2} w_q^2$ for large outputs.

Purpose. Use one number to score both the *structure* (which observations share a leaf) and the *outputs*, so that gradients, Hessians and regularisation are used *while growing* the tree, not after.

Impact. Compared with the GBM variants, which build the structure by unweighted squared-error splits on $y - \hat{p}$ and only afterwards compute Newton outputs, XGBoost's split search already sees the curvature; it is faster (one pass) and better regularised.

Completing the square: the objective is a weighted regression

For observation i in leaf q, $\frac{1}{2}h(w + g/h)^2 = \frac{1}{2}hw^2 + gw + g^2/(2h)$, hence

$$g^{(i)} w_q + \tfrac{1}{2} h^{(i)} w_q^2 = \tfrac{1}{2} h^{(i)} \left(r_{\text{XGB}}^{(i)} - w_q\right)^2 - \frac{(g^{(i)})^2}{2h^{(i)}}, \qquad r_{\text{XGB}}^{(i)} = -\frac{g^{(i)}}{h^{(i)}} = \frac{y^{(i)} - \hat{p}^{(i)}}{\hat{p}^{(i)}(1 - \hat{p}^{(i)})} \quad \text{[on your sheet]}$$

and the dropped term does not involve w. So $\mathcal{L}_m$ is a *weighted least-squares* problem: each observation asks the tree to output r_{XGB}, its own one-step Newton correction, and its request is weighted by h. An unsure observation (h large) has a modest target and a loud voice; a confidently wrong one has a huge target ($\approx 1/\hat{p}$) and a whisper of a weight; their product $h \cdot r_{\text{XGB}} = y - \hat{p}$ is bounded by 1. "XGBoost uses the Hessian in the split search" means the tree is grown by weighted squared-error splits on these targets.

Optimal leaf value and the gain of a split

Summing over a leaf: $\phi_q(w) = G_q w + \frac{1}{2}(H_q + \lambda) w^2$, minimised at $w_q^* = -G_q/(H_q + \lambda)$ with value $\phi_q(w_q^*) = -\frac{1}{2} G_q^2/(H_q + \lambda)$ [on your sheet], "how much loss this leaf can remove with its best output" (more negative is better). Replacing a parent (G, H) by children (G_L, H_L), (G_R, H_R) with $G = G_L + G_R$, $H = H_L + H_R$ changes the data-fit part by $\phi_L + \phi_R - \phi_{\text{parent}}$ and the leaf

count by one, so

$$\text{gain} = \tfrac{1}{2}\Big[\frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{G^2}{H+\lambda}\Big] - \gamma \quad \text{[on your sheet]}$$

and the split is accepted iff gain > 0. The two penalties act differently: λ sits inside every denominator and *shrinks* all outputs and all three terms, most strongly where H is small; γ is a flat per-leaf price that *discourages* marginal splits. One line: λ shrinks leaf outputs, γ discourages unnecessary splits.

Does the split pay for its leaf?

Parent $(G, H) = (-4, 3)$, children $(-3, 1)$ and $(-1, 2)$, $\lambda = 1$, $\gamma = 0.2$. Parent: $w^* = 4/4 = 1$, $\phi = -\frac{1}{2} \cdot 16/4 = -2$. Left: $w^* = 3/2$, $\phi = -\frac{1}{2} \cdot 9/2 = -2.25$. Right: $w^* = 1/3$, $\phi = -\frac{1}{2} \cdot 1/3 = -0.167$. Children together -2.417: the fit improves by 0.417; gain $= 0.417 - 0.2 = 0.217 > 0$, accept. The split would be rejected for any $\gamma \geq 5/12 \approx 0.417$. Note why the children win: the parent mixes $G_L = -3$ with $G_R = -1$, same sign, so the parent's single output already serves both; opposite-sign gradients in one leaf cancel in G and splitting them is where the big gains are.

Sweeping λ and γ at fixed depth and rate

Setup. Flow classification, XGBoost with depth 4, $\eta = 0.1$, early stopping.

Held fixed. Depth, η, subsampling off.

What we manipulate. $\lambda \in \{0, 1, 10, 100\}$ and separately $\gamma \in \{0, 0.1, 1, 10\}$.

What we measure. Validation log loss at the early-stopping round, the average number of leaves per tree, and the distribution of leaf outputs.

What the plot shows. Raising λ compresses the histogram of leaf outputs towards 0 (and removes the extreme values from near-pure leaves) with the number of leaves nearly unchanged; raising γ cuts the number of leaves (trees stop splitting early) with the surviving outputs nearly unchanged. Validation loss is U-shaped in both.

How to read it. The two knobs have different fingerprints: λ is continuous shrinkage of outputs, γ is a discrete tax on structure. Minimum child weight is a third knob with a third fingerprint: it removes leaves whose H is small, exactly the explosive ones.

Mixing up λ, γ and the slides' λJ

The slides use λJ for a penalty on the number of leaves when introducing trees and later λ for the L_2 penalty on leaf outputs; XGBoost's per-leaf price is γ. Students write "λ discourages splits": wrong, λ shrinks outputs (continuous), γ taxes leaves (discrete, exactly γ per binary split since a split adds one leaf). A second slip: believing XGBoost "is" a different algorithm from gradient boosting; it is gradient boosting with a second-order, regularised objective used for structure and outputs, plus engineering (histogram splits, sparse handling, parallel split evaluation, GPU). Rounds remain sequential.

The four leaf-value mechanisms

Algorithm	Structure target (split search)	Leaf value mechanism
Basic boosting	raw error $y - \hat{p}$	exact numerical line search
Unregularised GBM	raw error $y - \hat{p}$	Newton step $\sum(-g)/\sum h$
Regularised GBM	raw error $y - \hat{p}$	regularised step $\sum(-g)/(\sum h + \lambda)$
XGBoost	Newton target $-g/h$, weighted by h	same regularised step, found in the same weighted pass that chose the splits; gain includes γ

The five knobs of boosted trees

Knob	What it manipulates	Fingerprint when raised	Coupled with
M (rounds)	number of corrections	training loss down for ever; validation turns up	η, depth
η	size of each correction	slower, smoother, slightly better at its own M^*	M
depth / leaves	interaction order a tree can express	fewer rounds to the minimum; overfits if deep	M, γ
λ	shrinks every leaf output, most where H small	outputs compressed, leaves unchanged	minimum child weight
γ	price per leaf	fewer leaves, outputs unchanged	depth

Key insight. XGBoost grows the tree with the same regularised quadratic it uses for the outputs: every observation asks for $-g/h$ with weight h, λ shrinks the answer and γ taxes every extra leaf.

How the professor asks it

Q. *Starting from the leaf contribution $\phi_q(w) = G_q w + \frac{1}{2}(H_q + \lambda)w^2$, derive the optimal output, the loss it removes, and the gain of splitting a parent into two children; explain why γ enters as a plain subtraction.*
A. $\phi'(w) = G_q + (H_q + \lambda)w = 0$ gives $w_q^* = -G_q/(H_q + \lambda)$; substituting, $\phi(w^*) = -G_q^2/(H_q + \lambda) + \frac{1}{2}G_q^2/(H_q + \lambda) = -\frac{1}{2}G_q^2/(H_q + \lambda)$. Replacing one leaf by two changes the fit from ϕ_{parent} to $\phi_L + \phi_R$, a decrease of $\frac{1}{2}[G_L^2/(H_L + \lambda) + G_R^2/(H_R + \lambda) - G^2/(H + \lambda)]$, and raises the leaf count by exactly one, so the penalty rises by $\gamma(Q + 1) - \gamma Q = \gamma$: a flat threshold the improvement must clear.

Q. *Parent $(G, H) = (-2, 4)$, children $(-2.5, 2)$ and $(0.5, 2)$, $\lambda = 1$, $\gamma = 0.3$. Accept the split? Interpret the signs of G_L and G_R.*
A. Parent $\phi = -\frac{1}{2} \cdot 4/5 = -0.4$. Left $\phi = -\frac{1}{2} \cdot 6.25/3 = -1.042$. Right $\phi = -\frac{1}{2} \cdot 0.25/3 = -0.042$. Children -1.083, improvement 0.683, gain $0.683 - 0.3 = 0.383 > 0$, accept. The parent's $G = -2$ hides a left group pulling strongly one way (-2.5) and a right group pulling the other way $(+0.5)$; in one leaf they partly cancel and the single output $w^* = 0.4$ serves neither well. Separating opposite-sign gradients is exactly where splits earn their keep.

Q. *Explain in one paragraph what "XGBoost uses the Hessian in the split search" means and why the GBM variants do not.*
A. Completing the square turns the second-order objective into a weighted least-squares problem with per-observation target $-g/h$ and weight h; XGBoost evaluates candidate splits by the gain formula, which depends on G and H of each child, so curvature shapes the tree. The GBM variants grow the tree by unweighted squared-error splits on the raw residual $y - \hat{p}$ and use h only afterwards for the frozen leaves' Newton outputs; their split search is blind to curvature. Consequence: XGBoost separates confident from unsure observations when that improves the quadratic objective, and its gain already includes λ and γ.

10 Multi-class boosting, AdaBoost and the libraries

K trees per round in softmax space

For K classes the ensemble keeps a vector of logits $F_1, \ldots, F_K$, maps it to probabilities with the softmax, and uses the multi-class cross-entropy $\ell = -\sum_k y_k \ln \hat{p}_k$ with one-hot y. Per observation, $g_k = \hat{p}_k - y_k$ and $\partial^2 \ell / \partial F_k \partial F_r = \hat{p}_k(\delta_{kr} - \hat{p}_r)$ [on your sheet]: $\hat{p}_k(1 - \hat{p}_k)$ on the diagonal, $-\hat{p}_k \hat{p}_r$ off it. Keeping only the diagonal, the objective becomes a *sum* of K independent binary-style objectives, so one tree is fitted per class per round with target $r_{\text{XGB},k} = (y_k - \hat{p}_k)/(\hat{p}_k(1 - \hat{p}_k))$, formally identical to the binary case.

Purpose. Reuse the binary machinery for any number of classes.

Impact. The softmax couples the classes (raising F_1 lowers every other $\hat{p}_k$); the diagonal approx-

imation ignores that coupling within a round and restores it when the softmax is recomputed from all updated logits at the end of the round. K times the work per round; the K fits can run in parallel, the rounds cannot.

Softmax derivatives

Write $\ell = -\sum_k y_k F_k + \ln \sum_j e^{F_j}$, using $\ln \hat{p}_k = F_k - \ln \sum_j e^{F_j}$ and $\sum_k y_k = 1$. Then $\partial \ell / \partial F_k = -y_k + e^{F_k} / \sum_j e^{F_j} = \hat{p}_k - y_k$. Differentiating $\hat{p}_k = e^{F_k} / \sum_j e^{F_j}$ with respect to F_r by the quotient rule gives $\hat{p}_k(\mathbf{1}[k = r] - \hat{p}_r)$.

AdaBoost versus gradient boosting versus the libraries

Name	Type	Main emphasis
AdaBoost	a boosting *algorithm*	raises the weights of misclassified observations so the next weak classifier focuses on them; the ensemble is a weighted vote of classifiers, not a sum of regression trees
Gradient boosting	a boosting *algorithm*	fits each tree to the negative gradient of a chosen loss; leaf values by line search or Newton
XGBoost	a *library*	second-order regularised objective for splits and outputs; histogram splits, sparsity, parallel split evaluation, GPU
LightGBM	a *library*	histogram-based splits and leaf-wise growth for large datasets
CatBoost	a *library*	ordered treatment of categorical features designed to limit target leakage

Listing AdaBoost, XGBoost, LightGBM and CatBoost as four algorithms of one kind

AdaBoost reweights *observations* and combines *classifiers* by weighted vote; gradient boosting fits *trees to gradients* and sums them. XGBoost, LightGBM and CatBoost are implementations of gradient-boosted trees with different engineering. Writing "XGBoost is more accurate than gradient boosting" compares a library with the algorithm it implements. Second slip: "K independent trees per round" means independent within the round under the diagonal approximation; the rounds stay sequential, so multi-class boosting is K times the work and parallelism only reduces elapsed time.

Key insight. Multi-class boosting is K binary boostings sharing one softmax, decoupled within a round by dropping the off-diagonal Hessian terms.

How the professor asks it

Q. *Why does multi-class XGBoost fit K trees per round, and in what sense are they independent?*
A. Each class has its own logit F_k and its own gradient $g_k = \hat{p}_k - y_k$, so each needs its own additive correction, hence one tree per class. The full softmax Hessian couples the classes through $-\hat{p}_k \hat{p}_r$, which would make the K leaf values of a round depend on each other; dropping those terms makes the approximate objective a sum of K separate binary-style objectives, so the K trees can be fitted separately and in parallel. The coupling returns when the softmax is recomputed jointly before the next round; the rounds are sequential.

Q. *A report says "we compared four boosting algorithms: AdaBoost, XGBoost, LightGBM and CatBoost". What is wrong with the sentence?*
A. Three of the four are libraries implementing the same algorithm, gradient-boosted trees, with different engineering (histogram splits, leaf-wise growth, ordered categorical encoding); AdaBoost is a different algorithm that reweights observations and combines weak classifiers by weighted vote. The comparison mixes an algorithm with implementations; differences among the three libraries are about speed, defaults and categorical handling, not about the boosting principle.

11 Bagging versus boosting, and when ensembles do not help

Bagging and random forests versus boosting

Question	Bagging / random forest	Boosting
training	parallel randomised fits	sequential corrections
main mechanism	average unstable predictions (variance)	reduce the current loss (mostly bias)
typical trees	deep or moderately deep	shallow (depth 2 to 6)
diversity source	bootstrap rows, feature subsets	changing residuals or gradients
main controls	tree strength versus correlation (m_{try}, leaf size), M until plateau	η, depth, rounds by early stopping, λ, γ
noise sensitivity	generally robust	can chase difficult or mislabelled cases
free validation	OOB error	none; hold out or cross-validate
parallelisation	natural across trees	limited by sequential dependence

These are tendencies: either method can affect both bias and variance.

Bagging against boosting on noisy labels

Setup. Flow data; randomly flip 10% of the training labels (not the validation labels).

Held fixed. The data, the validation set, early stopping for boosting, M on the plateau for the forest.

What we manipulate. Clean labels versus 10% flipped; forest versus boosted trees.

What we measure. Validation error of each ensemble, and for boosting the validation curve against rounds.

What the plot shows. With clean labels boosting usually edges out the forest. With flipped labels the forest degrades gently: each flipped point affects a third of the trees and is outvoted. Boosting degrades more: a flipped point is a large, persistent residual, every round tries to fit it, the leaf containing it gets an extreme output, and the validation curve turns up earlier and higher. Stronger λ, smaller η and larger minimum child weight soften this.

How to read it. Boosting treats every residual as signal; bagging treats every point as one vote. Noisy or mislabelled data favour averaging; clean data with structured residuals favour boosting.

Start from the source of the error

(1) Is one strong learner *unstable* across training samples? Bag it or use a random forest. (2) Can many small corrections improve a *structured residual pattern*? Gradient-boosted trees. (3) Do credible but different model families make *complementary* errors? Validated voting, averaging or stacking. (4) Is the dominant error caused by *missing information* or *deployment shift*? No ensemble will solve it; fix the data and the evaluation protocol.

Purpose. Choose the method from the diagnosis, not from a favourite.

Impact. Ensembles do not help when all models miss the same information ($\rho \approx 1$), when the models are too similar, when boosting starts fitting noise, outliers or wrong labels, when validation data have been reused so often that the choices overfit them, or when test or deployment data differ from the training data. Extra models also cost computation, memory and system complexity, which Lab 3 measures.

Credit scoring, two failure stories

A bank's churn model is a single deep tree whose predictions change every quarter when retrained; the fix was a random forest, which stabilised the ranking of customers. A fraud model's residuals showed a clear pattern (transactions just under the reporting threshold were systematically under-scored); boosted trees learned the correction in a few rounds. A third team tried both on a model whose main error was that merchant category was unavailable at scoring time; both ensembles reproduced the single model's error, and the only fix was to make the feature available.

Choosing by fashion and blaming the ensemble

"Boosting always wins on tabular data" leads to boosting a model whose error is a label-noise problem (it gets worse) or a shift problem (nothing changes). "A forest is safer" leads to bagging a stable linear model (nothing changes). The inflated quantity is the time spent; the deflated one is the trust in the method. Fix: run the instability experiment and the residual inspection first, then pick.

Key insight. Bag what is unstable, boost what has structured residuals, stack what is complementary, and fix the data when the error is shared.

How the professor asks it

Q. *For each situation say whether bagging, boosting, neither or something else is the right response and why: (a) a deep tree whose boundary changes drastically between bootstrap samples; (b) a model whose residuals are systematically positive for one subgroup; (c) a detector that fails on attacks through a gateway absent from the features; (d) a dataset with* 15% *mislabelled rows.*
A. (a) Bagging or a forest: the error is variance and averaging cancels it. (b) Boosting: the residual pattern is structured, and successive trees can learn the subgroup correction. (c) Neither: the error is shared by every model because the information is missing; add the feature and re-evaluate under a grouped or temporal split. (d) Prefer a forest and clean the labels: boosting chases mislabelled points as if they were signal; if boosting must be used, lower η, raise λ and minimum child weight and stop early.

Q. *Explain why boosting has no out-of-bag error and what you use instead.*
A. Every round fits a tree on the full training set (its residuals), so no observation is left out of any model and there is no set of models that never saw a given point; the OOB construction needs bootstrap omission. Use a held-out validation set or K-fold cross-validation to choose M by early stopping and the other knobs by grid search, and an untouched test set at the end.

12 Stacking

Stacking; the meta-model

Base models $m = 1, \dots, M$ (a logistic regression, a random forest, a boosted ensemble) produce predictions z_m, usually probabilities; a *meta-model* h_γ maps them to the final prediction, $\hat{y} = h_{\hat{\gamma}}(z_1, \dots, z_M)$. For binary classification the common meta-model is a logistic regression on the base probabilities, $\hat{p}_{\text{stack}} = \sigma(\beta_0 + \sum_m \beta_m z_m)$ [on your sheet], fitted on the true labels and on *out-of-fold* base predictions. Soft voting is the special case $\beta_m = 1/M$ up to the sigmoid.

Purpose. Let the data decide the weights: a large positive β_m trusts model m; a small or negative one down-weights an unhelpful or redundant model.

Impact. The meta-model has only M inputs and is kept simple and regularised so that it cannot overfit; the gain comes from complementary base models, not from a clever combiner.

Stacking with in-sample versus out-of-fold base predictions

Setup. Three base models including a deep random forest; a logistic meta-model.

Held fixed. The base models, the data, the validation set.

What we manipulate. How the meta-model's training inputs z are produced: (A) each base model predicts the *same* rows it was trained on; (B) $K = 5$ folds, each base model fitted on four folds and predicting the fifth, so that every row's z comes from a model that never saw it.

What we measure. The fitted β, the meta-model's training loss, and its validation loss.

What the plot shows. In (A) the forest's z equals the label almost exactly on its own training rows, so β_{forest} is huge and the others are near 0; training loss is near 0; validation loss is *worse* than the forest alone. In (B) the weights are moderate and spread, training loss is realistic, validation loss is below every base model.

How to read it. (A) is leakage of training information into the combination rule: the meta-model learns

to trust whichever model overfits most. Out-of-fold predictions are the cure. At prediction time the base models are refitted on all the training data, $(z_1, \ldots, z_M)$ is computed for the new input, and passed through the fitted meta-model.

Training the meta-model on in-sample predictions, and confusing OOF with OOB

The tempting shortcut is to fit the base models once on all data and reuse their predictions on the same rows as meta-features. A deep tree or forest predicts its own training rows almost perfectly, so the meta-model's fit is unrealistically optimistic and it over-weights the most overfitted member; validation performance drops below the best base model. Fix: out-of-fold predictions from an explicit fold split. These are not OOB predictions: OOB is a bootstrap by-product inside a bagged ensemble, out-of-fold is a fold split that works for any base model, including boosting and logistic regression.

Key insight. Stacking is soft voting with learned weights, and it is honest only if the weights are learned from predictions the base models made on rows they never saw.

How the professor asks it

Q. *Describe the training and the prediction phases of a stacked classifier and explain the role of out-of-fold predictions.*
A. Training: split the training set into K folds; for each fold fit every base model on the other folds and predict the held-out fold; join the held-out predictions so every row has $z_m^{(i)}$ from a model that never saw it; fit the meta-model (regularised logistic regression) on $(z^{(i)}, y^{(i)})$. Prediction: refit each base model on all the training data, compute $(z_1, \ldots, z_M)$ for the new input, apply the meta-model. Out-of-fold predictions keep the meta-model from learning "trust the model that memorised its training rows", which in-sample predictions would teach it.

Q. *Give a counterexample showing that stacking can be worse than its best base model, and the condition under which it is better.*
A. Worse: train the meta-model on in-sample predictions with a deep forest among the bases; β_{forest} dominates, the stack reproduces the forest's overfitting and loses to a well-regularised logistic base on validation. Better: when the bases make complementary errors (a linear model good on the bulk, a forest good on interactions) and the meta-weights are fitted on out-of-fold predictions with regularisation, the learned weighting beats each base and beats equal-weight soft voting.

13 Mixture of experts

Mixture of experts; gate; routing

Experts $p_m(y \mid x)$ are combined by a *gating model* $g_m(x) \geq 0$, $\sum_m g_m(x) = 1$: $p(y \mid x) = \sum_m g_m(x) p_m(y \mid x)$ [on your sheet]. A linear gate scores expert m by $a_m(x) = \beta_m^\top x + b_m$ (a small network for nonlinear routing) and normalises with a softmax, $g = \text{softmax}(a(x))$ [on your sheet]. *Soft routing* uses all experts with their weights; *top-k routing* evaluates only the k experts with the largest gate scores. Gate and experts are trained *jointly* by minimising the negative log-likelihood of the mixture, $-\sum_i \log \left[\sum_m g_m(x^{(i)}) p_m(y^{(i)} \mid x^{(i)}) \right]$ [on your sheet].

Purpose. Let different experts own different regions of the input space, with the regions discovered during training rather than specified in advance.

Impact. Unlike stacking, where β_m is the same for every input, here the weights change with x; top-k routing is also a computational device (conditional computation): with many large experts and $k = 2$, only two are evaluated per input, which is how very large language and vision models use mixtures of experts.

Watching experts specialise, and collapse

Setup. Two-feature flow data with three visibly different attack regimes; three small experts and a linear gate.

Held fixed. The architecture, the loss.

What we manipulate. Training with and without a load-balancing penalty on the gate.

What we measure. Over epochs: the share of inputs routed mainly to each expert, and each expert's validation loss on its own region.

What the plot shows. Without the penalty: one expert's share climbs to nearly 100% while the others idle, the "mixture" degenerates to a single model. With it: the three shares settle near a third each, the gate map shows three regions matching the regimes, and the mixture's validation loss is below any single expert.

How to read it. This is *expert collapse*: an expert that receives more gate weight early learns faster, gets better, attracts still more inputs, and the rich get richer. Load-balancing terms encourage the gate to spread its weight.

Stacking versus mixture of experts

	Stacking	Mixture of experts
weights	global β_m, same for every x	$g_m(x)$, input-dependent
training	base models first, meta-model after, on out-of-fold predictions	gate and experts jointly on the mixture likelihood
specialisation	none; one global trust level per model	experts own regions discovered in training
risk	leakage through in-sample predictions	expert collapse
compute	all bases evaluated	top-k routing evaluates few experts

Key insight. A mixture of experts learns *who* to trust *where*; without load balancing it learns to trust one expert everywhere.

How the professor asks it

Q. *What distinguishes a mixture of experts from stacking, and what is expert collapse?*
A. Stacking learns one global combination $h(z_1, \ldots, z_M)$ after the bases are fitted; a mixture learns input-dependent weights $g_m(x)$ jointly with the experts, so experts can specialise by region of feature space. Expert collapse: an expert favoured early improves faster, attracts more weight, and ends up handling almost all inputs while the others stay untrained; a load-balancing penalty on the gate counters it. Consequence: the mixture needs joint training and balancing, where stacking only needs out-of-fold predictions.

14 Where ensembles are used, and Lab 3

Method, application, reason

Method	Representative applications	Why it is used
Voting / averaging	medical prediction, sensor fusion, deep-network ensembles	combines different models; more stable predictions
Random forest	intrusion detection, remote sensing, bioinformatics, tabular problems	strong baseline with nonlinearity, interactions, little preprocessing
Gradient-boosted trees	credit scoring, fraud detection, churn, ranking on tabular data	high accuracy from successive corrections
Stacking	data-science competitions, systems combining model families	learns when one model should complement another
Mixture of experts	large language and vision models, multi-domain prediction	specialisation and conditional computation

Lab 3 at a glance

Tree ensembles on the cybersecurity flow data of Labs 1 and 2, one fixed stratified split and identical rows for every model, so that differences reflect models and not data. Binary classification: bagged trees and a random forest, asking how much aggregation stabilises the model. Boosting: gradient boosting and XGBoost, asking how rate, depth and rounds interact. Model comparison against the best Lab 2 models, asking which tree errors are corrected. Multiclass: which attack classes remain difficult. Port information: with and without destination port. Efficiency: fit time, prediction time and model size, asking what higher accuracy costs.

How the professor asks it

Q. *Why does Lab 3 insist on the same fixed stratified split and the same rows for every model?*
A. So that a difference in validation or test score can be attributed to the model and not to the sample: with different rows, a forest and a boosted model would be scored on different attacks and the comparison would mix model variance with data variance. Stratification keeps the rare attack classes present in every subset. The same discipline applies to OOB versus validation comparisons: a forest's OOB error and a boosted model's validation error are not on the same rows and should not be compared directly.

Coverage of the textbook chapter

Every item of the textbook chapter, and where it is treated here.

- Why combine: wisdom of the crowd, instability, three families: §1.
- Averaging, hard and soft voting, 0.64 example, calibration gotcha: §2.
- Theorem: variance of an average of correlated errors; $0.232\sigma^2$; bias not removed: §2.
- Decision tree definition, optimal leaf constant, Gini, greedy split by hand, why trees in ensembles: §3.
- Bootstrap sample, count table, out-of-bag fraction theorem, OOB prediction and error, tune the ensemble not the trees: §4.
- Random forest definition, probability a feature is offered, strength versus correlation, controls, forest-level selection: §5.
- MDI credit and forest-level MDI, bias gotcha; permutation importance, masking and causality gotcha: §6.
- Additive model, residual-fitting theorem, gradient boosting algorithm, one round by hand, deep versus shallow, the triad and early stopping: §7.
- Logit boosting, g and h theorem, classification algorithm and line search, Newton leaf value with λ, worked leaf: §8.
- XGBoost objective, weighted-regression theorem, leaf value and gain theorem, worked gain, four mechanisms, further controls: §9.
- Multi-class derivatives, one multi-class round, AdaBoost and libraries gotcha: §10.
- Bagging versus boosting table, start from the source of error, when ensembles do not help: §11.

- Stacking definition, out-of-fold technique, in-sample gotcha: §12.
- Mixture of experts, joint training, expert collapse: §13.
- Applications table, Lab 3, notes on the slides (sign conventions, λJ versus γQ): §14, §8, §9.

The ten mistakes of this chapter

1. Expecting an ensemble to remove a *shared* error (missing feature, shift): averaging cancels only what differs.
2. Quoting σ^2/M for correlated models; the floor is $\rho\sigma^2$ and more trees cannot pass it.
3. Soft-voting with a miscalibrated member, which then decides every vote; or hard-voting away a deserved confidence.
4. Reporting the OOB error as a test result, or selecting on it repeatedly and reporting the minimum.
5. Tuning each tree of a forest separately, or giving the trees all features to make them "stronger".
6. Reading feature importance as causality, or ignoring that correlated twins mask each other and that MDI favours many split points.
7. Choosing the number of boosting rounds by training loss, or comparing learning rates at a fixed M.
8. Sign slips between $r = y - \hat{p}$ and $g = \hat{p} - y$; forgetting that $h \to 0$ makes Newton steps explode and that λ and minimum child weight exist for that.
9. Saying λ discourages splits: λ shrinks outputs, γ taxes leaves; calling XGBoost, LightGBM, CatBoost and AdaBoost "four algorithms".
10. Training a stacking meta-model on in-sample base predictions; confusing out-of-fold with out-of-bag; expecting an OOB error from boosting.

Chapter 5 in ten lines

1. A flexible single fit has high variance; averaging competent models with different errors cancels part of it; shared errors survive.
2. $\mathrm{Var}(\bar{e}) = \sigma^2(\rho + (1-\rho)/M)$: independent errors give σ^2/M, correlation leaves a floor $\rho\sigma^2$; every trick lowers ρ.
3. Trees: boxes with one constant each (mean, median, majority), greedy Gini splits, flexible and unstable, the ideal ensemble member.
4. Bagging: bootstrap samples (63.2% in, 36.8% out), average or vote; OOB error is free internal validation, not a test set.
5. Random forest: m_{try} random candidates per split decorrelate the trees; weaker trees, stronger forest; tune at forest level; add trees until the OOB plateau.
6. Importance: MDI (training credits, biased to many thresholds) and permutation (error increase on OOB rows); masking by correlation; never causality.
7. Boosting: $F_m = F_{m-1} + \eta T_m$, each tree fitted to the negative gradient; squared error gives residuals; M, η, depth are coupled and chosen with early stopping on validation.
8. Classification boosts the logit: $g = \hat{p} - y$, $h = \hat{p}(1-\hat{p})$; Newton leaf $\sum(-g)/(\sum h + \lambda)$; small $\sum h$ explodes without λ.
9. XGBoost: one regularised second-order objective for splits and outputs; target $-g/h$ with weight h; $w^* = -G/(H+\lambda)$; gain minus γ per leaf.
10. Stacking learns global weights from out-of-fold predictions; a mixture of experts learns input-dependent weights jointly with the experts; bag the unstable, boost the structured residual, and fix the data when the error is shared.